

Chapter

1

INTRODUCTION

Commercial usage of computers now spans the last sixty years. Computers were very slow in the initial years and lacked sophistication. Since then, their computational power and sophistication increased rapidly, while their prices dropped dramatically. To get an idea of the kind of improvements that have occurred to computers, consider the following analogy. If similar improvements could have occurred to aircrafts, now personal mini-airplanes should have become available, costing as much as a bicycle, and flying at over 1000 times the speed of the supersonic jets. To say it in other words, the rapid strides in computing technologies are unparalleled in any other field of human endeavour.

Let us now reflect the impact of the astounding progress made to the hardware technologies on the software. The more powerful a computer is, the more sophisticated programs can it run. Therefore, with every increase in the raw computing capabilities of computers, software engineers have been called upon to solve increasingly larger and complex problems, and that too in cost-effective and efficient ways. Software engineers have coped up with this challenge by innovating and building upon their past programming experiences.

The innovations and past experiences towards writing good quality programs cost-effectively, have contributed to the emergence of the software engineering discipline.

Let us now examine the scope of the software engineering discipline more closely.

What is software engineering?

A popular definition of software engineering is: "A systematic collection of good program development practices and techniques". Good program

development techniques have resulted from research innovations as well as from the lessons learnt by programmers through years of programming experiences. An alternative definition of software engineering is: “An *engineering approach* to develop software”. Based on these two point of views, we can define software engineering as follows:

Software engineering discusses systematic and cost-effective techniques for software development. These techniques help develop software using an engineering approach.

Let us now try to figure out what exactly is meant by an *engineering approach* to develop software. We explain this using an analogy. Suppose you have asked a petty contractor to build a small house for you. Petty contractors are not really experts in house building.

They normally carry out minor repair works and at most undertake very small building works such as the construction of boundary walls. Now faced with the task of building a complete house, your petty contractor would draw upon all his knowledge regarding house building. Yet, he may often be clueless regarding what to do. For example, he might not know the optimal proportion in which cement and sand should be mixed to realise sufficient strength for supporting the roof. In such situations, he would have to fall back upon his intuitions. He would normally succeed in his work, if the house you asked him to construct is sufficiently small. Of course, the house constructed by him may not look as good as one constructed by a professional, may lack proper planning, and display several defects and imperfections. It may even cost more and take longer to build.

Now, suppose you entrust your petty contractor to build a large 50-storeyed commercial complex for you. He might exercise prudence, and politely refuse to undertake your request. On the other hand, he might be ambitious and agree to undertake the task. In the later case, he is sure to fail. The failure might come in several forms—the building might collapse during the construction stage itself due to his ignorance of the basic theories concerning the strengths of materials; the construction might get unduly delayed, since he may not prepare proper estimates and detailed plans regarding the types and quantities of raw materials required, the times at which these are required, etc. In short, to be successful in constructing a building of large magnitude, one needs a good understanding of various civil and architectural engineering techniques such as analysis, estimation, prototyping, planning, designing, and testing. Similar is the case with the software development

projects. For sufficiently small-sized problems, one might proceed according to one's intuition and succeed; though the solution may have several imperfections, cost more, take longer to complete, etc. But, failure is almost certain, if one without a sound understanding of the software engineering principles undertakes a large-scale software development work.

Is software engineering a science or an art?

Several people hold the opinion that writing good quality programs is an art. In this context, let us examine whether software engineering is really a form of art or is it akin to other engineering disciplines. There exist several fundamental issues that set engineering disciplines such as software engineering and civil engineering apart from both science and arts disciplines. Let us now examine where software engineering stands based on an investigation into these issues:

- Just as any other engineering discipline, software engineering makes heavy use of the knowledge that has accrued from the experiences of a large number of practitioners. These past experiences have been systematically organised and wherever possible theoretical basis to the empirical observations have been provided. Whenever no reasonable theoretical justification could be provided, the past experiences have been adopted as rule of thumb. In contrast, all scientific solutions are constructed through rigorous application of provable principles.
- As is usual in all engineering disciplines, in software engineering several conflicting goals are encountered while solving a problem. In such situations, several alternate solutions are first proposed. An appropriate solution is chosen out of the candidate solutions based on various trade-offs that need to be made on account of issues of cost, maintainability, and usability. Therefore, while arriving at the final solution, several iterations are possible.
- Engineering disciplines such as software engineering make use of only well-understood and well-documented principles. Art, on the other hand, is often based on making subjective judgement based on qualitative attributes and using ill-understood principles.

From the above, we can easily infer that software engineering is in many ways similar to other engineering disciplines such as civil engineering or electronics engineering.

1.1 EVOLUTION—FROM AN ART FORM TO AN ENGINEERING DISCIPLINE

In this section, we review how starting from an esoteric art form, the software engineering discipline has evolved over the years.

1.1.1 Evolution of an Art into an Engineering Discipline

Software engineering principles have evolved over the last sixty years with contributions from numerous researchers and software professionals. Over the years, it has emerged from a pure art to a craft, and finally to an engineering discipline.

The early programmers used an *ad hoc* programming style. This style of program development is now variously being referred to as *exploratory*, *build and fix*, and *code and fix* styles.

In a build and fix style, a program is quickly developed without making any specification, plan, or design. The different imperfections that are subsequently noticed are fixed.

The exploratory programming style is an informal style in the sense that there are no set rules or recommendations that a programmer has to adhere to—every programmer himself evolves his own software development techniques solely guided by his own intuition, experience, whims, and fancies. The exploratory style comes naturally to all first time programmers. Later in this chapter we point out that except for trivial problems, the exploratory style usually yields poor quality and unmaintainable code and also makes program development very expensive as well as time-consuming.

As we have already pointed out, the build and fix style was widely adopted by the programmers in the early years of computing history. We can consider the exploratory program development style as an art—since this style, as is the case with any art, is mostly guided by intuition. There are many stories about programmers in the past who were like proficient artists and could write good programs using an essentially build and fix model and some esoteric knowledge. The bad programmers were left to wonder how could some programmers effortlessly write elegant and correct programs each time. In contrast, the programmers working in modern software industry rarely make use of any esoteric knowledge and develop software by applying some well-understood principles.

1.1.2 Evolution Pattern for Engineering Disciplines

If we analyse the evolution of the software development styles over the last sixty years, we can easily notice that it has evolved from an esoteric art form to a craft form, and then has slowly emerged as an engineering discipline. As a matter of fact, this pattern of evolution is not very different from that seen in other engineering disciplines. Irrespective of whether it is iron making, paper making, software development, or building construction; evolution of technology has followed strikingly similar patterns. This pattern of technology development has schematically been shown in Figure 1.1. It can be seen from Figure 1.1 that every technology in the initial years starts as a form of art. Over time, it graduates to a craft and finally emerges as an engineering discipline. Let us illustrate this fact using an example. Consider the evolution of the iron making technology. In ancient times, only a few people knew how to make iron. Those who knew iron making, kept it a closely-guarded secret. This esoteric knowledge got transferred from generation to generation as a family secret. Slowly, over time technology graduated from an art to a craft form where tradesmen shared their knowledge with their apprentices and the knowledge pool continued to grow. Much later, through a systematic organisation and documentation of knowledge, and incorporation of scientific basis, modern steel making technology emerged. The story of the evolution of the software engineering discipline is not much different. As we have already pointed out, in the early days of programming, there were good programmers and bad programmers. The good programmers knew certain principles (or tricks) that helped them write good programs, which they seldom shared with the bad programmers. Program writing in later years was akin to a craft. Over the next several years, all good principles (or tricks) that were organised into a body of knowledge that forms the discipline of software engineering.

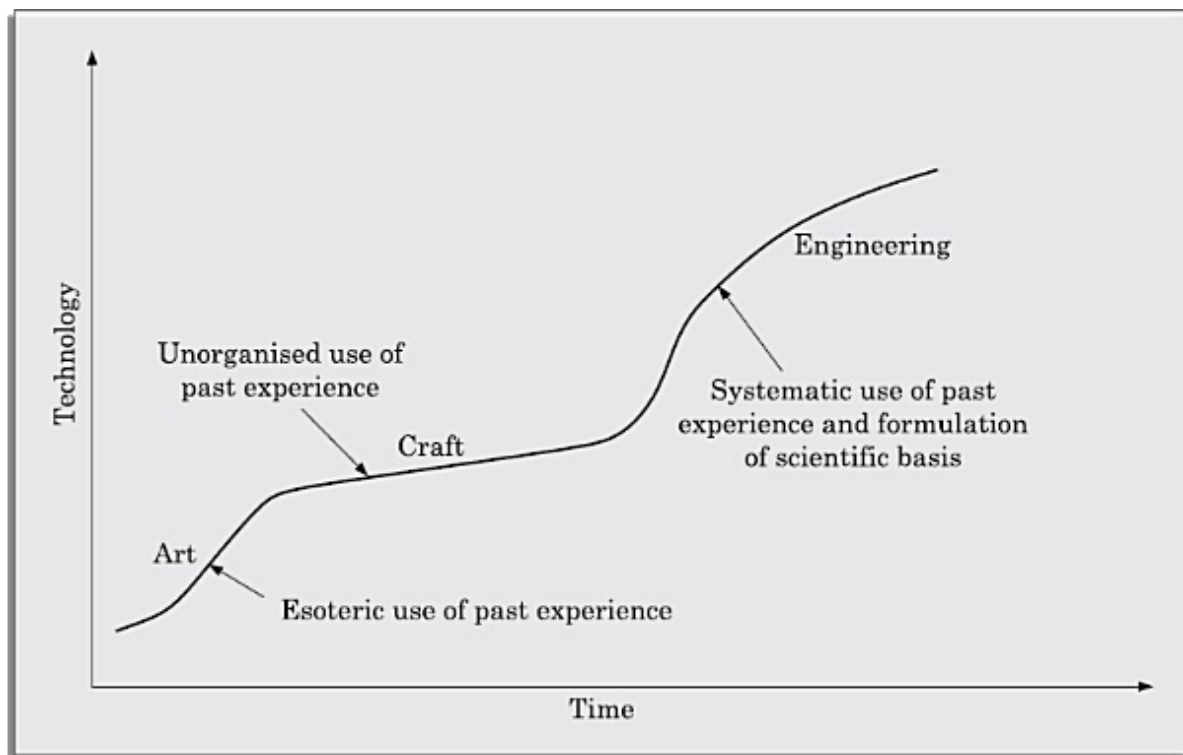


Figure 1.1: Evolution of technology with time.

Software engineering principles are now being widely used in industry, and new principles are still continuing to emerge at a very rapid rate—making this discipline highly dynamic. In spite of its wide acceptance, critics point out that many of the methodologies and guidelines provided by the software engineering discipline lack scientific basis, are subjective, and often inadequate. Yet, there is no denying the fact that adopting software engineering techniques facilitates development of high quality software in a cost-effective and timely manner. Software engineering practices have proven to be indispensable to the development of large software products—though exploratory styles are often used successfully to develop small programs such as those written by students as classroom assignments.

1.1.3 A Solution to the Software Crisis

At present, software engineering appears to be among the few options that are available to tackle the present software crisis. But, what exactly is the present software crisis? What are its symptoms, causes, and possible solutions? To understand the present software crisis, consider the following facts. The expenses that organisations all over the world are incurring on software purchases as compared to the expenses incurred on hardware purchases have been showing an worrying trend over the years (see Figure 1.2). As can be seen in the

figure, organisations are spending increasingly larger portions of their budget on software as compared to that on hardware. Among all the symptoms of the present software crisis, the trend of increasing software costs is probably the most vexing.

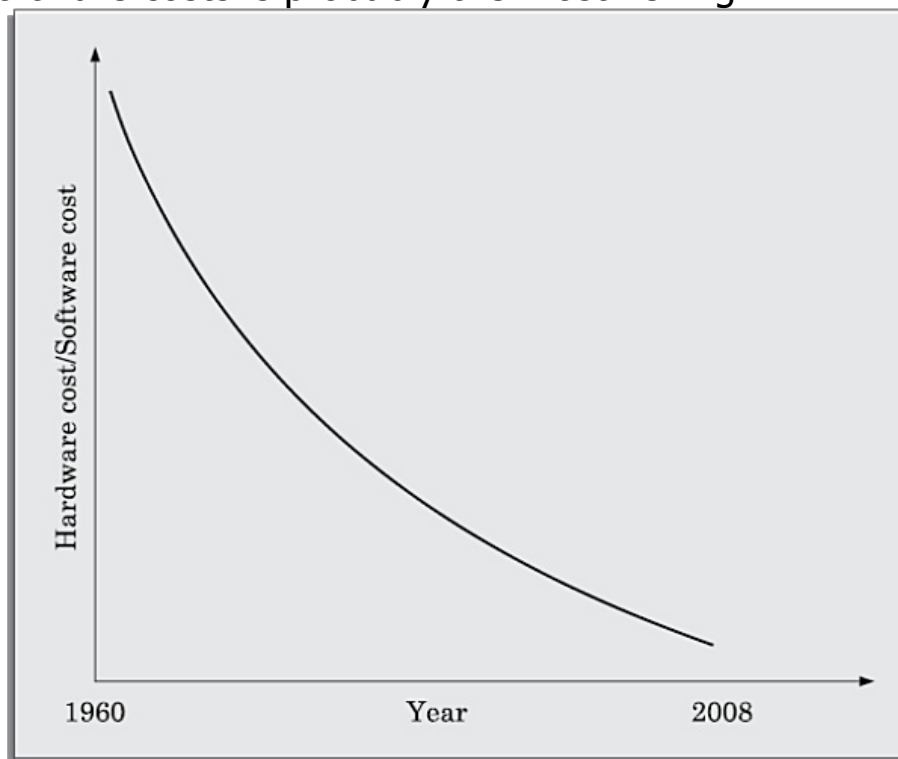


Figure 1.2: Relative changes of hardware and software costs over time.

Not only are the software products becoming progressively more expensive than hardware, but they also present a host of other problems to the customers—software products are difficult to alter, debug, and enhance; use resources non-optimally; often fail to meet the user requirements; are far from being reliable; frequently crash; and are often delivered late.

At present, many organisations are actually spending much more on software than on hardware. If this trend continues, we might soon have a rather amusing scenario. Not long ago, when you bought any hardware product, the essential software that ran on it came free with it. But, unless some sort of revolution happens, in not very distant future, hardware prices would become insignificant compared to software prices—when you buy any software product the hardware on which the software runs would come free with the software!!!

The symptoms of software crisis are not hard to observe. But, what are the factors that have contributed to the present software crisis? Apparently, there are many factors, the important ones being—rapidly increasing problem size, lack of adequate training in software engineering techniques, increasing skill

shortage, and low productivity improvements. What is the remedy? It is believed that a satisfactory solution to the present software crisis can possibly come from a spread of software engineering practices among the developers, coupled with further advancements to the software engineering discipline itself.

With this brief discussion on the evolution and impact of the discipline of software engineering, we now examine some basic concepts pertaining to the different types of software development projects that are undertaken by software companies.

1.2 SOFTWARE DEVELOPMENT PROJECTS

Before discussing about the various types of development projects that are being undertaken by software development companies, let us first understand the important ways in which professional software differs from toy software such as those written by a student in his first programming assignment.

Programs *versus* Products

Many toy software are being developed by individuals such as students for their classroom assignments and hobbyists for their personal use. These are usually small in size and support limited functionalities. Further, the author of a program is usually the sole user of the software and himself maintains the code. These toy software therefore usually lack good user-interface and proper documentation. Besides these may have poor maintainability, efficiency, and reliability. Since these toy software do not have any supporting documents such as users' manual, maintenance manual, design document, test documents, etc., we call these toy software as *programs*.

In contrast, professional software usually have multiple users and, therefore, have good user-interface, proper users' manuals, and good documentation support. Since, a software product has a large number of users, it is systematically designed, carefully implemented, and thoroughly tested. In addition, a professionally written software usually consists not only of the program code but also of all associated documents such as requirements specification document, design document, test document, users' manuals, etc. A further difference is that professional software are often too large and complex to be developed by any single individual. It is usually developed by a group of developers working in a team.

A professional software is developed by a group of software developers working together in a team. It is therefore necessary for them to use some systematic development methodology. Otherwise, they would find it very difficult to interface and understand each other's work, and produce a coherent set of documents.

Even though software engineering principles are primarily intended for use in development of professional software, many results of software engineering can effectively be used for development of small programs as well. However, when developing small programs for personal use, rigid adherence to software engineering principles is often not worthwhile. An ant can be killed using a gun, but it would be ridiculously inefficient and inappropriate. CAR Hoare [1994] observed that rigorously using software engineering principles to develop toy programs is very much like employing civil and architectural engineering principles to build sand castles for children to play.

1.2.1 Types of Software Development Projects

A software development company is typically structured into a large number of teams that handle various types of software development projects. These software development projects concern the development of either software product or some software service. In the following subsections, we distinguish between these two types of software development projects.

Software products

We all know of a variety of software such as Microsoft's Windows and the Office suite, Oracle DBMS, software accompanying a camcorder or a laser printer, etc. These software are available off-the-shelf for purchase and are used by a diverse range of customers. These are called *generic software products* since many users essentially use the same software. These can be purchased off-the-shelf by the customers. When a software development company wishes to develop a generic product, it first determines the features or functionalities that would be useful to a large cross section of users. Based on these, the development team draws up the product specification on its own. Of course, it may base its design discretion on feedbacks collected from a large number of users. Typically, each software product is targetted to some market segment (set of users). Many companies find it

advantageous to develop *product lines* that target slightly different market segments based on variations of essentially the same software. For example, Microsoft targets desktops and laptops through its *Windows 8* operating system, while it targets high-end mobile handsets through its *Windows mobile* operating system, and targets servers through its *Windows server* operating system.

Software services

A software service usually involves either development of a *customised software* or development of some specific part of a software in an outsourced mode. A *customised software* is developed according to the specification drawn up by one or at most a few customers. These need to be developed in a short time frame (typically a couple of months), and at the same time the development cost must be low. Usually, a developing company develops customised software by tailoring some of its existing software. For example, when an academic institution wishes to have a software that would automate its important activities such as student registration, grading, and fee collection; companies would normally develop such a software as a customised product. This means that for developing a customised software, the developing company would normally tailor one of its existing software products that it might have developed in the past for some other academic institution.

In a customised software development project, a large part of the software is reused from the code of related software that the company might have already developed. Usually, only a small part of the software that is specific to some client is developed. For example, suppose a software development organisation has developed an academic automation software that automates the student registration, grading, Establishment, hostel and other aspects of an academic institution. When a new educational institution requests for developing a software for automation of its activities, a large part of the existing software would be reused. However, a small part of the existing code may be modified to take into account small variations in the required features. For example, a software might have been developed for an academic institute that offers only regular residential programs, the educational institute that has now requested for a software to automate its activities also offers a distance mode post graduate program where the teaching and sessional evaluations are done by the local centres.

Another type of software service is *outsourced software*. Sometimes, it can

*****ebook converter DEMO - www.ebook-converter.com*****

make good commercial sense for a company developing a large project to outsource some parts of its development work to other companies. The reasons behind such a decision may be many. For example, a company might consider the outsourcing option, if it feels that it does not have sufficient expertise to develop some specific parts of the software; or if it determines that some parts can be developed cost-effectively by another company. Since an outsourced project is a small part of some larger project, outsourced projects are usually small in size and need to be completed within a few months or a few weeks of time.

The types of development projects that are being undertaken by a company can have an impact on its profitability. For example, a company that has developed a generic software product usually gets an uninterrupted stream of revenue that is spread over several years. However, this entails substantial upfront investment in developing the software and any return on this investment is subject to the risk of customer acceptance. On the other hand, outsourced projects are usually less risky, but fetch only one time revenue to the developing company.

1.2.2 Software Projects Being Undertaken by Indian Companies

Indian software companies have excelled in executing software services projects and have made a name for themselves all over the world. Of late, the Indian companies have slowly started to focus on product development as well. Can you recall the names of a few software products developed by Indian software companies? Let us try to hypothesise the reason for this situation. Generic product development entails certain amount of business risk. A company needs to invest upfront and there is substantial risks concerning whether the investments would turn profitable. Possibly, the Indian companies were risk averse.

Till recently, the world-wide sales revenue of software products and services were evenly matched. But, of late the services segment has been growing at a faster pace due to the advent of application service provisioning and cloud computing. We discuss these issues in Chapter 15.

1.3 EXPLORATORY STYLE OF SOFTWARE DEVELOPMENT

We have already discussed that the *exploratory program development style* refers to an informal development style where the programmer makes use of his own intuition to develop a program rather than making use of

the systematic body of knowledge categorized under the software engineering discipline. The exploratory development style gives complete freedom to the programmer to choose the activities using which to develop software. Though the exploratory style imposes no rules a typical development starts after an initial briefing from the customer. Based on this briefing, the developers start coding to develop a working program. The software is tested and the bugs found are fixed. This cycle of testing and bug fixing continues till the software works satisfactorily for the customer. A schematic of this work sequence in a build and fix style has been shown graphically in Figure 1.3. Observe that coding starts after an initial customer briefing about what is required. After the program development is complete, a test and fix cycle continues till the program becomes acceptable to the customer.

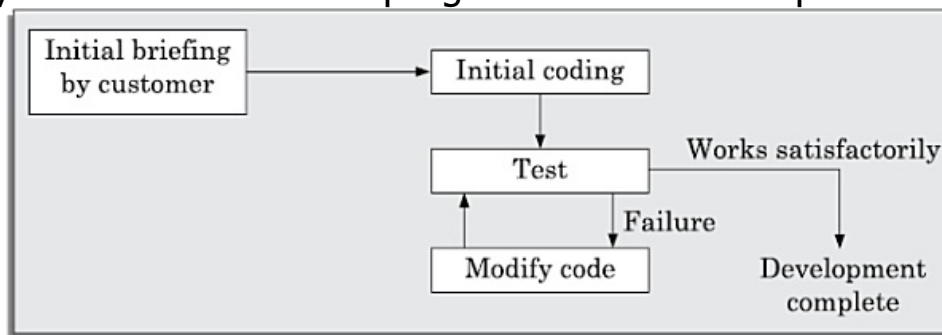


Figure 1.3: Exploratory program development.

An exploratory development style can be successful when used for developing very small programs, and not for professional software. We had examined this issue with the help of the petty contractor analogy. Now let us examine this issue more carefully.

What is wrong with the exploratory style of software development?

Though the exploratory software development style is intuitively obvious, no software team can remain competitive if it uses this style of software development. Let us investigate the reasons behind this. In an exploratory development scenario, let us examine how do the effort and time required to develop a professional software increases with the increase in program size. Let us first consider that exploratory style is being used to develop a professional software. The increase in development effort and time with problem size has been indicated in Figure 1.4. Observe the thick line plot that represents the case in which the exploratory style is used to develop a program. It can be seen that as the program size increases, the required effort and time increases almost exponentially. For large problems, it would

take too long and cost too much to be practically meaningful to develop the program using the exploratory style of development. The exploratory development approach is said to break down after the size of the program to be developed increases beyond certain value. For example, using the exploratory style, you may easily solve a problem that requires writing only 1000 or 2000 lines of source code. But, if you are asked to solve a problem that would require writing one million lines of source code, you may never be able to complete it using the exploratory style; irrespective of the amount time or effort you might invest to solve it. Now observe the thin solid line plot in Figure 1.4 which represents the case when development is carried out using software engineering principles. In this case, it becomes possible to solve a problem with effort and time that is almost linear in program size. On the other hand, if programs could be written automatically by machines, then the increase in effort and time with size would be even closer to a linear (dotted line plot) increase with size.

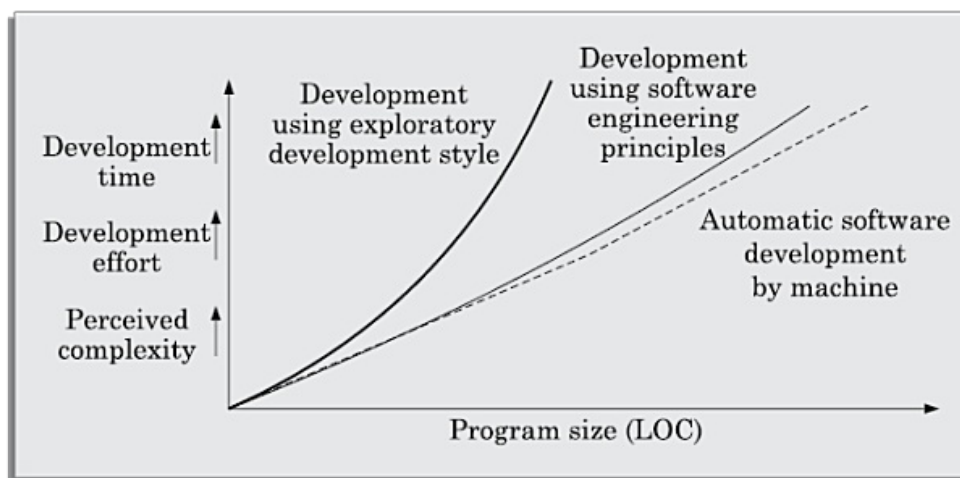


Figure 1.4: Increase in development time and effort with problem size.

Now let us try to understand why does the effort required to develop a program grow exponentially with program size when the exploratory style is used and then this approach to develop a program completely breaks down when the program size becomes large? To get an insight into the answer to this question, we need to have some knowledge of the human cognitive limitations (see the discussion on human psychology in subsection 1.3.1). As we shall see, the perceived (or psychological) complexity of a problem grows exponentially with its size. Please note that the perceived complexity of a problem is not related to the time or space complexity issues with which you are likely to be familiar with from a basic course on algorithms.

experienced by a programmer while solving the problem using the exploratory development style.

Even if the exploratory style causes the perceived difficulty of a problem to grow exponentially due to human cognitive limitations, how do the software engineering principles help to contain this exponential rise in complexity with problem size and hold it down to almost a linear increase? We will discuss in subsection 1.3.2 that software engineering principle help achieve this by profusely making use of the abstraction and decomposition techniques to overcome the human cognitive limitations. You may still wonder that when software engineering principles are used, why does the curve not become completely linear? The answer is that it is very difficult to apply the decomposition and abstraction principles to completely overcome the problem complexity.

Summary of the shortcomings of the exploratory style of software development:

We briefly summarise the important shortcomings of using the exploratory development style to develop a professional software:

- The foremost difficulty is the exponential growth of development time and effort with problem size and large-sized software becomes almost impossible using this style of development.
- The exploratory style usually results in unmaintainable code. The reason for this is that any code developed without proper design would result in highly unstructured and poor quality code.
- It becomes very difficult to use the exploratory style in a team development environment. In the exploratory style, the development work is undertaken without any proper design and documentation. Therefore it becomes very difficult to meaningfully partition the work among a set of developers who can work concurrently. On the other hand, team development is indispensable for developing modern software—most software mandate huge development efforts, necessitating team effort for developing these. Besides poor quality code, lack of proper documentation makes any later maintenance of the code very difficult.

1.3.1 Perceived Problem Complexity: An Interpretation Based on

Human Cognition Mechanism

The rapid increase of the perceived complexity of a problem with increase in problem size can be explained from an interpretation of the human cognition mechanism. A simple understanding of the human cognitive mechanism would also give us an insight into why the exploratory style of development leads to an undue increase in the time and effort required to develop a programming solution. It can also explain why it becomes practically infeasible to solve problems larger than a certain size while using an exploratory style; whereas using software engineering principles, the required effort grows almost linearly with size (as indicated by the thin solid line in Figure 1.4).

Psychologists say that the human memory can be thought to consist of two distinct parts[Miller 56]: short-term and long-term memories. A schematic representation of these two types of memories and their roles in human cognition mechanism has been shown in Figure 1.5. In Figure 1.5, the block labelled sensory organs represents the five human senses sight, hearing, touch, smell, and taste. The block labelled actuator represents neuromotor organs such as hand, finger, feet, etc. We now elaborate this human cognition model in the following subsection.

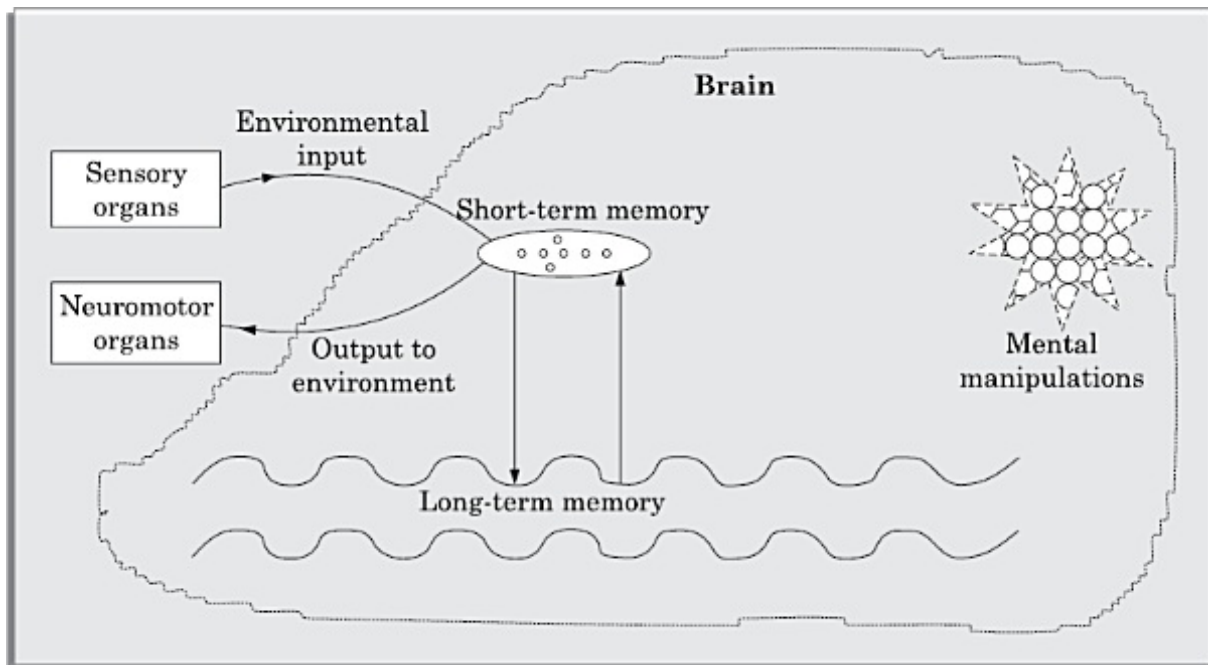


Figure 1.5: Human cognition mechanism model.

Short-term memory: The short-term memory, as the name itself suggests, can store information for a short while—usually up to a few seconds, and at most for a few minutes. The short-term memory is also sometimes referred to as

the *working memory*. The information stored in the short-term memory is immediately accessible for processing by the brain. The short-term memory of an average person can store up to seven items; but in extreme cases it can vary anywhere from five to nine items (7 ± 2). As shown in Figure 1.5, the short-term memory participates in all interactions of the human mind with its environment.

It should be clear that the short-term memory plays a very crucial part in the human cognition mechanism. All information collected through the sensory organs are first stored in the short-term memory. The short-term memory is also used by the brain to drive the neuromotor organs. The mental manipulation unit also gets its inputs from the short-term memory and stores back any output it produces. Further, information retrieved from the long-term memory first gets stored in the short-term memory. For example, if you are asked the question: "If it is 10AM now, how many hours are remaining today?" First, 10AM would be stored in the short-term memory. Next, the information that a day is 24 hours long would be fetched from the long-term memory into the short-term memory. The mental manipulation unit would compute the difference ($24-10$), and 14 hours would get stored in the short-term memory. As you can notice, this model is very similar to the organisation of a computer in terms of cache, main memory, and processor.

An item stored in the short-term memory can get lost either due to decay with time or displacement by newer information. This restricts the duration for which an item is stored in the short-term memory to few tens of seconds. However, an item can be retained longer in the short-term memory by recycling. That is, when we repeat or refresh an item consciously, we can remember it for a much longer duration. Certain information stored in the short-term memory, under certain circumstances gets stored in the long-term memory.

Long-term memory: Unlike the short-term memory, the size of the long-term memory is not known to have a definite upper bound. The size of the long-term memory can vary from several million items to several billion items, largely depending on how actively a person exercises his mental faculty. An item once stored in the long-term memory, is usually retained for several years. But, how do items get stored in the long-term memory? Items present in the short-term memory can get stored in the long-term memory either through large number of refreshments (repetitions) or by forming links with already existing items in the long-term memory. For example, you possibly remember your own telephone number because you might have repeated

(refreshed) it for a large number of times in your short-term memory. Let us now take an example of a situation where you may form links to existing items in the long-term memory to remember certain information. Suppose you want to remember the 10 digit mobile number 9433795369. To remember it by rote may be intimidating. But, suppose you consider the number as split into 9433 7953 69 and notice that 94 is the code for BSNL, 33 is the code for Kolkata, suppose 79 is your year of birth, and 53 is your roll number, and the rest of the two numbers are each one less than the corresponding digits of the previous number; you have effectively established links with already stored items, making it easier to remember the number.

Item: We have so far only mentioned the number of items that the long-term and the short-term memories can store. But, what exactly is an item? An *item* is any set of related information. According to this definition, a character such as *a* or a digit such as '5' can each be considered as an item. A word, a sentence, a story, or even a picture can each be a single item. Each item normally occupies one place in memory. The definition of an item as any set of related information implies that when you are able to establish some simple relationship between several different items, the information that should normally occupy several places can be stored using only one place in the memory. This phenomenon of forming one item from several items is referred to as *chunking* by psychologists. For example, if you are given the binary number 110010101001—it may prove very hard for you to understand and remember. But, the octal form of the number 6251 (i.e., the representation (110)(010)(101)(001)) may be much easier to understand and remember since we have managed to create chunks of three items each.

Evidence of short-term memory: Evidences of short-term memory manifest themselves in many of our day-to-day experiences. As an example of the short-term memory, consider the following situation. Suppose, you look up a number from the telephone directory and start dialling it. If you find the number to be busy, you would dial the number again after a few seconds—in this case, you would be able to do so almost effortlessly without having to look up the directory. But, after several hours or days since you dialled the number last, you may not remember the number at all, and would need to consult the directory again.

The magical number 7: Miller called the number seven as the *magical number* [Miller 56] since if a person deals with seven or less number of unrelated information at a time these would be easily accommodated in the short-term

memory. So, he can easily understand it. As the number of items that one has to deal with increases beyond seven, it becomes exceedingly difficult to understand it. This observation can easily be extended to writing programs.

When the number of details (or variables) that one has to track to solve a problem increases beyond seven, it exceeds the capacity of the short-term memory and it becomes exceedingly more difficult for a human mind to grasp the problem.

A small program having just a few variables is within the easy grasp of an individual. As the number of independent variables in the program increases, it quickly exceeds the grasping power of an individual and would require an unduly large effort to master the problem. This outlines a possible reason behind the exponential nature of the effort-size plot (thick line) shown in Figure 1.4. Please note that the situation depicted in Figure 1.4 arises mostly due to the human cognitive limitations. Instead of a human, if a machine could be writing (generating) a program, the slope of the curve would be linear, as the cache size (short-term memory) of a computer is quite large. But, why does the effort-size curve become almost linear when software engineering principles are deployed? This is because software engineering principles extensively use the techniques that are designed specifically to overcome the human cognitive limitations. We discuss this issue in the next subsection.

1.3.2 Principles Deployed by Software Engineering to Overcome Human Cognitive Limitations

We shall see throughout this book that a central theme of most of software engineering principles is the use of techniques to effectively tackle the problems that arise due to human cognitive limitations.

Two important principles that are deployed by software engineering to overcome the problems arising due to human cognitive limitations are—abstraction and decomposition.

In the following subsections, with the help of Figure 1.6(a) and (b), we explain the essence of these two important principles and how they help to overcome the human cognitive limitations. In the rest of this book, we shall time and again encounter the use of these two fundamental principles in various forms and flavours in the different software development activities. A thorough understanding of these two principles is therefore needed.

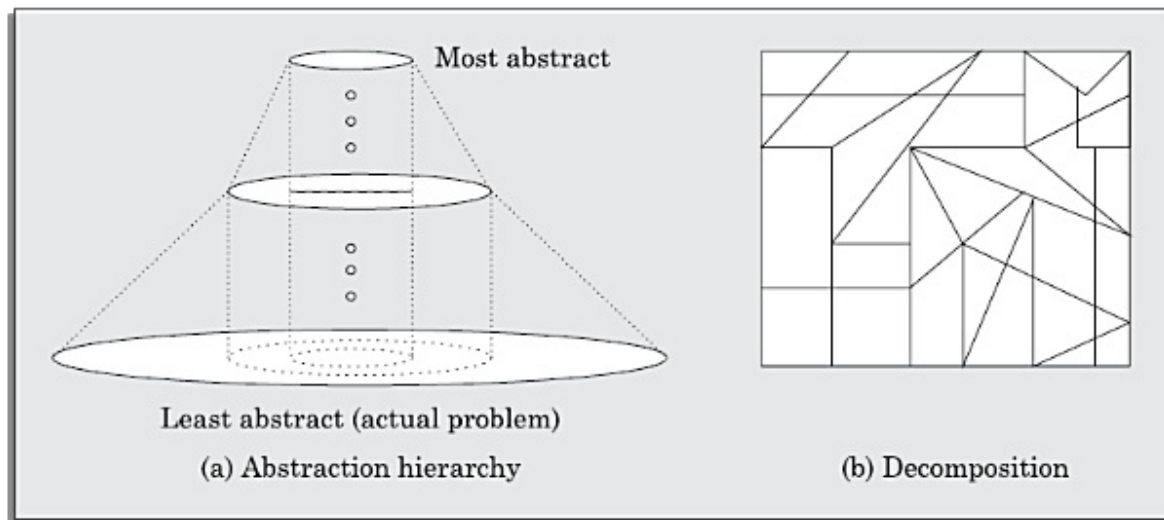


Figure 1.6: Schematic representation.

Abstraction

Abstraction refers to construction of a simpler version of a problem by ignoring the details. The principle of constructing an abstraction is popularly known as *modelling* (or *model construction*).

Abstraction is the simplification of a problem by focusing on only one aspect of the problem while omitting all other aspects.

When using the principle of abstraction to understand a complex problem, we focus our attention on only one or two specific aspects of the problem and ignore the rest. Whenever we omit some details of a problem to construct an abstraction, we construct a *model* of the problem. In every day life, we use the principle of abstraction frequently to understand a problem or to assess a situation. Consider the following two examples.

- Suppose you are asked to develop an overall understanding of some country. No one in his right mind would start this task by meeting all the citizens of the country, visiting every house, and examining every tree of the country, etc. You would probably take the help of several types of abstractions to do this. You would possibly start by referring to and understanding various types of maps for that country. A map, in fact, is an abstract representation of a country. It ignores detailed information such as the specific persons who inhabit it, houses, schools, play grounds, trees, etc. Again, there are two important types of maps—physical and political maps. A *physical map* shows the physical features of an area; such as mountains, lakes, rivers, coastlines, and so

on. On the other hand, the *political map* shows states, capitals, and national boundaries, etc. The physical map is an abstract model of the country and ignores the state and district boundaries. The political map, on the other hand, is another abstraction of the country that ignores the physical characteristics such as elevation of lands, vegetation, etc. It can be seen that, for the same object (e.g. country), several abstractions are possible. In each abstraction, some aspects of the object is ignored. We understand a problem by abstracting out different aspects of a problem (constructing different types of models) and understanding them. It is not very difficult to realise that proper use of the principle of abstraction can be a very effective help to master even intimidating problems.

- Consider the following situation. Suppose you are asked to develop an understanding of all the living beings inhabiting the earth. If you use the naive approach, you would start taking up one living being after another who inhabit the earth and start understanding them. Even after putting in tremendous effort, you would make little progress and left confused since there are billions of living things on earth and the information would be just too much for any one to handle. Instead, what can be done is to build and understand an abstraction hierarchy of all living beings as shown in Figure 1.7. At the top level, we understand that there are essentially three fundamentally different types of living beings—plants, animals, and fungi. Slowly more details are added about each type at each successive level, until we reach the level of the different species at the leaf level of the abstraction tree.

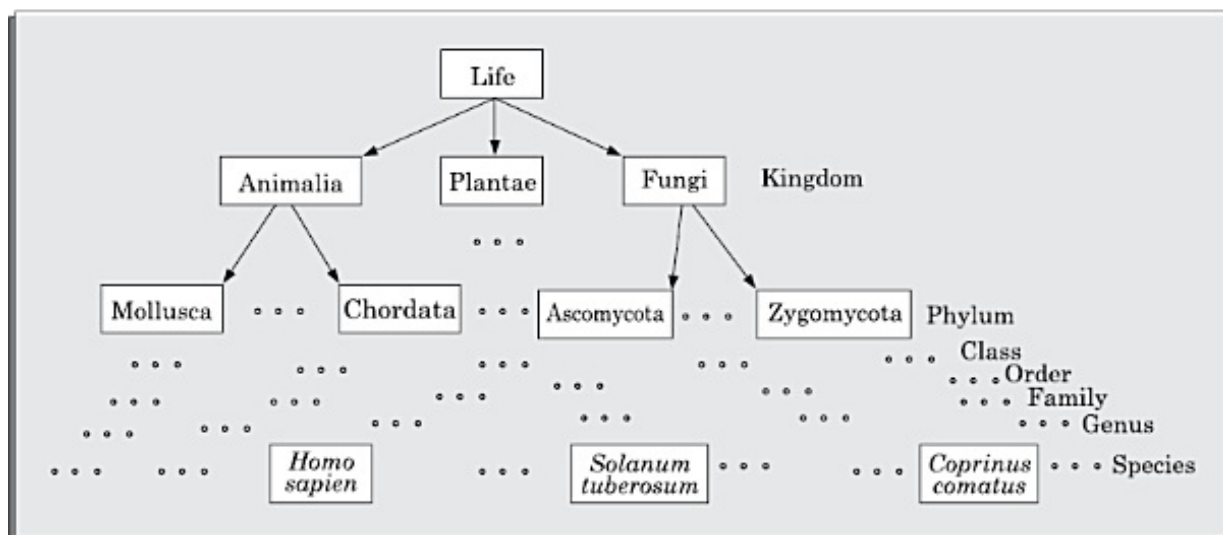


Figure 1.7: An abstraction hierarchy classifying living organisms.

A single level of abstraction can be sufficient for rather simple problems. However, more complex problems would need to be modelled as a hierarchy of abstractions. A schematic representation of an abstraction hierarchy has been shown in Figure 1.6(a). The most abstract representation would have only a few items and would be the easiest to understand. After one understands the simplest representation, one would try to understand the next level of abstraction where at most five or seven new information are added and so on until the lowest level is understood. By the time, one reaches the lowest level, he would have mastered the entire problem.

Decomposition

Decomposition is another important principle that is available in the repertoire of a software engineer to handle problem complexity. This principle is profusely made use by several software engineering techniques to contain the exponential growth of the perceived problem complexity. The decomposition principle is popularly known as the *divide and conquer* principle.

The decomposition principle advocates decomposing the problem into many small independent parts. The small parts are then taken up one by one and solved separately. The idea is that each small part would be easy to grasp and understand and can be easily solved. The full problem is solved when all the parts are solved.

A popular way to demonstrate the decomposition principle is by trying to break a large bunch of sticks tied together and then breaking them individually. Figure 1.6(b) shows the decomposition of a large problem into many small parts. However, it is very important to understand that any arbitrary decomposition of a problem into small parts would not help. The different parts after decomposition should be more or less independent of each other. That is, to solve one part you should not have to refer and understand other parts. If to solve one part you would have to understand other parts, then this would boil down to understanding all the parts together. This would effectively reduce the problem to the original problem before decomposition (the case when all the sticks tied together). Therefore, it is not sufficient to just decompose the problem in any way, but the decomposition should be such that the different decomposed parts must be more or less independent of each other.

As an example of a use of the principle of decomposition, consider the following. You would understand a book better when the contents are decomposed (organised) into more or less independent chapters. That is,

each chapter focuses on a separate topic, rather than when the book mixes up all topics together throughout all the pages. Similarly, each chapter should be decomposed into sections such that each section discusses a different issue. Each section should be decomposed into subsections and so on. If various subsections are nearly independent of each other, the subsections can be understood one by one rather than keeping on cross referencing to various subsections across the book to understand one.

Why study software engineering?

Let us examine the skills that you could acquire from a study of the software engineering principles. The following two are possibly the most important skill you could be acquiring after completing a study of software engineering:

- The skill to participate in development of large software. You can meaningfully participate in a team effort to develop a large software only after learning the systematic techniques that are being used in the industry.
- You would learn how to effectively handle complexity in a software development problem. In particular, you would learn how to apply the principles of abstraction and decomposition to handle complexity during various stages in software development such as specification, design, construction, and testing.

Besides the above two important skills, you would also be learning the techniques of software requirements specification user interface development, quality assurance, testing, project management, maintenance, etc.

As we had already mentioned, small programs can also be written without using software engineering principles. However even if you intend to write small programs, the software engineering principles could help you to achieve higher productivity and at the same time enable you to produce better quality programs.

1.4 EMERGENCE OF SOFTWARE ENGINEERING

We have already pointed out that software engineering techniques have evolved over many years in the past. This evolution is the result of a series of innovations and accumulation of experience about writing

good quality programs. Since these innovations and programming experiences are too numerous, let us briefly examine only a few of these innovations and programming experiences which have contributed to the development of the software engineering discipline.

1.4.1 Early Computer Programming

Early commercial computers were very slow and too elementary as compared to today's standards. Even simple processing tasks took considerable computation time on those computers. No wonder that programs at that time were very small in size and lacked sophistication. Those programs were usually written in assembly languages. Program lengths were typically limited to about a few hundreds of lines of monolithic assembly code. Every programmer developed his own individualistic style of writing programs according to his intuition and used this style *ad hoc* while writing different programs. In simple words, programmers wrote programs without formulating any proper solution strategy, plan, or design a jump to the terminal and start coding immediately on hearing out the problem. They then went on fixing any problems that they observed until they had a program that worked reasonably well. We have already designated this style of programming as the *build and fix* (or the *exploratory programming*) style.

1.4.2 High-level Language Programming

Computers became faster with the introduction of the semiconductor technology in the early 1960s. Faster semiconductor transistors replaced the prevalent vacuum tube-based circuits in a computer. With the availability of more powerful computers, it became possible to solve larger and more complex problems. At this time, high-level languages such as FORTRAN, ALGOL, and COBOL were introduced. This considerably reduced the effort required to develop software and helped programmers to write larger programs (why?). Writing each high-level programming construct in effect enables the programmer to write several machine instructions. Also, the machine details (registers, flags, etc.) are abstracted from the programmer. However, programmers were still using the exploratory style of software development. Typical programs were limited to sizes of around a few thousands of lines of source code.

1.4.3 Control Flow-based Design

As the size and complexity of programs kept on increasing, the exploratory programming style proved to be insufficient. Programmers found it increasingly difficult not only to write cost-effective and correct programs, but also to understand and maintain programs written by others. To cope up with this problem, experienced programmers advised other programmers to pay particular attention to the design of a program's *control flow structure*.

A program's control flow structure indicates the sequence in which the program's instructions are executed.

In order to help develop programs having good control flow structures, the *flow charting technique* was developed. Even today, the flow charting technique is being used to represent and design algorithms; though the popularity of flow charting represent and design programs has waned to a great extent due to the emergence of more advanced techniques.

Figure 1.8 illustrates two alternate ways of writing program code for the same problem. The flow chart representations for the two program segments of Figure 1.8 are drawn in Figure 1.9. Observe that the control flow structure of the program segment in Figure 1.9(b) is much more simpler than that of Figure 1.9(a). By examining the code, it can be seen that Figure 1.9(a) is much harder to understand as compared to Figure 1.9(b). This example corroborates the fact that if the flow chart representation is simple, then the corresponding code should be simple. You can draw the flow chart representations of several other problems to convince yourself that a program with complex flow chart representation is indeed more difficult to understand and maintain.

<pre> 1 if(customer_savings_balance>withdrawal_request) { 2 100: issue_money=TRUE; 3 GOTO 110; 4 } 5 else if(privileged_customer==TRUE) 6 GOTO 100; 7 110: activate_cash_dispenser(withdrawal_request); 8 GOTO 130; 9 120: print(error); 10 130: end-transaction(); </pre> (a) An example unstructured program	<pre> 1 if(privileged_customer (customer_savings_balance>withdrawal_request)){ 2 activate_cash_dispenser(withdrawal_request); 3 } 4 else print(error); 5 end-transaction(); </pre> (b) Corresponding structured program
--	---

Figure 1.8: An example of (a) Unstructured program (b) Corresponding structured program.

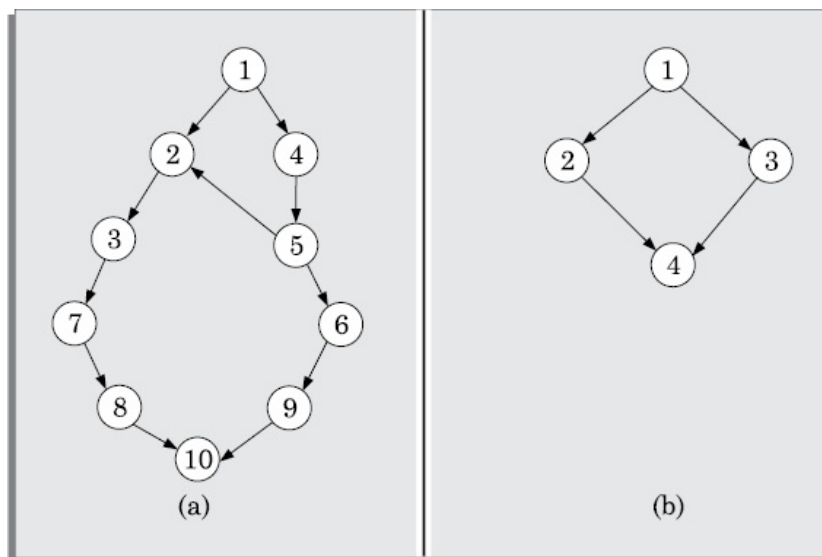


Figure 1.9: Control flow graphs of the programs of Figures 1.8(a) and (b).

Let us now try to understand why a program having good control flow structure would be easier to develop and understand. In other words, let us understand why a program with a complex flow chart representation is difficult to understand? The main reason behind this situation is that normally one understands a program by mentally tracing its execution sequence (i.e. statement sequences) to understand how the output is produced from the input values. That is, we can start from a statement producing an output, and trace back the statements in the program and understand how they produce the output by transforming the input data. Alternatively, we may start with the input data and check by running through the program how each statement processes (transforms) the input data until the output is produced. For example, for the program of Fig 1.9(a) you would have to understand the execution of the program along the paths 1-2-3-7-8-10, 1-4-5-6-9-10, and 1-4-5-2-3-7-8-10. A program having a messy control flow (i.e. flow chart) structure, would have a large number of execution paths (see Figure 1.10). Consequently, it would become extremely difficult to determine all the execution paths, and tracing the execution sequence along all the paths trying to understand them can be nightmarish. It is therefore evident that a program having a messy flow chart representation would indeed be difficult to understand and debug.

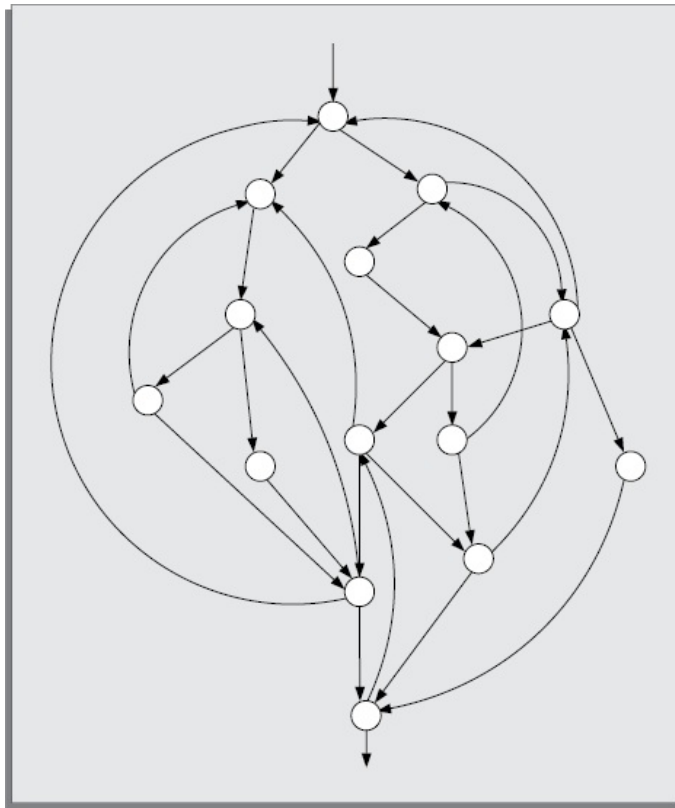


Figure 1.10: CFG of a program having too many GO TO statements.

Are GO TO statements the culprits?

In a landmark paper, Dijkstra [1968] published his (now famous) article “GO TO Statements Considered Harmful”. He pointed out that unbridled use of GO TO statements is the main culprit in making the control structure of a program messy. To understand his argument, examine Figure 1.10 which shows the flow chart representation of a program in which the programmer has used rather too many GO TO statements. GO TO statements alter the flow of control arbitrarily, resulting in too many paths. But, then why does use of too many GO TO statements makes a program hard to understand?

A programmer trying to understand a program would have to mentally trace and understand the processing that take place along all the paths of the program making program understanding and debugging extremely complicated.

Soon it became widely accepted that good programs should have very simple control structures. It is possible to distinguish good programs from bad programs by just visually examining their flow chart representations. The use of flow charts to design good control flow structures of programs became wide spread.

Structured programming—a logical extension

The need to restrict the use of GO TO statements was recognised by everybody. However, many programmers were still using assembly languages. JUMP instructions are frequently used for program branching in assembly languages. Therefore, programmers with assembly language programming background considered the use of GO TO statements in programs inevitable. However, it was conclusively proved by Bohm and Jacopini that only three programming constructs—sequence, selection, and iteration—were sufficient to express any programming logic. This was an important result—it is considered important even today. An example of a sequence statement is an assignment statement of the form $a=b$;. Examples of selection and iteration statements are the if-then-else and the do-while statements respectively. Gradually, everyone accepted that it is indeed possible to solve any programming problem without using GO TO statements and that indiscriminate use of GO TO statements should be avoided. This formed the basis of the structured programming methodology.

A program is called structured when it uses only the sequence, selection, and iteration types of constructs and is modular.

Structured programs avoid unstructured control flows by restricting the use of GO TO statements. Structured programming is facilitated, if the programming language being used supports single-entry, single-exit program constructs such as if-then-else, do-while, etc. Thus, an important feature of structured programs is the design of good control structures. An example illustrating this key difference between structured and unstructured programs is shown in Figure 1.8. The program in Figure 1.8(a) makes use of too many GO TO statements, whereas the program in Figure 1.8(b) makes use of none. The flow chart of the program making use of GO TO statements is obviously much more complex as can be seen in Figure 1.9.

Besides the control structure aspects, the term *structured program* is being used to denote a couple of other program features as well. A structured program should be modular. A modular program is one which is decomposed into a set of modules¹ such that the modules should have low interdependency among each other. We discuss the concept of modular programs in Chapter 5.

But, what are the main advantages of writing structured programs compared to the unstructured ones? Research experiences have shown that

programmers commit less number of errors while using structured if-then-else and do-while statements than when using test-and-branch code constructs. Besides being less error-prone, structured programs are normally more readable, easier to maintain, and require less effort to develop compared to unstructured programs. The virtues of structured programming became widely accepted and the structured programming concepts are being used even today. However, violations to the structured programming feature is usually permitted in certain specific programming situations, such as exception handling, etc.

Very soon several languages such as PASCAL, MODULA, C, etc., became available which were specifically designed to support structured programming. These programming languages facilitated writing modular programs and programs having good control structures. Therefore, messy control structure was no longer a big problem. So, the focus shifted from designing good control structures to designing good data structures for programs.

1.4.4 Data Structure-oriented Design

Computers became even more powerful with the advent of *integrated circuits* (ICs) in the early seventies. These could now be used to solve more complex problems. Software developers were tasked to develop larger and more complicated software, which often required writing in excess of several tens of thousands of lines of source code. The control flow-based program development techniques could not be used satisfactorily any more to write those programs, and more effective program development techniques were needed.

It was soon discovered that while developing a program, it is much more important to pay attention to the design of the important data structures of the program than to the design of its control structure. Design techniques based on this principle are called *data structure-oriented* design techniques.

Using data structure-oriented design techniques, first a program's data structures are designed. The code structure is designed based on the data structure.

In the next step, the program design is derived from the data structure. An example of a data structure-oriented design technique is the Jackson's Structured Programming (JSP) technique developed by Michael Jackson [1975]. In JSP methodology, a program's data structure is first designed using the notations for sequence, selection, and iteration. The JSP methodology

provides an interesting technique to derive the program structure from its data structure representation. Several other data structure-based design techniques were also developed. Some of these techniques became very popular and were extensively used. Another technique that needs special mention is the Warnier-Orr Methodology [1977, 1981]. However, we will not discuss these techniques in this text because now-a-days these techniques are rarely used in the industry and have been replaced by the data flow-based and the object-oriented techniques.

1.4.5 Data Flow-oriented Design

As computers became still faster and more powerful with the introduction of *very large scale integrated* (VLSI) Circuits and some new architectural concepts, more complex and sophisticated software were needed to solve further challenging problems. Therefore, software developers looked out for more effective techniques for designing software and soon *data flow-oriented techniques* were proposed.

The data flow-oriented techniques advocate that the major data items handled by a system must be identified and the processing required on these data items to produce the desired outputs should be determined.

The functions (also called as *processes*) and the data items that are exchanged between the different functions are represented in a diagram known as a *data flow diagram* (DFD). The program structure can be designed from the DFD representation of the problem.

DFDs: A crucial program representation for procedural program design

DFD has proven to be a generic technique which is being used to model all types of systems, and not just software systems. For example, Figure 1.11 shows the data-flow representation of an automated car assembly plant. If you have never visited an automated car assembly plant, a brief description of an automated car assembly plant would be necessary. In an automated car assembly plant, there are several processing stations (also called *workstations*) which are located along side of a conveyor belt (also called an *assembly line*). Each workstation is specialised to do jobs such as fitting of wheels, fitting the engine, spray painting the car, etc. As the partially assembled program moves along the assembly line, different workstations perform their respective jobs on the partially assembled software. Each circle in the DFD

model of Figure 1.11 represents a workstation (called a *process* or *bubble*). Each workstation consumes certain input items and produces certain output items. As a car under assembly arrives at a workstation, it fetches the necessary items to be fitted from the corresponding stores (represented by two parallel horizontal lines), and as soon as the fitting work is complete passes on to the next workstation. It is easy to understand the DFD model of the car assembly plant shown in Figure 1.11 even without knowing anything regarding DFDs. In this regard, we can say that a major advantage of the DFDs is their simplicity. In Chapter 6, we shall study how to construct the DFD model of a software system. Once you develop the DFD model of a problem, data flow-oriented design techniques provide a rather straight forward methodology to transform the DFD representation of a problem into an appropriate software design. We shall study the data flow-based design techniques in Chapter 6.

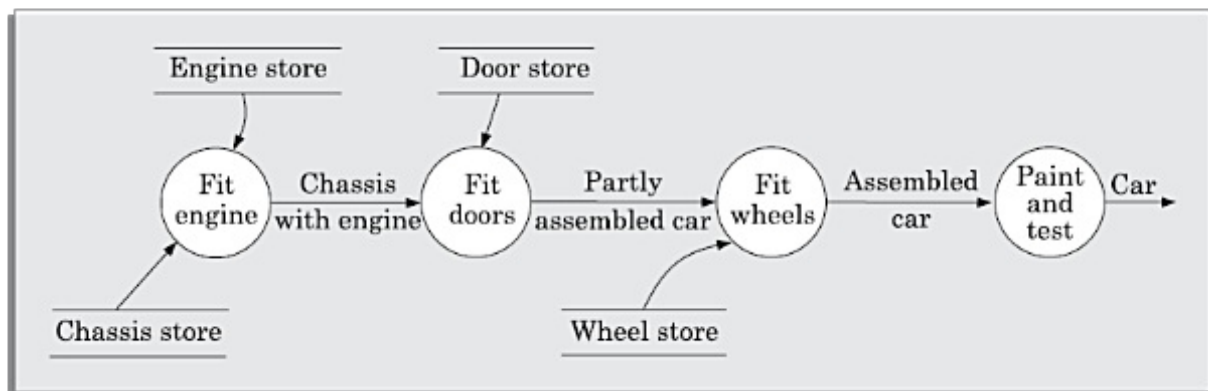


Figure 1.11: Data flow model of a car assembly plant.

1.4.6 Object-oriented Design

Data flow-oriented techniques evolved into object-oriented design (OOD) techniques in the late seventies. Object-oriented design technique is an intuitively appealing approach, where the natural objects (such as employees, pay-roll-register, etc.) relevant to a problem are first identified and then the relationships among the objects such as composition, reference, and inheritance are determined. Each object essentially acts as a *data hiding* (also known as *data abstraction*) entity. Object-oriented techniques have gained wide spread acceptance because of their simplicity, the scope for code and design reuse, promise of lower development time, lower development cost, more robust code, and easier maintenance. OOD techniques are discussed in Chapters 7 and 8.

1.4.7 What Next?

In this section, we have so far discussed how software design techniques have evolved since the early days of programming. We pictorially summarise this evolution of the software design techniques in Figure 1.12. It can be observed that in almost every passing decade, revolutionary ideas were put forward to design larger and more sophisticated programs, and at the same time the quality of the design solutions improved. But, what would the next improvement to the design techniques be? It is very difficult to speculate about the developments that may occur in the future. However, we have already seen that in the past, the design techniques have evolved each time to meet the challenges faced in developing contemporary software. Therefore, the next development would most probably occur to help meet the challenges being faced by the modern software designers. To get an indication of the techniques that are likely to emerge, let us first examine what are the current challenges in designing software. First, program sizes are further increasing as compared to what was being developed a decade back. Second, many of the present day software are required to work in a client-server environment through a web browser-based access (called *web-based software*). At the same time, embedded devices are experiencing an unprecedented growth and rapid customer acceptance in the last decade. It is there for necessary for developing applications for small hand held devices and embedded processors. We examine later in this text how aspect-oriented programming, client- server based design, and embedded software design techniques have emerged rapidly. In the current decade, service-orientation has emerged as a recent direction of software engineering due to the popularity of web-based applications and public clouds.

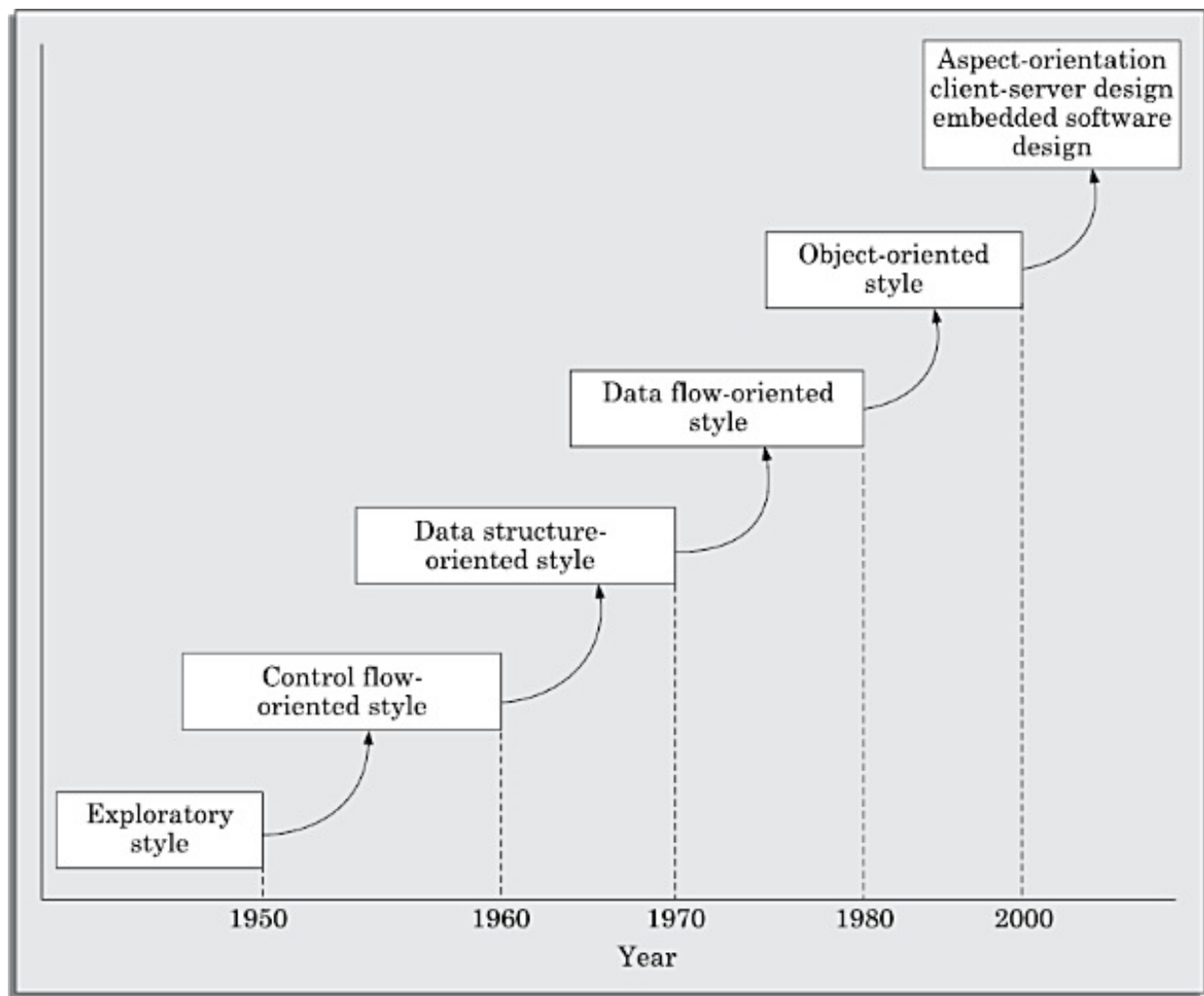


Figure 1.12: Evolution of software design techniques.

1.4.8 Other Developments

It can be seen that remarkable improvements to the prevalent software design technique occurred almost every passing decade. The improvements to the software design methodologies over the last five decades have indeed been remarkable. In addition to the advancements made to the software design techniques, several other new concepts and techniques for effective software development were also introduced. These new techniques include life cycle models, specification techniques, project management techniques, testing techniques, debugging techniques, quality assurance techniques, software measurement techniques, *computer aided software engineering* (CASE) tools, etc. The development of these techniques accelerated the growth of software engineering as a discipline. We shall discuss these techniques in the later chapters.

1.5 NOTABLE CHANGES IN SOFTWARE DEVELOPMENT PRACTICES

Before we discuss the details of various software engineering principles, it is worthwhile to examine the glaring differences that you would notice when you observe an exploratory style of software development and another development effort based on modern software engineering practices. The following noteworthy differences between these two software development approaches would be immediately observable.

- An important difference is that the exploratory software development style is based on *error correction (build and fix)* while the software engineering techniques are based on the principles of *error prevention*. Inherent in the software engineering principles is the realisation that it is much more cost-effective to prevent errors from occurring than to correct them as and when they are detected. Even when mistakes are committed during development, software engineering principles emphasize detection of errors as detected only during the final product testing. In contrast, the modern practice of software development is to develop the software through several well-defined stages such as requirements specification, design, coding, testing, etc., and attempts are made to detect and fix as many errors as possible in the same phase in which they are made.
- In the exploratory style, coding was considered synonymous with software development. For instance, this naive way of developing a software believed in developing a working system as quickly as possible and then successively modifying it until it performed satisfactorily. Exploratory programmers literally dive at the computer to get started with their programs even before they fully learn about the problem!!! It was recognised that exploratory programming not only turns out to be prohibitively costly for non-trivial problems, but also produces hard-to-maintain programs. Even minor modifications to such programs later can become nightmarish. In the modern software development style, coding is regarded as only a small part of the overall software development activities. There are several development activities such as design and testing which may demand much more effort than coding.
- A lot of attention is now being paid to requirements specification. Significant effort is being devoted to develop a clear and correct

specification of the problem before any development activity starts. Unless the requirements specification is able to correctly capture the exact customer requirements, large number of rework would be necessary at a later stage. Such rework would result in higher cost of development and customer dissatisfaction.

- Now there is a distinct design phase where standard design techniques are employed to yield coherent and complete design models.
- Periodic reviews are being carried out during all stages of the development process. The main objective of carrying out reviews is *phase containment of errors*, i.e. detect and correct errors as soon as possible. Phase containment of errors is an important software engineering principle. We will discuss this technique in Chapter 2.
- Today, software testing has become very systematic and standard testing techniques are available. Testing activity has also become all encompassing, as test cases are being developed right from the requirements specification stage.
- There is better visibility of the software through various developmental activities.

-

By visibility we mean production of good quality, consistent and peer reviewed documents at the end of every software development activity.

- In the past, very little attention was being paid to producing good quality and consistent documents. In the exploratory style, the design and test activities, even if carried out (in whatever way), were not documented satisfactorily. Today, consciously good quality documents are being developed during software development. This has made fault diagnosis and maintenance far more smoother. We will see in Chapter 3 that in addition to facilitating product maintenance, increased visibility makes management of a software project easier.
- Now, projects are being thoroughly planned. The primary objective of project planning is to ensure that the various development activities take place at the correct time and no activity is halted due to the want of some resource. Project planning normally includes preparation of various types of estimates, resource scheduling, and development of project tracking plans. Several techniques and automation tools for tasks such as configuration management, cost estimation, scheduling,

etc., are being used for effective software project management.

- Several metrics (quantitative measurements) of the products and the product development activities are being collected to help in software project management and software quality assurance.

1.6 COMPUTER SYSTEMS ENGINEERING

In all the discussions so far, we assumed that the software being developed would run on some general-purpose hardware platform such as a desktop computer or a server. But, in several situations it may be necessary to develop special hardware on which the software would run. Examples of such systems are numerous, and include a robot, a factory automation system, and a cell phone. In a cell phone, there is a special processor and other specialised devices such as a speaker and a microphone. It can run only the programs written specifically for it. Development of such systems entails development of both software and specific hardware that would run the software. Computer systems engineering addresses development of such systems requiring development of both software and specific hardware to run the software. Thus, systems engineering encompasses software engineering.

The general model of systems engineering is shown schematically in Figure 1.13. One of the important stages in systems engineering is the stage in which decision is made regarding the parts of the problems that are to be implemented in hardware and the ones that would be implemented in software. This has been represented by the box captioned hardware-software partitioning in Figure 1.13. While partitioning the functions between hardware and software, several trade-offs such as flexibility, cost, speed of operation, etc., need to be considered. The functionality implemented in hardware runs faster. On the other hand, functionalities implemented in software are easier to extend. Further, it is difficult to implement complex functions in hardware. Also, functions implemented in hardware incur extra space, weight, manufacturing cost, and power overhead.

After the hardware-software partitioning stage, development of hardware and software are carried out concurrently (shown as concurrent branches in Figure 1.13). In system engineering, testing the software during development becomes a tricky issue, the hardware on which the software would run and tested would still be under development—remember that the hardware and the software are being developed at the same time. To test the software

during development, it usually becomes necessary to develop simulators that mimic the features of the hardware being developed. The software is tested using these simulators. Once both hardware and software development are complete, these are integrated and tested. The project management activity is required through out the duration of system development as shown in Figure 1.13. In this text, we have confined our attention to software engineering only.

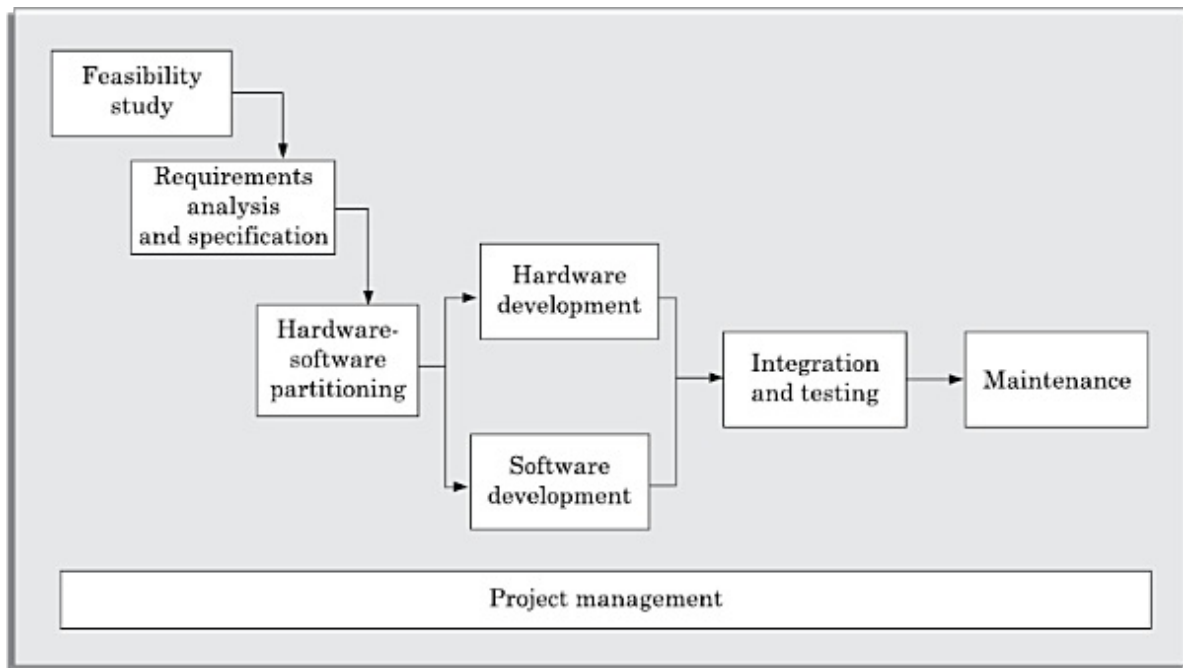


Figure 1.13: Computer systems engineering.

SUMMARY

- We first defined the scope of software engineering. We came up with two alternate but equivalent definitions:
 - The systematic collection of decades of programming experience together with the innovations made by researchers towards developing high quality software in a cost- effective manner.
 - The engineering approach to develop software.
- The exploratory (also called build and fix) style of program development is used by novice programmers. The exploratory style is characterized by quickly developing the program code and then modifying it successively till the program works. This approach turns out not only to be a very costly and inefficient way of developing

software, but yields a product that is unreliable and difficult to maintain. Also, the exploratory style is very difficult to use when software is developed through team effort. A still larger handicap of the exploratory style of programming is that it breaks down when used to develop large programs.

- Unless one makes use of software engineering principles, the increase in effort and time with size of the program would be exponential—making it virtually impossible for a person to develop large programs.
- To handle complexity in a problem, all software engineering principles make extensive use of the following two techniques:
 - Abstraction (modelling), and
 - Decomposition (Divide and conquer).
- Software engineering techniques are essential for development of large software products where a group of engineers work in a team to develop the product. However, most of the principles of software engineering are useful even while developing small programs.
- A program is called structured, when it is decomposed into a set of modules and each module in turn is decomposed into functions. Additionally, structured programs avoid the use of GO TO statements and use only structured programming constructs.
- Computer systems engineering deals with the development of complete systems, necessitating integrated development of both software and hardware parts. Computer systems engineering encompasses software engineering.
- We shall delve into various software engineering principles starting from the next chapter. But, before that here is a word of caution. Those who have written large-sized programs, can better appreciate many of the principles of software engineering. Students with less or no programming experience would have to take our words for it and work harder with the topics. However, it is a fact that unless somebody has seen an elephant (read problems encountered during program development) at least once, any amount of describing and explaining would not result in the kind of understanding that somebody who has previously seen an elephant (developed a program) would get.

EXERCISES

1. Choose the correct option:

(a) Which of the following is not a symptom of the present software crisis:

- (i) Software is expensive.
- (ii) It takes too long to build a software product.
- (iii) Software is delivered late.
- (iv) Software products are required to perform very complex tasks.

(b) The goal of structured programming is which one of the following:

- (i) To have well indented programs.
- (ii) To be able to infer the flow of control from the compiled code.
- (iii) To be able to infer the flow of control from the program text.
- (iv) To avoid the use of GO TO statements.

(c) Unrestricted use of GO TO statements is normally avoided while writing a program, since:

- (i) It increases the running time of programs.
- (ii) It increases memory requirements of programs.
- (iii) It results in larger executable code sizes.
- (iv) It makes debugging difficult.

(d) Why is writing easily modifiable code important?

- (i) Easily modifiable code results in quicker run time.
- (ii) Most real world programs require change at some point of time or other.
- (iii) Most text editors make it mandatory to write modifiable code.
- (iv) Several people may be writing different parts of code at the same time.

2. What is the principal aim of the software engineering discipline? What does the discipline of software engineering discuss?

3. Why do you think systematic software development using the software engineering principle is any different than art or craft?

4. Distinguish between a program and a professionally developed software.

5. Distinguish among a program, a software product and a software service. Give one example of each. Discuss the difference of the characteristics of development projects for each of these.

6. What is a software product line? Give an example of a software product line. How is a software product line development any different from a software product development.

7. What are the main types of projects that are undertaken by software

development companies? Give examples of these types of projects and point out the important characteristic differences between these types of projects.

8. Do you agree with the following statement—"The focus of exploratory programming is error correction while the software engineering principles emphasise error prevention"? Give the reasonings behind your answer.
9. What difficulties would a software development company face, if it tries to use the exploratory (build and fix) program development style in its development projects? Explain your answer.
10. What are the symptoms of the present software crisis? What factors have contributed to the making of the present software crisis? What are the possible solutions to the present software crisis?
11. Explain why the effort, time, and cost required to develop a program using the build and fix style increase exponentially with the size of the program? How do software engineering principles help tackle this rapid rise in development time and cost?
12. Distinguish between software products and services. Give examples of each.
13. What are the different types of projects that are being undertaken by software development houses? Which of these type of projects is the forte of Indian software development organisations? Identify any possible reasons as to why the other has not been focused by the Indian software development organisations.
14. Name the basic techniques used by the software engineering techniques to handle complexity in a problem.
15. What do you understand by the exploratory (also known as the build and fix) style of software development? Graphically depict the activities that a programmer typically carries out while developing a programming solution using the exploratory style. In your diagram also show the order in which the activities are carried out. What are the shortcomings of this style of program development?
16. List the major differences between the exploratory and modern software development practices.
17. What is the difference between the actual complexity of solving a problem and its perceived complexity? What causes the difference between the two to arise?
18. What do you understand by the term perceived complexity of a problem? How is it different from computational complexity? How can the

perceived complexity of a problem be reduced?

19. Why is the number 7 considered as a magic number in software engineering? How is it useful software engineering?
20. What do you understand by the principles of abstraction and decomposition? Why are these two principles considered important in software engineering? Explain the problems that these two principles target to solve? Support your answer using suitable examples.
21. What do you understand by control flow structure of a program? Why is it difficult to understand a program having a messy control flow structure? How can a good control flow structure for a program be designed?
22. What is a flow chart? How is the flow charting technique useful during software development?
23. What do you understand by visibility of design and code? How does increased visibility help in systematic software development? (We shall revisit this question in Chapter 3)
24. What do you understand by the term—structured programming? How do modern programming languages such as PASCAL and C facilitate writing structured programs? What are the advantages of writing structured programs vis-a-vis unstructured programs?
25. What is a high-level programming language? Why does a programmer using a high-level programming language have a higher productivity as compared to when using machine language for application development?
26. What are the three basic types of program constructs necessary to develop the program for any given problem? Give examples of these three constructs from any high-level language you know.
27. What do you understand by a program module? What are the important characteristics of a program module?
28. Explain how do the use of software engineering principles help to develop software products cost-effectively and timely. Elaborate your answer by using suitable examples.
29. What is the basic difference between a control flow-oriented and a data flow-oriented design technique? Can you think of any reason as to why a data flow-oriented design technique is likely to produce better designs than a control flow-oriented design technique? (We shall revisit this question while discussing the design techniques in Chapter 6.)
30. Name the two fundamental principles that are used extensively in software engineering to tackle the complexity in developing large programs? Explain these two principles. By using suitable examples

explain how these two principles help tackle the complexity associated with developing large programs.

31. What does the *control flow graph* (CFG) of a program represent? Draw the CFG of the following program:

```
main() {  
    int y=1;  
    if(y<0)  
        if(y>0) y=3;  
        else y=0;  
    printf("%d\n", y);  
}
```

32. Discuss the possible reasons behind supersession of the data structure-oriented design methods by the control flow-oriented design methods.

33. What is a data structure-oriented software design methodology? How is it different from the data flow-oriented design methodology?

34. Discuss the major advantages of the *object-oriented design* (OOD) methodologies over the data flow-oriented design methodologies.

35. Explain how the software design techniques have evolved in the past. How do you think shall the software design techniques evolve in the near future?

36. What is computer systems engineering? How is it different from software engineering?

Give examples of some types of product development projects for which systems engineering is appropriate.

37. What do you mean by software service? Explain the important differences between the characteristics of a software service development project and a software product development project.

1 In this text, we shall use the terms *module* and *module structure* to loosely mean the following—A *module* is a collection of procedures and data structures. The data structures in a module are accessible only to the procedures defined inside the module. A module forms an independently compilable unit and may be linked to other modules to form a complete application. The term *module structure* will be used to denote the way in which different modules invoke each other's procedures.

Chapter

2

SOFTWARE LIFE CYCLE MODELS

In Chapter 1, we discussed a few basic issues in software engineering. We pointed out a few important differences between the exploratory program development style and the software engineering approach. Please recollect from our discussions in Chapter 1 that the exploratory style is also known as the *build and fix* programming. In build and fix programming, a programmer typically starts to write the program immediately after he has formed an informal understanding of the requirements. Once program writing is complete, he gets down to fix anything that does not meet the user's expectations. Usually, a large number of code fixes are required even for toy programs. This pushes up the development costs and pulls down the quality of the program. Further, this approach usually turns out to be a recipe for project failure when used to develop non-trivial programs requiring team effort. In contrast to the build and fix style, the software engineering approaches emphasise software development through a well-defined and ordered set of activities. These activities are graphically modelled (represented) as well as textually described and are variously called as *software life cycle model*, *software development life cycle (SDLC) model*, and *software development process model*. Several life cycle models have so far been proposed. However, in this Chapter we confine our attention to only a few important and commonly used ones.

In this chapter, we first discuss a few basic concepts associated with life cycle models. Subsequently, we discuss the important activities that have been prescribed to be carried out in the classical waterfall model. This is intended to provide an insight into the activities that are carried out as part of every life cycle model. In fact, the classical waterfall model can be considered as a basic model and all other life cycle models as extensions of this model to cater to specific project situations. After discussing the waterfall

model, we discuss a few derivatives of this model. Subsequently we discuss the spiral model that generalises various life cycle models. Finally, we discuss a few recently proposed life cycle models that are categorized under the umbrella term *agile model*. Of late, agile models are finding increasing acceptance among developers and researchers.

The genesis of the agile model can be traced to the radical changes to the types of project that are being undertaken at present, rather than to any radical innovations to the life cycle models themselves. The projects have changed from large multi-year product development projects to small services projects now

2.1 A FEW BASIC CONCEPTS

In this section, we present a few basic concepts concerning the life cycle models.

Software life cycle

It is well known that all living organisms undergo a life cycle. For example when a seed is planted, it germinates, grows into a full tree, and finally dies. Based on this concept of a biological life cycle, the term *software life cycle* has been defined to imply the different stages (or phases) over which a software evolves from an initial customer request for it, to a fully developed software, and finally to a stage where it is no longer useful to any user, and then it is discarded.

As we have already pointed out, the life cycle of every software starts with a request for it by one or more customers. At this stage, the customers are usually not clear about all the features that would be needed, neither can they completely describe the identified features in concrete terms, and can only vaguely describe what is needed. This stage where the customer feels a need for the software and forms rough ideas about the required features is known as the *inception* stage. Starting with the inception stage, a software evolves through a series of identifiable stages (also called phases) on account of the development activities carried out by the developers, until it is fully developed and is released to the customers.

Once installed and made available for use, the users start to use the software. This signals the start of the operation (also called *maintenance*) phase. As the users use the software, not only do they request for fixing any failures that they might encounter, but they also continually suggest several

improvements and modifications to the software. Thus, the maintenance phase usually involves continually making changes to the software to accommodate the bug-fix and change requests from the user. The operation phase is usually the longest of all phases and constitutes the useful life of a software. Finally the software is retired, when the users do not find it any longer useful due to reasons such as changed business scenario, availability of a new software having improved features and working, changed computing platforms, etc. This forms the essence of the life cycle of every software. Based on this description, we can define the software life cycle as follows:

The life cycle of a software represents the series of identifiable stages through which it evolves during its life time.

With this knowledge of a software life cycle, we discuss the concept of a software life cycle model and explore why it is necessary to follow a life cycle model in professional software development environments.

Software development life cycle (SDLC) model

In any systematic software development scenario, certain well-defined activities need to be performed by the development team and possibly by the customers as well, for the software to evolve from one stage in its life cycle to the next. For example, for a software to evolve from the requirements specification stage to the design stage, the developers need to elicit requirements from the customers, analyse those requirements, and formally document the requirements in the form of an SRS document.

A *software development life cycle* (SDLC) model (also called *software life cycle model* and *software development process model*) describes the different activities that need to be carried out for the software to evolve in its life cycle. Throughout our discussion, we shall use the terms *software development life cycle* (SDLC) and *software development process* interchangeably. However, some authors distinguish an SDLC from a software development process. In their usage, a software development process describes the life cycle activities more precisely and elaborately, as compared to an SDLC. Also, a development process may not only describe various activities that are carried out over the life cycle, but also prescribe a specific methodologies to carry out the activities, and also recommends the the specific documents and other artifacts that should be produced at the end of each phase. In this sense, the term SDLC can be considered to be a more generic term, as compared to the

development process and several development processes may fit the same SDLC.

An SDLC is represented graphically by drawing various stages of the life cycle and showing the transitions among the phases. This graphical model is usually accompanied by a textual description of various activities that need to be carried out during a phase before that phase can be considered to be complete. In simple words, we can define an SDLC as follows:

An SDLC graphically depicts the different phases through which a software evolves. It is usually accompanied by a textual description of the different activities that need to be carried out during each phase.

Process *versus* methodology

Though the terms *process* and *methodology* are at time used interchangeably, there is a subtle difference between the two. First, the term process has a broader scope and addresses either all the activities taking place during software development, or certain coarse grained activities such as design (e.g. design process), testing (test process), etc. Further, a software process not only identifies the specific activities that need to be carried out, but may also prescribe certain methodology for carrying out each activity. For example, a design process may recommend that in the design stage, the high-level design activity be carried out using Hatley and Pirbhai's structured analysis and design methodology. A methodology, on the other hand, prescribes a set of steps for carrying out a specific life cycle activity. It may also include the rationale and philosophical assumptions behind the set of steps through which the activity is accomplished.

A software development process has a much broader scope as compared to a software development methodology. A process usually describes all the activities starting from the inception of a software to its maintenance and retirement stages, or at least a chunk of activities in the life cycle. It also recommends specific methodologies for carrying out each activity. A methodology, in contrast, describes the steps to carry out only a single or at best a few individual activities.

Why use a development process?

The primary advantage of using a development process is that it encourages development of software in a systematic and disciplined manner. Adhering to a process is especially important to the development of professional software needing team effort. When software is developed by a team rather than by

an individual programmer, use of a life cycle model becomes indispensable for successful completion of the project.

Software development organisations have realised that adherence to a suitable life cycle model helps to produce good quality software and that helps minimise the chances of time and cost overruns.

Suppose a single programmer is developing a small program. For example, a student may be developing code for a class room assignment. The student might succeed even when he does not strictly follow a specific development process and adopts a build and fix style of development. However, it is a different ball game when a professional software is being developed by a team of programmers. Let us now understand the difficulties that may arise if a team does not use any development process, and the team members are given complete freedom to develop their assigned part of the software as per their own discretion. Several types of problems may arise. We illustrate one of the problems using an example. Suppose, a software development problem has been divided into several parts and these parts are assigned to the team members. From then on, suppose the team members are allowed the freedom to develop the parts assigned to them in whatever way they like. It is possible that one member might start writing the code for his part while making assumptions about the input results required from the other parts, another might decide to prepare the test documents first, and some other developer might start to carry out the design for the part assigned to him. In this case, severe problems can arise in interfacing the different parts and in managing the overall development. Therefore, *ad hoc* development turns out to be a sure way to have a failed project. Believe it or not, this is exactly what has caused many project failures in the past!

When a software is developed by a team, it is necessary to have a precise understanding among the team members as to—when to do what. In the absence of such an understanding, if each member at any time would do whatever activity he feels like doing. This would be an open invitation to developmental chaos and project failure. The use of a suitable life cycle model is crucial to the successful completion of a team-based development project. But, do we need an SDLC model for developing a small program. In this context, we need to distinguish between programming-in-the-small and programming-in-the-large.

Programming-in-the-small refers to development of a toy program by a single programmer. Whereas programming-in-the-large refers to development of a

professional software through team effort. While development of a software of the former type could succeed even while an individual programmer uses a build and fix style of development, use of a suitable SDLC is essential for a professional software development project involving team effort to succeed.

Why document a development process?

It is not enough for an organisation to just have a well-defined development process, but the development process needs to be properly documented. To understand the reason for this, let us consider that a development organisation does not document its development process. In this case, its developers develop only an informal understanding of the development process. An informal understanding of the development process among the team members can create several problems during development. We have identified a few important problems that may crop up when a development process is not adequately documented. Those problems are as follows:

- A documented process model ensures that every activity in the life cycle is accurately defined. Also, wherever necessary the methodologies for carrying out the respective activities are described. Without documentation, the activities and their ordering tend to be loosely defined, leading to confusion and misinterpretation by different teams in the organisation. For example, code reviews may informally and inadequately be carried out since there is no documented methodology as to how the code review should be done. Another difficulty is that for loosely defined activities, the developers tend to use their subjective judgments. As an example, unless it is explicitly prescribed, the team members would subjectively decide as to whether the test cases should be designed just after the requirements phase, after the design phase, or after the coding phase. Also, they would debate whether the test cases should be documented at all and the rigour with it should be documented.
- An undocumented process gives a clear indication to the members of the development teams about the lack of seriousness on the part of the management of the organisation about following the process. Therefore, an undocumented process serves as a hint to the developers to loosely follow the process. The symptoms of an undocumented process are easily visible—designs are shabbily done, reviews are not carried out rigorously, etc.

- A project team might often have to tailor a standard process model for use in a specific project. It is easier to tailor a documented process model, when it is required to modify certain activities or phases of the life cycle. For example, consider a project situation that requires the testing activities to be outsourced to another organisation. In this case, A documented process model would help to identify where exactly the required tailoring should occur.
- A documented process model, as we discuss later, is a mandatory requirement of the modern quality assurance standards such as ISO 9000 and SEI CMM. This means that unless a software organisation has a documented process, it would not qualify for accreditation with any of the quality standards. In the absence of a quality certification for the organisation, the customers would be suspicious of its capability of developing quality software and the organisation might find it difficult to win tenders for software development.

A documented development process forms a common understanding of the activities to be carried out among the software developers and helps them to develop software in a systematic and disciplined manner. A documented development process model, besides preventing the misinterpretations that might occur when the development process is not adequately documented, also helps to identify inconsistencies, redundancies, and omissions in the development process.

Nowadays, good software development organisations normally document their development process in the form of a booklet. They expect the developers recruited fresh to their organisation to first master their software development process during a short induction training that they are made to undergo.

Phase entry and exit criteria

A good SDLC besides clearly identifying the different phases in the life cycle, should unambiguously define the entry and exit criteria for each phase. The phase entry (or exit) criteria is usually expressed as a set of conditions that needs to be satisfied for the phase to start (or to complete). As an example, the phase exit criteria for the software requirements specification phase, can be that the *software requirements specification* (SRS) document is ready, has been reviewed internally, and also has been reviewed and approved by the customer. Only after these criteria are satisfied, the next phase can start.

If the entry and exit criteria for various phases are not well-defined, then that would leave enough scope for ambiguity in starting and ending various phases, and cause lot of confusion among the developers. Sometimes they might prematurely stop the activities in a phase, and some other times they might continue working on a phase much after when the phase should have been over. The decision regarding whether a phase is complete or not becomes subjective and it becomes difficult for the project manager to accurately tell how much has the development progressed. When the phase entry and exit criteria are not well-defined, the developers might close the activities of a phase much before they are actually complete, giving a false impression of rapid progress. In this case, it becomes very difficult for the project manager to determine the exact status of development and track the progress of the project. This usually leads to a problem that is usually identified as the *99 per cent complete syndrome*. This syndrome appears when there the software project manager has no definite way of assessing the progress of a project, the optimistic team members feel that their work is 99 per cent complete even when their work is far from completion—making all projections made by the project manager about the project completion time to be highly inaccurate.

2.2 WATERFALL MODEL AND ITS EXTENSIONS

The waterfall model and its derivatives were extremely popular in the 1970s and still are heavily being used across many development projects. The waterfall model is possibly the most obvious and intuitive way in which software can be developed through team effort. We can think of the waterfall model as a generic model that has been extended in many ways for catering to certain specific software development situations to realise all other software life cycle models. For this reason, after discussing the classical and iterative waterfall models, we discuss its various extensions.

2.2.1 Classical Waterfall Model

Classical waterfall model is intuitively the most obvious way to develop software. It is simple but idealistic. In fact, it is hard to put this model into use in any non-trivial software development project. One might wonder if this model is hard to use in practical development projects, then why study it at all? The reason is that all other life cycle models can be thought of as being extensions of the classical waterfall model.

Therefore, it makes sense to first understand the classical waterfall model, in order to be able to develop a proper understanding of other life cycle models. Besides, we shall see later in this text that this model though not used for software development; is implicitly used while documenting software.

The classical waterfall model divides the life cycle into a set of phases as shown in Figure 2.1. It can be easily observed from this figure that the diagrammatic representation of the classical waterfall model resembles a multi-level waterfall. This resemblance justifies the name of the model.

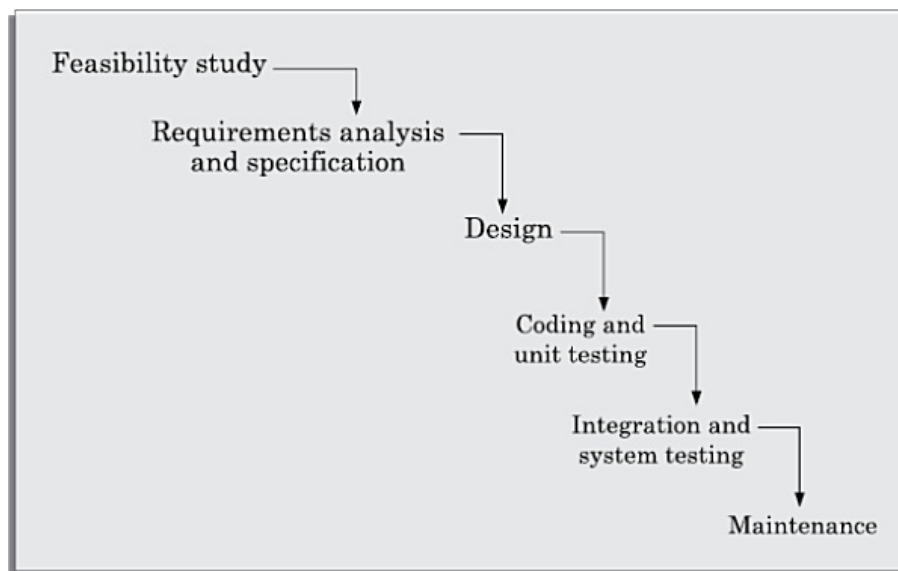


Figure 2.1: Classical waterfall model.

Phases of the classical waterfall model

The different phases of the classical waterfall model have been shown in Figure 2.1. As shown in Figure 2.1, the different phases are—feasibility study, requirements analysis and specification, design, coding and unit testing, integration and system testing, and maintenance. The phases starting from the feasibility study to the integration and system testing phase are known as the *development phases*. A software is developed during the development phases, and at the completion of the development phases, the software is delivered to the customer. After the delivery of software, customers start to use the software signalling the commencement of the operation phase. As the customers start to use the software, changes to it become necessary on account of bug fixes and feature extensions, causing maintenance works to be undertaken. Therefore, the last phase is also known as the *maintenance phase* of the life cycle. It needs to be kept in mind that some of the text

books have different number and names of the phases.

An activity that spans all phases of software development is *project management*. Since it spans the entire project duration, no specific phase is named after it. Project management, nevertheless, is an important activity in the life cycle and deals with managing the software development and maintenance activities.

In the waterfall model, different life cycle phases typically require relatively different amounts of efforts to be put in by the development team. The relative amounts of effort spent on different phases for a typical software has been shown in Figure 2.2. Observe from Figure 2.2 that among all the life cycle phases, the maintenance phase normally requires the maximum effort. On the average, about 60 per cent of the total effort put in by the development team in the entire life cycle is spent on the maintenance activities alone.

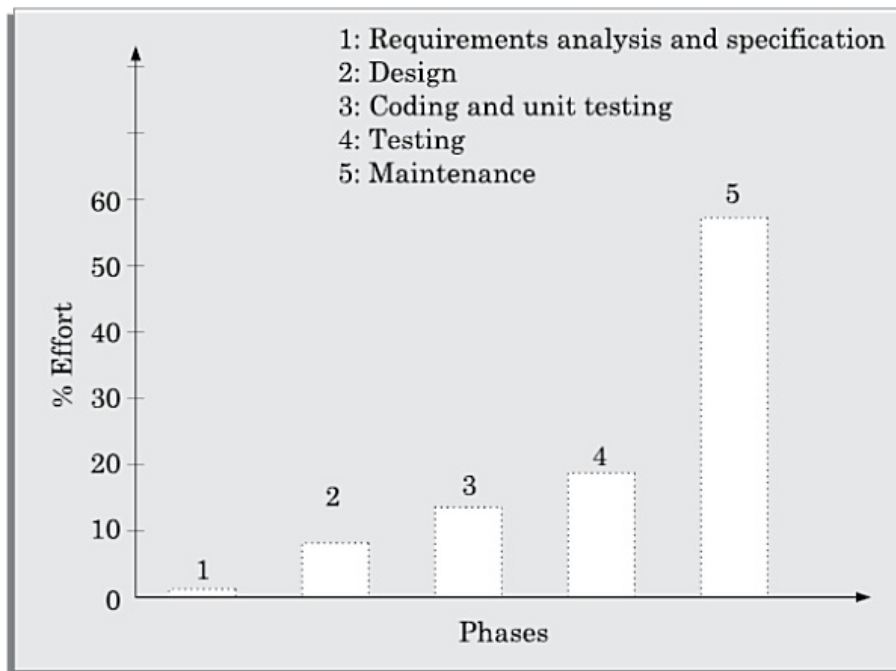


Figure 2.2: Relative effort distribution among different phases of a typical product.

However, among the development phases, the integration and system testing phase requires the maximum effort in a typical development project. In the following subsection, we briefly describe the activities that are carried out in the different phases of the classical waterfall model.

Feasibility study

The main focus of the feasibility study stage is to determine whether it would be *financially* and *technically feasible* to develop the software. The

feasibility study involves carrying out several activities such as collection of basic information relating to the software such as the different data items that would be input to the system, the processing required to be carried out on these data, the output data required to be produced by the system, as well as various constraints on the development. These collected data are analysed to perform at the following:

Development of an overall understanding of the problem: It is necessary to first develop an overall understanding of what the customer requires to be developed. For this, only the the important requirements of the customer need to be understood and the details of various requirements such as the screen layouts required in the *graphical user interface* (GUI), specific formulas or algorithms required for producing the required results, and the databases schema to be used are ignored.

Formulation of the various possible strategies for solving the problem: In this activity, various possible high-level solution schemes to the problem are determined. For example, solution in a client-server framework and a standalone application framework may be explored.

Evaluation of the different solution strategies: The different identified solution schemes are analysed to evaluate their benefits and shortcomings. Such evaluation often requires making approximate estimates of the resources required, cost of development, and development time required. The different solutions are compared based on the estimations that have been worked out. Once the best solution is identified, all activities in the later phases are carried out as per this solution. At this stage, it may also be determined that none of the solutions is feasible due to high cost, resource constraints, or some technical reasons. This scenario would, of course, require the project to be abandoned.

We can summarise the outcome of the feasibility study phase by noting that other than deciding whether to take up a project or not, at this stage very high-level decisions regarding the solution strategy is defined. Therefore, feasibility study is a very crucial stage in software development. The following is a case study of the feasibility study undertaken by an organisation. It is intended to give a feel of the activities and issues involved in the feasibility study phase of a typical software project.

Case study 2.1

A mining company named Galaxy Mining Company Ltd. (GMC Ltd.) has mines located at various places in India. It has about fifty different mine sites spread across eight states. The company employs a large number of miners at each mine site. Mining being a risky profession, the company intends to operate a special provident fund, which would exist in addition to the standard provident fund that the miners already enjoy. The main objective of having the special provident fund (SPF) would be to quickly distribute some compensation before the PF amount is paid.

According to this scheme, each mine site would deduct SPF installments from each miner every month and deposit the same to the central special provident fund commissioner (CSPFC). The CSPFC will maintain all details regarding the SPF installments collected from the miners.

GMC Ltd. requested a reputed software vendor Adventure Software Inc. to undertake the task of developing the software for automating the maintenance of SPF records of all employees. GMC Ltd. has realised that besides saving manpower on book-keeping work, the software would help in speedy settlement of claim cases. GMC Ltd. indicated that the amount it can at best afford Rs. 1 million for this software to be developed and installed.

Adventure Software Inc. deputed their project manager to carry out the feasibility study. The project manager discussed with the top managers of GMC Ltd. to get an overview of the project. He also discussed with the field PF officers at various mine sites to determine the exact details of the project. The project manager identified two broad approaches to solve the problem. One is to have a central database which would be accessed and updated via a satellite connection to various mine sites. The other approach is to have local databases at each mine site and to update the central database periodically through a dial-up connection. This periodic updates can be done on a daily or hourly basis depending on the delay acceptable to GMC Ltd. in invoking various functions of the software. He found that the second approach is very affordable and more fault-tolerant as the local mine sites can operate even when the communication link temporarily fails. In this approach, when a link fails, only the update of the central database gets delayed. Whereas in the first approach, all SPF work gets stalled at a mine site for the entire duration of link failure. The project manager quickly analysed the overall database functionalities required, the user-interface issues, and the software handling communication with the mine sites. From this analysis, he estimated the approximate cost to develop the software. He found that a solution involving maintaining local databases at the mine sites and periodically updating a central database is financially and technically feasible. The project manager discussed this solution with the president of GMC Ltd., who indicated that the proposed solution would be acceptable to them.

Requirements analysis and specification

The aim of the requirements analysis and specification phase is to understand the exact requirements of the customer and to document

them properly. This phase consists of two distinct activities, namely requirements gathering and analysis, and requirements specification. In the following subsections, we give an overview of these two activities:

- **Requirements gathering and analysis:** The goal of the requirements gathering activity is to collect all relevant information regarding the software to be developed from the customer with a view to clearly understand the requirements. For this, first requirements are gathered from the customer and then the gathered requirements are analysed. The goal of the requirements analysis activity is to weed out the incompleteness and inconsistencies in these gathered requirements. Note that a *inconsistent* requirement is one in which some part of the requirement contradicts with some other part. On the other hand, a *incomplete* requirement is one in which some parts of the actual requirements have been omitted.
- **Requirements specification:** After the requirement gathering and analysis activities are complete, the identified requirements are documented. This is called a *software requirements specification* (SRS) document. The SRS document is written using end-user terminology. This makes the SRS document understandable to the customer. Therefore, understandability of the SRS document is an important issue. The SRS document normally serves as a contract between the development team and the customer. Any future dispute between the customer and the developers can be settled by examining the SRS document. The SRS document is therefore an important document which must be thoroughly understood by the development team, and reviewed jointly with the customer. The SRS document not only forms the basis for carrying out all the development activities, but several documents such as users' manuals, system test plan, etc. are prepared directly based on it. In Chapter 4, we examine the requirements analysis activity and various issues involved in developing a good SRS document in more detail.

Design

The goal of the design phase is to transform the requirements specified in the SRS document into a structure that is suitable for implementation in some programming language. In technical terms, during the design phase the *software architecture* is derived from the SRS document. Two

distinctly different design approaches are popularly being used at present—the procedural and object-oriented design approaches. In the following, we briefly discuss the essence of these two approaches. These two approaches are discussed in detail in Chapters 6, 7, and 8.

- **Procedural design approach:** The traditional design approach is in use in many software development projects at the present time. This traditional design technique is based on the data flow-oriented design approach. It consists of two important activities; first *structured analysis* of the requirements specification is carried out where the detailed structure of the problem is examined. This is followed by a *structured design* step where the results of structured analysis are transformed into the software design.

During structured analysis, the functional requirements specified in the SRS document are decomposed into subfunctions and the data-flow among these subfunctions is analysed and represented diagrammatically in the form of DFDs. The DFD technique is discussed in Chapter 6. Structured design is undertaken once the structured analysis activity is complete. Structured design consists of two main activities—architectural design (also called *high-level design*) and detailed design (also called *Low-level design*). High-level design involves decomposing the system into modules, and representing the interfaces and the invocation relationships among the modules. A high-level software design is some times referred to as the *software architecture*. During the detailed design activity, internals of the individual modules such as the data structures and algorithms of the modules are designed and documented.

- **Object-oriented design approach:** In this technique, various objects that occur in the problem domain and the solution domain are first identified and the different relationships that exist among these objects are identified. The object structure is further refined to obtain the detailed design. The OOD approach is credited to have several benefits such as lower development time and effort, and better maintainability of the software. The object-oriented design technique is discussed in Chapters 7 and 8.

Coding and unit testing

The purpose of the coding and unit testing phase is to translate a software design into source code and to ensure that individually each function is working correctly. The coding phase is also sometimes called the *implementation phase*, since the design is implemented into a workable solution in this phase. Each component of the design is implemented as a program module. The end-product of this phase is a set of program modules that have been individually unit tested. The main objective of unit testing is to determine the correct working of the individual modules. The specific activities carried out during unit testing include designing test cases, testing, debugging to fix problems, and management of test cases. We shall discuss the coding and unit testing techniques in Chapter 10.

Integration and system testing

Integration of different modules is undertaken soon after they have been coded and unit tested. During the integration and system testing phase, the different modules are integrated in a planned manner. Various modules making up a software are almost never integrated in one shot (can you guess the reason for this?). Integration of various modules are normally carried out incrementally over a number of steps. During each integration step, previously planned modules are added to the partially integrated system and the resultant system is tested. Finally, after all the modules have been successfully integrated and tested, the full working system is obtained. System testing is carried out on this fully working system.

Integration testing is carried out to verify that the interfaces among different units are working satisfactorily. On the other hand, the goal of system testing is to ensure that the developed system conforms to the requirements that have been laid out in the SRS document.

System testing usually consists of three different kinds of testing activities:

- **-testing:** testing is the system testing performed by the development team.
- **-testing:** This is the system testing performed by a friendly set of customers.
- **Acceptance testing:** After the software has been delivered, the customer performs system testing to determine whether to accept the delivered software or to reject it.

We discuss various integration and system testing techniques in more detail in Chapter 10.

Maintenance

The total effort spent on maintenance of a typical software during its operation phase is much more than that required for developing the software itself. Many studies carried out in the past confirm this and indicate that the ratio of relative effort of developing a typical software product and the total effort spent on its maintenance is roughly 40:60. Maintenance is required in the following three types of situations:

- **Corrective maintenance:** This type of maintenance is carried out to correct errors that were not discovered during the product development phase.
- **Perfective maintenance:** This type of maintenance is carried out to improve the performance of the system, or to enhance the functionalities of the system based on customer's requests.
- **Adaptive maintenance:** Adaptive maintenance is usually required for porting the software to work in a new environment. For example, porting may be required to get the software to work on a new computer platform or with a new operating system.

Various maintenance activities have been discussed in more detail in Chapter 13.

Shortcomings of the classical waterfall model

The classical waterfall model is a very simple and intuitive model. However, it suffers from several shortcomings. Let us identify some of the important shortcomings of the classical waterfall model:

No feedback paths: In classical waterfall model, the evolution of a software from one phase to the next is analogous to a waterfall. Just as water in a waterfall after having flowed down cannot flow back, once a phase is complete, the activities carried out in it and any artifacts produced in this phase are considered to be final and are closed for any rework. This requires that all activities during a phase are flawlessly carried out.

The classical waterfall model is idealistic in the sense that it assumes that no error is ever committed by the developers during any of the life cycle phases, and therefore, incorporates no mechanism for error correction.

Contrary to a fundamental assumption made by the classical waterfall model, in practical development environments, the developers do commit a large number of errors in almost every activity they carry out during various phases of the life cycle. After all, programmers are humans and as the old adage says *to err is humane*. The cause for errors can be many—oversight, wrong interpretations, use of incorrect solution scheme, communication gap, etc. These defects usually get detected much later in the life cycle. For example, a design defect might go unnoticed till the coding or testing phase. Once a defect is detected at a later time, the developers need to redo some of the work done during that phase and also redo the work of later phases that are affected by the rework. Therefore, in any non-trivial software development project, it becomes nearly impossible to strictly follow the classical waterfall model of software development.

Difficult to accommodate change requests: This model assumes that all customer requirements can be completely and correctly defined at the beginning of the project. There is much emphasis on creating an unambiguous and complete set of requirements. But, it is hard to achieve this even in ideal project scenarios. The customers' requirements usually keep on changing with time. But, in this model it becomes difficult to accommodate any requirement change requests made by the customer after the requirements specification phase is complete, and this often becomes a source of customer discontent.

Inefficient error corrections: This model defers integration of code and testing tasks until it is very late when the problems are harder to resolve.

No overlapping of phases: This model recommends that the phases be carried out sequentially—new phase can start only after the previous one completes. However, it is rarely possible to adhere to this recommendation and it leads to a large number of team members to idle for extended periods. For example, for efficient utilisation of manpower, the testing team might need to design the system test cases immediately after requirements specification is complete. (We shall discuss in Chapter 10 that the system test cases are designed solely based on the SRS document). In this case, the activities of the design and testing phases overlap. Consequently, it is safe to say that in a practical software development scenario, rather than having a precise point in time at which a phase transition occurs, the different phases need to overlap for cost and efficiency reasons.

Is the classical waterfall model useful at all?

We have already pointed out that it is hard to use the classical waterfall model in real projects. In any practical development environment, as the software takes shape, several iterations through the different waterfall stages become necessary for correction of errors committed during various phases. Therefore, the classical waterfall model is hardly usable for software development. But, as suggested by Parnas [1972] the final documents for the product should be written as if the product was developed using a pure classical waterfall.

Irrespective of the life cycle model that is actually followed for a product development, the final documents are always written to reflect a classical waterfall model of development, so that comprehension of the documents becomes easier for any one reading the document.

The rationale behind preparation of documents based on the classical waterfall model can be explained using Hoare's metaphor of mathematical theorem [1994] proving—A mathematician presents a proof as a single chain of deductions, even though the proof might have come from a convoluted set of partial attempts, blind alleys and backtracks. Imagine how difficult it would be to understand, if a mathematician presents a proof by retaining all the backtracking, mistake corrections, and solution refinements he made while working out the proof.

2.2.2 Iterative Waterfall Model

We had pointed out in the previous section that in a practical software development project, the classical waterfall model is hard to use. We had branded the classical waterfall model as an idealistic model. In this context, the iterative waterfall model can be thought of as incorporating the necessary changes to the classical waterfall model to make it usable in practical software development projects.

The main change brought about by the iterative waterfall model to the classical waterfall model is in the form of providing feedback paths from every phase to its preceding phases.

The feedback paths introduced by the iterative waterfall model are shown in Figure 2.3. The feedback paths allow for correcting errors committed by a programmer during some phase, as and when these are detected in a later phase. For example, if during the testing phase a design error is identified, then the feedback path allows the design to be reworked and the changes to

be reflected in the design documents and all other subsequent documents. Please notice that in Figure 2.3 there is no feedback path to the feasibility stage. This is because once a team having accepted to take up a project, does not give up the project easily due to legal and moral reasons.

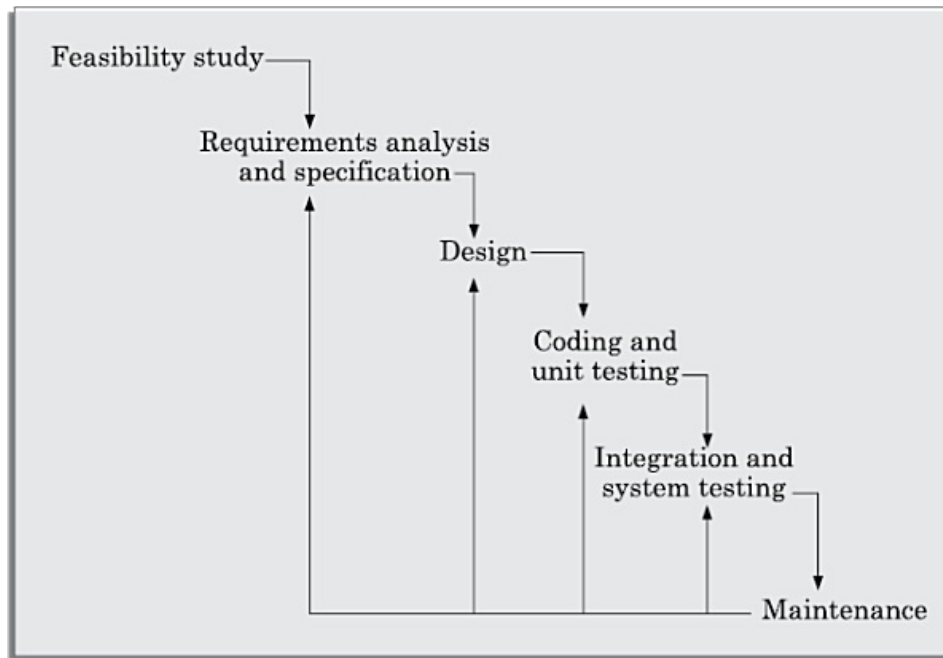


Figure 2.3: Iterative waterfall model.

Almost every life cycle model that we discuss are iterative in nature, except the classical waterfall model and the V-model—which are sequential in nature. In a sequential model, once a phase is complete, no work product of that phase are changed later.

Phase containment of errors

No matter how careful a programmer may be, he might end up committing some mistake or other while carrying out a life cycle activity. These mistakes result in errors (also called *faults* or *bugs*) in the work product. It is advantageous to detect these errors in the same phase in which they take place, since early detection of bugs reduces the effort and time required for correcting those. For example, if a design problem is detected in the design phase itself, then the problem can be taken care of much more easily than if the error is identified, say, at the end of the testing phase. In the later case, it would be necessary not only to rework the design, but also to appropriately redo the relevant coding as well as the testing activities, thereby incurring higher cost. It may not always be possible to detect all the errors in the same phase in which they are made. Nevertheless, the errors should be detected as

early as possible.

The principle of detecting errors as close to their points of commitment as possible is known as *phase containment of errors*.

For achieving phase containment of errors, how can the developers detect almost all error that they commit in the same phase? After all, the end product of many phases are text or graphical documents, e.g. SRS document, design document, test plan document, etc. A popular technique is to rigorously review the documents produced at the end of a phase.

Phase overlap

Even though the strict waterfall model envisages sharp transitions to occur from one phase to the next (see Figure 2.3), in practice the activities of different phases overlap (as shown in Figure 2.4) due to two main reasons:

- In spite of the best effort to detect errors in the same phase in which they are committed, some errors escape detection and are detected in a later phase. These subsequently detected errors cause the activities of some already completed phases to be reworked. If we consider such rework after a phase is complete, we can say that the activities pertaining to a phase do not end at the completion of the phase, but overlap with other phases as shown in Figure 2.4.
- An important reason for phase overlap is that usually the work required to be carried out in a phase is divided among the team members. Some members may complete their part of the work earlier than other members. If strict phase transitions are maintained, then the team members who complete their work early would idle waiting for the phase to be complete, and are said to be in a *blocking state*. Thus the developers who complete early would idle while waiting for their team mates to complete their assigned work. Clearly this is a cause for wastage of resources and a source of cost escalation and inefficiency. As a result, in real projects, the phases are allowed to overlap. That is, once a developer completes his work assignment for a phase, proceeds to start the work for the next phase, without waiting for all his team members to complete their respective work allocations.

Considering these situations, the effort distribution for different phases with

time would be as shown in Figure 2.4.

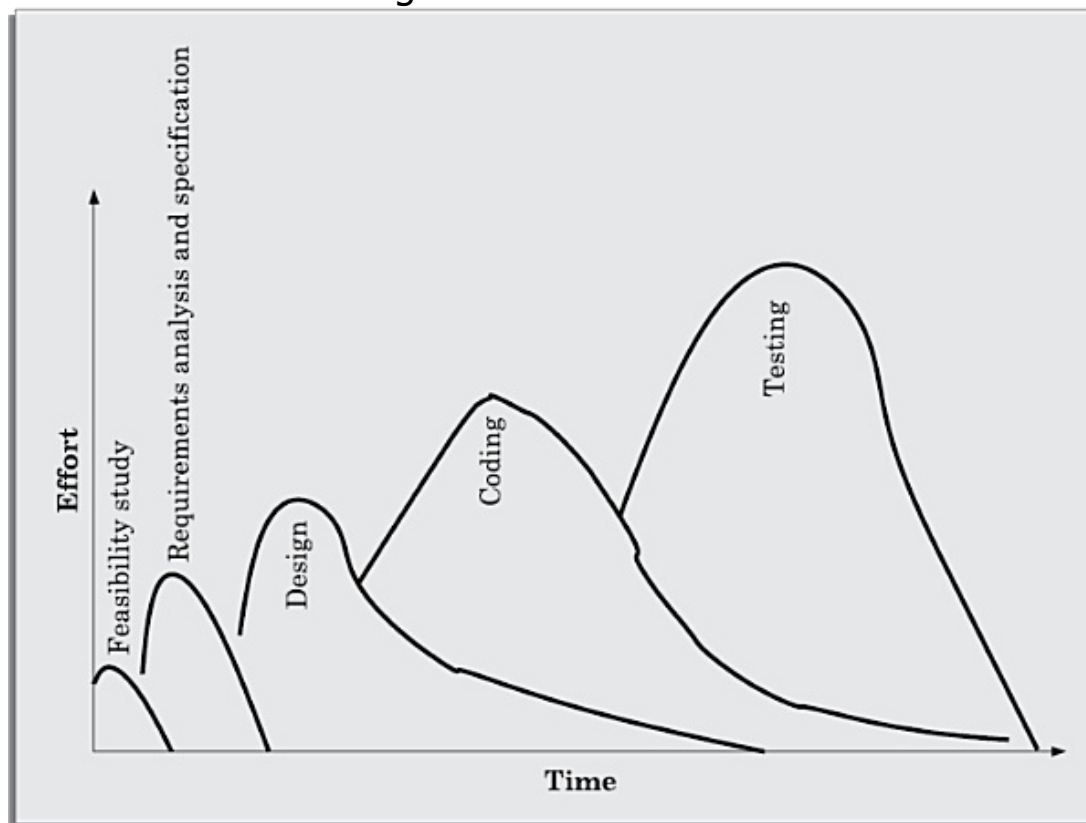


Figure 2.4: Distribution of effort for various phases in the iterative waterfall model.

Shortcomings of the iterative waterfall model

The iterative waterfall model is a simple and intuitive software development model. It was used satisfactorily during 1970s and 1980s. However, the characteristics of software development projects have changed drastically over years. In the 1970s and 1960s, software development projects spanned several years and mostly involved generic software product development. The projects are now shorter, and involve Customised software development. Further, software was earlier developed from scratch. Now the emphasis is on as much reuse of code and other project artifacts as possible. Waterfall-based models have worked satisfactorily over last many years in the past. The situation has changed substantially now. As pointed out in the first chapter several decades back, every software was developed from scratch. Now, not only software has become very large and complex, very few (if at all any) software project is being developed from scratch. The software services (customised software) are poised to become the dominant types of projects. In the present software development projects, use of waterfall model causes several problems. In this

context, the agile models have been proposed about a decade back that attempt to overcome the important shortcomings of the waterfall model by suggesting certain radical modification to the waterfall style of software development. In Section 2.4, we discuss the agile model. Some of the glaring shortcomings of the waterfall model when used in the present-day software development projects are as following:

Difficult to accommodate change requests: A major problem with the waterfall model is that the requirements need to be frozen before the development starts. Based on the frozen requirements, detailed plans are made for the activities to be carried out during the design, coding, and testing phases. Since activities are planned for the entire duration, substantial effort and resources are invested in the activities as developing the complete requirements specification, design for the complete functionality and so on. Therefore, accommodating even small change requests after the development activities are underway not only requires overhauling the plan, but also the artifacts that have already been developed.

Once requirements have been frozen, the waterfall model provides no scope for any modifications to the requirements.

While the waterfall model is inflexible to later changes to the requirements, evidence gathered from several projects points to the fact that later changes to requirements are almost inevitable. Even for projects with highly experienced professionals at all levels, as well as computer savvy customers, requirements are often missed as well as misinterpreted. Unless change requests are encouraged, the developed functionalities would be misfit to the true customer requirements. Requirement changes can arise due to a variety of reasons including the following—requirements were not clear to the customer, requirements were misunderstood, business process of the customer may have changed after the SRS document was signed off, etc. In fact, customers get clearer understanding of their requirements only after working on a fully developed and installed system.

The basic assumption made in the iterative waterfall model that methodical requirements gathering and analysis alone would comprehensively and correctly identify all the requirements by the end of the requirements phase is flawed.

Incremental delivery not supported: In the iterative waterfall model, the full software is completely developed and tested before it is delivered to the customer. There is no provision for any intermediate deliveries to occur. This

is problematic because the complete application may take several months or years to be completed and delivered to the customer. By the time the software is delivered, installed, and becomes ready for use, the customer's business process might have changed substantially. This makes the developed application a poor fit to the customer's requirements.

Phase overlap not supported: For most real life projects, it becomes difficult to follow the rigid phase sequence prescribed by the waterfall model. By the term *a rigid phase sequence*, we mean that a phase can start only after the previous phase is complete in all respects. As already discussed, strict adherence to the waterfall model creates *blocking states*. The waterfall model is usually adapted for use in real-life projects by allowing overlapping of various phases as shown in Figure 2.4.

Error correction unduly expensive: In waterfall model, validation is delayed till the complete development of the software. As a result, the defects that are noticed at the time of validation incur expensive rework and result in cost escalation and delayed delivery.

Limited customer interactions: This model supports very limited customer interactions. It is generally accepted that software developed in isolation from the customer is the cause of many problems. In fact, interactions occur only at the start of the project and at project completion. As a result, the developed software usually turns out to be a misfit to the customer's actual requirements.

Heavy weight: The waterfall model overemphasises documentation. A significant portion of the time of the developers is spent in preparing documents, and revising them as changes occur over the life cycle. Heavy documentation though useful during maintenance and for carrying out review, is a source of team inefficiency.

No support for risk handling and code reuse: It becomes difficult to use the waterfall model in projects that are susceptible to various types of risks, or those involving significant reuse of existing development artifacts. Please recollect that software services types of projects usually involve significant reuse.

2.2.3 V-Model

A popular development process model, V-model is a variant of the waterfall model. As is the case with the waterfall model, this model gets its name from its visual appearance (see Figure 2.5). In this model verification and

validation activities are carried out throughout the development life cycle, and therefore the chances bugs in the work products considerably reduce. This model is therefore generally considered to be suitable for use in projects concerned with development of safety-critical software that are required to have high reliability.

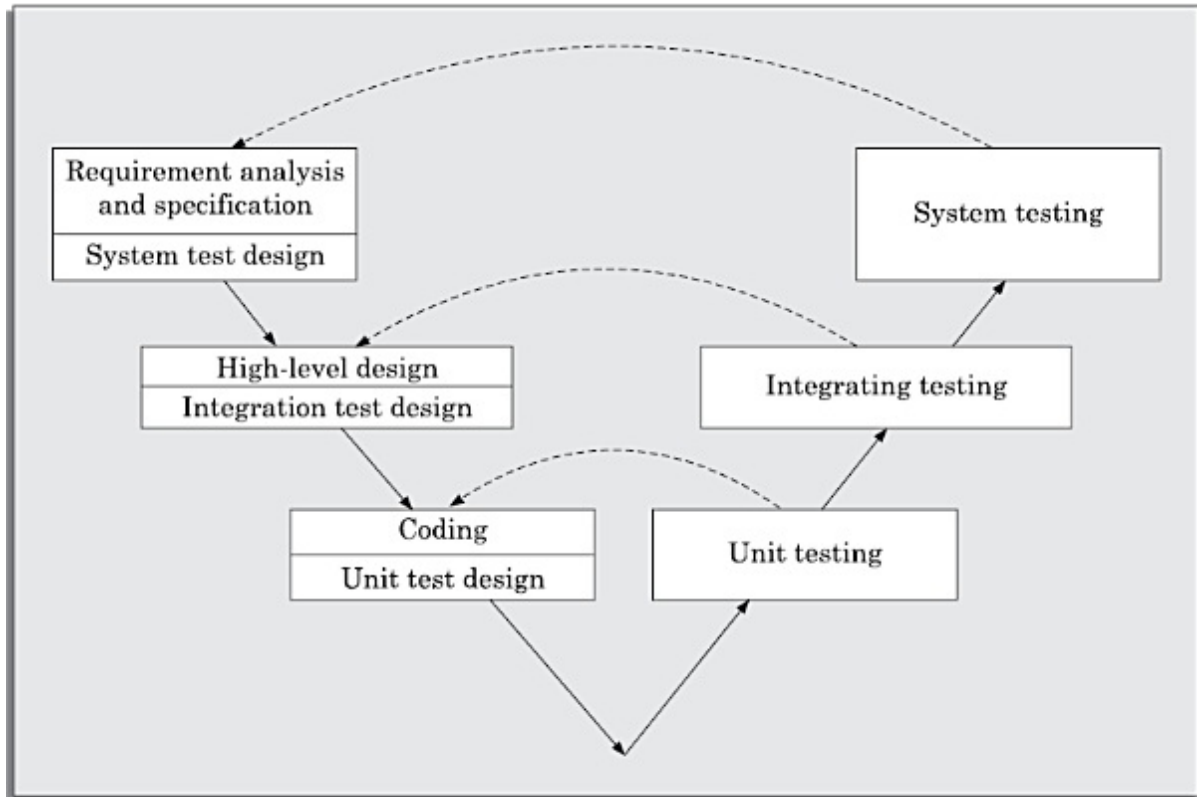


Figure 2.5: V-model.

As shown in Figure 2.5, there are two main phases—development and validation phases. The left half of the model comprises the development phases and the right half comprises the validation phases.

- In each development phase, along with the development of a work product, test case design and the plan for testing the work product are carried out, whereas the actual testing is carried out in the validation phase. This validation plan created during the development phases is carried out in the corresponding validation phase which have been shown by dotted arcs in Figure 2.5.
- In the validation phase, testing is carried out in three steps—unit, integration, and system testing. The purpose of these three different steps of testing during the validation phase is to detect defects that arise in the corresponding phases of software development—

requirements analysis and specification, design, and coding respectively.

V-model *versus* waterfall model

We have already pointed out that the V-model can be considered to be an extension of the waterfall model. However, there are major differences between the two. As already mentioned, in contrast to the iterative waterfall model where testing activities are confined to the testing phase only, in the V-model testing activities are spread over the entire life cycle. As shown in Figure 2.5, during the requirements specification phase, the system test suite design activity takes place. During the design phase, the integration test cases are designed. During coding, the unit test cases are designed. Thus, we can say that in this model, development and validation activities proceed hand in hand.

Advantages of V-model

The important advantages of the V-model over the iterative waterfall model are as following:

- In the V-model, much of the testing activities (test case design, test planning, etc.) are carried out in parallel with the development activities. Therefore, before testing phase starts significant part of the testing activities, including test case design and test planning, is already complete. Therefore, this model usually leads to a shorter testing phase and an overall faster product development as compared to the iterative model.
- Since test cases are designed when the schedule pressure has not built up, the quality of the test cases are usually better.
- The test team is reasonably kept occupied throughout the development cycle in contrast to the waterfall model where the testers are active only during the testing phase. This leads to more efficient manpower utilisation.
- In the V-model, the test team is associated with the project from the beginning. Therefore they build up a good understanding of the development artifacts, and this in turn, helps them to carry out effective testing of the software. In contrast, in the waterfall model often the test team comes on board late in the development cycle,

since no testing activities are carried out before the start of the implementation and testing phase.

Disadvantages of V-model

Being a derivative of the classical waterfall model, this model inherits most of the weaknesses of the waterfall model.

2.2.4 Prototyping Model

The prototype model is also a popular life cycle model. The prototyping model can be considered to be an extension of the waterfall model. This model suggests building a working *prototype* of the system, before development of the actual software. A prototype is a toy and crude implementation of a system. It has limited functional capabilities, low reliability, or inefficient performance as compared to the actual software. A prototype can be built very quickly by using several shortcuts. The shortcuts usually involve developing inefficient, inaccurate, or dummy functions. The shortcut implementation of a function, for example, may produce the desired results by using a table look-up rather than by performing the actual computations. Normally the term *rapid prototyping* is used when software tools are used for prototype construction. For example, tools based on *fourth generation languages* (4GL) may be used to construct the prototype for the GUI parts.

Necessity of the prototyping model

The prototyping model is advantageous to use for specific types of projects. In the following, we identify three types of projects for which the prototyping model can be followed to advantage:

- It is advantageous to use the prototyping model for development of the *graphical user interface* (GUI) part of an application. Through the use of a prototype, it becomes easier to illustrate the input data formats, messages, reports, and the interactive dialogs to the customer. This is a valuable mechanism for gaining better understanding of the customers' needs. In this regard, the prototype model turns out to be especially useful in developing the *graphical user interface* (GUI) part of a system. For the user, it becomes much easier

to form an opinion regarding what would be more suitable by experimenting with a working user interface, rather than trying to imagine the working of a hypothetical user interface.

The GUI part of a software system is almost always developed using the prototyping model.

- The prototyping model is especially useful when the exact technical solutions are unclear to the development team. A prototype can help them to critically examine the technical issues associated with product development. For example, consider a situation where the development team has to write a command language interpreter as part of a graphical user interface development. Suppose none of the team members has ever written a compiler before. Then, this lack of familiarity with a required development technology is a technical risk. This risk can be resolved by developing a prototype compiler for a very small language to understand the issues associated with writing a compiler for a command language. Once they feel confident in writing compiler for the small language, they can use this knowledge to develop the compiler for the command language. Often, major design decisions depend on issues such as the response time of a hardware controller, or the efficiency of a sorting algorithm, etc. In such circumstances, a prototype is often the best way to resolve the technical issues.
- An important reason for developing a prototype is that it is impossible to “get it right” the first time. As advocated by Brooks [1975], one must plan to throw away the software in order to develop a good software later. Thus, the prototyping model can be deployed when development of highly optimised and efficient software is required.

From the above discussions, we can conclude the following:

The prototyping model is considered to be useful for the development of not only the GUI parts of a software, but also for a software project for which certain technical issues are not clear to the development team.

Life cycle activities of prototyping model

The prototyping model of software development is graphically shown in

Figure 2.6. As shown in Figure 2.6, software is developed through two major activities—prototype construction and iterative waterfall-based software development.

Prototype development: Prototype development starts with an initial requirements gathering phase. A quick design is carried out and a prototype is built. The developed prototype is submitted to the customer for evaluation. Based on the customer feedback, the requirements are refined and the prototype is suitably modified. This cycle of obtaining customer feedback and modifying the prototype continues till the customer approves the prototype.

Iterative development: Once the customer approves the prototype, the actual software is developed using the iterative waterfall approach. In spite of the availability of a working prototype, the SRS document is usually needed to be developed since the SRS document is invaluable for carrying out traceability analysis, verification, and test case design during later phases. However, for GUI parts, the requirements analysis and specification phase becomes redundant since the working prototype that has been approved by the customer serves as an animated requirements specification.

The code for the prototype is usually thrown away. However, the experience gathered from developing the prototype helps a great deal in developing the actual system.

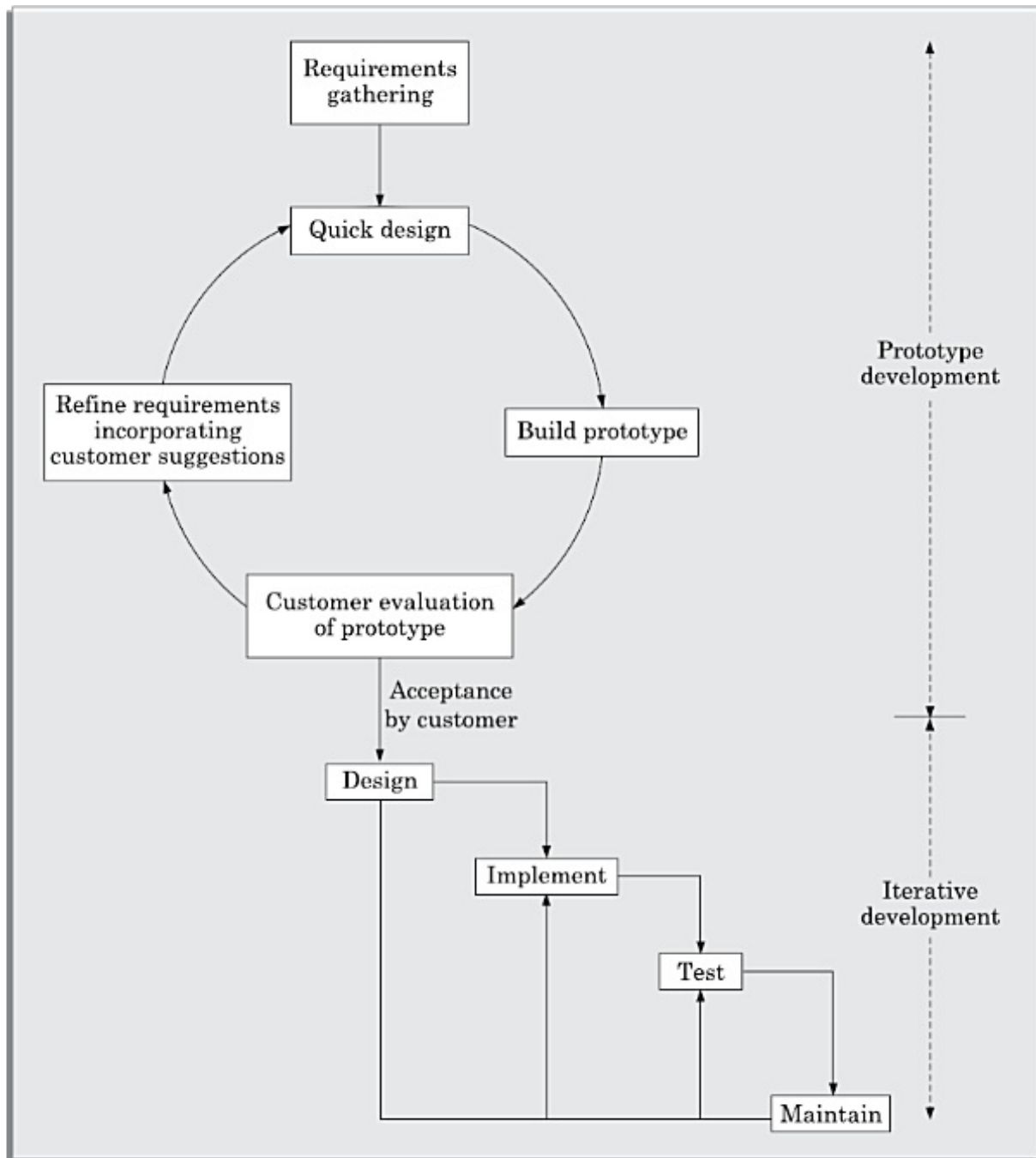


Figure 2.6: Prototyping model of software development.

Even though the construction of a throwaway prototype might involve incurring additional cost, for systems with unclear customer requirements and for systems with unresolved technical issues, the overall development cost usually turns out to be lower compared to an equivalent system developed using the iterative waterfall model.

By constructing the prototype and submitting it for user evaluation, many customer requirements get properly defined and technical issues get resolved by experimenting with the prototype. This minimises later change requests

from the customer and the associated redesign costs.

Strengths of the prototyping model

This model is the most appropriate for projects that suffer from technical and requirements risks. A constructed prototype helps overcome these risks.

Weaknesses of the prototyping model

The prototype model can increase the cost of development for projects that are routine development work and do not suffer from any significant risks. Even when a project is susceptible to risks, the prototyping model is effective only for those projects for which the risks can be identified upfront before the development starts. Since the prototype is constructed only at the start of the project, the prototyping model is ineffective for risks identified later during the development cycle. The prototyping model would not be appropriate for projects for which the risks can only be identified after the development is underway.

2.2.5 Incremental Development Model

This life cycle model is sometimes referred to as the *successive versions* model and sometimes as the incremental model. In this life cycle model, first a simple working system implementing only a few basic features is built and delivered to the customer. Over many successive iterations successive versions are implemented and delivered to the customer until the desired system is realised. The incremental development model has been shown in Figure 2.7.

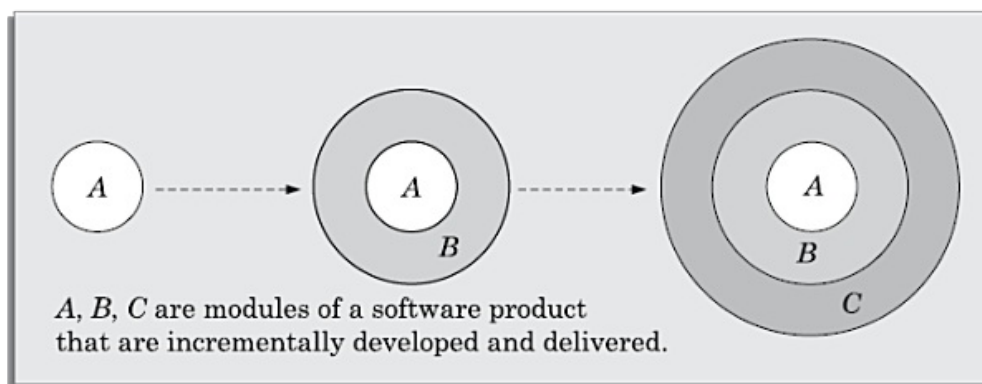


Figure 2.7: Incremental software development.

Life cycle activities of incremental development model

In the incremental life cycle model, the requirements of the software are first broken down into several modules or features that can be incrementally constructed and delivered. This has been pictorially depicted in Figure 2.7. At any time, plan is made only for the next increment and no long-term plans are made. Therefore, it becomes easier to accommodate change requests from the customers.

The development team first undertakes to develop the core features of the system. The core or basic features are those that do not need to invoke any services from the other features. On the other hand, non-core features need services from the core features. Once the initial core features are developed, these are refined into increasing levels of capability by adding new functionalities in successive versions. Each incremental version is usually developed using an iterative waterfall model of development. The incremental model is schematically shown in Figure 2.8. As each successive version of the software is constructed and delivered to the customer, the customer feedback is obtained on the delivered version and these feedbacks are incorporated in the next version. Each delivered version of the software incorporates additional features over the previous version and also refines the features that were already delivered to the customer.

The incremental model has schematically been shown in Figure 2.8. After the requirements gathering and specification, the requirements are split into several versions. Starting with the core (version 1), in each successive increment, the next version is constructed using an iterative waterfall model of development and deployed at the customer site. After the last (shown as version n) has been developed and deployed at the client site, the full software is deployed.

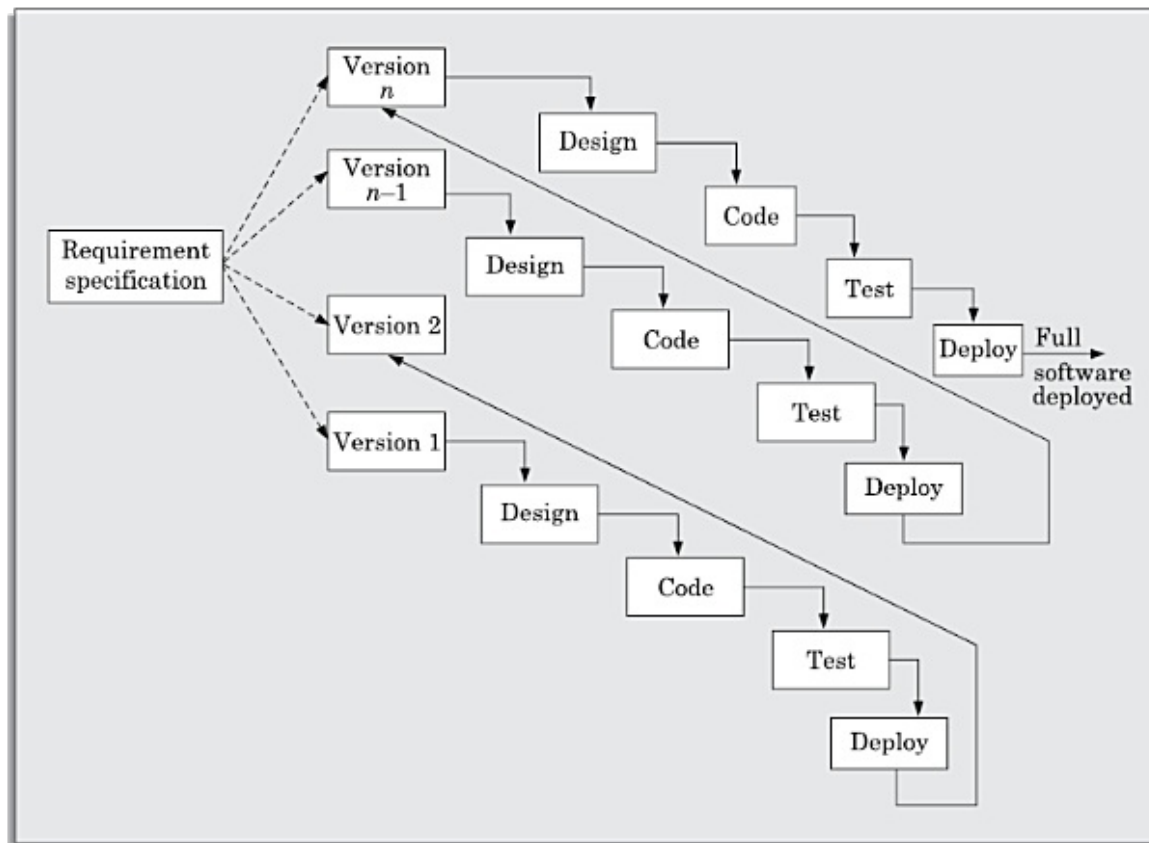


Figure 2.8: Incremental model of software development.

Advantages

The incremental development model offers several advantages. Two important ones are the following:

- **Error reduction:** The core modules are used by the customer from the beginning and therefore these get tested thoroughly. This reduces chances of errors in the core modules of the final product, leading to greater reliability of the software.
- **Incremental resource deployment:** This model obviates the need for the customer to commit large resources at one go for development of the system. It also saves the developing organisation from deploying large resources and manpower for a project in one go.

2.2.6 Evolutionary Model

This model has many of the features of the incremental model. As in case of the incremental model, the software is developed over a number of increments. At each increment, a concept (feature) is implemented and is deployed at the client site. The software is

successively refined and feature-enriched until the full software is realised. The principal idea behind the evolutionary life cycle model is conveyed by its name. In the incremental development model, complete requirements are first developed and the SRS document prepared. In contrast, in the evolutionary model, the requirements, plan, estimates, and solution evolve over the iterations, rather than fully defined and frozen in a major up-front specification effort before the development iterations begin. Such evolution is consistent with the pattern of unpredictable feature discovery and feature changes that take place in new product development.

Though the evolutionary model can also be viewed as an extension of the waterfall model, but it incorporates a major paradigm shift that has been widely adopted in many recent life cycle models. Due to obvious reasons, the evolutionary software development process is sometimes referred to as *design a little, build a little, test a little, deploy a little* model. This means that after the requirements have been specified, the design, build, test, and deployment activities are iterated. A schematic representation of the evolutionary model of development has been shown in Figure 2.9.

Advantages

The evolutionary model of development has several advantages. Two important advantages of using this model are the following:

- **Effective elicitation of actual customer requirements:** In this model, the user gets a chance to experiment with a partially developed software much before the complete requirements are developed. Therefore, the evolutionary model helps to accurately elicit user requirements with the help of feedback obtained on the delivery of different versions of the software. As a result, the change requests after delivery of the complete software gets substantially reduced.
- **Easy handling change requests:** In this model, handling change requests is easier as no long term plans are made. Consequently, reworks required due to change requests are normally much smaller compared to the sequential models.

Disadvantages

The main disadvantages of the successive versions model are as follows:

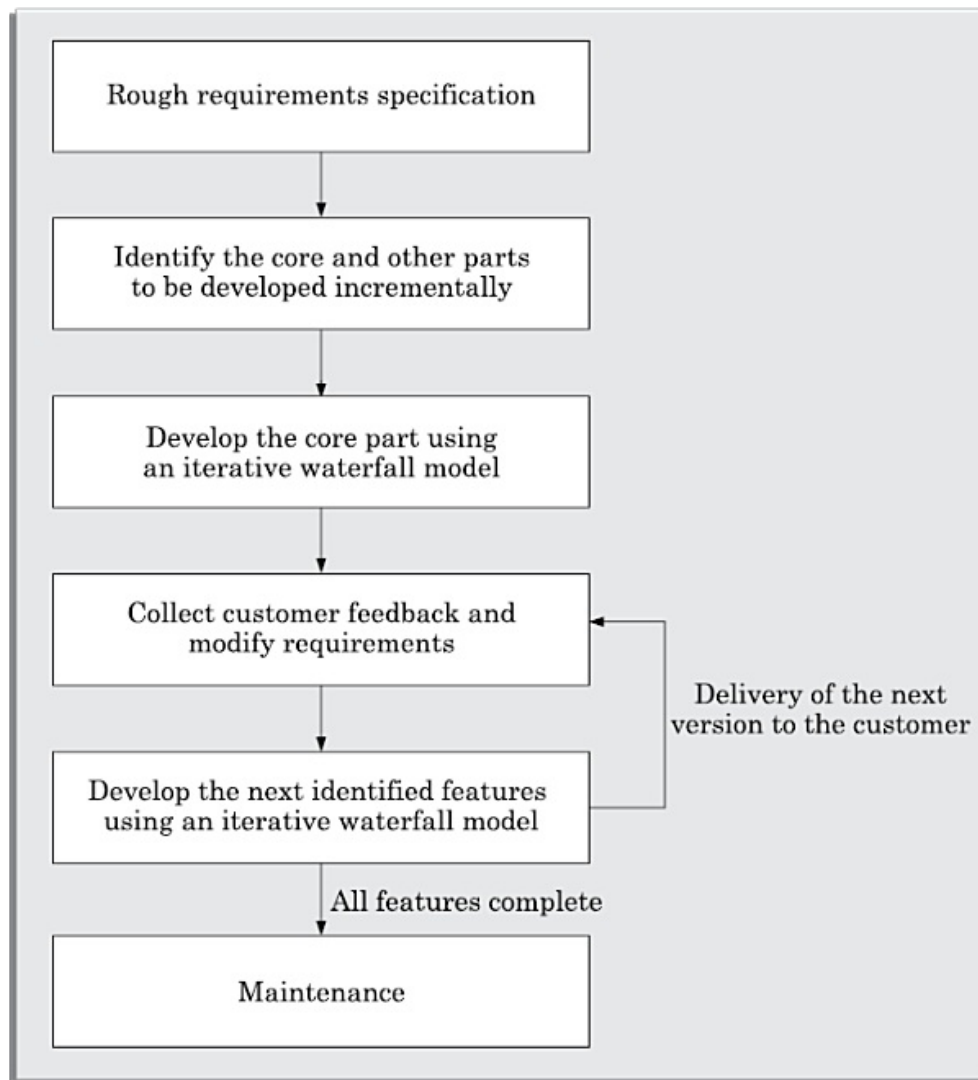


Figure 2.9: Evolutionary model of software development.

- **Feature division into incremental parts can be non-trivial:** For many development projects, especially for small-sized projects, it is difficult to divide the required features into several parts that can be incrementally implemented and delivered. Further, even for larger problems, often the features are so intertwined and dependent on each other that even an expert would need considerable effort to plan the incremental deliveries.
- **Ad hoc design:** Since at a time design for only the current increment is done, the design can become ad hoc without specific attention being paid to maintainability and optimality. Obviously, for moderate sized problems and for those for which the customer requirements are clear, the iterative waterfall model can yield a better solution.

Applicability of the evolutionary model

The evolutionary model is normally useful for very large products, where it is easier to find modules for incremental implementation. Often evolutionary model is used when the customer prefers to receive the product in increments so that he can start using the different features as and when they are delivered rather than waiting all the time for the full product to be developed and delivered. Another important category of projects for which the evolutionary model is suitable, is projects using object-oriented development.

The evolutionary model is well-suited to use in object-oriented software development projects.

Evolutionary model is appropriate for object-oriented development project, since it is easy to partition the software into stand alone units in terms of the classes. Also, classes are more or less self contained units that can be developed independently.

2.3 RAPID APPLICATION DEVELOPMENT (RAD)

The *rapid application development* (RAD) model was proposed in the early nineties in an attempt to overcome the rigidity of the waterfall model (and its derivatives) that makes it difficult to accommodate any change requests from the customer. It proposed a few radical extensions to the waterfall model. This model has the features of both prototyping and evolutionary models. It deploys an evolutionary delivery model to obtain and incorporate the customer feedbacks on incrementally delivered versions.

In this model prototypes are constructed, and incrementally the features are developed and delivered to the customer. But unlike the prototyping model, the prototypes are not thrown away but are enhanced and used in the software construction

The major goals of the RAD model are as follows:

- To decrease the time taken and the cost incurred to develop software systems.
- To limit the costs of accommodating change requests.
- To reduce the communication gap between the customer and the developers.

Main motivation

In the iterative waterfall model, the customer requirements need to be gathered, analysed, documented, and signed off upfront, before any development could start. However, often clients do not know what they exactly wanted until they saw a working system. It has now become well accepted among the practitioners that only through the process commenting on an installed application that the exact requirements can be brought out. But in the iterative waterfall model, the customers do not get to see the software, until the development is complete in all respects and the software has been delivered and installed. Naturally, the delivered software often does not meet the customer expectations and many change request are generated by the customer. The changes are incorporated through subsequent maintenance efforts. This made the cost of accommodating the changes extremely high and it usually took a long time to have a good solution in place that could reasonably meet the requirements of the customers. The RAD model tries to overcome this problem by inviting and incorporating customer feedback on successively developed and refined prototypes.

2.3.1 Working of RAD

In the RAD model, development takes place in a series of short cycles or iterations. At any time, the development team focuses on the present iteration only, and therefore plans are made for one increment at a time. The time planned for each iteration is called a *time box*. Each iteration is planned to enhance the implemented functionality of the application by only a small amount. During each time box, a quick-and-dirty prototype-style software for some functionality is developed. The customer evaluates the prototype and gives feedback on the specific improvements that may be necessary. The prototype is refined based on the customer feedback. Please note that the prototype is not meant to be released to the customer for regular use though.

The development team almost always includes a customer representative to clarify the requirements. This is intended to make the system tuned to the exact customer requirements and also to bridge the communication gap between the customer and the development team. The development team usually consists of about five to six members, including a customer representative.

How does RAD facilitate accommodation of change requests?

The customers usually suggest changes to a specific feature only after they have used it. Since the features are delivered in small increments, the customers are able to give their change requests pertaining to a feature already delivered. Incorporation of such change requests just after the delivery of an incremental feature saves cost as this is carried out before large investments have been made in development and testing of a large number of features.

How does RAD facilitate faster development?

The decrease in development time and cost, and at the same time an increased flexibility to incorporate changes are achieved in the RAD model in two main ways—minimal use of planning and heavy reuse of any existing code through rapid prototyping. The lack of long-term and detailed planning gives the flexibility to accommodate later requirements changes. Reuse of existing code has been adopted as an important mechanism of reducing the development cost.

RAD model emphasises code reuse as an important means for completing a project faster. In fact, the adopters of the RAD model were the earliest to embrace object-oriented languages and practices. Further, RAD advocates use of specialised tools to facilitate fast creation of working prototypes. These specialised tools usually support the following features:

- Visual style of development.
- Use of reusable components.

2.3.2 Applicability of RAD Model

The following are some of the characteristics of an application that indicate its suitability to RAD style of development:

- **Customised software:** As already pointed out a customised software is developed for one or two customers only by adapting an existing software. In customised software development projects, substantial reuse is usually made of code from pre-existing software. For example, a company might have developed a software for automating the data processing activities at one or more educational institutes. When any other institute requests for an automation package to be developed, typically only a few aspects need to be tailored—since among different educational

institutes, most of the data processing activities such as student registration, grading, fee collection, estate management, accounting, maintenance of staff service records etc. are similar to a large extent. Projects involving such tailoring can be carried out speedily and cost-effectively using the RAD model.

- **Non-critical software:** The RAD model suggests that a quick and dirty software should first be developed and later this should be refined into the final software for delivery. Therefore, the developed product is usually far from being optimal in performance and reliability. In this regard, for well understood development projects and where the scope of reuse is rather restricted, the Iterative waterfall model may provide a better solution.
- **Highly constrained project schedule:** RAD aims to reduce development time at the expense of good documentation, performance, and reliability. Naturally, for projects with very aggressive time schedules, RAD model should be preferred.
- **Large software:** Only for software supporting many features (large software) can incremental development and delivery be meaningfully carried out.

Application characteristics that render RAD unsuitable

The RAD style of development is not advisable if a development project has one or more of the following characteristics:

- **Generic products (wide distribution):** As we have already pointed out in Chapter 1, software products are generic in nature and usually have wide distribution. For such systems, optimal performance and reliability are imperative in a competitive market. As it has already been discussed, the RAD model of development may not yield systems having optimal performance and reliability.
- **Requirement of optimal performance and/or reliability:** For certain categories of products, optimal performance or reliability is required. Examples of such systems include an operating system (high reliability required) and a flight simulator software (high performance required). If such systems are to be developed using the RAD model, the desired product performance and reliability may not be realised.
- **Lack of similar products:** If a company has not developed similar

software, then it would hardly be able to reuse much of the existing artifacts. In the absence of sufficient plug-in components, it becomes difficult to develop rapid prototypes through reuse, and use of RAD model becomes meaningless.

- **Monolithic entity:** For certain software, especially small-sized software, it may be hard to divide the required features into parts that can be incrementally developed and delivered. In this case, it becomes difficult to develop a software incrementally.

2.3.3 Comparison of RAD with Other Models

In this section, we compare the relative advantages and disadvantages of RAD with other life cycle models.

RAD versus prototyping model

In the prototyping model, the developed prototype is primarily used by the development team to gain insights into the problem, choose between alternatives, and elicit customer feedback. The code developed during prototype construction is usually thrown away. In contrast, in RAD it is the developed prototype that evolves into the deliverable software.

Though RAD is expected to lead to faster software development compared to the traditional models (such as the prototyping model), though the quality and reliability would be inferior.

RAD versus iterative waterfall model

In the iterative waterfall model, all the functionalities of a software are developed together. On the other hand, in the RAD model the product functionalities are developed incrementally through heavy code and design reuse. Further, in the RAD model customer feedback is obtained on the developed prototype after each iteration and based on this the prototype is refined. Thus, it becomes easy to accommodate any request for requirements changes. However, the iterative waterfall model does not support any mechanism to accommodate any requirement change requests. The iterative waterfall model does have some important advantages that include the following. Use of the iterative waterfall model leads to production of good quality documentation which can help during software maintenance. Also, the developed software usually has better quality and reliability than that

developed using RAD.

RAD *versus* evolutionary model

Incremental development is the hallmark of both evolutionary and RAD models. However, in RAD each increment results in essentially a quick and dirty prototype, whereas in the evolutionary model each increment is systematically developed using the iterative waterfall model. Also in the RAD model, software is developed in much shorter increments compared the evolutionary model. In other words, the incremental functionalities that are developed are of fairly larger granularity in the evolutionary model.

2.4 AGILE DEVELOPMENT MODELS

As already pointed out, though the iterative waterfall model has been very popular during the 1970s and 1980s, developers face several problems while using it on present day software projects. The main difficulties included handling change requests from customers during product development, and the unreasonably high cost and time that is incurred while developing customised applications. Capers Jones carried out research involving 800 real-life software development projects, and concluded that on the average 40 per cent of the requirements is arrived after the development has already begun. In this context, over the last two decade or so, several life cycle models have been proposed to overcome the important shortcomings of the waterfall-based models that become conspicuous when used in modern software development projects.

Over the last two decades or so, projects using iterative waterfall-based life cycle models are becoming rare due to the rapid shift in the characteristics of the software development projects over time. Two changes that are becoming noticeable are rapid shift from development of software products to development of customised software and the increased emphasis and scope for reuse.

In the following, a few reasons why the waterfall-based development was becoming difficult to use in project in recent times:

- In the traditional iterative waterfall-based software development models, the requirements for the system are determined at the start of a development project and are assumed to be fixed from that point on. Later changes to the requirements after the SRS document has been

completed are discouraged. If at all any later requirement changes becomes unavoidable, then the cost of accommodating it becomes prohibitively high. On the other hand, accumulated experience indicates that customers frequently change their requirements during the development period due to a variety of reasons.

- As pointed out in Chapter 1, over the last two decades or so, customised applications (services) has become common place and the sales revenue generated world wide from services already exceeds that of the software products. Clearly, iterative waterfall model is not suitable for development of such software. Since customization essentially involves reusing most of the parts of an existing application and consists of only carrying out minor modifications by writing minimal amounts of code. For such development projects, the need for more appropriate development models was deeply felt, and many researchers started to investigate in this direction.
- Waterfall model is called a *heavy weight* model, since there is too much emphasis on producing documentation and usage of tools. This is often a source of inefficiency and causes the project completion time to be much longer in comparison to the customer expectations.
- Waterfall model prescribes almost no customer interactions after the requirements have been specified. In fact, in the waterfall model of software development, customer interactions are largely confined to the project initiation and project completion stages.

The agile software development model was proposed in the mid-1990s to overcome the serious shortcomings of the waterfall model of development identified above. The agile model was primarily designed to help a project to adapt to change requests quickly.¹ Thus, a major aim of the agile models is to facilitate quick project completion. But, how is agility achieved in these models? Agility is achieved by fitting the process to the project, i.e. removing activities that may not be necessary for a specific project. Also, anything that wastes time and effort is avoided.

Please note that agile model is being used as an umbrella term to refer to a group of development processes. These processes share certain common characteristics, but do have certain subtle differences among themselves. A few popular agile SDLC models are the following:

- Crystal

- Atern (formerly DSDM)
- Feature-driven development
- Scrum
- Extreme programming (XP)
- Lean development
- Unified process

In the agile model, the requirements are decomposed into many small parts that can be incrementally developed. The agile model adopts an iterative approach. Each incremental part is developed over an iteration. Each iteration is intended to be small and easily manageable and lasting for a couple of weeks only. At a time, only one increment is planned, developed, and then deployed at the customer site. No long-term plans are made. The time to complete an iteration is called a *time box*. The implication of the term *time box* is that the end date for an iteration does not change. That is, the delivery date is considered sacrosanct. The development team can, however, decide to reduce the delivered functionality during a time box if necessary.

A central principle of the agile model is the delivery of an increment to the customer after each time box. A few other principles that are central to the agile model are discussed below.

2.4.1 Essential Idea behind Agile Models

For establishing close contact with the customer during development and to gain a clear understanding of the domain-specific issues, each agile project usually includes a customer representative in the team. At the end of each iteration, stakeholders and the customer representative review the progress made and re-evaluate the requirements. A distinguishing characteristics of the agile models is frequent delivery of software increments to the customer.

Agile model emphasise face-to-face communication over written documents. It is recommended that the development team size be deliberately kept small (5–9 people) to help the team members meaningfully engage in face-to-face communication and have collaborative work environment. It is implicit then that the agile model is suited to the development of small projects. However, if a large project is required to be developed using the agile model, it is likely that the collaborating teams might work at different locations. In this case, the different teams are needed to maintain as much daily contact as possible through video conferencing,

telephone, e-mail, etc.

The agile model emphasises incremental release of working software as the primary measure of progress.

The following important principles behind the agile model were publicised in the agile manifesto in 2001:

- Working software over comprehensive documentation.
- Frequent delivery of incremental versions of the software to the customer in intervals of few weeks.
- Requirement change requests from the customer are encouraged and are efficiently incorporated.
- Having competent team members and enhancing interactions among them is considered much more important than issues such as usage of sophisticated tools or strict adherence to a documented process. It is advocated that enhanced communication among the development team members can be realised through face-to-face communication rather than through exchange of formal documents.
- Continuous interaction with the customer is considered much more important rather than effective contract negotiation. A customer representatives is required to be a part of the development team, thus facilitating close, daily co-operation between customers and developers.

Agile development projects usually deploy pair programming.

In pair programming, two programmers work together at one work station. One types in code while the other reviews the code as it is typed in. The two programmers switch their roles every hour or so.

Several studies indicate that programmers working in pairs produce compact well-written programs and commit fewer errors as compared to programmers working alone.

Advantages and disadvantages of agile methods

The agile methods derive much of their agility by relying on the tacit knowledge of the team members about the development project and informal communications to clarify issues, rather than spending significant amounts of time in preparing formal documents and

reviewing them. Though this eliminates some overhead, but lack of adequate documentation may lead to several types of problems, which are as follows:

- Lack of formal documents leaves scope for confusion and important decisions taken during different phases can be misinterpreted at later points of time by different team members.
- In the absence of any formal documents, it becomes difficult to get important project decisions such as design decisions to be reviewed by external experts.
- When the project completes and the developers disperse, maintenance can become a problem.

2.4.2 Agile *versus* Other Models

In the following subsections, we compare the characteristics of the agile model with other models of development.

Agile model *versus* iterative waterfall model

The waterfall model is highly structured and systematically steps through requirements-capture, analysis, specification, design, coding, and testing stages in a planned sequence. Progress is generally measured in terms of the number of completed and reviewed artifacts such as requirement specifications, design documents, test plans, code reviews, etc. for which review is complete. In contrast, while using an agile model, progress is measured in terms of the developed and delivered functionalities. In agile model, delivery of working versions of a software is made in several increments. However, as regards to similarity it can be said that agile teams use the waterfall model on a small scale, repeating the entire waterfall cycle in every iteration.

If a project being developed using waterfall model is cancelled mid-way during development, then there is nothing to show from the abandoned project beyond several documents. With agile model, even if a project is cancelled midway, it still leaves the customer with some worthwhile code, that might possibly have already been put into live operation.

Agile *versus* exploratory programming

Though a few similarities do exist between the agile and exploratory programming styles, there are vast differences between the two as

well. Agile development model's frequent re-evaluation of plans, emphasis on face-to-face communication, and relatively sparse use of documentation are similar to that of the exploratory style. Agile teams, however, do follow defined and disciplined processes and carry out systematic requirements capture, rigorous designs, compared to chaotic coding in exploratory programming.

Agile model *versus* RAD model

The important differences between the agile and the RAD models are the following:

- Agile model does not recommend developing prototypes, but emphasises systematic development of each incremental feature. In contrast, the central theme of RAD is based on designing quick-and-dirty prototypes, which are then refined into production quality code.
- Agile projects logically break down the solution into features that are incrementally developed and delivered. The RAD approach does not recommend this. Instead, developers using the RAD model focus on developing all the features of an application by first doing it badly and then successively improving the code over time.
- Agile teams only demonstrate completed work to the customer. In contrast, RAD teams demonstrate to customers screen mock ups, and prototypes, that may be based on simplifications such as table look-ups rather than actual computations.

2.4.3 Extreme Programming Model

Extreme programming (XP) is an important process model under the agile umbrella and was proposed by Kent Beck in 1999. The name of this model reflects the fact that it recommends taking these *best practices* that have worked well in the past in program development projects to extreme levels. This model is based on a rather simple philosophy: "If something is known to be beneficial, why not put it to constant use?" Based on this principle, it puts forward several key practices that need to be practised to the extreme. Please note that most of the key practices that it emphasises were already recognised as good practices for some time.

Good practices that need to be practised to the extreme

In the following subsections, we mention some of the good practices that have been recognised in the extreme programming model and the suggested way to maximise their use:

Code review: It is good since it helps detect and correct problems most efficiently. It suggests *pair programming* as the way to achieve continuous review. In pair programming, coding is carried out by pairs of programmers. The programmers take turn in writing programs and while one writes the other reviews code that is being written.

Testing: Testing code helps to remove bugs and improves its reliability. XP suggests *test-driven development* (TDD) to continually write and execute test cases. In the TDD approach, test cases are written even before any code is written.

Incremental development: Incremental development is good, since it helps to get customer feedback, and extent of features delivered is a reliable indicator of progress. It suggests that the team should come up with new increments every few days.

Simplicity: Simplicity makes it easier to develop good quality code, as well as to test and debug it. Therefore, one should try to create the simplest code that makes the basic functionality being written to work. For creating the simplest code, one can ignore the aspects such as efficiency, reliability, maintainability, etc. Once the simplest thing works, other aspects can be introduced through refactoring.

Design: Since having a good quality design is important to develop a good quality solution, everybody should design daily. This can be achieved through *refactoring*, whereby a working code is improved for efficiency and maintainability.

Integration testing: It is important since it helps identify the bugs at the interfaces of different functionalities. To this end, extreme programming suggests that the developers should achieve continuous integration, by building and performing integration testing several times a day.

Basic idea of extreme programming model

XP is based on frequent releases (called *iteration*), during which the developers implement “user stories”. User stories are similar to use cases, but are more informal and are simpler. A user story is the conversational description by the user about a feature of the required

system. For example, a user story about a library software can be:

- A library member can issue a book.
- A library member can query about the availability of a book.
- A library member should be able to return a borrowed book.

A user story is a simplistic statement of a customer about a functionality he needs, it does not mention about finer details such as the different scenarios that can occur, the precondition (state at which the system) to be satisfied before the feature can be invoked, etc.

On the basis of user stories, the project team proposes “metaphors”—a common vision of how the system would work. The development team may decide to construct a *spike* for some feature. A *spike*, is a very simple program that is constructed to explore the suitability of a solution being proposed. A spike can be considered to be similar to a prototype.

XP prescribes several basic activities to be part of the software development process. We discuss these activities in the following subsections:

Coding: XP argues that code is the crucial part of any system development process, since without code it is not possible to have a working system. Therefore, utmost care and attention need to be placed on coding activity. However, the concept of code as used in XP has a slightly different meaning from what is traditionally understood. For example, coding activity includes drawing diagrams (modelling) that will be transformed to code, scripting a web-based system, and choosing among several alternative solutions.

Testing: XP places high importance on testing and considers it be the primary means for developing a fault-free software.

Listening: The developers need to carefully listen to the customers if they have to develop a good quality software. Programmers may not necessarily be having an in-depth knowledge of the the specific domain of the system under development. On the other hand, customers usually have this domain knowledge. Therefore, for the programmers to properly understand what the functionality of the system should be, they have to listen to the customer.

Designing: Without proper design, a system implementation becomes too complex and the dependencies within the system become too numerous and it becomes very difficult to comprehend the solution, and thereby making maintenance prohibitively expensive. A good design should result in

elimination of complex dependencies within a system. Thus, effective use of a suitable design technique is emphasised.

Feedback: It espouses the wisdom: "A system staying out of users is trouble waiting to happen". It recognises the importance of user feedback in understanding the exact customer requirements. The time that elapses between the development of a version and collection of feedback on it is critical to learning and making changes. It argues that frequent contact with the customer makes the development effective.

Simplicity: A corner-stone of XP is based on the principle: "build something simple that will work today, rather than trying to build something that would take time and yet may never be used". This in essence means that attention should be focused on specific features that are immediately needed and making them work, rather than devoting time and energy on speculations about future requirements.

XP is in favour of making the solution to a problem as simple as possible. In contrast, the traditional system development methods recommend planning for reusability and future extensibility of code and design at the expense of higher code and design complexity.

Applicability of extreme programming model

The following are some of the project characteristics that indicate the suitability of a project for development using extreme programming model:

Projects involving new technology or research projects: In this case, the requirements change rapidly and unforeseen technical problems need to be resolved.

Small projects: Extreme programming was proposed in the context of small teams as face to face meeting is easier to achieve.

Project characteristics not suited to development using agile models

The following are some of the project characteristics that indicate unsuitability of agile development model for use in a development project:

- **Stable requirements:** Conventional development models are more suited to use in projects characterised by stable requirements. For such

projects, it is known that few changes, if at all, will occur. Therefore, process models such as iterative waterfall model that involve making long-term plans during project initiation can meaningfully be used.

- **Mission critical or safety critical systems:** In the development of such systems, the traditional SDLC models are usually preferred to ensure reliability.

2.4.4 Scrum Model

In the scrum model, a project is divided into small parts of work that can be incrementally developed and delivered over time boxes that are called *sprints*. The software therefore gets developed over a series of manageable chunks. Each sprint typically takes only a couple of weeks to complete. At the end of each sprint, stakeholders and team members meet to assess the progress made and the stakeholders suggest to the development team any changes needed to features that have already been developed and any overall improvements that they might feel necessary.

In the scrum model, the team members assume three fundamental roles—software owner, scrum master, and team member. The software owner is responsible for communicating the customers vision of the software to the development team. The scrum master acts as a liaison between the software owner and the team, thereby facilitating the development work.

2.5 SPIRAL MODEL

This model gets its name from the appearance of its diagrammatic representation that looks like a spiral with many loops (see Figure 2.10). The exact number of loops of the spiral is not fixed and can vary from project to project. The number of loops shown in Figure 2.10 is just an example. Each loop of the spiral is called a *phase* of the software process. The exact number of phases through which the product is developed can be varied by the project manager depending upon the project risks. A prominent feature of the spiral model is handling unforeseen risks that can show up much after the project has started. In this context, please recollect that the prototyping model can be used effectively only when the risks in a project can be identified upfront before the development work starts. As we shall discuss, this model achieves this by incorporating much more flexibility compared to SDLC

other models.

While the prototyping model does provide explicit support for risk handling, the risks are assumed to have been identified completely before the project start. This is required since the prototype is constructed only at the start of the project. In contrast, in the spiral model prototypes are built at the start of every phase. Each phase of the model is represented as a loop in its diagrammatic representation. Over each loop, one or more features of the product are elaborated and analysed and the risks at that point of time are identified and are resolved through prototyping. Based on this, the identified features are implemented.

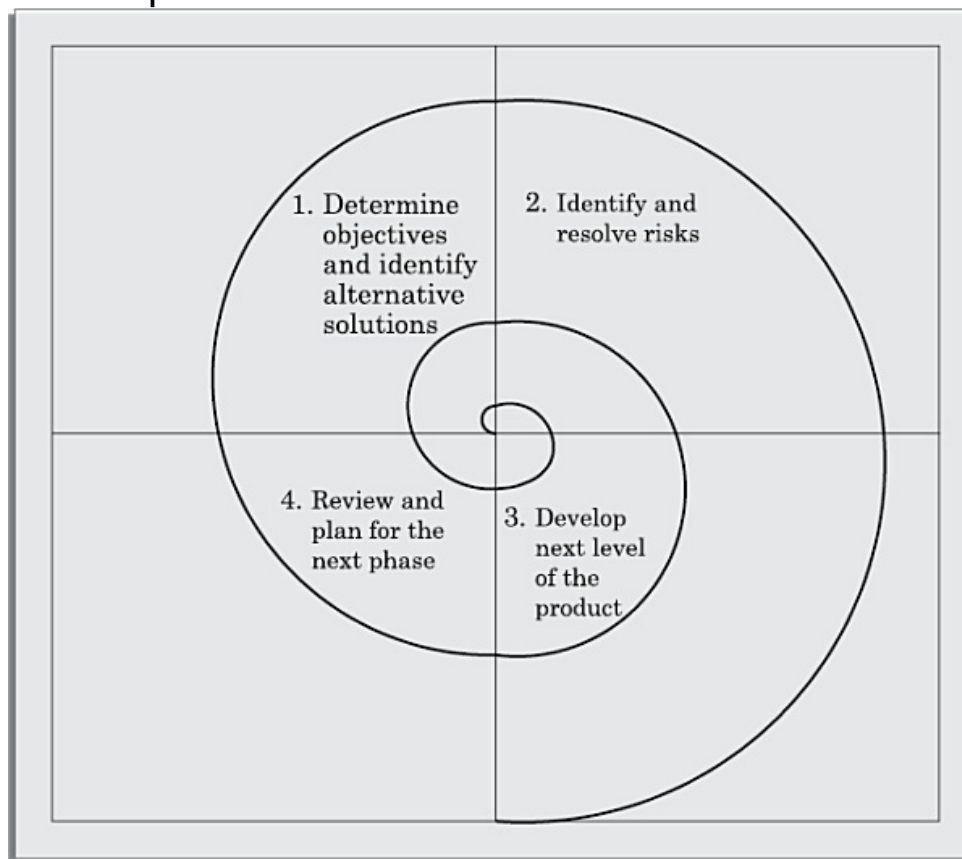


Figure 2.10: Spiral model of software development.

Risk handling in spiral model

A risk is essentially any adverse circumstance that might hamper the successful completion of a software project. As an example, consider a project for which a risk can be that data access from a remote database might be too slow to be acceptable by the customer. This risk can be resolved by building a prototype of the data access subsystem and experimenting with the exact access rate. If the data access rate is too slow, possibly a caching scheme can be implemented or a faster

communication scheme can be deployed to overcome the slow data access rate. Such risk resolutions are easier done by using a prototype as the pros and cons of an alternate solution scheme can be evaluated faster and inexpensively, as compared to experimenting using the actual software application being developed. The spiral model supports coping up with risks by providing the scope to build a prototype at every phase of software development.

2.5.1 Phases of the Spiral Model

Each phase in this model is split into four sectors (or quadrants) as shown in Figure 2.10. In the first quadrant, a few features of the software are identified to be taken up for immediate development based on how crucial it is to the overall software development. With each iteration around the spiral (beginning at the center and moving outwards), progressively more complete versions of the software get built. In other words, implementation of the identified features forms a phase.

Quadrant 1: The objectives are investigated, elaborated, and analysed. Based on this, the risks involved in meeting the phase objectives are identified. In this quadrant, alternative solutions possible for the phase under consideration are proposed.

Quadrant 2: During the second quadrant, the alternative solutions are evaluated to select the best possible solution. To be able to do this, the solutions are evaluated by developing an appropriate prototype.

Quadrant 3: Activities during the third quadrant consist of developing and verifying the next level of the software. At the end of the third quadrant, the identified features have been implemented and the next version of the software is available.

Quadrant 4: Activities during the fourth quadrant concern reviewing the results of the stages traversed so far (i.e. the developed version of the software) with the customer and planning the next iteration of the spiral.

The radius of the spiral at any point represents the cost incurred in the project so far, and the angular dimension represents the progress made so far in the current phase.

In the spiral model of development, the project manager dynamically determines the number of phases as the project progresses. Therefore, in this model, the project manager plays the crucial role of tuning the model to

specific projects.

To make the model more efficient, the different features of the software that can be developed simultaneously through parallel cycles are identified. To keep our discussion simple, we shall not delve into parallel cycles in the spiral model.

Advantages/pros and disadvantages/cons of the spiral model

There are a few disadvantages of the spiral model that restrict its use to a only a few types of projects. To the developers of a project, the spiral model usually appears as a complex model to follow, since it is risk-driven and is more complicated phase structure than the other models we discussed. It would therefore be counterproductive to use, unless there are knowledgeable and experienced staff in the project. Also, it is not very suitable for use in the development of outsourced projects, since the software risks need to be continually assessed as it is developed.

In spite of the disadvantages of the spiral model that we pointed out, for certain categories of projects, the advantages of the spiral model can outweigh its disadvantages.

For projects having many unknown risks that might show up as the development proceeds, the spiral model would be the most appropriate development model to follow.

In this regard, it is much more powerful than the prototyping model. Prototyping model can meaningfully be used when all the risks associated with a project are known beforehand. All these risks are resolved by building a prototype before the actual software development starts.

Spiral model as a meta model

As compared to the previously discussed models, the spiral model can be viewed as a *meta model*, since it subsumes all the discussed models. For example, a single loop spiral actually represents the waterfall model. The spiral model uses the approach of the prototyping model by first building a prototype in each phase before the actual development starts. This prototypes are used as a risk reduction mechanism. The spiral model incorporates the systematic step- wise approach of the waterfall model. Also, the spiral model can be considered as supporting the evolutionary model—the

iterations along the spiral can be considered as evolutionary levels through which the complete system is built. This enables the developer to understand and resolve the risks at each evolutionary level (i.e. iteration along the spiral).

Case study 2.2

Galaxy Inc. undertook the development of a satellite-based communication between mobile handsets that can be anywhere on the earth. In contrast to the traditional cell phones, by using a satellite-based mobile phone a call can be established as long as both the source and destination phones are in the coverage areas of some base stations. The system would function through about six dozens of satellites orbiting the earth. The satellites would directly pick up the signals from a handset and beam signal to the destination handset. Since the foot prints of the revolving satellites would cover the entire earth, communication between any two points on the earth, even between remote places such as those in the Arctic ocean and Antarctica, would also be possible. However, the risks in the project are many, including determining how the calls among the satellites can be handed-off when they are themselves revolving at a very high speed. In the absence of any published material and availability of staff with experience in development of similar products, many of the risks cannot be identified at the start of the project and are likely to crop up as the project progresses. The software would require several million lines of code to be written. Galaxy Inc. decided to deploy the spiral model for software development after hiring highly qualified staff. To speed up the software development, independent parts of the software were developed through parallel cycles on the spiral. The cost and delivery schedule were refined many times, as the project progressed. The project was successfully completed after five years from start date

2.6 A COMPARISON OF DIFFERENT LIFE CYCLE MODELS

The classical waterfall model can be considered as the basic model and all other life cycle models as embellishments of this model. However, the classical waterfall model cannot be used in practical development projects, since this model supports no mechanism to correct the errors that are committed during any of the phases but detected at a later phase. This problem is overcome by the iterative waterfall model through the provision of feedback paths.

The iterative waterfall model is probably the most widely used software development model so far. This model is simple to understand and use. However, this model is suitable only for well-understood problems, and is not suitable for development of very large projects and projects that suffer from large number of risks.

The prototyping model is suitable for projects for which either the user requirements or the underlying technical aspects are not well understood, however all the risks can be identified before the project starts. This model is especially popular for development of the user interface part of projects.

The evolutionary approach is suitable for large problems which can be decomposed into a set of modules for incremental development and delivery. This model is also used widely for object-oriented development projects. Of course, this model can only be used if incremental delivery of the system is acceptable to the customer.

The spiral model is considered a *meta model* and encompasses all other life cycle models. Flexibility and risk handling are inherently built into this model. The spiral model is suitable for development of technically challenging and large software that are prone to several kinds of risks that are difficult to anticipate at the start of the project. However, this model is much more complex than the other models—this is probably a factor deterring its use in ordinary projects.

Let us now compare the prototyping model with the spiral model. The prototyping model can be used if the risks are few and can be determined at the start of the project. The spiral model, on the other hand, is useful when the risks are difficult to anticipate at the beginning of the project, but are likely to crop up as the development proceeds.

Let us compare the different life cycle models from the viewpoint of the customer. Initially, customer confidence is usually high on the development team irrespective of the development model followed. During the lengthy development process, customer confidence normally drops off, as no working software is yet visible. Developers answer customer queries using technical slang, and delays are announced. This gives rise to customer resentment. On the other hand, an evolutionary approach lets the customer experiment with a working software much earlier than the monolithic approaches. Another important advantage of the incremental model is that it reduces the customer's trauma of getting used to an entirely new system. The gradual introduction of the software via incremental phases provides time to the customer to adjust to the new software. Also, from the customer's financial view point, incremental development does not require a large upfront capital outlay. The customer can order the incremental versions as and when he can afford them.

2.6.1 Selecting an Appropriate Life Cycle Model for a Project

We have discussed the advantages and disadvantages of the various life cycle models. However, how to select a suitable life cycle model for a specific project? The answer to this question would depend on several factors. A suitable life cycle model can possibly be selected based on an analysis of issues such as the following:

Characteristics of the software to be developed: The choice of the life cycle model to a large extent depends on the nature of the software that is being developed. For small services projects, the agile model is favoured. On the other hand, for product and embedded software development, the iterative waterfall model can be preferred. An evolutionary model is a suitable model for object-oriented development projects.

Characteristics of the development team: The skill-level of the team members is a significant factor in deciding about the life cycle model to use. If the development team is experienced in developing similar software, then even an embedded software can be developed using an iterative waterfall model. If the development team is entirely novice, then even a simple data processing application may require a prototyping model to be adopted.

Characteristics of the customer: If the customer is not quite familiar with computers, then the requirements are likely to change frequently as it would be difficult to form complete, consistent, and unambiguous requirements. Thus, a prototyping model may be necessary to reduce later change requests from the customers.

SUMMARY

- During the development of any type of software, adherence to a suitable process model has become universally accepted by software development organisations. Adoption of a suitable life cycle model is now accepted as a primary necessity for successful completion of projects.
- We discussed only the central ideas behind some important process models. Good software development organisations carefully and elaborately document the precise process model they follow and typically include the following in the document:
 - Identification of the different phases.
 - Identification of the different activities in each phase and the order in which they are carried out.

- The phase entry and exit criteria for different phases.
- The methodology followed to carry out the different activities.
- Adherence to a software life cycle model encourages the team members to perform various development activities in a systematic and disciplined manner. It also makes management of software development projects easier.
- The principle of detecting errors as close to their point of introduction as possible is known as phase containment of errors. Phase containment minimises the cost to fix errors.
- The classical waterfall model can be considered as the basic model and all other life cycle models are embellishments of this model. Iterative waterfall model has been the most widely used life cycle model so far, though the usage of RAD and agile models have been increasing.
- Different life cycle models have their own advantages and disadvantages. Therefore, an appropriate life cycle model should be chosen for the problem at hand. After choosing a basic life cycle model, software development organisations usually tailor the standard life cycle models according to their needs.
- Even though an organisation may follow whichever life cycle model is appropriate to a project, the final document should reflect as if the software was developed using the classical waterfall model. This makes it easier for the maintainers to understand the software documents.

EXERCISES

1. Choose the correct option for each of the following questions:
 - (a) Which one of the following disadvantages may be experienced when a systematic development process model is adopted in preference over a build-and-fix style of development?
 - (i) Increased documentation overhead
 - (ii) Increased development cost
 - (iii) Decreased maintainability
 - (iv) Increased development time
 - (b) A software process model represents which one of the following:
 - (i) The way in which software is developed
 - (ii) The way in which software processes data
 - (iii) The way in which software is used
 - (iv) The way in which software may fail

Chapter

4

REQUIREMENTS ANALYSIS AND SPECIFICATION

All plan-driven life cycle models prescribe that before starting to develop a software, the exact requirements of the customer must be understood and documented. In the past, many projects have suffered because the developers started to implement something without determining whether they were building what the customers exactly wanted. Starting development work without properly understanding and documenting the requirements increases the number of iterative changes in the later life cycle phases, and thereby alarmingly pushes up the development costs. This also sets the ground for customer dissatisfaction and bitter customer-developer disputes and protracted legal battles. No wonder that experienced developers consider the requirements analysis and specification to be a very important phase of software development life cycle and undertake it with utmost care.

Experienced developers take considerable time to understand the exact requirements of the customer and to meticulously document those. They know that without a clear understanding of the problem and proper documentation of the same, it is impossible to develop a satisfactory solution.

For any type of software development project, availability of a good quality requirements document has been acknowledged to be a key factor in the successful completion of the project. A good requirements document not only helps to form a clear understanding of various features required from the software, but also serves as the basis for various activities carried out during later life cycle phases. When software is developed in a contract mode for some other organisation (that is, an outsourced project), the crucial role played by documentation of the precise requirements cannot be overstated. Even when an organisation develops a generic software product, the situation is not very different since some personnel from the organisation's own

marketing department act as the customer. Therefore, for all types of software development projects, proper formulation of requirements and their effective documentation is vital. However, for very small software service projects, the agile methods advocate incremental development of the requirements.

An overview of requirements analysis and specification phase

The requirements analysis and specification phase starts after the feasibility study stage is complete and the project has been found to be financially viable and technically feasible.

The requirements analysis and specification phase ends when the requirements specification document has been developed and reviewed. The requirements specification document is usually called as the *software requirements specification* (SRS) document. The goal of the requirements analysis and specification phase can be stated in a nutshell as follows.

The goal of the requirements analysis and specification phase is to clearly understand the customer requirements and to systematically organise the requirements into a document called the Software Requirements Specification (SRS) document.

Who carries out requirements analysis and specification?

Requirements analysis and specification activity is usually carried out by a few experienced members of the development team and it normally requires them to spend some time at the customer site. The engineers who gather and analyse customer requirements and then write the requirements specification document are known as *system analysts* in the software industry parlance. System analysts collect data pertaining to the product to be developed and analyse the collected data to conceptualise what exactly needs to be done. After understanding the precise user requirements, the analysts analyse the requirements to weed out inconsistencies, anomalies and incompleteness. They then proceed to write the *software requirements specification* (SRS) document.

The SRS document is the final outcome of the requirements analysis and specification phase.

How is the SRS document validated?

Once the SRS document is ready, it is first reviewed internally by the

project team to ensure that it accurately captures all the user requirements, and that it is understandable, consistent, unambiguous, and complete. The SRS document is then given to the customer for review. After the customer has reviewed the SRS document and agrees to it, it forms the basis for all future development activities and also serves as a contract document between the customer and the development organisation.

What are the main activities carried out during requirements analysis and specification phase?

Requirements analysis and specification phase mainly involves carrying out the following two important activities:

- Requirements gathering and analysis
- Requirements specification

In the next section, we will discuss the requirements gathering and analysis activity and in the subsequent section we will discuss the requirements specification activity.

4.1 REQUIREMENTS GATHERING AND ANALYSIS

The complete set of requirements are almost never available in the form of a single document from the customer. In fact, it would be unrealistic to expect the customers to produce a comprehensive document containing a precise description of what he wants. Further, the complete requirements are rarely obtainable from any single customer representative. Therefore, the requirements have to be gathered by the analyst from several sources in bits and pieces. These gathered requirements need to be analysed to remove several types of problems that frequently occur in the requirements that have been gathered piecemeal from different sources.

We can conceptually divide the requirements gathering and analysis activity into two separate tasks:

- Requirements gathering
- Requirements analysis

We discuss these two tasks in the following subsections.

4.1.1 Requirements Gathering

Requirements gathering is also popularly known as *requirements elicitation*.

The primary objective of the requirements gathering task is to collect the requirements from the *stakeholders*.

A stakeholder is a source of the requirements and is usually a person, or a group of persons who either directly or indirectly are concerned with the software.

Requirements gathering may sound like a simple task. However, in practice it is very difficult to gather all the necessary information from a large number of stakeholders and from information scattered across several pieces of documents. Gathering requirements turns out to be especially challenging if there is no working model of the software being developed.

Suppose a customer wants to automate some activity in his organisation that is currently being carried out manually. In this case, a working model of the system (that is, the manual system) exists. Availability of a working model is usually of great help in requirements gathering. For example, if the project involves automating the existing accounting activities of an organisation, then the task of the system analyst becomes a lot easier as he can immediately obtain the input and output forms and the details of the operational procedures. In this context, consider that it is required to develop a software to automate the book-keeping activities involved in the operation of a certain office. In this case, the analyst would have to study the input and output forms and then understand how the outputs are produced from the input data. However, if a project involves developing something new for which no working model exists, then the requirements gathering activity becomes all the more difficult. In the absence of a working system, much more imagination and creativity is required on the part of the system analyst.

Typically even before visiting the customer site, requirements gathering activity is started by studying the existing documents to collect all possible information about the system to be developed. During visit to the customer site, the analysts normally interview the end-users and customer representatives, ¹carry out requirements gathering activities such as questionnaire surveys, task analysis, scenario analysis, and form analysis.

Given that many customers are not computer savvy, they describe their requirements very vaguely. Good analysts share their experience and expertise with the customer and give his suggestions to define certain functionalities more comprehensively, make the functionalities more general and more complete. In the following, we briefly discuss the important ways in which an experienced analyst gathers requirements:

1. Studying existing documentation: The analyst usually studies all the

available documents regarding the system to be developed before visiting the customer site. Customers usually provide statement of purpose (SoP) document to the developers. Typically these documents might discuss issues such as the context in which the software is required, the basic purpose, the stakeholders, features of any similar software developed elsewhere, etc.

2. Interview: Typically, there are many different categories of users of a software. Each category of users typically requires a different set of features from the software. Therefore, it is important for the analyst to first identify the different categories of users and then determine the requirements of each. For example, the different categories of users of a library automation software could be the library members, the librarians, and the accountants. The library members would like to use the software to query availability of books and issue and return books. The librarians might like to use the software to determine books that are overdue, create member accounts, delete member accounts, etc. The accounts personnel might use the software to invoke functionalities concerning financial aspects such as the total fee collected from the members, book procurement expenditures, staff salary expenditures, etc.

To systematise this method of requirements gathering, the Delphi technique can be followed. In this technique, the analyst consolidates the requirements as understood by him in a document and then circulates it for the comments of the various categories of users. Based on their feedback, he refines his document. This procedure is repeated till the different users agree on the set of requirements.

3. Task analysis: The users usually have a black-box view of a software and consider the software as something that provides a set of services (functionalities). A service supported by a software is also called a *task*. We can therefore say that the software performs various tasks of the users. In this context, the analyst tries to identify and understand the different tasks to be performed by the software. For each identified task, the analyst tries to formulate the different steps necessary to realise the required functionality in consultation with the users. For example, for the issue book service, the steps may be—authenticate user, check the number of books issued to the customer and determine if the maximum number of books that this member can borrow has been reached, check whether the book has been reserved, post the book issue details in the member's record, and finally print out a book issue slip that can be presented by the member at the security counter to take the book out of the library premises.

Task analysis helps the analyst to understand the nitty-gritty of various user tasks and to represent each task as a hierarchy of subtasks.

Scenario analysis: A task can have many scenarios of operation. The different scenarios of a task may take place when the task is invoked under different situations. For different types of scenarios of a task, the behaviour of the software can be different. For example, the possible scenarios for the book issue task of a library automation software may be:

- Book is issued successfully to the member and the book issue slip is printed.
- The book is reserved, and hence cannot be issued to the member.
- The maximum number of books that can be issued to the member is already reached, and no more books can be issued to the member.

For various identified tasks, the possible scenarios of execution are identified and the details of each scenario is identified in consultation with the users. For each of the identified scenarios, details regarding system response, the exact conditions under which the scenario occurs, etc. are determined in consultation with the user.

Form analysis: Form analysis is an important and effective requirements gathering activity that is undertaken by the analyst, when the project involves automating an existing manual system. During the operation of a manual system, normally several forms are required to be filled up by the stakeholders, and in turn they receive several notifications (usually manually filled forms). In form analysis the existing forms and the formats of the notifications produced are analysed to determine the data input to the system and the data that are output from the system. For the different sets of data input to the system, how these input data would be used by the system to produce the corresponding output data is determined from the users.

Case study 4.1 Requirements gathering for automation of the office work at the CSE department

The academic, inventory, and financial information at the CSE (Computer Science and Engineering) department of a certain institute was being carried out manually by two office clerks, a store keeper, and two attendants. The department has a student strength of 500 and a teacher strength of 30. The head of the department (HoD) wants to automate the office work. Considering the low budget that he has at his disposal, he entrusted the work to a team of student volunteers.

For requirements gathering, a member of the team who was responsible for requirements analysis and specification (analyst) was first briefed by the HoD about the specific activities to be automated. The HoD mentioned that three main aspects of the office work needs to be automated—stores-related activities, student grading activities, and student leave management activities. It was necessary for the analyst to meet the other categories of users. The HoD introduced the analyst (a student) to the office staff. The analyst first discussed with the two clerks regarding their specific responsibilities (tasks) that were required to be automated. For each task, they asked the clerks to brief them about the steps through which these are carried out. The analyst also enquired about the various scenarios that might arise for each task. The analyst collected all types of forms that were being used by the student and the staff of the department to register various types of information with the office (e.g. student course registration, course grading) or requests for some specific service (e.g. issue of items from store). He also collected samples of various types of documents (outputs) the clerks were preparing. Some of these had specific printed forms that the clerks filled up manually, and others were entered using a spreadsheet, and then printed out on a laser printer. For each output form, the analyst consulted the clerks regarding how these different entries are generated from the input data.

The analyst met the store keeper and enquired about the material issue procedures, store ledger entry procedures, and the procedures for raising indents on various vendors. He also collected copies of all the relevant forms that were being used by the store keeper. The analyst also interviewed the student and faculty representatives. Since it was needed to automate the existing activities of an working office, the analyst could without much difficulty obtain the exact formats of the input data, output data, and the precise description of the existing office procedures.

4.1.2 Requirements Analysis

After requirements gathering is complete, the analyst analyses the gathered requirements to form a clear understanding of the exact customer requirements and to weed out any problems in the gathered requirements. It is natural to expect that the data collected from various stakeholders to contain several contradictions, ambiguities, and incompleteness, since each stakeholder typically has only a partial and incomplete view of the software. Therefore, it is necessary to identify all the problems in the requirements and resolve them through further discussions with the customer.

The main purpose of the requirements analysis activity is to analyse the gathered requirements to remove all ambiguities, incompleteness, and inconsistencies from the gathered customer requirements and to obtain a clear understanding of the software to be developed.

For carrying out requirements analysis effectively, the analyst first needs to

develop a clear grasp of the problem. The following basic questions pertaining to the project should be clearly understood by the analyst before carrying out analysis:

- What is the problem?
- Why is it important to solve the problem?
- What exactly are the data input to the system and what exactly are the data output by the system?
- What are the possible procedures that need to be followed to solve the problem?
- What are the likely complexities that might arise while solving the problem?
- If there are external software or hardware with which the developed software has to interface, then what should be the data interchange formats with the external systems?

After the analyst has understood the exact customer requirements, he proceeds to identify and resolve the various problems that he detects in the gathered requirements.

During requirements analysis, the analyst needs to identify and resolve three main types of problems in the requirements:

- Anomaly
- Inconsistency
- Incompleteness

Let us examine these different types of requirements problems in detail.

Anomaly: It is an anomaly is an ambiguity in a requirement. When a requirement is anomalous, several interpretations of that requirement are possible. Any anomaly in any of the requirements can lead to the development of an incorrect system, since an anomalous requirement can be interpreted in the several ways during development. The following are two examples of anomalous requirements:

Example 4.1 While gathering the requirements for a process control application, the following requirement was expressed by a certain stakeholder: When the temperature becomes high, the heater should be switched off. Please note that words such as "high", "low", "good", "bad" etc. are indications of ambiguous requirements as these lack quantification and can be subjectively interpreted. If the threshold above which the temperature

can be considered to be high is not specified, then it can be interpreted differently by different developers.

Example 4.2 In the case study 4.1, suppose one office clerk described the following requirement: during the final grade computation, if any student scores a sufficiently low grade in a semester, then his parents would need to be informed. This is clearly an ambiguous requirement as it lacks any well defined criterion as to what can be considered as a “sufficiently low grade”.

Inconsistency: Two requirements are said to be inconsistent, if one of the requirements contradicts the other. The following are two examples of inconsistent requirements:

Example 4.3 Consider the following two requirements that were collected from two different stakeholders in a process control application development project.

- The furnace should be switched-off when the temperature of the furnace rises above 500° C.
- When the temperature of the furnace rises above 500° C, the water shower should be switched-on and the furnace should remain on.

The requirements expressed by the two stakeholders are clearly inconsistent.

Example 4.4 In the case study 4.1 suppose one of the clerks gave the following requirement—a student securing fail grades in three or more subjects must repeat the courses over an entire semester, and he cannot credit any other courses while repeating the courses. Suppose another clerk expressed the following requirement—there is no provision for any student to repeat a semester; the student should clear the subject by taking it as an extra subject in any later semester. There is a clear inconsistency between the requirements given by the two stakeholders.

Incompleteness: An incomplete set of requirements is one in which some requirements have been overlooked. The lack of these features would be felt by the customer much later, possibly while using the software. Often, incompleteness is caused by the inability of the customer to visualise the system that is to be developed and to anticipate all the features that would be required. An experienced analyst can detect most of these missing features and suggest them to the customer for his consideration and approval for incorporation in the requirements. The following are two examples of

incomplete requirements:

Example 4.5 Suppose for the case study 4.1, one of the clerks expressed the following—If a student secures a *grade point average* (GPA) of less than 6, then the parents of the student must be intimated about the regrettable performance through a (postal) letter as well as through e-mail. However, on an examination of all requirements, it was found that there is no provision by which either the postal or e-mail address of the parents of the students can be entered into the system. The feature that would allow entering the e-mail ids and postal addresses of the parents of the students was missing, thereby making the requirements incomplete.

Example 4.6 In a chemical plant automation software, suppose one of the requirements is that if the internal temperature of the reactor exceeds 200 °C then an alarm bell must be sounded. However, on an examination of all requirements, it was found that there is no provision for resetting the alarm bell after the temperature has been brought down in any of the requirements. This is clearly an incomplete requirement.

Can an analyst detect all the problems existing in the gathered requirements?

Many of the inconsistencies, anomalies, and incompleteness are detected effortlessly, while some others require a focused study of the specific requirements. A few problems in the requirements can, however, be very subtle and escape even the most experienced eyes. Many of these subtle anomalies and inconsistencies can be detected, if the requirements are specified and analysed using a formal method. Once a system has been formally specified, it can be systematically (and even automatically) analysed to remove all problems from the specification. We will discuss the basic concepts of formal system specification in Section 4.3. Though the use of formal techniques is not widespread, the current practice is to formally specify only the safety-critical² parts of a system.

4.2 SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

After the analyst has gathered all the required information regarding the software to be developed, and has removed all incompleteness, inconsistencies, and anomalies from the specification, he starts to systematically organise the requirements in the form of an SRS

document. The SRS document usually contains all the user requirements in a structured though an informal form.

Among all the documents produced during a software development life cycle, SRS document is probably the most important document and is the toughest to write. One reason for this difficulty is that the SRS document is expected to cater to the needs of a wide variety of audience. In the following subsection, we discuss the different categories of users of an SRS document and their needs from it.

4.2.1 Users of SRS Document

Usually a large number of different people need the SRS document for very different purposes. Some of the important categories of users of the SRS document and their needs for use are as follows:

Users, customers, and marketing personnel: These stakeholders need to refer to the SRS document to ensure that the system as described in the document will meet their needs. Remember that the customer may not be the user of the software, but may be some one employed or designated by the user. For generic products, the marketing personnel need to understand the requirements that they can explain to the customers.

Software developers: The software developers refer to the SRS document to make sure that they are developing exactly what is required by the customer.

Test engineers: The test engineers use the SRS document to understand the functionalities, and based on this write the test cases to validate its working. They need that the required functionality should be clearly described, and the input and output data should have been identified precisely.

User documentation writers: The user documentation writers need to read the SRS document to ensure that they understand the features of the product well enough to be able to write the users' manuals.

Project managers: The project managers refer to the SRS document to ensure that they can estimate the cost of the project easily by referring to the SRS document and that it contains all the information required to plan the project.

Maintenance engineers: The SRS document helps the maintenance engineers to understand the functionalities supported by the system. A clear knowledge of the functionalities can help them to understand the design and

code. Also, a proper understanding of the functionalities supported enables them to determine the specific modifications to the system's functionalities would be needed for a specific purpose.

Many software engineers in a project consider the SRS document to be a reference document. However, it is often more appropriate to think of the SRS document as the documentation of a contract between the development team and the customer. In fact, the SRS document can be used to resolve any disagreements between the developers and the customers that may arise in the future. The SRS document can even be used as a legal document to settle disputes between the customers and the developers in a court of law. Once the customer agrees to the SRS document, the development team proceeds to develop the software and ensure that it conforms to all the requirements mentioned in the SRS document.

4.2.2 Why Spend Time and Resource to Develop an SRS Document?

A well-formulated SRS document finds a variety of usage other than the primary intended usage as a basis for starting the software development work. In the following subsection, we identify the important uses of a well-formulated SRS document:

Forms an agreement between the customers and the developers: A good SRS document sets the stage for the customers to form their expectation about the software and the developers about what is expected from the software.

Reduces future reworks: The process of preparation of the SRS document forces the stakeholders to rigorously think about all of the requirements before design and development get underway. This reduces later redesign, recoding, and retesting. Careful review of the SRS document can reveal omissions, misunderstandings, and inconsistencies early in the development cycle.

Provides a basis for estimating costs and schedules: Project managers usually estimate the size of the software from an analysis of the SRS document. Based on this estimate they make other estimations such as the effort required to develop the software and the total cost of development. The SRS document also serves as a basis for price negotiations with the customer. The project manager also uses the SRS document for work scheduling.

Provides a baseline for validation and verification: The SRS document

provides a baseline against which compliance of the developed software can be checked. It is also used by the test engineers to create the *test plan*.

Facilitates future extensions: The SRS document usually serves as a basis for planning future enhancements.

Before we discuss about how to write an SRS document, we first discuss the characteristics of a good SRS document and the pitfalls that one must consciously avoid while writing an SRS document.

4.2.3 Characteristics of a Good SRS Document

The skill of writing a good SRS document usually comes from the experience gained from writing SRS documents for many projects. However, the analyst should be aware of the desirable qualities that every good SRS document should possess. IEEE Recommended Practice for Software Requirements Specifications[IEEE830] describes the content and qualities of a good software requirements specification (SRS). Some of the identified desirable qualities of an SRS document are the following:

- **Concise:** The SRS document should be concise and at the same time unambiguous, consistent, and complete. Verbose and irrelevant descriptions reduce readability and also increase the possibilities of errors in the document.
- **Implementation-independent:** The SRS should be free of design and implementation decisions unless those decisions reflect actual requirements. It should only specify what the system should do and refrain from stating how to do these. This means that the SRS document should specify the externally visible behaviour of the system and not discuss the implementation issues. This view with which a requirements specification is written, has been shown in Figure 4.1. Observe that in Figure 4.1, the SRS document describes the output produced for the different types of input and a description of the processing required to produce the output from the input (shown in ellipses) and the internal working of the software is not discussed at all.

The SRS document should describe the system to be developed as a black box, and should specify only the externally visible behaviour of the system. For this reason, the SRS document is also called the *black-box* specification of the software being

developed.

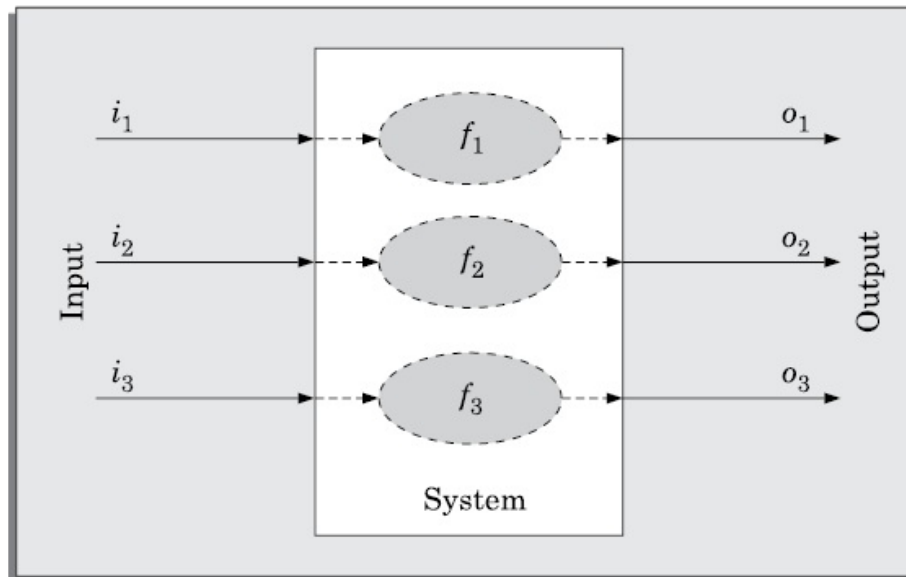


Figure 4.1: The black-box view of a system as performing a set of functions.

Traceable: It should be possible to trace a specific requirement to the design elements that implement it and *vice versa*. Similarly, it should be possible to trace a requirement to the code segments that implement it and the test cases that test this requirement and *vice versa*. Traceability is also important to verify the results of a phase with respect to the previous phase and to analyse the impact of changing a requirement on the design elements and the code.

Modifiable: Customers frequently change the requirements during the software development development due to a variety of reasons. Therefore, in practice the SRS document undergoes several revisions during software development. Also, an SRS document is often modified after the project completes to accommodate future enhancements and evolution. To cope up with the requirements changes, the SRS document should be easily modifiable. For this, an SRS document should be well-structured. A well-structured document is easy to understand and modify. Having the description of a requirement scattered across many places in the SRS document may not be wrong—but it tends to make the requirement difficult to understand and also any modification to the requirement would become difficult as it would require changes to be made at large number of places in the document.

Identification of response to undesired events: The SRS document should discuss the system responses to various undesired events and

exceptional conditions that may arise.

Verifiable: All requirements of the system as documented in the SRS document should be verifiable. This means that it should be possible to design test cases based on the description of the functionality as to whether or not requirements have been met in an implementation. A requirement such as “the system should be user friendly” is not verifiable. On the other hand, the requirement—“When the name of a book is entered, the software should display whether the book is available for issue or it has been loaned out” is verifiable. Any feature of the required system that is not verifiable should be listed separately in the goals of the implementation section of the SRS document.

4.2.4 Attributes of Bad SRS Documents

SRS documents written by novices frequently suffer from a variety of problems. As discussed earlier, the most damaging problems are incompleteness, ambiguity, and contradictions. There are many other types problems that a specification document might suffer from. By knowing these problems, one can try to avoid them while writing an SRS document. Some of the important categories of problems that many SRS documents suffer from are as follows:

Over-specification: It occurs when the analyst tries to address the “how to” aspects in the SRS document. For example, in the library automation problem, one should not specify whether the library membership records need to be stored indexed on the member’s first name or on the library member’s identification (ID) number. Over-specification restricts the freedom of the designers in arriving at a good design solution.

Forward references: One should not refer to aspects that are discussed much later in the SRS document. Forward referencing seriously reduces readability of the specification.

Wishful thinking: This type of problems concern description of aspects which would be difficult to implement.

Noise: The term noise refers to presence of material not directly relevant to the software development process. For example, in the register customer function, suppose the analyst writes that customer registration department is manned by clerks who report for work between 8am and 5pm, 7 days a week. This information can be called *noise* as it would hardly be of any use to the software developers and would unnecessarily clutter the SRS document,

diverting the attention from the crucial points.

Several other “sins” of SRS documents can be listed and used to guard against writing a bad SRS document and is also used as a checklist to review an SRS document.

4.2.5 Important Categories of Customer Requirements

A good SRS document, should properly categorize and organise the requirements into different sections [IEEE830]. As per the IEEE 830 guidelines, the important categories of user requirements are the following.

An SRS document should clearly document the following aspects of a software:

- Functional requirements
- Non-functional requirements
 - Design and implementation constraints
 - External interfaces required
 - Other non-functional requirements
- Goals of implementation.

In the following subsections, we briefly describe the different categories of requirements.

Functional requirements

The functional requirements capture the functionalities required by the users from the system. We have already pointed out in Chapter 2 that it is useful to consider a software as offering a set of functions $\{f_i\}$ to the user. These functions can be considered similar to a mathematical function $f : I \rightarrow O$, meaning that a function transforms an element (i_i) in the input domain (I) to a value (o_i) in the output (O). This functional view of a system is shown schematically in Figure 4.1. Each function f_i of the system can be considered as reading certain data i_i , and then transforming a set of input data (i_i) to the corresponding set of output data (o_i). The functional requirements of the system, should clearly describe each functionality that the system would support along with the corresponding input and output data set. Considering that the functional requirements are a crucial part of the SRS document, we discuss functional requirements in more detail in Section 4.2.6. Section 4.2.7 discusses how the functional requirements can be identified from a problem description.

Finally, Section 4.2.8 discusses how the functional requirements can be

documented effectively.

Non-functional requirements

The non-functional requirements are non-negotiable obligations that must be supported by the software. The non-functional requirements capture those requirements of the customer that cannot be expressed as functions (i.e., accepting input data and producing output data). Non-functional requirements usually address aspects concerning external interfaces, user interfaces, maintainability, portability, usability, maximum number of concurrent users, timing, and throughput (transactions per second, etc.). The non-functional requirements can be critical in the sense that any failure by the developed software to achieve some minimum defined level in these requirements can be considered as a failure and make the software unacceptable by the customer.

The IEEE 830 standard recommends that out of the various non-functional requirements, the external interfaces, and the design and implementation constraints should be documented in two different sections. The remaining non-functional requirements should be documented later in a section and these should include the performance and security requirements.

In the following subsections, we discuss the different categories of non-functional requirements that are described under three different sections:

Design and implementation constraints: Design and implementation constraints are an important category of non-functional requirements describe any items or issues that will limit the options available to the developers. Some of the example constraints can be—corporate or regulatory policies that needs to be honoured; hardware limitations; interfaces with other applications; specific technologies, tools, and databases to be used; specific communications protocols to be used; security considerations; design conventions or programming standards to be followed, etc. Consider an example of a constraint that can be included in this section—Oracle DBMS needs to be used as this would facilitate easy interfacing with other applications that are already operational in the organisation.

External interfaces required: Examples of external interfaces are—hardware, software and communication interfaces, user interfaces, report formats, etc. To specify the user interfaces, each interface between the software and the users must be described. The description may include sample screen images, any GUI standards or style guides that are to be

followed, screen layout constraints, standard buttons and functions (e.g., help) that will appear on every screen, keyboard shortcuts, error message display standards, and so on. One example of a user interface requirement of a software can be that it should be usable by factory shop floor workers who may not even have a high school degree. The details of the user interface design such as screen designs, menu structure, navigation diagram, etc. should be documented in a separate user interface specification document.

Other non-functional requirements: This section contains a description of non-functional requirements that are neither design constraints nor are external interface requirements. An important example is a performance requirement such as the number of transactions completed per unit time. Besides performance requirements, the other non-functional requirements to be described in this section may include reliability issues, accuracy of results, and security issues.

Goals of implementation

The 'goals of implementation' part of the SRS document offers some general suggestions regarding the software to be developed. These are not binding on the developers, and they may take these suggestions into account if possible. For example, the developers may use these suggestions while choosing among different design solutions.

A goal, in contrast to the functional and non-functional requirements, is not checked by the customer for conformance at the time of acceptance testing.

The goals of implementation section might document issues such as easier revisions to the system functionalities that may be required in the future, easier support for new devices to be supported in the future, reusability issues, etc. These are the items which the developers might keep in their mind during development so that the developed system may meet some aspects that are not required immediately. It is useful to remember that anything that would be tested by the user and the acceptance of the system would depend on the outcome of this task, is usually considered as a requirement to be fulfilled by the system and not a goal and *vice versa*.

How to classify the different types of requirements?

We should be clear regarding the aspects of the system requirement that are to be documented as the functional requirement, the ones to be documented as non-functional requirement, and the ones to be

documented as the goals of implementation. Aspects which can be expressed as transformation of some input data to some output data (i.e., the functions of the system) should be documented as the functional requirement. Any other requirements whose compliance by the developed system can be verified by inspecting the system are documented as non-functional requirements. Aspects whose compliance by the developed system need not be verified but are merely included as suggestions to the developers are documented as goals of the implementation.

The difference between non-functional requirements and guidelines is the following. Non-functional requirements would be tested for compliance, before the developed product is accepted by the customer whereas guideline, on the other hand, are customer request that are desirable to be done, but would not be tested during product acceptance.

Functional requirements form the basis for most design and test methodologies. Therefore, unless the functional requirements are properly identified and documented, the design and testing activities cannot be carried out satisfactorily. We discuss how to document the functional requirements in the next section.

4.2.6 Functional Requirements

In order to document the functional requirements of a system, it is necessary to first learn to identify the high-level functions of the systems by reading the informal documentation of the gathered requirements. The high-level functions would be split into smaller subrequirements. Each high-level function is an instance of use of the system (use case) by the user in some way.

A high-level function is one using which the user can get some useful piece of work done.

However, the above is not a very accurate definition of a high-level function. For example, how useful must a piece of work be performed by the system for it to be called 'a useful piece of work' ? Can the printing of the statements of the ATM transaction during withdrawal of money from an ATM be called a useful piece of work? Printing of ATM transaction should not be considered a high-level requirement, because the user does not specifically request for this activity. The receipt gets printed automatically as part of the withdraw money function. Usually, the user invokes (requests) the services of each high-level requirement. It may therefore be possible to treat print

receipt as part of the withdraw money function rather than treating it as a high-level function. It is therefore required that for some of the high-level functions, we might have to debate whether we wish to consider it as a high-level function or not. However, it would become possible to identify most of the high-level functions without much difficulty after practising the solution to a few exercise problems.

Each high-level requirement typically involves accepting some data from the user through a user interface, transforming it to the required response, and then displaying the system response in proper format. For example, in a library automation software, a high-level functional requirement might be search-book. This function involves accepting a book name or a set of key words from the user, running a matching algorithm on the book list, and finally outputting the matched books. The generated system response can be in several forms, e.g., display on the terminal, a print out, some data transferred to the other systems, etc. However, in degenerate cases, a high-level requirement may not involve any data input to the system or production of displayable results. For example, it may involve switch on a light, or starting a motor in an embedded application.

Are high-level functions of a system similar to mathematical functions?

We all know that a mathematical function transforms input data to output data. A high-level function transforms certain input data to output data. However, except for very simple high-level functions, a function rarely reads all its required data in one go and rarely outputs all the results in one shot. In fact, a high-level function usually involves a series of interactions between the system and one or more users. An example of the interactions that may occur in a single high-level requirement has been shown in Figure 4.2. In Figure 4.2, the user inputs have been represented by rectangles and the response produced by the system by circles. Observe that the rectangles and circles alternate in the execution of a single high-level function of the system, indicating a series of requests from the user and the corresponding responses from the system. Typically, there is some initial data input by the user. After accepting this, the system may display some response (called *system action*). Based on this, the user may input further data, and so on.

For any given high-level function, there can be different interaction

sequences or scenarios due to users selecting different options or entering different data items.

In Figure 4.2, the different scenarios occur depending on the amount entered for withdrawal. The different scenarios are essentially different behaviour exhibited by the system for the same high-level function. Typically, each user input and the corresponding system action may be considered as a sub-requirement of a high-level requirement. Thus, each high-level requirement can consist of several sub-requirements.

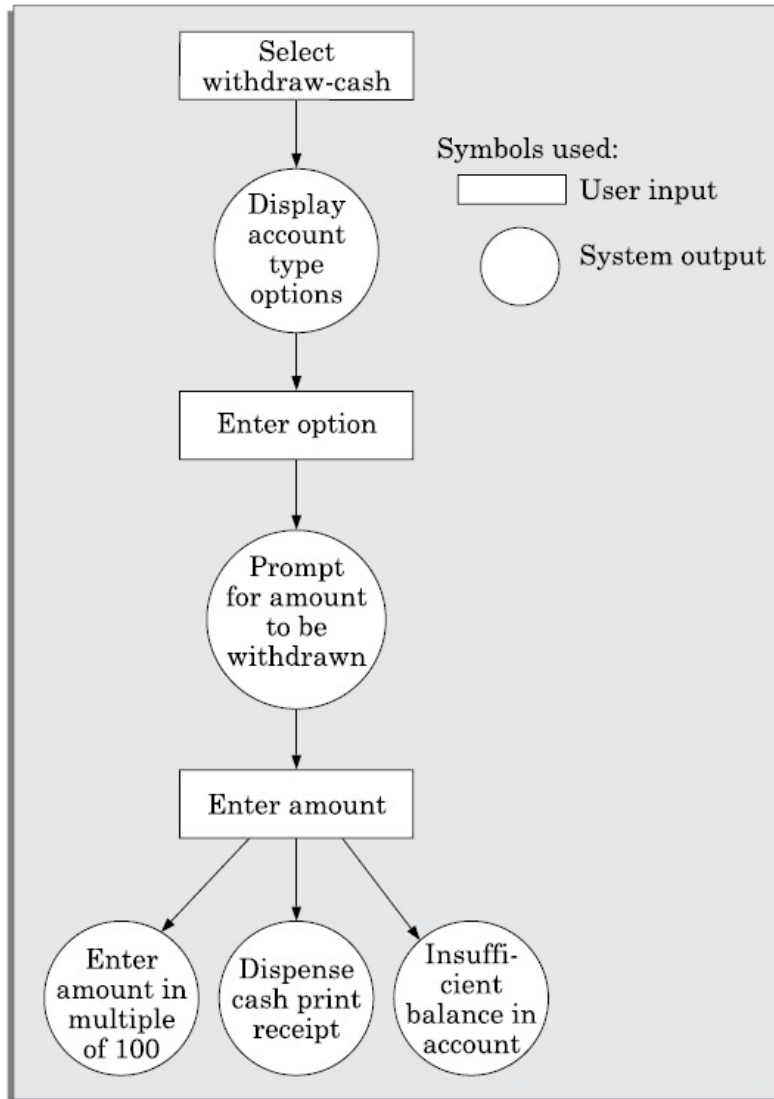


Figure 4.2: User and system interactions in high-level functional requirement.

Is it possible to determine all input and output data precisely?

In a requirements specification document, it is desirable to define the precise data input to the system and the precise data output by the system. Sometimes, the exact data items may be very difficult to

identify. This is especially the case, when no working model of the system to be developed exists. In such cases, the data in a high-level requirement should be described using high-level terms and it may be very difficult to identify the exact components of this data accurately. Another aspect that must be kept in mind is that the data might be input to the system in stages at different points in execution. For example, consider the withdraw-cash function of an *automated teller machine* (ATM) of Figure 4.2. Since during the course of execution of the withdraw-cash function, the user would have to input the type of account, the amount to be withdrawn, it is very difficult to form a single high-level name that would accurately describe both the input data. However, the input data for the subfunctions can be more accurately described.

4.2.7 How to Identify the Functional Requirements?

The high-level functional requirements often need to be identified either from an informal problem description document or from a conceptual understanding of the problem.

Each high-level requirement characterises a way of system usage (service invocation) by some user to perform some meaningful piece of work.

Remember that there can be many types of users of a system and their requirements from the system may be very different. So, it is often useful to first identify the different types of users who might use the system and then try to identify the different services expected from the software by different types of users.

The decision regarding which functionality of the system can be taken to be a high-level functional requirement and the one that can be considered as part of another function (that is, a subfunction) leaves scope for some subjectivity. For example, consider the issue-book function in a Library Automation System. Suppose, when a user invokes the issue-book function, the system would require the user to enter the details of each book to be issued. Should the entry of the book details be considered as a high-level function, or as only a part of the issue-book function? Many times, the choice is obvious. But, sometimes it requires making non-trivial decisions.

4.2.8 How to Document the Functional Requirements?

Once all the high-level functional requirements have been identified and

the requirements problems have been eliminated, these are documented. A function can be documented by identifying the state at which the data is to be input to the system, its input data domain, the output data domain, and the type of processing to be carried on the input data to obtain the output data. We now illustrate the specification of the functional requirements through two examples. Let us first try to document the withdraw-cash function of an *automated teller machine* (ATM) system in the following. The withdraw-cash is a high-level requirement. It has several sub-requirements corresponding to the different user interactions. These user interaction sequences may vary from one invocation from another depending on some conditions. These different interaction sequences capture the different *scenarios*. To accurately describe a functional requirement, we must document all the different scenarios that may occur.

Example 4.7 (Withdraw cash from ATM): An initial informal description of a required functionality is usually given by the customer as a *statement of purpose* (SoP). An SoP serves as a starting point for the analyst and he proceeds with the requirements gathering activity after a basic understanding of the SoP. However, the functionalities of withdraw cash from ATM is intuitively obvious to any one who has used a bank ATM. So, we are not including an informal description of withdraw cash functionality here and in the following, we documents this functional requirement.

R.1: Withdraw cash

Description: The withdraw cash function first determines the type of account that the user has and the account number from which the user wishes to withdraw cash. It checks the balance to determine whether the requested amount is available in the account. If enough balance is available, it outputs the required cash, otherwise it generates an error message.

R.1.1: Select withdraw amount option

Input: "Withdraw amount" option selected *Output:* User prompted to enter the account type

R.1.2: Select account type

Input: User selects option from any one of the followings—savings/checking/deposit.

Output: Prompt to enter amount

R.1.3: Get required amount

Input: Amount to be withdrawn in integer values greater than 100 and less than 10,000 in multiples of 100.

Output: The requested cash and printed transaction statement.

Processing: The amount is debited from the user's account if sufficient balance is available, otherwise an error message displayed.

Example 4.8 (Search book availability in library): An initial informal description of a required functionality is usually given by the customer as a *statement of purpose* (SoP) based on which an later requirements gathering, the analyst understand the functionality. However, the functionalities of *search book availability* is intuitively obvious to any one who has used a library. So, we are not including an informal description of *search book availability* functionality here and in the following, we documents this functional requirement.

R.1: Search book

Description Once the user selects the search option, he would be asked to enter the keywords. The system would search the book in the book list based on the key words entered. After making the search, the system should output the details of all books whose title or author name match any of the key words entered. The book details to be displayed include: title, author name, publisher name, year of publication, ISBN number, catalog number, and the location in the library.

R.1.1: Select search option

Input: "Search" option

Output: User prompted to enter the key words

R.1.2: Search and display

Input: Key words

Output: Details of all books whose title or author name matches any of the key words entered by the user. The book details displayed would include—title of the book, author name, ISBN number, catalog number, year of publication, number of copies available, and the location in the library.

Processing: Search the book list based on the key words:

R.2: Renew book

Description: When the "renew" option is selected, the user is asked to enter his membership number and password. After password validation, the list of

the books borrowed by him are displayed. The user can renew any of his borrowed books by indicating them. A requested book cannot be renewed if it is reserved by another user. In this case, an error message would be displayed.

R.2.1: Select renew option

State: The user has logged in and the main menu has been displayed.

Input: "Renew" option selection.

Output: Prompt message to the user to enter his membership number and password.

R.2.2: Login

State: The renew option has been selected.

Input: Membership number and password.

Output: List of the books borrowed by the user is displayed, and user is prompted to select the books to be renewed, if the password is valid. If the password is invalid, the user is asked to re-enter the password.

Processing: Password validation, search the books issued to the user from the borrower's list and display.

Next function: R.2.3 if password is valid and R.2.2 if password is invalid.

R.2.3: Renew selected books

Input: User choice for books to be renewed out of the books borrowed by him.

Output: Confirmation of the books successfully renewed and apology message for the books that could not be renewed.

Processing: Check if any one has reserved any of the requested books. Renew the books selected by the user in the borrower's list, if no one has reserved those books.

In order to properly identify the high-level requirements, a lot of common sense and the ability to visualise various scenarios that might arise in the operation of a function are required. Please note that when any of the aspects of a requirement, such as the state, processing description, next function to be executed, etc. are obvious, we have omitted it. We have to make a trade-off between cluttering the document with trivial details versus missing out some important descriptions.

Specification of large software: If there are large number of functional requirements (much larger than seen), should they just be written in a long numbered list of requirements? A better way to organise the functional

requirements in this case would be to split the requirements into sections of related requirements. For example, the functional requirements of a academic institute automation software can be split into sections such as accounts, academics, inventory, publications, etc. When there are too many functional requirements, these should be properly arranged into sections. For example the following can be sections in the trade house automation software:

- Customer management
- Account management
- Purchase management
- Vendor management
- Inventory management

Level of details in specification: Even for experienced analysts, a common dilemma is in specifying too little or specifying too much. In practice, we would have to specify only the important input/output interactions in a functionality along with the processing required to generate the output from the input. However, if the interaction sequence is specified in too much detail, then it becomes an unnecessary constraint on the developers and restricts their choice in solution. On the other hand, if the interaction sequence is not sufficiently detailed, it may lead to ambiguities and result in improper implementation.

4.2.9 Traceability

Traceability means that it would be possible to identify (trace) the specific design component which implements a given requirement, the code part that corresponds to a given design component, and test cases that test a given requirement. Thus, any given code component can be traced to the corresponding design component, and a design component can be traced to a specific requirement that it implements and *vice versa*. Traceability analysis is an important concept and is frequently used during software development. For example, by doing a traceability analysis, we can tell whether all the requirements have been satisfactorily addressed in all phases. It can also be used to assess the impact of a requirements change. That is, traceability makes it easy to identify which parts of the design and code would be affected, when certain requirement change occurs. It can also be used to study the impact of a bug that is known to exist in a code part on various

requirements, etc.

To achieve traceability, it is necessary that each functional requirement should be numbered uniquely and consistently. Proper numbering of the requirements makes it possible for different documents to uniquely refer to specific requirements. An example scheme of numbering the functional requirements is shown in Examples 4.7 and 4.8, where the functional requirements have been numbered R.1, R.2, etc. and the subrequirements for the requirement R.1 have been numbered R.1.1, R.1.2, etc.

4.2.10 Organisation of the SRS Document

In this section, we discuss the organisation of an SRS document as prescribed by the IEEE 830 standard[IEEE 830]. Please note that IEEE 830 standard has been intended to serve only as a guideline for organizing a requirements specification document into sections and allows the flexibility of tailoring it, as may be required for specific projects. Depending on the type of project being handled, some sections can be omitted, introduced, or interchanged as may be considered prudent by the analyst. However, organisation of the SRS document to a large extent depends on the preferences of the system analyst himself, and he is often guided in this by the policies and standards being followed by the development company. Also, the organisation of the document and the issues discussed in it to a large extent depend on the type of the product being developed. However, irrespective of the company's principles and product type, the three basic issues that any SRS document should discuss are—functional requirements, non-functional requirements, and guidelines for system implementation.

The introduction section should describe the context in which the system is being developed, and provide an overall description of the system, and the environmental characteristics. The introduction section may include the hardware that the system will run on, the devices that the system will interact with and the user skill-levels. Description of the user skill-level is important, since the command language design and the presentation styles of the various documents depend to a large extent on the types of the users it is targeted for. For example, if the skill-levels of the users is "novice", it would mean that the user interface has to be very simple and rugged, whereas if the user-level is "advanced", several short cut techniques and advanced features may be provided in the user interface.

It is desirable to describe the formats for the input commands, input data, output reports, and if necessary the modes of interaction. We have already

discussed how the contents of the Sections on the functional requirements, the non-functional requirements, and the goals of implementation should be written. In the following subsections, we outline the important sections that an SRS document should contain as suggested by the IEEE 830 standard, for each section of the document, we also briefly discuss the aspects that should be discussed in it.

Introduction

Purpose: This section should describe where the software would be deployed and how the software would be used.

Project scope: This section should briefly describe the overall context within which the software is being developed. For example, the parts of a problem that are being automated and the parts that would need to be automated during future evolution of the software.

Environmental characteristics: This section should briefly outline the environment (hardware and other software) with which the software will interact.

Overall description of organisation of SRS document

Product perspective: This section needs to briefly state as to whether the software is intended to be a replacement for a certain existing systems, or it is a new software. If the software being developed would be used as a component of a larger system, a simple schematic diagram can be given to show the major components of the overall system, subsystem interconnections, and external interfaces can be helpful.

Product features: This section should summarize the major ways in which the software would be used. Details should be provided in Section 3 of the document. So, only a brief summary should be presented here.

User classes: Various user classes that are expected to use this software are identified and described here. The different classes of users are identified by the types of functionalities that they are expected to invoke, or their levels of expertise in using computers.

Operating environment: This section should discuss in some detail the hardware platform on which the software would run, the operating system, and other application software with which the developed software would interact.

Design and implementation constraints: In this section, the different

constraints on the design and implementation are discussed. These might include—corporate or regulatory policies; hardware limitations (timing requirements, memory requirements); interfaces to other applications; specific technologies, tools, and databases to be used; specific programming language to be used; specific communication protocols to be used; security considerations; design conventions or programming standards.

User documentation: This section should list out the types of user documentation, such as user manuals, on-line help, and trouble-shooting manuals that will be delivered to the customer along with the software.

Functional requirements for organisation of SRS document

This section can classify the functionalities either based on the specific functionalities invoked by different users, or the functionalities that are available in different modes, etc., depending what may be appropriate.

1. User class 1
 - (a) Functional requirement 1.1
 - (b) Functional requirement 1.2
2. User class 2
 - (a) Functional requirement 2.1
 - (b) Functional requirement 2.2

External interface requirements

User interfaces: This section should describe a high-level description of various interfaces and various principles to be followed. The user interface description may include sample screen images, any GUI standards or style guides that are to be followed, screen layout constraints, standard push buttons (e.g., help) that will appear on every screen, keyboard shortcuts, error message display standards, etc. The details of the user interface design should be documented in a separate user interface specification document.

Hardware interfaces: This section should describe the interface between the software and the hardware components of the system. This section may include the description of the supported device types, the nature of the data and control interactions between the software and the hardware, and the communication protocols to be used.

Software interfaces: This section should describe the connections between this software and other specific software components, including databases,

operating systems, tools, libraries, and integrated commercial components, etc. Identify the data items that would be input to the software and the data that would be output should be identified and the purpose of each should be described.

Communications interfaces: This section should describe the requirements associated with any type of communications required by the software, such as e-mail, web access, network server communications protocols, etc. This section should define any pertinent message formatting to be used. It should also identify any communication standards that will be used, such as TCP sockets, FTP, HTTP, or SHTTP. Specify any communication security or encryption issues that may be relevant, and also the data transfer rates, and synchronisation mechanisms.

Other non-functional requirements for organisation of SRS document

This section should describe the non-functional requirements other than the design and implementation constraints and the external interface requirements that have been described in Sections 2 and 4 respectively.

Performance requirements: Aspects such as number of transaction to be completed per second should be specified here. Some performance requirements may be specific to individual functional requirements or features. These should also be specified here.

Safety requirements: Those requirements that are concerned with possible loss or damage that could result from the use of the software are specified here. For example, recovery after power failure, handling software and hardware failures, etc. may be documented here.

Security requirements: This section should specify any requirements regarding security or privacy requirements on data used or created by the software. Any user identity authentication requirements should be described here. It should also refer to any external policies or regulations concerning the security issues. Define any security or privacy certifications that must be satisfied.

For software that have distinct modes of operation, in the functional requirements section, the different modes of operation can be listed and in each mode the specific functionalities that are available for invocation can be organised as follows.

Functional requirements

1. Operation mode 1
 - (a) Functional requirement 1.1
 - (b) Functional requirement 1.2
2. Operation mode 2
 - (a) Functional requirement 2.1
 - (b) Functional requirement 2.2

Specification of the behaviour may not be necessary for all systems. It is usually necessary for those systems in which the system behaviour depends on the state in which the system is, and the system transits among a set of states depending on some prespecified conditions and events. The behaviour of a system can be specified using either the *finite state machine* (FSM) formalism and any other alternate formalisms. The FSMs can be used to specify the possible states (modes) of the system and the transition among these states due to occurrence of events.

Example 4.9 (Personal library software): It is proposed to develop a software that would be used by individuals to manage their personal collection of books. The following is an informal description of the requirements of this software as worked out by the marketing department. Develop the functional and non-functional requirements for the software.

A person can have up to a few hundreds of books. The details of all the books such as name of the book, year of publication, date of purchase, price, and publisher would be entered by the owner. A book should be assigned a unique serial number by the computer. This number would be written by the owner using a pen on the inside page of the book. Only a registered friend can be lent a book. While registering a friend, the following data would have to be supplied—name of the friend, his address, land line number, and mobile number. Whenever a book issue request is given, the name of the friend to whom the book is to be issued and the unique id of the book is entered. At this, the various books outstanding against the borrower along with the date borrowed are displayed for information of the owner. If the owner wishes to go ahead with the issue of the book, then the date of issue, the title of the book, and the unique identification number of the book are stored. When a friend returns a book, the date of return is stored and the book is removed from his borrowing list. Upon query, the software should display the name, address, and telephone numbers of each friend against whom books are outstanding along with the titles of the outstanding books and the date on

which those were issued. The software should allow the owner to update the details of a friend such as his address, phone, telephone number, etc. It should be possible for the owner to delete all the data pertaining to a friend who is no more active in using the library. The records should be stored using a free (public domain) data base management system. The software should run on both Windows and Unix machines.

Whenever the owner of the library software borrows a book from his friends, would enter the details regarding the title of the book, and the date borrowed and the friend from whom he borrowed it. Similarly, the return details of books would be entered. The software should be able to display all the books borrowed from various friends upon request by the owner.

It should be possible for any one to query about the availability of a particular book through a web browser from any location. The owner should be able to query the total number of books in the personal library, and the total amount he has invested in his library. It should also be possible for him to view the number of books borrowed and returned by any (or all) friend(s) over any specified time.

Functional requirements

The software needs to support three categories of functionalities as described below:

1. Manage own books

1.1 Register book

Description: To register a book in the personal library, the details of a book, such as name, year of publication, date of purchase, price and publisher are entered. This is stored in the database and a unique serial number is generated.

Input: Book details

Output: Unique serial number

R.1.2: Issue book

Description: A friend can be issued book only if he is registered. The various books outstanding against him along with the date borrowed are first displayed.

R.1.2.1: Display outstanding books

Description: First a friend's name and the serial number of the book to be issued are entered. Then the books outstanding against the friend should be displayed.

Input: Friend name

Output: List of outstanding books along with the date on which each was borrowed.

R.1.2.2: Confirm issue book

If the owner confirms, then the book should be issued to him and the relevant records should be updated.

Input: Owner confirmation for book issue. *Output:* Confirmation of book issue.

R.1.3: Query outstanding books

Description: Details of friends who have books outstanding against their name is displayed.

Input: User selection

Output: The display includes the name, address and telephone numbers of each friend against whom books are outstanding along with the titles of the outstanding books and the date on which those were issued.

R.1.4: Query book

Description: Any user should be able to query a particular book from anywhere using a web browser.

Input: Name of the book.

Output: Availability of the book and whether the book is issued out.

R.1.5: Return book

Description: Upon return of a book by a friend, the date of return is stored and the book is removed from the borrowing list of the concerned friend.

Input: Name of the book.

Output: Confirmation message.

2. Manage friend details

R.2.1: Register friend

Description: A friend must be registered before he can be issued books. After the registration data is entered correctly, the data should be stored and a confirmation message should be displayed.

Input: Friend details including name of the friend, address, land line number and mobile number.

Output: Confirmation of registration status.

R.2.2: Update friend details

Description: When a friend's registration information changes, the same must be updated in the computer.

R.2.2.1: Display current details

Input: Friend name.

Output: Currently stored details.

R2.2.2: Update friend details

Input: Changes needed.

Output: Updated details with confirmation of the changes.

R.3.3: Delete a friend record

Description: Delete records of inactive members.

Input: Friend name.

Output: Confirmation message.

3. Manage borrowed books

R.3.1: Register borrowed books

Description: The books borrowed by the user of the personal library are registered.

Input: Title of the book and the date borrowed.

Output: Confirmation of the registration status.

R.3.2: Deregister borrowed books

Description: A borrowed book is deregistered when it is returned.

Input: Book name.

Output: Confirmation of deregistration.

R.3.3: Display borrowed books

Description: The data about the books borrowed by the owner are displayed.

Input: User selection.

Output: List of books borrowed from other friends.

4. Manage statistics

R.4.1: Display book count

Description: The total number of books in the personal library should be displayed.

Input: User selection.

Output: Count of books.

R4.2: Display amount invested

Description: The total amount invested in the personal library is displayed.

Input: User selection.

Output: Total amount invested.

R.4.2: Display number of transactions *Description:* The total numbers of books issued and returned over a specific period by one (or all) friend(s) is displayed.

Input: Start of period and end of period.

Output: Total number of books issued and total number of books returned.

Non-functional requirements

N.1: Database: A data base management system that is available free of cost in the public domain should be used.

N.2: Platform: Both Windows and Unix versions of the software need to be developed. **N.3: Web-support:** It should be possible to invoke the query book functionality from any place by using a web browser.

Observation: Since there are many functional requirements, the requirements have been organised into four sections: Manage own books, manage friends, manage borrowed books, and manage statistics. Now each section has less than 7 functional requirements. This would not only enhance the readability of the document, but would also help in design.

4.2.11 Techniques for Representing Complex Logic

A good SRS document should properly characterise the conditions under which different scenarios of interaction occur (see Section 4.2.5). That is, a high-level function might involve different steps to be undertaken as a consequence of some decisions made after each step. Sometimes the conditions can be complex and numerous and several alternative interaction and processing sequences may exist depending on the outcome of the corresponding condition checking. A simple text description in such cases can be difficult to comprehend and analyse. In such situations, a decision tree or a decision table can be used to represent the logic and the processing involved. Also, when the decision making in a functional requirement has been represented as a decision table, it becomes easy to automatically or at least manually design test

cases for it. However, use of decision trees or tables would be superfluous in cases where the number of alternatives are few, or the decision logic is straightforward. In such cases, a simple text description would suffice.

There are two main techniques available to analyse and represent complex processing logic—decision trees and decision tables. Once the decision making logic is captured in the form of trees or tables, the test cases to validate these logic can be automatically obtained. It should, however, be noted that decision trees and decision tables have much broader applicability than just specifying complex processing logic in an SRS document. For instance, decision trees and decision tables find applications in information theory and switching theory.

Decision tree

A decision tree gives a graphic view of the processing logic involved in decision making and the corresponding actions taken. Decision tables specify which variables are to be tested, and based on this what actions are to be taken depending upon the outcome of the decision making logic, and the order in which decision making is performed.

The edges of a decision tree represent conditions and the leaf nodes represent the actions to be performed depending on the outcome of testing the conditions. Instead of discussing how to draw a decision tree for a given processing logic, we shall explain through a simple example how to represent the processing logic in the form of a decision tree.

Example 4.10 A library membership management software (LMS) should support the following three options—new member, renewal, and cancel membership. When the *new member* option is selected, the software should ask the member's name, address, and phone number. If proper information is entered, the software should create a membership record for the new member and print a bill for the annual membership charge and the security deposit payable. If the *renewal* option is chosen, the LMS should ask the member's name and his membership number and check whether he is a valid member. If the member details entered are valid, then the membership expiry date in the membership record should be updated and the annual membership charge payable by the member should be printed. If the membership details entered are invalid, an error message should be displayed. If the *cancel membership* option is selected and the name of a valid member is entered, then the membership is cancelled, a choke for the

balance amount due to the member is printed and his membership record is deleted. The decision tree representation for this problem is shown in Figure 4.3.

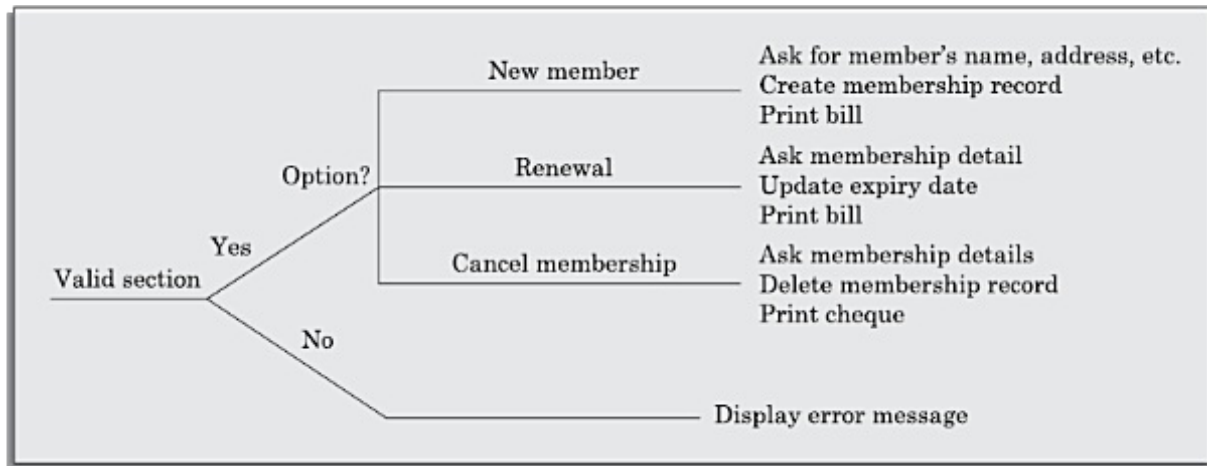


Figure 4.3: Decision Tree for LMS.

Observe from Figure 4.3 that the internal nodes represent conditions, the edges of the tree correspond to the outcome of the corresponding conditions. The leaf nodes represent the actions to be performed by the system. In the decision tree of Figure 4.3, first the user selection is checked. Based on whether the selection is valid, either further condition checking is undertaken or an error message is displayed. Observe that the order of condition checking is explicitly represented.

Decision table

A decision table shows the decision making logic and the corresponding actions taken in a tabular or a matrix form. The upper rows of the table specify the variables or conditions to be evaluated and the lower rows specify the actions to be taken when an evaluation test is satisfied. A column in the table is called a *rule*. A rule implies that if a certain condition combination is true, then the corresponding action is executed. The decision table for the

LMS problem of Example 4.10 is as shown in Table 4.1.

Table 4.1: Decision Table for the LMS Problem

Conditions				
Valid selection	NO	YES	YES	YES
New member	-	YES	NO	NO
Renewal	-	NO	YES	NO
Cancellation	-	NO	NO	YES
Actions				

Display error message	×		
Ask member's name, etc.	×		
Build customer record	×		
Generate bill	×	×	
Ask membership details		×	×
Update expiry date		×	
Print cheque			×
Delete record			×

Decision table *versus* decision tree

Even though both decision tables and decision trees can be used to represent complex program logic, they can be distinguishable on the following three considerations:

Readability: Decision trees are easier to read and understand when the number of conditions are small. On the other hand, a decision table causes the analyst to look at every possible combination of conditions which he might otherwise omit.

Explicit representation of the order of decision making: In contrast to the decision trees, the order of decision making is abstracted out in decision tables. A situation where decision tree is more useful is when multilevel decision making is required. Decision trees can more intuitively represent multilevel decision making hierarchically, whereas decision tables can only represent a single decision to select the appropriate action for execution.

Representing complex decision logic: Decision trees become very complex to understand when the number of conditions and actions increase. It may even be to draw the tree on a single page. When very large number of decisions are involved, the decision table representation may be preferred.

4.3 FORMAL SYSTEM SPECIFICATION

In recent years, formal techniques³ have emerged as a central issue in software engineering. This is not accidental; the importance of precise specification, modelling, and verification is recognised to be important in most engineering disciplines. Formal methods provide us with tools to precisely describe a system and show that a system is correctly implemented. We say a system is correctly implemented when it satisfies its given specification. The specification of a system can be given either as a list of its desirable properties (property-oriented approach) or as an abstract model of the system (model-oriented

approach). These two approaches are discussed here. Before discussing representative examples of these two types of formal specification techniques, we first discuss a few basic concepts in formal specification. We will first highlight some important concepts in formal methods, and examine the merits and demerits of using formal techniques.

4.3.1 What is a Formal Technique?

A formal technique is a mathematical method to specify a hardware and/or software system, verify whether a specification is realisable, verify that an implementation satisfies its specification, prove properties of a system without necessarily running the system, etc. The mathematical basis of a formal method is provided by its specification language. More precisely, a formal specification language consists of two sets—*syn* and *sem*, and a relation *sat* between them. The set *syn* is called the *syntactic domain*, the set *sem* is called the *semantic domain*, and the relation *sat* is called the *satisfaction relation*. For a given specification *syn*, and model of the system *sem*, if *sat* (*syn*, *sem*), then *syn* is said to be the *specification of sem*, and *sem* is said to be the *specificand of syn*.

The generally accepted paradigm for system development is through a hierarchy of abstractions. Each stage in this hierarchy is an implementation of its preceding stage and a specification of the succeeding stage. The different stages in this system development activity are requirements specification, functional design, architectural design, detailed design, coding, implementation, etc. In general, formal techniques can be used at every stage of the system development activity to verify that the output of one stage conforms to the output of the previous stage.

Syntactic domains

The syntactic domain of a formal specification language consists of an alphabet of symbols and a set of formation rules to construct well-formed formulas from the alphabet. The well-formed formulas are used to specify a system.

Semantic domains

Formal techniques can have considerably different semantic domains. Abstract data type specification languages are used to specify algebras, theories, and programs. Programming languages are used to specify functions from input to output values. Concurrent and distributed

system specification languages are used to specify state sequences, event sequences, state-transition sequences, synchronisation trees, partial orders, state machines, etc.

Satisfaction relation

Given the model of a system, it is important to determine whether an element of the semantic domain satisfies the specifications. This satisfaction is determined by using a homomorphism known as *semantic abstraction function*. The semantic abstraction function maps the elements of the semantic domain into equivalent classes. There can be different specifications describing different aspects of a system model, possibly using different specification languages. Some of these specifications describe the system's behaviour and the others describe the system's structure. Consequently, two broad classes of semantic abstraction functions are defined—those that *preserve* a system's behaviour and those that *preserve* a system's structure.

Model *versus* property-oriented methods

Formal methods are usually classified into two broad categories—the so-called *model-oriented* and the property-oriented approaches. In a *model-oriented* style, one defines a system's behaviour directly by constructing a model of the system in terms of mathematical structures such as tuples, relations, functions, sets, sequences, etc. In the *property-oriented* style, the system's behaviour is defined indirectly by stating its properties, usually in the form of a set of axioms that the system must satisfy. Let us consider a simple producer/consumer example. In a *property-oriented* style, we would probably start by listing the properties of the system like—the consumer can start consuming only after the producer has produced an item, the producer starts to produce an item only after the consumer has consumed the last item, etc. Two examples of property-oriented specification styles are axiomatic specification and algebraic specification.

In a *model-oriented* style, we would start by defining the basic operations, p (produce) and c (consume). Then we can state that $S \vdash p \Rightarrow S$, $S \vdash c \Rightarrow S$. Thus model-oriented approaches essentially specify a program by writing another, presumably simpler program. A few notable examples of popular model-oriented specification techniques are Z, CSP, CCS, etc.

It is alleged that property-oriented approaches are more suitable for *requirements specification*, and that the model-oriented approaches are more suited to *system design specification*. The reason for this distinction is the fact that property-oriented approaches specify a system behaviour not by what they say of the system but by what they do not say of the system. Thus, property-oriented specifications permit a large number of possible implementations. Furthermore, property-oriented approaches specify a system by a conjunction of axioms, thereby making it easier to alter/augment specifications at a later stage. On the other hand, model-oriented methods do not support logical conjunctions and disjunctions, and thus even minor changes to a specification may lead to overhauling an entire specification. Since the initial customer requirements undergo several changes as the development proceeds, the property-oriented style is generally preferred for requirements specification. Later in this chapter, we have discussed two property-oriented specification techniques.

4.3.2 Operational Semantics

Informally, the *operational semantics* of a formal method is the way computations are represented. There are different types of operational semantics according to what is meant by a *single run* of the system and how the runs are grouped together to describe the *behaviour* of the system. In the following subsection we discuss some of the commonly used operational semantics.

Linear semantics: In this approach, a *run* of a system is described by a sequence (possibly infinite) of events or states. The concurrent activities of the system are represented by non-deterministic interleavings of the atomic actions. For example, a concurrent activity $a \parallel b$ is represented by the set of sequential activities $a; b$ and $b; a$. This is a simple but rather unnatural representation of concurrency. The behaviour of a system in this model consists of the set of all its runs. To make this model more realistic, usually *justice* and *fairness* restrictions are imposed on computations to exclude the unwanted interleavings.

Branching semantics: In this approach, the behaviour of a system is represented by a directed graph. The nodes of the graph represent the possible states in the evolution of a system. The descendants of each node of the graph represent the states which can be generated by any of the atomic actions enabled at that state. Although this semantic model distinguishes the

branching points in a computation, still it represents concurrency by interleaving.

Maximally parallel semantics: In this approach, all the concurrent actions enabled at any state are assumed to be taken together. This is again not a natural model of concurrency since it implicitly assumes the availability of all the required computational resources.

Partial order semantics: Under this view, the semantics ascribed to a system is a *structure of states* satisfying a partial order relation among the states (events). The partial order represents a *precedence ordering* among events, and constrains some events to occur only after some other events have occurred; while the occurrence of other events (called *concurrent events*) is considered to be incomparable. This fact identifies concurrency as a phenomenon not translatable to any interleaved representation.

Merits and limitations of formal methods

In addition to facilitating precise formulation of specifications, formal methods possess several positive features, some of which are discussed as follows:

- Formal specifications encourage rigour. It is often the case that the very process of construction of a rigorous specification is more important than the formal specification itself. The construction of a rigorous specification clarifies several aspects of system behaviour that are not obvious in an informal specification. It is widely acknowledged that it is cost-effective to spend more efforts at the specification stage, otherwise, many flaws would go unnoticed only to be detected at the later stages of software development that would lead to iterative changes to occur in the development life cycle. According to an estimate, for large and complex systems like distributed real-time systems 80 per cent of project costs and most of the cost overruns result from the iterative changes required in a system development process due to inappropriate formulation of requirements specification. Thus, the additional effort required to construct a rigorous specification is well worth the trouble.
- Formal methods usually have a well-founded mathematical basis. Thus, formal specifications are not only more precise, but also mathematically sound and can be used to reason about the properties

of a specification and to rigorously prove that an implementation satisfies its specifications. Informal specifications may be useful in understanding a system and its documentation, but they cannot serve as a basis of verification. Even carefully written specifications are prone to error, and experience has shown that unverified specifications are comparable in reliability to unverified programs. automatically avoided when one formally specifies a system.

- The mathematical basis of the formal methods makes it possible for automating the analysis of specifications. For example, a tableau-based technique has been used to automatically check the consistency of specifications. Also, automatic theorem proving techniques can be used to verify that an implementation satisfies its specifications. The possibility of automatic verification is one of the most important advantages of formal methods.
- Formal specifications can be executed to obtain immediate feedback on the features of the specified system. This concept of executable specifications is related to rapid prototyping. Informally, a prototype is a “toy” working model of a system that can provide immediate feedback on the behaviour of the specified system, and is especially useful in checking the completeness of specifications.

It is clear that formal methods provide mathematically sound frameworks within which large, complex systems can be specified, developed and verified in a systematic rather than in an ad hoc manner. However, formal methods suffer from several shortcomings, some of which are as following:

- Formal methods are difficult to learn and use.
- The basic incompleteness results of first-order logic suggest that it is impossible to check *absolute* correctness of systems using theorem proving techniques.
- Formal techniques are not able to handle complex problems. This shortcoming results from the fact that, even moderately complicated problems blow up the complexity of formal specification and their analysis. Also, a large unstructured set of mathematical formulas is difficult to comprehend.

It has been pointed out by several researchers that formal specifications neither replace nor make the informal descriptions obsolete but complement

them. In fact, the comprehensibility of formal specifications is greatly enhanced when the specifications are accompanied by an informal description. What is suggested is the use of formal techniques as a broad guideline for the use of the informal techniques. An interesting example of such an approach is reported by Jones in [1980]. In this approach, the use of a formal method identifies the necessary verification steps that need to be carried out, but it is legitimate to apply informal reasoning in presentation of correctness arguments and transformations. Any doubt or query relating to an informal argument is to be resolved by formal proofs.

In the following two sections, we discuss the axiomatic and algebraic specification styles. Both these techniques can be classified as the property-oriented specification techniques.

4.4 AXIOMATIC SPECIFICATION

In axiomatic specification of a system, first-order logic is used to write the pre- and post- conditions to specify the operations of the system in the form of axioms. The pre-conditions basically capture the conditions that must be satisfied before an operation can successfully be invoked. In essence, the pre-conditions capture the requirements on the input parameters of a function. The post-conditions are the conditions that must be satisfied when a function post-conditions are essentially constraints on the results produced for the function execution to be considered successful.

How to develop an axiomatic specifications?

The following are the sequence of steps that can be followed to systematically develop the axiomatic specifications of a function:

- Establish the range of input values over which the function should behave correctly. Establish the constraints on the input parameters as a predicate.
- Specify a predicate defining the condition which must hold on the output of the function if it behaved properly.
- Establish the changes made to the function's input parameters after execution of the function. Pure mathematical functions do not change their input and therefore this type assertion is not necessary for pure functions.
- Combine all of the above into pre- and post-conditions of the function.

We now illustrate how simple abstract data types can be algebraically specified through two simple examples.

Example 4.11 Specify the pre- and post-conditions of a function that takes a real number as argument and returns half the input value if the input is less than or equal to 100, or else returns double the value.

$$f(x : \text{real}) : \text{real}$$

$$\text{pre} : x \in \mathbb{R}$$

$$\text{post} : \{(x \leq 100) \wedge (f(x) = x/2)\} \vee \{(x > 100) \wedge (f(x) = 2 * x)\}$$

Example 4.12 Axiomatically specify a function named search which takes an integer array and an integer key value as its arguments and returns the index in the array where the key value is present.

$$\text{search}(X : \text{intArray}, \text{key} : \text{integer}) : \text{integer}$$

$$\text{pre} : \exists i \in [X \text{ first} \dots X \text{ last}], X[i] = \text{key}$$

$$\text{post} : \{(X \sqcap [\text{search}(X, \text{key})] = \text{key}) \wedge (X = X \sqcap)\}$$

Please note that we have followed the convention that if a function changes any of its input parameters, and if that parameter is named X , then we refer to it after the function completes execution as X' . One practical application of the axiomatic specification is in program documentation. Engineers developing code for a function specify the function by noting down the pre and post conditions of the function in the function header. Another application of the axiomatic specifications is in proving program properties by composing the pre and post-conditions of a number of functions.

4.5 ALGEBRAIC SPECIFICATION

In the algebraic specification technique, an object class or type is specified in terms of relationships existing between the operations defined on that type. It was first brought into prominence by Guttag [1980,1985] in specification of abstract data types. Various notations of algebraic specifications have evolved, including those based on OBJ and Larch languages.

Essentially, algebraic specifications define a system as a *heterogeneous algebra*. A heterogeneous algebra is a collection of different sets on which several operations are defined. Traditional algebras are homogeneous. A homogeneous algebra consists of a single set and several operations defined in this set; e.g. $\{I, +, -, *, /\}$. In contrast, alphabetic strings S together with

operations of concatenation and length $\{S, I, \text{con}, \text{len}\}$, is not a homogeneous algebra, since the range of the length operation is the set of integers.

Each set of symbols in a heterogeneous algebra is called a *sort* of the algebra. To define a heterogeneous algebra, besides defining the sorts, we need to specify the involved operations, their signatures, and their domains and ranges. Using algebraic specification, we define the meaning of a set of interface procedure by using *equations*. An algebraic specification is usually presented in four sections.

Types section: In this section, the sorts (or the data types) being used is specified.

Exception section: This section gives the names of the exceptional conditions that might occur when different operations are carried out. These exception conditions are used in the later sections of an algebraic specification.

Syntax section: This section defines the signatures of the interface procedures. The collection of sets that form input domain of an operator and the sort where the output is produced are called the *signature* of the operator. For example, PUSH takes a stack and an element as its input and returns a new stack that has been created.

Equations section: This section gives a set of *rewrite rules* (or equations) defining the meaning of the interface procedures in terms of each other. In general, this section is allowed to contain conditional expressions.

By convention each equation is implicitly universally quantified over all possible values of the variables. This means that the equation holds for all possible values of the variable. Names not mentioned in the syntax section such *r* or *e* are variables. The first step in defining an algebraic specification is to identify the set of required operations. After having identified the required operators, it is helpful to classify them as either basic constructor operators, extra constructor operators, basic inspector operators, or extra inspection operators. The definition of these categories of operators is as follows:

Basic construction operators: These operators are used to create or modify entities of a type. The basic construction operators are essential to generate all possible element of the type being specified. For example, 'create' and 'append' are basic construction operators in Example 4.13.

Extra construction operators: These are the construction operators other

than the basic construction operators. For example, the operator 'remove' in Example 4.13 is an extra construction operator, because even without using 'remove' it is possible to generate all values of the type being specified.

Basic inspection operators: These operators evaluate attributes of a type without modifying them, e.g., eval, get, etc. Let S be the set of operators whose range is not the data type being specified—these are the inspection operators. The set of the basic operators S_1 is a subset of S , such that each operator from $S - S_1$ can be expressed in terms of the operators from S_1 .

Extra inspection operators: These are the inspection operators that are not basic inspectors. A simple way to determine whether an operator is a constructor (basic or extra) or an inspector (basic or extra) is to check the syntax expression for the operator. If the type being specified appears on the right hand side of the expression then it is a constructor, otherwise it is an inspection operator. For example, in Example 4.13, create is a constructor because point appears on the right hand side of the expression and point is the data type being specified. But, xcoord is an inspection operator since it does not modify the point type.

A good rule of thumb while writing an algebraic specification, is to first establish which are the constructor (basic and extra) and inspection operators (basic and extra). Then write down an axiom for composition of each basic construction operator over each basic inspection operator and extra constructor operator. Also, write down an axiom for each of the extra inspector in terms of any of the basic inspectors. Thus, if there are m_1 basic constructors, m_2 extra constructors, n_1 basic inspectors, and n_2 extra inspectors, we should have $m_1 \times (m_2 + n_1) + n_2$ axioms. However, it should be clearly noted that these $m_1 \times (m_2 + n_1) + n_2$ axioms are the minimum required and many more axioms may be needed to make the specification complete. Using a complete set of rewrite rules, it is possible to simplify an arbitrary sequence of operations on the interface procedures.

While developing the rewrite rules, different persons can come up with different sets of equations. However, while developing the equations one has to be careful that the equations should be able to handle all meaningful composition of operators, and they should have the unique termination and finite termination properties. These two properties of the rewrite rules are discussed later in this section.

Example 4.13 Let us specify a data type point supporting the operations create, xcoord, ycoord, isequal; where the operations have their usual

meaning.

Types:

defines point

uses boolean, integer

Syntax:

1. $\text{create} : \text{integer} \times \text{integer} \rightarrow \text{point}$
2. $\text{xcoord} : \text{point} \rightarrow \text{integer}$
3. $\text{ycoord} : \text{point} \rightarrow \text{integer}$
4. $\text{isequal} : \text{point} \times \text{point} \rightarrow \text{boolean}$

Equations:

1. $\text{xcoord}(\text{create}(x, y)) = x$
2. $\text{ycoord}(\text{create}(x, y)) = y$
3. $\text{isequal}(\text{create}(x_1, y_1), \text{create}(x_2, y_2)) = ((x_1 = x_2) \text{ and } (y_1 = y_2))$

In this example, we have only one basic constructor (create), and three basic inspectors (xcoord, ycoord, and isequal). Therefore, we have only 3 equations.

The rewrite rules let you determine the meaning of any sequence of calls on the point type. Consider the following expression: *isequal (create (xcoord (create(2, 3)), 5), create (ycoord (create(2, 3)), 5))*. By applying the rewrite rule 1, you can simplify the given expression as *isequal (create (2, 5), create (ycoord (create(2, 3)), 5))*. By using rewrite rule 2, you can further simplify this as *isequal (create (2, 5), create (3, 5))*. This is false by rewrite rule 3.

Properties of algebraic specifications

Three important properties that every algebraic specification should possess are:

Completeness: This property ensures that using the equations, it should be possible to reduce any arbitrary sequence of operations on the interface procedures. When the equations are not complete, at some step during the reduction process, we might not be able to reduce the expression arrived at that step by using any of the equations. There is no simple procedure to ensure that an algebraic specification is complete.

Finite termination property: This property essentially addresses the following question: Do applications of the rewrite rules to arbitrary expressions involving the interface procedures always terminate? For arbitrary algebraic equations, convergence (finite termination) is undecidable.

But, if the right hand side of each rewrite rule has fewer terms than the left, then the rewrite process must terminate.

Unique termination property: This property indicates whether application of rewrite rules in different orders always result in the same answer. Essentially, to determine this property, the answer to the following question needs to be checked—Can all possible sequence of choices in application of the rewrite rules to an arbitrary expression involving the interface procedures always give the same answer? Checking the unique termination property is a very difficult problem.

Example 4.14 Let us specify a FIFO queue supporting the operations create, append, remove, first, and isempty; where the operations have their usual meaning.

Types:

```
defines queue
uses boolean, element
```

Exception:

```
underflow, novalue
```

Syntax:

1. create : $\emptyset \rightarrow \text{queue}$
2. append : $\text{queue} \times \text{element} \rightarrow \text{queue}$
3. remove : $\text{queue} \rightarrow \text{queue} + \{\text{underflow}\}$
4. first : $\text{queue} \rightarrow \text{element} + \{\text{novalue}\}$
5. isempty : $\text{queue} \rightarrow \text{boolean}$

Equations:

1. isempty(create()) = true
2. isempty(append(q, e)) = f else
3. first(create()) = novalue
4. first(append(q, e)) = if isempty(q) then e else first(q)
5. remove(create()) = underflow
6. remove(append(q, e)) = if isempty(q) then create() else append(remove(q), e)

In this example, we have two basic construction operators (create and append). We have one extra construction operator (remove). We have considered remove to be an extra construction operator because all values of the queue can be realised, even without having the remove operator. We have two basic inspectors (first and isempty). Therefore we have $2 \times 3 = 6$

equations.

4.5.1 Auxiliary Functions

Sometimes while specifying a system, one needs to introduce extra functions not part of the system to define the meaning of some interface procedures. These are called *auxiliary functions*. In the following, we discuss an example where it becomes necessary to use an auxiliary function to be able to specify a system.

Example 4.15 Let us specify a bounded FIFO queue having a maximum size of *MaxSize* and supporting the operations *create*, *append*, *remove*, *first*, and *isempty*; where the operations have their usual meaning.

Types:

defines queue
uses boolean, element, integer

Exception:

underflow, novalue, overflow

Syntax:

1. *create* : $\emptyset \rightarrow \text{queue}$
2. *append* : $\text{queue} \times \text{element} \rightarrow \text{queue} + \{\text{overflow}\}$
3. *size* : $\text{queue} \rightarrow \text{integer}$
4. *remove* : $\text{queue} \rightarrow \text{queue} + \{\text{underflow}\}$
5. *first* : $\text{queue} \rightarrow \text{element} + \{\text{novalue}\}$
6. *isempty* : $\text{queue} \rightarrow \text{boolean}$

Equations:

1. *first*(*create*()) = novalue
2. *first*(*append*(*q*, *e*)) = if *size*(*q*) = *MaxSize* then overflow else if *isempty*(*q*) then e else *first*(*q*)
3. *remove*(*create*()) = underflow
4. *remove*(*append*(*q*, *e*)) = if *isempty*(*q*) then *create*() else
 .
 if *size*(*q*) = *MaxSize* then overflow else *append*(*remove*(*q*), *e*)
5. *size*(*create*()) = 0
6. *size*(*append*(*q*, *e*)) = if *size*(*q*) = *MaxSize* then overflow else *size*(*q*) + 1
7. *isempty*(*q*) = if (*size*(*q*) = 0) then true else false

In this example, we have used the auxiliary function *size* to enable us to

specify that during appending an element, overflow might occur if the queue size exceeds `MaxSize`. However, after we have introduced the auxiliary function `size`, we find that the operator `isempty` can no longer be considered as a basic inspector because `isempty` can be expressed in terms of `size`. Therefore, we have removed the axioms for the operator `isempty` used in Example 4.15, and have instead used an axiom to express `isempty` in terms of `size`. We have added two axioms to express `size` in terms of the basic construction operators (`create` and `append`).

4.5.2 Structured Specification

Developing algebraic specifications is time consuming. Therefore efforts have been made to devise ways to ease the task of developing algebraic specifications. The following are some of the techniques that have successfully been used to reduce the effort in writing the specifications.

Incremental specification: The idea behind incremental specification is to first develop the specifications of the simple types and then specify more complex types by using the specifications of the simple types.

Specification instantiation: This involves taking an existing specification which has been developed using a generic parameter and instantiating it with some other sort.

Pros and Cons of algebraic specifications

Algebraic specifications have a strong mathematical basis and can be viewed as heterogeneous algebra. Therefore, they are unambiguous and precise. Using an algebraic specification, the effect of any arbitrary sequence of operations involving the interface procedures can automatically be studied. A major shortcoming of algebraic specifications is that they cannot deal with side effects. Therefore, algebraic specifications are difficult to integrate with typical programming languages. Also, algebraic specifications are hard to understand.

4.6 EXECUTABLE SPECIFICATION AND 4GL

When the specification of a system is expressed formally or is described by using a programming language, then it becomes possible to directly execute the specification without having to design and write code for

Chapter 5

SOFTWARE DESIGN

During the software design phase, the design document is produced, based on the customer requirements as documented in the SRS document. We can state the main objectives of the design phase, in other words, as follows.

The activities carried out during the design phase (called as design process) transform the SRS document into the design document.

This view of a design process has been shown schematically in Figure 5.1. As shown in Figure 5.1, the design process starts using the SRS document and completes with the production of the design document. The design document produced at the end of the design phase should be implementable using a programming language in the subsequent (coding) phase.

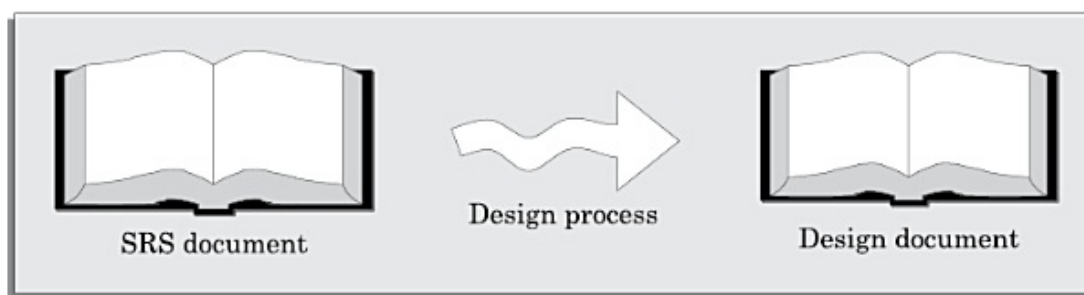


Figure 5.1: The design process.

5.1 OVERVIEW OF THE DESIGN PROCESS

The design process essentially transforms the SRS document into a design document. In the following sections and subsections, we will discuss a few important issues associated with the design process.

5.1.1 Outcome of the Design Process

The following items are designed and documented during the design phase.

Different modules required: The different modules in the solution should be clearly identified. Each module is a collection of functions and the data shared by the functions of the module. Each module should accomplish some well-defined task out of the overall responsibility of the software. Each module should be named according to the task it performs. For example, in an academic automation software, the module consisting of the functions and data necessary to accomplish the task of registration of the students should be named handle student registration.

Control relationships among modules: A control relationship between two modules essentially arises due to function calls across the two modules. The control relationships existing among various modules should be identified in the design document.

Interfaces among different modules: The interfaces between two modules identifies the exact data items that are exchanged between the two modules when one module invokes a function of the other module.

Data structures of the individual modules: Each module normally stores some data that the functions of the module need to share to accomplish the overall responsibility of the module. Suitable data structures for storing and managing the data of a module need to be properly designed and documented.

Algorithms required to implement the individual modules: Each function in a module usually performs some processing activity. The algorithms required to accomplish the processing activities of various modules need to be carefully designed and documented with due considerations given to the accuracy of the results, space and time complexities.

Starting with the SRS document (as shown in Figure 5.1), the design documents are produced through iterations over a series of steps that we are going to discuss in this chapter and the subsequent three chapters. The design documents are reviewed by the members of the development team to ensure that the design solution conforms to the requirements specification.

5.1.2 Classification of Design Activities

A good software design is seldom realised by using a single step procedure, rather it requires iterating over a series of steps called the design activities. Let us first classify the design activities before discussing them in detail. Depending on the order in which various design activities are performed, we can broadly classify them into two

important stages.

- Preliminary (or high-level) design, and
- Detailed design.

The meaning and scope of these two stages can vary considerably from one design methodology to another. However, for the traditional function-oriented design approach, it is possible to define the objectives of the high-level design as follows:

Through high-level design, a problem is decomposed into a set of modules. The control relationships among the modules are identified, and also the interfaces among various modules are identified.

The outcome of high-level design is called the program structure or the software architecture. High-level design is a crucial step in the overall design of a software. When the high-level design is complete, the problem should have been decomposed into many small functionally independent modules that are cohesive, have low coupling among themselves, and are arranged in a hierarchy. Many different types of notations have been used to represent a high-level design. A notation that is widely being used for procedural development is a tree-like diagram called the structure chart. Another popular design representation techniques called UML that is being used to document object-oriented design, involves developing several types of diagrams to document the object-oriented design of a systems. Though other notations such as Jackson diagram [1975] or Warnier-Orr [1977, 1981] diagram are available to document a software design, we confine our attention in this text to structure charts and UML diagrams only.

Once the high-level design is complete, detailed design is undertaken.

During detailed design each module is examined carefully to design its data structures and the algorithms.

The outcome of the detailed design stage is usually documented in the form of a module specification (MSPEC) document. After the high-level design is complete, the problem would have been decomposed into small modules, and the data structures and algorithms to be used described using MSPEC and can be easily grasped by programmers for initiating coding. In this text, we do not discuss MSPECs and confine our attention to high-level design only.

5.1.3 Classification of Design Methodologies

The design activities vary considerably based on the specific design

methodology being used. A large number of software design methodologies are available. We can roughly classify these methodologies into procedural and object-oriented approaches. These two approaches are two fundamentally different design paradigms. In this chapter, we shall discuss the important characteristics of these two fundamental design approaches. Over the next three chapters, we shall study these two approaches in detail.

Do design techniques result in unique solutions?

Even while using the same design methodology, different designers usually arrive at very different design solutions. The reason is that a design technique often requires the designer to make many subjective decisions and work out compromises to contradictory objectives. As a result, it is possible that even the same designer can work out many different solutions to the same problem. Therefore, obtaining a good design would involve trying out several alternatives (or candidate solutions) and picking out the best one. However, a fundamental question that arises at this point is—how to distinguish superior design solution from an inferior one? Unless we know what a good software design is and how to distinguish a superior design solution from an inferior one, we can not possibly design one. We investigate this issue in the next section.

Analysis versus design

Analysis and design activities differ in goal and scope.

The goal of any analysis technique is to elaborate the customer requirements through careful thinking and at the same time consciously avoiding making any decisions regarding the exact way the system is to be implemented.

The analysis results are generic and does not consider implementation or the issues associated with specific platforms. The analysis model is usually documented using some graphical formalism. In case of the function-oriented approach that we are going to discuss, the analysis model would be documented using data flow diagrams (DFDs), whereas the design would be documented using structure chart. On the other hand, for object-oriented approach, both the design model and the analysis model will be documented using unified modelling language (UML). The analysis model would normally be very difficult to implement using a programming language.

The design model is obtained from the analysis model through

transformations over a series of steps. In contrast to the analysis model, the design model reflects several decisions taken regarding the exact way system is to be implemented. The design model should be detailed enough to be easily implementable using a programming language.

5.2 HOW TO CHARACTERISE A GOOD SOFTWARE DESIGN?

Coming up with an accurate characterisation of a good software design that would hold across diverse problem domains is certainly not easy. In fact, the definition of a “good” software design can vary depending on the exact application being designed. For example, “memory size used up by a program” may be an important issue to Characterise a good solution for embedded software development—since embedded applications are often required to work under severely limited memory sizes due to cost, space, or power consumption considerations. For embedded applications, factors such as design comprehensibility may take a back seat while judging the goodness of design. Thus for embedded applications, one may sacrifice design comprehensibility to achieve code compactness. Similarly, it is not usually true that a criterion that is crucial for some application, needs to be almost completely ignored for another application. It is therefore clear that the criteria used to judge a design solution can vary widely across different types of applications. Not only do the criteria used to judge a design solution depend on the exact application being designed, but to make the matter worse, there is no general agreement among software engineers and researchers on the exact criteria to use for judging a design even for a specific category of application. However, most researchers and software engineers agree on a few desirable characteristics that every good software design for general applications must possess. These characteristics are listed below:

Correctness: A good design should first of all be correct. That is, it should correctly implement all the functionalities of the system.

Understandability: A good design should be easily understandable. Unless a design solution is easily understandable, it would be difficult to implement and maintain it.

Efficiency: A good design solution should adequately address resource, time, and cost optimisation issues.

Maintainability: A good design should be easy to change. This is an

important requirement, since change requests usually keep coming from the customer even after product release.

5.2.1 Understandability of a Design: A Major Concern

While performing the design of a certain problem, assume that we have arrived at a large number of design solutions and need to choose the best one. Obviously all incorrect designs have to be discarded first. Out of the correct design solutions, how can we identify the best one?

Given that we are choosing from only correct design solutions, understandability of a design solution is possibly the most important issue to be considered while judging the goodness of a design.

Recollect from our discussions in Chapter 1 that a good design should help overcome the human cognitive limitations that arise due to limited short-term memory. A large problem overwhelms the human mind, and a poor design would make the matter worse. Unless a design solution is easily understandable, it could lead to an implementation having a large number of defects and at the same time tremendously pushing up the development costs. Therefore, a good design solution should be simple and easily understandable. A design that is easy to understand is also easy to develop and maintain. A complex design would lead to severely increased life cycle costs. Unless a design is easily understandable, it would require tremendous effort to implement, test, debug, and maintain it. We had already pointed out in Chapter 2 that about 60 per cent of the total effort in the life cycle of a typical product is spent on maintenance. If the software is not easy to understand, not only would it lead to increased development costs, the effort required to maintain the product would also increase manifold. Besides, a design solution that is difficult to understand would lead to a program that is full of bugs and is unreliable. Recollect that we had already discussed in Chapter 1 that understandability of a design solution can be enhanced through clever applications of the principles of abstraction and decomposition.

An understandable design is modular and layered

How can the understandability of two different designs be compared, so that we can pick the better one? To be able to compare the understandability of two design solutions, we should at least have an understanding of the general features that an easily understandable design should possess. A design solution should have the following

characteristics to be easily understandable:

- It should assign consistent and meaningful names to various design components.
- It should make use of the principles of decomposition and abstraction in good measures to simplify the design.

We had discussed the essential concepts behind the principles of abstraction and decomposition principles in Chapter 1. But, how can the abstraction and decomposition principles are used in arriving at a design solution? These two principles are exploited by design methodologies to make a design modular and layered. (Though there are also a few other forms in which the abstraction and decomposition principles can be used in the design solution, we discuss those later). We can now define the characteristics of an easily understandable design as follows: A design solution is understandable, if it is modular and the modules are arranged in distinct layers.

A design solution should be modular and layered to be understandable.

We now elaborate the concepts of modularity and layering of modules:

Modularity

A modular design is an effective decomposition of a problem. It is a basic characteristic of any good design solution. A modular design, in simple words, implies that the problem has been decomposed into a set of modules that have only limited interactions with each other. Decomposition of a problem into modules facilitates taking advantage of the divide and conquer principle. If different modules have either no interactions or little interactions with each other, then each module can be understood separately. This reduces the perceived complexity of the design solution greatly. To understand why this is so, remember that it may be very difficult to break a bunch of sticks which have been tied together, but very easy to break the sticks individually.

It is not difficult to argue that modularity is an important characteristic of a good design solution. But, even with this, how can we compare the modularity of two alternate design solutions? From an inspection of the module structure, it is at least possible to intuitively form an idea as to which design is more modular. For example, consider two alternate design solutions

to a problem that are represented in Figure 5.2, in which the modules M1 , M2 etc. have been drawn as rectangles. The invocation of a module by another module has been shown as an arrow. It can easily be seen that the design solution of Figure 5.2(a) would be easier to understand since the interactions among the different modules is low. But, can we quantitatively measure the modularity of a design solution? Unless we are able to quantitatively measure the modularity of a design solution, it will be hard to say which design solution is more modular than another. Unfortunately, there are no quantitative metrics available yet to directly measure the modularity of a design. However, we can quantitatively characterise the modularity of a design solution based on the cohesion and coupling existing in the design.

A design solution is said to be highly modular, if the different modules in the solution have high cohesion and their inter-module couplings are low.

A software design with high cohesion and low coupling among modules is the effective problem decomposition we discussed in Chapter 1. Such a design would lead to increased productivity during program development by bringing down the perceived problem complexity.

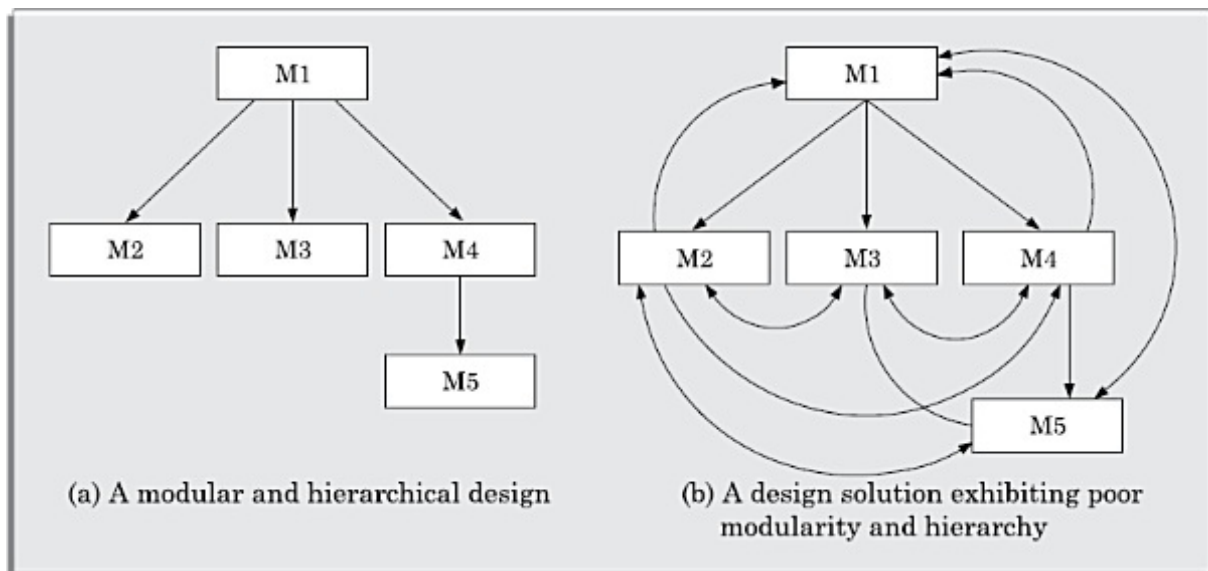


Figure 5.2: Two design solutions to the same problem.

Based on this classification, we would be able to easily judge the cohesion and coupling existing in a design solution. From a knowledge of the cohesion and coupling in a design, we can form our own opinion about the modularity of the design solution. We shall define the concepts of cohesion and coupling and the various classes of cohesion and coupling in Section 5.3. Let us now discuss the other important characteristic of a good design solution—layered

design.

Layered design

A layered design is one in which when the call relations among different modules are represented graphically, it would result in a tree-like diagram with clear layering. In a layered design solution, the modules are arranged in a hierarchy of layers. A module can only invoke functions of the modules in the layer immediately below it. The higher layer modules can be considered to be similar to managers that invoke (order) the lower layer modules to get certain tasks done. A layered design can be considered to be implementing control abstraction, since a module at a lower layer is unaware of (about how to call) the higher layer modules.

A layered design can make the design solution easily understandable, since to understand the working of a module, one would at best have to understand how the immediately lower layer modules work without having to worry about the functioning of the upper layer modules.

When a failure is detected while executing a module, it is obvious that the modules below it can possibly be the source of the error. This greatly simplifies debugging since one would need to concentrate only on a few modules to detect the error. We shall elaborate these concepts governing layered design of modules in Section 5.4.

5.3 COHESION AND COUPLING

We have so far discussed that effective problem decomposition is an important characteristic of a good design. Good module decomposition is indicated through high cohesion of the individual modules and low coupling of the modules with each other. Let us now define what is meant by cohesion and coupling.

Cohesion is a measure of the functional strength of a module, whereas the coupling between two modules is a measure of the degree of interaction (or interdependence) between the two modules.

In this section, we first elaborate the concepts of cohesion and coupling. Subsequently, we discuss the classification of cohesion and coupling.

Coupling: Intuitively, we can think of coupling as follows. Two modules are said to be highly coupled, if either of the following two situations arise:

- If the function calls between two modules involve passing large chunks

of shared data, the modules are tightly coupled.

- If the interactions occur through some shared data, then also we say that they are highly coupled.

If two modules either do not interact with each other at all or at best interact by passing no data or only a few primitive data items, they are said to have low coupling.

Cohesion: To understand cohesion, let us first understand an analogy. Suppose you listened to a talk by some speaker. You would call the speech to be cohesive, if all the sentences of the speech played some role in giving the talk a single and focused theme. Now, we can extend this to a module in a design solution. When the functions of the module co-operate with each other for performing a single objective, then the module has good cohesion. If the functions of the module do very different things and do not co-operate with each other to perform a single piece of work, then the module has very poor cohesion.

Functional independence

By the term functional independence, we mean that a module performs a single task and needs very little interaction with other modules.

A module that is highly cohesive and also has low coupling with other modules is said to be functionally independent of the other modules.

Functional independence is a key to any good design primarily due to the following advantages it offers:

Error isolation: Whenever an error exists in a module, functional independence reduces the chances of the error propagating to the other modules. The reason behind this is that if a module is functionally independent, its interaction with other modules is low. Therefore, an error existing in the module is very unlikely to affect the functioning of other modules.

Further, once a failure is detected, error isolation makes it very easy to locate the error. On the other hand, when a module is not functionally independent, once a failure is detected in a functionality provided by the module, the error can be potentially in any of the large number of modules and propagated to the functioning of the module.

Scope of reuse: Reuse of a module for the development of other applications becomes easier. The reasons for this is as follows. A functionally

independent module performs some well-defined and precise task and the interfaces of the module with other modules are very few and simple. A functionally independent module can therefore be easily taken out and reused in a different program. On the other hand, if a module interacts with several other modules or the functions of a module perform very different tasks, then it would be difficult to reuse it. This is especially so, if the module accesses the data (or code) internal to other modules.

Understandability: When modules are functionally independent, complexity of the design is greatly reduced. This is because of the fact that different modules can be understood in isolation, since the modules are independent of each other. We have already pointed out in Section 5.2 that understandability is a major advantage of a modular design. Besides the three we have listed here, there are many other advantages of a modular design as well. We shall not list those here, and leave it as an assignment to the reader to identify them.

5.3.1 Classification of Cohesiveness

Cohesiveness of a module is the degree to which the different functions of the module co-operate to work towards a single objective. The different modules of a design can possess different degrees of freedom. However, the different classes of cohesion that modules can possess are depicted in Figure 5.3. The cohesiveness increases from coincidental to functional cohesion. That is, coincidental is the worst type of cohesion and functional is the best cohesion possible. These different classes of cohesion are elaborated below.

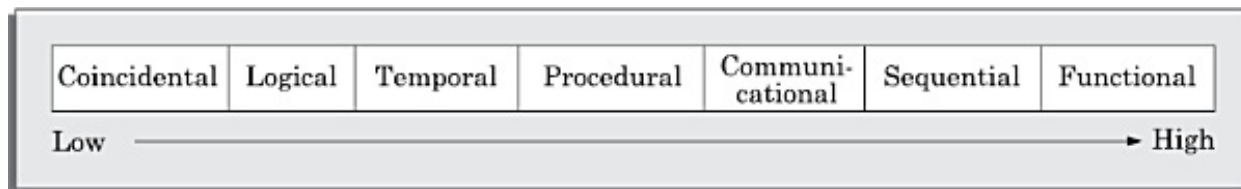


Figure 5.3: Classification of cohesion.

Coincidental cohesion: A module is said to have coincidental cohesion, if it performs a set of tasks that relate to each other very loosely, if at all. In this case, we can say that the module contains a random collection of functions. It is likely that the functions have been placed in the module out of pure coincidence rather than through some thought or design. The designs made by novice programmers often possess this category of cohesion, since they often bundle functions to modules rather arbitrarily. An example of a module with coincidental cohesion

has been shown in Figure 5.4(a). Observe that the different functions of the module carry out very different and unrelated activities starting from issuing of library books to creating library member records on one hand, and handling librarian leave request on the other.

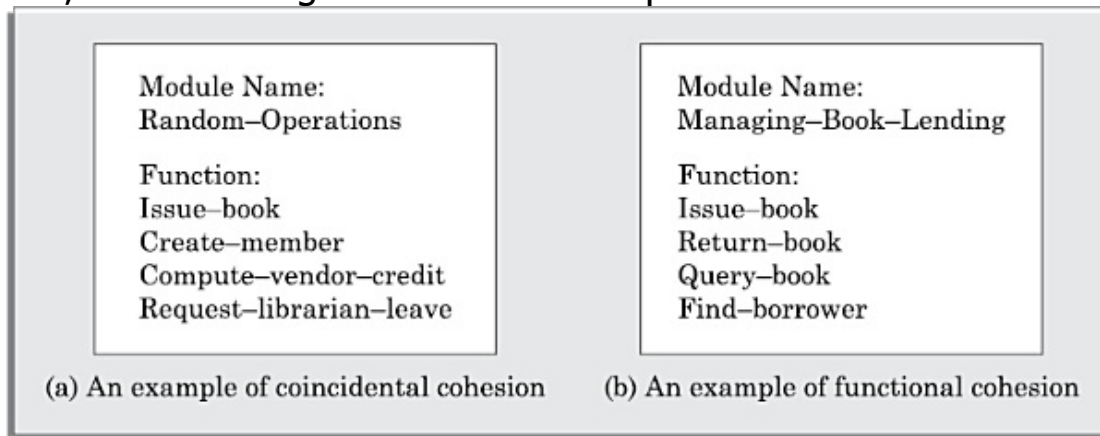


Figure 5.4: Examples of cohesion.

Logical cohesion: A module is said to be logically cohesive, if all elements of the module perform similar operations, such as error handling, data input, data output, etc. As an example of logical cohesion, consider a module that contains a set of print functions to generate various types of output reports such as grade sheets, salary slips, annual reports, etc.

Temporal cohesion: When a module contains functions that are related by the fact that these functions are executed in the same time span, then the module is said to possess temporal cohesion. As an example, consider the following situation. When a computer is booted, several functions need to be performed. These include initialisation of memory and devices, loading the operating system, etc. When a single module performs all these tasks, then the module can be said to exhibit temporal cohesion. Other examples of modules having temporal cohesion are the following. Similarly, a module would exhibit temporal cohesion, if it comprises functions for performing initialisation, or start-up, or shut-down of some process.

Procedural cohesion: A module is said to possess procedural cohesion, if the set of functions of the module are executed one after the other, though these functions may work towards entirely different purposes and operate on very different data. Consider the activities associated with order processing in a trading house. The functions login(), place-order(), check-order(), print-bill(), place-order-on-vendor(), update-inventory(), and logout() all do different thing and operate on different data. However, they are normally

executed one after the other during typical order processing by a sales clerk.

Communicational cohesion: A module is said to have communicational cohesion, if all functions of the module refer to or update the same data structure. As an example of procedural cohesion, consider a module named student in which the different functions in the module such as admitStudent, enterMarks, printGradeSheet, etc. access and manipulate data stored in an array named studentRecords defined within the module.

Sequential cohesion: A module is said to possess sequential cohesion, if the different functions of the module execute in a sequence, and the output from one function is input to the next in the sequence. As an example consider the following situation. In an on-line store consider that after a customer requests for some item, it is first determined if the item is in stock. In this case, if the functions create-order(), check-item-availability(), place-order-on-vendor() are placed in a single module, then the module would exhibit sequential cohesion. Observe that the function create-order() creates an order that is processed by the function check-item-availability() (whether the items are available in the required quantities in the inventory) is input to place-order-on-vendor().

Functional cohesion: A module is said to possess functional cohesion, if different functions of the module co-operate to complete a single task. For example, a module containing all the functions required to manage employees' pay-roll displays functional cohesion. In this case, all the functions of the module (e.g., computeOvertime(), computeWorkHours(), computeDeductions(), etc.) work together to generate the payslips of the employees. Another example of a module possessing functional cohesion has been shown in Figure 5.4(b). In this example, the functions issue-book(), return-book(), query-book(), and find-borrower(), together manage all activities concerned with book lending. When a module possesses functional cohesion, then we should be able to describe what the module does using only one simple sentence. For example, for the module of Figure 5.4(a), we can describe the overall responsibility of the module by saying "It manages the book lending procedure of the library."

A simple way to determine the cohesiveness of any given module is as follows. First examine what do the functions of the module perform. Then, try to write down a sentence to describe the overall work performed by the module. If you need a compound sentence to describe the functionality of the module, then it has sequential or communicational cohesion. If you need words such as "first", "next", "after", "then", etc., then it possesses sequential

or temporal cohesion. If it needs words such as “initialise”, “setup”, “shut down”, etc., to define its functionality, then it has temporal cohesion.

We can now make the following observation. A cohesive module is one in which the functions interact among themselves heavily to achieve a single goal. As a result, if any of these functions is removed to a different module, the coupling would increase as the functions would now interact across two different modules.

5.3.2 Classification of Coupling

The coupling between two modules indicates the degree of interdependence between them. Intuitively, if two modules interchange large amounts of data, then they are highly interdependent or coupled. We can alternately state this concept as follows.

The degree of coupling between two modules depends on their interface complexity.

The interface complexity is determined based on the number of parameters and the complexity of the parameters that are interchanged while one module invokes the functions of the other module.

Let us now classify the different types of coupling that can exist between two modules. Between any two interacting modules, any of the following five different types of coupling can exist. These different types of coupling, in increasing order of their severities have also been shown in Figure 5.5.

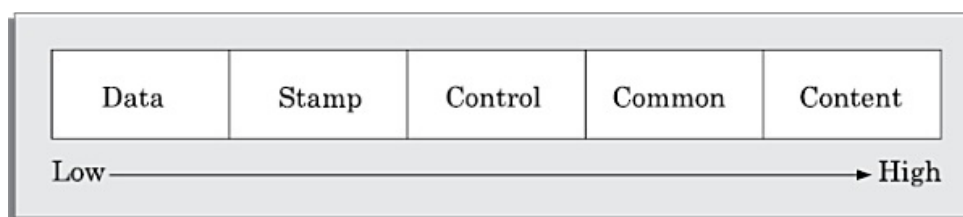


Figure 5.5: Classification of coupling.

Data coupling: Two modules are data coupled, if they communicate using an elementary data item that is passed as a parameter between the two, e.g. an integer, a float, a character, etc. This data item should be problem related and not used for control purposes.

Stamp coupling: Two modules are stamp coupled, if they communicate using a composite data item such as a record in PASCAL or a structure in C.

Control coupling: Control coupling exists between two modules, if data from one module is used to direct the order of instruction execution in another. An example of control coupling is a flag set in one module and

tested in another module.

Common coupling: Two modules are common coupled, if they share some global data items.

Content coupling: Content coupling exists between two modules, if they share code. That is, a jump from one module into the code of another module can occur. Modern high-level programming languages such as C do not support such jumps across modules.

The different types of coupling are shown schematically in Figure 5.5. The degree of coupling increases from data coupling to content coupling. High coupling among modules not only makes a design solution difficult to understand and maintain, but it also increases development effort and also makes it very difficult to get these modules developed independently by different team members.

5.4 LAYERED ARRANGEMENT OF MODULES

The control hierarchy represents the organisation of program components in terms of their call relationships. Thus we can say that the control hierarchy of a design is determined by the order in which different modules call each other. Many different types of notations have been used to represent the control hierarchy. The most common notation is a tree-like diagram known as a structure chart which we shall study in some detail in Chapter 6. However, other notations such as Warnier-Orr [1977, 1981] or Jackson diagrams [1975] may also be used. Since, Warnier-Orr and Jackson's notations are not widely used nowadays, we shall discuss only structure charts in this text.

In a layered design solution, the modules are arranged into several layers based on their call relationships. A module is allowed to call only the modules that are at a lower layer. That is, a module should not call a module that is either at a higher layer or even in the same layer. Figure 5.6(a) shows a layered design, whereas Figure 5.6(b) shows a design that is not layered. Observe that the design solution shown in Figure 5.6(b), is actually not layered since all the modules can be considered to be in the same layer. In the following, we state the significance of a layered design and subsequently we explain it.

An important characteristic feature of a good design solution is layering of the modules. A layered design achieves control abstraction and is easier to understand and debug.

In a layered design, the top-most module in the hierarchy can be considered as a manager that only invokes the services of the lower level module to discharge its responsibility. The modules at the intermediate layers offer services to their higher layer by invoking the services of the lower layer modules and also by doing some work themselves to a limited extent. The modules at the lowest layer are the worker modules. These do not invoke services of any module and entirely carry out their responsibilities by themselves.

Understanding a layered design is easier since to understand one module, one would have to at best consider the modules at the lower layers (that is, the modules whose services it invokes). Besides, in a layered design errors are isolated, since an error in one module can affect only the higher layer modules. As a result, in case of any failure of a module, only the modules at the lower levels need to be investigated for the possible error. Thus, debugging time reduces significantly in a layered design. On the other hand, if the different modules call each other arbitrarily, then this situation would correspond to modules arranged in a single layer. Locating an error would be both difficult and time consuming. This is because, once a failure is observed, the cause of failure (i.e. error) can potentially be in any module, and all modules would have to be investigated for the error. In the following, we discuss some important concepts and terminologies associated with a layered design:

Superordinate and subordinate modules: In a control hierarchy, a module that controls another module is said to be superordinate to it. Conversely, a module controlled by another module is said to be subordinate to the controller.

Visibility: A module B is said to be visible to another module A, if A directly calls B. Thus, only the immediately lower layer modules are said to be visible to a module.

Control abstraction: In a layered design, a module should only invoke the functions of the modules that are in the layer immediately below it. In other words, the modules at the higher layers, should not be visible (that is, abstracted out) to the modules at the lower layers. This is referred to as control abstraction.

Depth and width: Depth and width of a control hierarchy provide an indication of the number of layers and the overall span of control respectively. For the design of Figure 5.6(a), the depth is 3 and width is also 3.

Fan-out: Fan-out is a measure of the number of modules that are directly controlled by a given module. In Figure 5.6(a), the fan-out of the module M1 is 3. A design in which the modules have very high fan-out numbers is not a good design. The reason for this is that a very high fan-out is an indication that the module lacks cohesion. A module having a large fan-out (greater than 7) is likely to implement several different functions and not just a single cohesive function.

Fan-in: Fan-in indicates the number of modules that directly invoke a given module. High fan-in represents code reuse and is in general, desirable in a good design. In Figure 5.6(a), the fan-in of the module M1 is 0, that of M2 is 1, and that of M5 is 2.

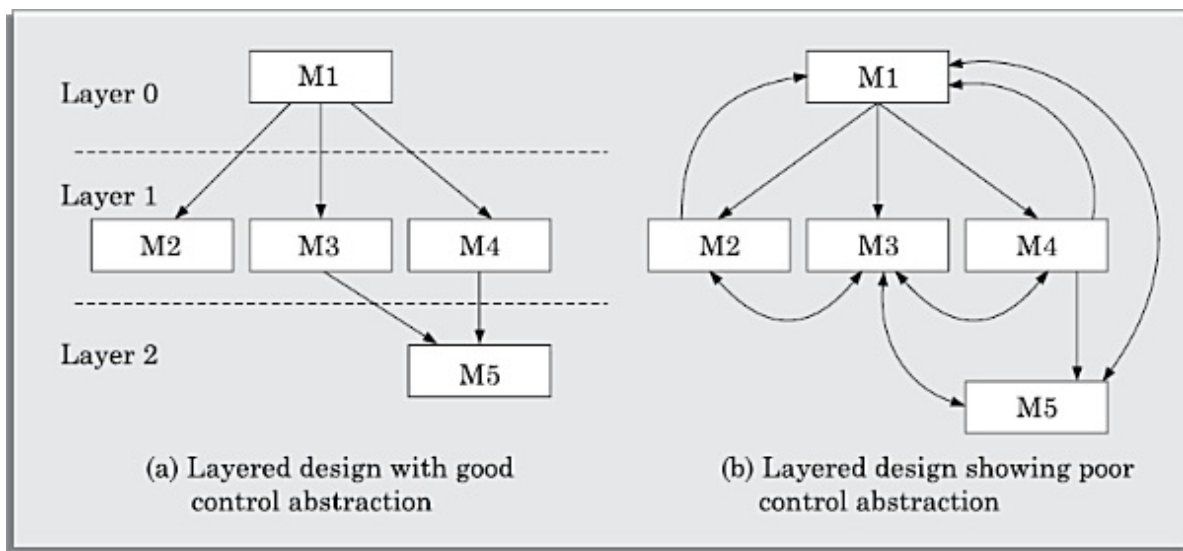


Figure 5.6: Examples of good and poor control abstraction.

5.5 APPROACHES TO SOFTWARE DESIGN

There are two fundamentally different approaches to software design that are in use today— function-oriented design, and object-oriented design. Though these two design approaches are radically different, they are complementary rather than competing techniques. The object-oriented approach is a relatively newer technology and is still evolving. For development of large programs, the object-oriented approach is becoming increasingly popular due to certain advantages that it offers. On the other hand, function-oriented designing is a mature technology and has a large following. Salient features of these two approaches are discussed in subsections 5.5.1 and 5.5.2 respectively.

5.5.1 Function-oriented Design

The following are the salient features of the function-oriented design approach:

Top-down decomposition: A system, to start with, is viewed as a black box that provides certain services (also known as high-level functions) to the users of the system.

In top-down decomposition, starting at a high-level view of the system, each high-level function is successively refined into more detailed functions.

For example, consider a function create-new-library member which essentially creates the record for a new member, assigns a unique membership number to him, and prints a bill towards his membership charge. This high-level function may be refined into the following subfunctions:

- assign-membership-number
- create-member-record
- print-bill

Each of these subfunctions may be split into more detailed subfunctions and so on.

Centralised system state: The system state can be defined as the values of certain data items that determine the response of the system to a user action or external event. For example, the set of books (i.e. whether borrowed by different users or available for issue) determines the state of a library automation system. Such data in procedural programs usually have global scope and are shared by many modules.

The system state is centralised and shared among different functions.

For example, in the library management system, several functions such as the following share data such as member-records for reference and updation:

- create-new-member
- delete-member
- update-member-record

A large number of function-oriented design approaches have been proposed in the past. A

few of the well-established function-oriented design approaches are as following:

- Structured design by Constantine and Yourdon, [1979]
- Jackson's structured design by Jackson [1975]
- Warnier-Orr methodology [1977, 1981]

- Step-wise refinement by Wirth [1971]
- Hatley and Pirbhai's Methodology [1987]

5.5.2 Object-oriented Design

In the object-oriented design (OOD) approach, a system is viewed as being made up of a collection of objects (i.e. entities). Each object is associated with a set of functions that are called its methods. Each object contains its own data and is responsible for managing it. The data internal to an object cannot be accessed directly by other objects and only through invocation of the methods of the object. The system state is decentralised since there is no globally shared data in the system and data is stored in each object. For example, in a library automation software, each library member may be a separate object with its own data and functions to operate on the stored data. The methods defined for one object cannot directly refer to or change the data of other objects.

The object-oriented design paradigm makes extensive use of the principles of abstraction and decomposition as explained below. Objects decompose a system into functionally independent modules. Objects can also be considered as instances of abstract data types (ADTs). The ADT concept did not originate from the object-oriented approach. In fact, ADT concept was extensively used in the ADA programming language introduced in the 1970s. ADT is an important concept that forms an important pillar of object-orientation. Let us now discuss the important concepts behind an ADT. There are, in fact, three important concepts associated with an ADT—data abstraction, data structure, data type. We discuss these in the following subsection:

Data abstraction: The principle of data abstraction implies that how data is exactly stored is abstracted away. This means that any entity external to the object (that is, an instance of an ADT) would have no knowledge about how data is exactly stored, organised, and manipulated inside the object. The entities external to the object can access the data internal to an object only by calling certain well-defined methods supported by the object. Consider an ADT such as a stack. The data of a stack object may internally be stored in an array, a linearly linked list, or a bidirectional linked list. The external entities have no knowledge of this and can access data of a stack object only through the supported operations such as push and pop.

Data structure: A data structure is constructed from a collection of primitive data items. Just as a civil engineer builds a large civil engineering structure using primitive building materials such as bricks, iron rods, and cement; a programmer can construct a data structure as an organised collection of primitive data items such as integer, floating point numbers, characters, etc.

Data type: A type is a programming language terminology that refers to anything that can be instantiated. For example, int, float, char etc., are the basic data types supported by C programming language. Thus, we can say that ADTs are user defined data types.

In object-orientation, classes are ADTs. But, what is the advantage of developing an application using ADTs? Let us examine the three main advantages of using ADTs in programs:

- The data of objects are encapsulated within the methods. The encapsulation principle is also known as data hiding. The encapsulation principle requires that data can be accessed and manipulated only through the methods supported by the object and not directly. This localises the errors. The reason for this is as follows. No program element is allowed to change a data, except through invocation of one of the methods. So, any error can easily be traced to the code segment changing the value. That is, the method that changes a data item, making it erroneous can be easily identified.
- An ADT-based design displays high cohesion and low coupling. Therefore, object-oriented designs are highly modular.
- Since the principle of abstraction is used, it makes the design solution easily understandable and helps to manage complexity.

Similar objects constitute a class. In other words, each object is a member of some class. Classes may inherit features from a super class. Conceptually, objects communicate by message passing. Objects have their own internal data. Thus an object may exist in different states depending the values of the internal data. In different states, an object may behave differently. We shall elaborate these concepts in Chapter 7 and subsequently we discuss an object-oriented design methodology in Chapter 8.

Object-oriented v e r s u s function-oriented design approaches

The following are some of the important differences between the

function-oriented and object-oriented design:

- Unlike function-oriented design methods in OOD, the basic abstraction is not the services available to the users of the system such as issue-book, display-book-details, find-issued-books, etc., but real-world entities such as member, book, book-register, etc. For example in OOD, an employee pay-roll software is not developed by designing functions such as update-employee-record, get-employee-address, etc., but by designing objects such as employees, departments, etc.
- In OOD, state information exists in the form of data distributed among several objects of the system. In contrast, in a procedural design, the state information is available in a centralised shared data store. For example, while developing an employee pay-roll system, the employee data such as the names of the employees, their code numbers, basic salaries, etc., are usually implemented as global data in a traditional programming system; whereas in an object-oriented design, these data are distributed among different employee objects of the system. Objects communicate by message passing. Therefore, one object may discover the state information of another object by sending a message to it. Of course, somewhere or other the real-world functions must be implemented.
- Function-oriented techniques group functions together if, as a group, they constitute a higher level function. On the other hand, object-oriented techniques group functions together on the basis of the data they operate on.

To illustrate the differences between the object-oriented and the function-oriented design approaches, let us consider an example—that of an automated fire-alarm system for a large building.

Automated fire-alarm system—customer requirements

The owner of a large multi-storied building wants to have a computerised fire alarm system designed, developed, and installed in his building. Smoke detectors and fire alarms would be placed in each room of the building. The fire alarm system would monitor the status of these smoke detectors. Whenever a fire condition is reported by any of the smoke detectors, the fire alarm system should determine the location at which the fire has been sensed and then sound the alarms

only in the neighbouring locations. The fire alarm system should also flash an alarm message on the computer console. Fire fighting personnel would man the console round the clock. After a fire condition has been successfully handled, the fire alarm system should support resetting the alarms by the fire fighting personnel.

Function-oriented approach: In this approach, the different high-level functions are first identified, and then the data structures are designed.

```
/* Global data (system state) accessible by various functions */
    BOOL  detector_status[MAX_ROOMS];
    int    detector_locs[MAX_ROOMS];
    BOOL  alarm-status[MAX_ROOMS]; /* alarm activated when status is set */
    int    alarm_locs[MAX_ROOMS]; /* room number where alarm is located */
    int    neighbour-alarms[MAX_ROOMS][10]; /* each detector has at most */
                                           /* 10 neighbouring alarm locations */

    int    sprinkler[MAX_ROOMS];
```

The functions which operate on the system state are:

```
interrogate_detectors();
get_detector_location();
determine_neighbour_alarm();
determine_neighbour_sprinkler();
ring_alarm();
activate_sprinkler();
reset_alarm();
reset_sprinkler();
report_fire_location();
```

Object-oriented approach: In the object-oriented approach, the different classes of objects are identified. Subsequently, the methods and data for each object are identified. Finally, an appropriate number of instances of each class is created.

```
class detector
attributes: status, location, neighbours
operations: create, sense-status, get-location,
            find-neighbours

class alarm
attributes: location, status
operations: create, ring-alarm, get_location, reset-
alarm

class sprinkler
```

```
attributes: location, status
operations: create, activate-sprinkler, get_location,
reset-sprinkler
```

We can now compare the function-oriented and the object-oriented approaches based on the two examples discussed above, and easily observe the following main differences:

- In a function-oriented program, the system state (data) is centralised and several functions access and modify this central data. In case of an object-oriented program, the state information (data) is distributed among various objects.
- In the object-oriented design, data is private in different objects and these are not available to the other objects for direct access and modification.
- The basic unit of designing an object-oriented program is objects, whereas it is functions and modules in procedural designing. Objects appear as nouns in the problem description; whereas functions appear as verbs.

At this point, we must emphasise that it is not necessary that an object-oriented design be implemented by using an object-oriented language only. However, an object-oriented language such as C++ and Java support the definition of all the basic mechanisms of class, inheritance, objects, methods, etc. and also support all key object-oriented concepts that we have just discussed. Thus, an object-oriented language facilitates the implementation of an OOD. However, an OOD can as well be implemented using a conventional procedural languages—though it may require more effort to implement an OOD using a procedural language as compared to the effort required for implementing the same design using an object-oriented language. In fact, the older C++ compilers were essentially pre-processors that translated C++ code into C code.

Even though object-oriented and function-oriented techniques are remarkably different approaches to software design, yet one does not replace the other; but they complement each other in some sense. For example, usually one applies the top-down function oriented techniques to design the internal methods of a class, once the classes are identified. In this case, though outwardly the system appears to have been developed in an object-oriented fashion, but inside each class there may be a small hierarchy of

functions designed in a top-down manner.

SUMMARY

- software design is typically carried out through two stages—high-level design, and detailed design. During high-level design, the important components (modules) of the system and their interactions are identified. During detailed design, the algorithms and data structures are identified.
- We discussed that there is no unique design solution to any problem and one needs to choose the best solution among a set of candidate solutions. To be able to achieve this, we identified the factors based on which a superior design can be distinguished from a inferior design.
- We discussed that understandability of a design is a major criterion determining the goodness of a design. We Characterised the understandability of design in terms of satisfactory usage of decomposition and abstraction principles. Later, we Characterised these in terms of cohesion, coupling, layering, control abstraction, fan-in, fan-out, etc.
- We identified two fundamentally different approaches to software design—function- oriented design and object-oriented design. We discussed the essential philosophy governing these two approaches and argued that these two approaches to software design are not really competing approaches but complementary approaches.

EXERCISES

1. Choose the correct option
 - (a) The extent of data interchange between two modules is called:
 - (i) Coupling
 - (ii) Cohesion
 - (iii) Structure
 - (iv) Union
 - (b) Which of the following type of cohesion can be considered as the strongest cohesion:
 - (i) Logical
 - (ii) Coincidental
 - (iii) Temporal
 - (iv) Functional

- (c) The modules in a good software design should have which of the following characteristics:
- (i) High cohesion, low coupling
 - (ii) Low cohesion, high coupling
 - (iii) Low cohesion, low coupling
 - (iv) High cohesion, high coupling
2. Do you agree with the following assertion? A design solution that is difficult to understand would lead to increased development and maintenance cost. Give reasonings for your answer.
3. What do you mean by the terms cohesion and coupling in the context of software design?
How are these concepts useful in arriving at a good design of a system?
4. What do you mean by a modular design? How can you determine whether a given design is modular or not?
5. Enumerate the different types of cohesion that a module in a design might exhibit. Give examples of each.
6. Enumerate the different types of coupling that might exist between two modules. Give examples of each.
7. Is it true that whenever you increase the cohesion of your design, coupling in the design would automatically decrease? Justify your answer by using suitable examples.
8. What according to you are the characteristics of a good software design?
9. What do you understand by the term functional independence in the context of software design? What are the advantages of functional independence? How can functional independence in a software design be achieved?
10. Explain how the principles of abstraction and decomposition are used to arrive at a good design.
11. What do you understand by information hiding in the context of software design?
Explain why a design approach based on the information hiding principle is likely to lead to a reusable and maintainable design. Illustrate your answer with a suitable example.
12. In the context of software development, distinguish between analysis and design with respect to intention, methodology, and the documentation technique used.
13. State whether the following statements are **TRUE** or **FALSE**. Give reasons for your answer.

- (a) The essence of any good function-oriented design technique is to map the functions performing similar activities into a module.
 - (b) Traditional procedural design is carried out top-down whereas object-oriented design is normally carried out bottom-up.
 - (c) Common coupling is the worst type of coupling between two modules.
 - (d) Temporal cohesion is the worst type of cohesion that a module can have.
 - (e) The extent to which two modules depend on each other determines the cohesion of the two modules.
14. Compare relative advantages of the object-oriented and function-oriented approaches to software design.
 15. Name a few well-established function-oriented software design techniques.
 16. Explain the important causes of and remedies for high coupling between two software modules.
 17. What problems are likely to arise if two modules have high coupling?
 18. What problems are likely to occur if a module has low cohesion?
 19. Distinguish between high-level and detailed designs. What documents should be produced on completion of high-level and detailed designs respectively?
 20. What is meant by the term cohesion in the context of software design? Is it true that in a good design, the modules should have low cohesion? Why?
 21. What is meant by the term coupling in the context of software design? Is it true that in a good design, the modules should have low coupling? Why?
 22. What do you mean by modular design? What are the different factors that affect the modularity of a design? How can you assess the modularity of a design? What are the advantages of a modular design?
 23. How would you improve a software design that displays very low cohesion and high coupling?
 24. Explain how the overall cohesion and coupling of a design would be impacted if all modules of the design are merged into a single module.
 25. Explain what do you understand by the terms decomposition and abstraction in the context of software design. How are these two principles used in arriving good procedural designs?
 26. What is an ADT? What advantages accrue when a software design

technique is based on ADTs? Explain why the object paradigm is said to be based on ADTs.

27. By using suitable examples explain the following terms associated with an abstract data type (ADT)—data abstraction, data structure, data type.
28. What do you understand by the term top-down decomposition in the context of function- oriented design? Explain your answer using a suitable example.
29. What do you understand by a layered software design? What are the advantages of a layered design? Explain your answer by using suitable examples.
30. What is the principal difference between the software design methodologies based on functional abstraction and those based on data abstraction? Name at least one popular design technique based on each of these two software design paradigms.
31. What are the main advantages of using an object-oriented approach to software design over a function-oriented approach?
32. Point out three important differences between the function oriented and the object- oriented approaches to software design. Corroborate your answer through suitable examples.
33. Identify the criteria that you would use to decide which one of two alternate function- oriented design solutions to a problem is superior.
34. Explain the main differences between architectural design, high-level-design, and detailed design of a software system.

Chapter

6

FUNCTION-ORIENTED SOFTWARE DESIGN

Function-oriented design techniques were proposed nearly four decades ago. These techniques are at the present time still very popular and are currently being used in many software development organisations. These techniques, to start with, view a system as a black-box that provides a set of services to the users of the software. These services provided by a software (e.g., issue book, serach book, etc., for a Library Automation Software to its users are also known as the high-level functions supported by the software. During the design process, these high-level functions are successively decomposed into more detailed functions.

The term top-down decomposition is often used to denote the successive decomposition of a set of high-level functions into more detailed functions.

After top-down decomposition has been carried out, the different identified functions are mapped to modules and a module structure is created. This module structure would possess all the characteristics of a good design identified in the last chapter.

In this text, we shall not focus on any specific design methodology. Instead, we shall discuss a methodology that has the essential features of several important function-oriented design methodologies. Such an approach shall enable us to easily assimilate any specific design methodology in the future whenever the need arises. Learning a specific methodology may become necessary for you later, since different software development houses follow different methodologies. After all, the different procedural design techniques can be considered as sister techniques that have only minor differences with respect to the methodology and notations. We shall call the design technique discussed in this text a structured analysis/structured design (SA/SD)

methodology. This technique draws heavily from the design methodologies proposed by the following authors:

- DeMarco and Yourdon [1978]
- Constantine and Yourdon [1979]
- Gane and Sarson [1979]
- Hatley and Pirbhai [1987]

The SA/SD technique can be used to perform the high-level design of a software. The details of SA/SD technique are discussed further.

6.1 OVERVIEW OF SA/SD METHODOLOGY

As the name itself implies, SA/SD methodology involves carrying out two distinct activities:

- Structured analysis (SA)
- Structured design (SD)

The roles of structured analysis (SA) and structured design (SD) have been shown schematically in Figure 6.1. Observe the following from the figure:

- During structured analysis, the SRS document is transformed into a data flow diagram (DFD) model.
- During structured design, the DFD model is transformed into a structure chart.

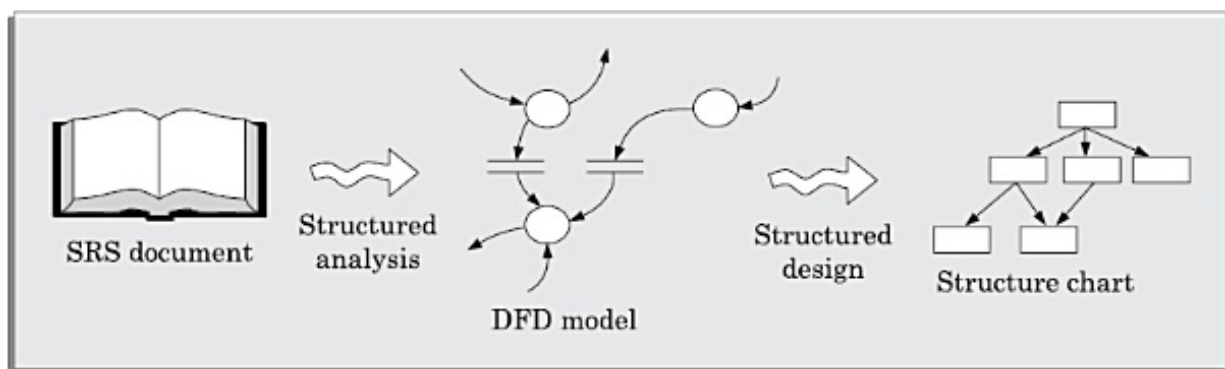


Figure 6.1: Structured analysis and structured design methodology.

As shown in Figure 6.1, the structured analysis activity transforms the SRS document into a graphic model called the DFD model. During structured analysis, functional decomposition of the system is achieved. That is, each

function that the system needs to perform is analysed and hierarchically decomposed into more detailed functions. On the other hand, during structured design, all functions identified during structured analysis are mapped to a module structure. This module structure is also called the high-level design or the software architecture for the given problem. This is represented using a structure chart.

The high-level design stage is normally followed by a detailed design stage. During the detailed design stage, the algorithms and data structures for the individual modules are designed. The detailed design can directly be implemented as a working system using a conventional programming language.

It is important to understand that the purpose of structured analysis is to capture the detailed structure of the system as perceived by the user, whereas the purpose of structured design is to define the structure of the solution that is suitable for implementation in some programming language.

The results of structured analysis can therefore, be easily understood by the user. In fact, the different functions and data in structured analysis are named using the user's terminology. The user can therefore even review the results of the structured analysis to ensure that it captures all his requirements.

In the following section, we first discuss how to carry out structured analysis to construct the DFD model. Subsequently, we discuss how the DFD model can be transformed into structured design.

6.2 STRUCTURED ANALYSIS

We have already mentioned that during structured analysis, the major processing tasks (high-level functions) of the system are analysed, and the data flow among these processing tasks are represented graphically. Significant contributions to the development of the structured analysis techniques have been made by Gane and Sarson [1979], and DeMarco and Yourdon [1978]. The structured analysis technique is based on the following underlying principles:

- Top-down decomposition approach.
- Application of divide and conquer principle. Through this each high-level function is independently decomposed into detailed functions.
- Graphical representation of the analysis results using data flow

diagrams (DFDs).

DFD representation of a problem, as we shall see shortly, is very easy to construct. Though extremely simple, it is a very powerful tool to tackle the complexity of industry standard problems.

A DFD is a hierarchical graphical model of a system that shows the different processing activities or functions that the system performs and the data interchange among those functions.

Please note that a DFD model only represents the data flow aspects and does not show the sequence of execution of the different functions and the conditions based on which a function may or may not be executed. In fact, it completely ignores aspects such as control flow, the specific algorithms used by the functions, etc. In the DFD terminology, each function is called a process or a bubble. It is useful to consider each function as a processing station (or process) that consumes some input data and produces some output data.

DFD is an elegant modelling technique that can be used not only to represent the results of structured analysis of a software problem, but also useful for several other applications such as showing the flow of documents or items in an organisation. Recall that in Chapter 1 we had given an example (see Figure 1.10) to illustrate how a DFD can be used to represent the processing activities and flow of material in an automated car assembling plant. We now elaborate how a DFD model can be constructed.

6.2.1 Data Flow Diagrams (DFDs)

The DFD (also known as the bubble chart) is a simple graphical formalism that can be used to represent a system in terms of the input data to the system, various processing carried out on those data, and the output data generated by the system. The main reason why the DFD technique is so popular is probably because of the fact that DFD is a very simple formalism— it is simple to understand and use. A DFD model uses a very limited number of primitive symbols (shown in Figure 6.2) to represent the functions performed by a system and the data flow among these functions.

Starting with a set of high-level functions that a system performs, a DFD model represents the subfunctions performed by the functions using a hierarchy of diagrams. We had pointed out while discussing the principle of abstraction in Section 1.3.2 that any hierarchical representation is an

effective means to tackle complexity. Human mind is such that it can easily understand any hierarchical model of a system—because in a hierarchical model, starting with a very abstract model of a system, various details of the system are slowly introduced through different levels of the hierarchy. The DFD technique is also based on a very simple set of intuitive concepts and rules. We now elaborate the different concepts associated with building a DFD model of a system.

Primitive symbols used for constructing DFDs

There are essentially five different types of symbols used for constructing DFDs. These primitive symbols are depicted in Figure 6.2. The meaning of these symbols are explained as follows:

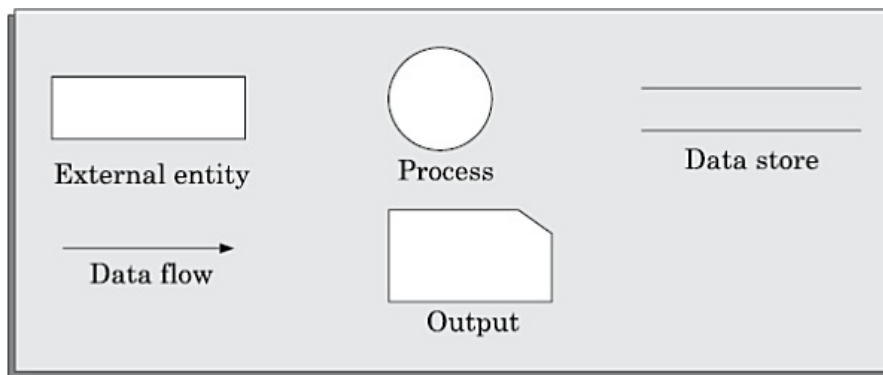


Figure 6.2: Symbols used for designing DFDs.

Function symbol: A function is represented using a circle. This symbol is called a process or a bubble. Bubbles are annotated with the names of the corresponding functions (see Figure 6.3).

External entity symbol: An external entity such as a librarian, a library member, etc. is represented by a rectangle. The external entities are essentially those physical entities external to the software system which interact with the system by inputting data to the system or by consuming the data produced by the system. In addition to the human users, the external entity symbols can be used to represent external hardware and software such as another application software that would interact with the software being modelled.

Data flow symbol: A directed arc (or an arrow) is used as a data flow symbol. A data flow symbol represents the data flow occurring between two processes or between an external entity and a process in the direction of the data flow arrow. Data flow symbols are usually annotated with the corresponding data names. For example the DFD in Figure 6.3(a) shows three data flows—the

data item number flowing from the process read-number to validate-number, data-item flowing into read-number, and valid-number flowing out of validate-number.

Data store symbol: A data store is represented using two parallel lines. It represents a logical file. That is, a data store symbol can represent either a data structure or a physical file on disk. Each data store is connected to a process by means of a data flow symbol. The direction of the data flow arrow shows whether data is being read from or written into a data store. An arrow flowing in or out of a data store implicitly represents the entire data of the data store and hence arrows connecting to a data store need not be annotated with the name of the corresponding data items. As an example of a data store, number is a data store in Figure 6.3(b).

Output symbol: The output symbol is as shown in Figure 6.2. The output symbol is used when a hard copy is produced.

The notations that we are following in this text are closer to the Yourdon's notations than to the other notations. You may sometimes find notations in other books that are slightly different than those discussed here. For example, the data store may look like a box with one end open. That is because, they may be following notations such as those of Gane and Sarson [1979].

Important concepts associated with constructing DFD models

Before we discuss how to construct the DFD model of a system, let us discuss some important concepts associated with DFDs:

Synchronous and asynchronous operations

If two bubbles are directly connected by a data flow arrow, then they are synchronous. This means that they operate at the same speed. An example of such an arrangement is shown in Figure 6.3(a). Here, the `validate-number` bubble can start processing only after the `read-number` bubble has supplied data to it; and the `read-number` bubble has to wait until the `validate-number` bubble has consumed its data.

However, if two bubbles are connected through a data store, as in Figure 6.3(b) then the speed of operation of the bubbles are independent. This statement can be explained using the following reasoning. The data produced by a producer bubble gets stored in the data store. It is therefore possible that the producer bubble stores several pieces of data items, even before the

consumer bubble consumes any of them.

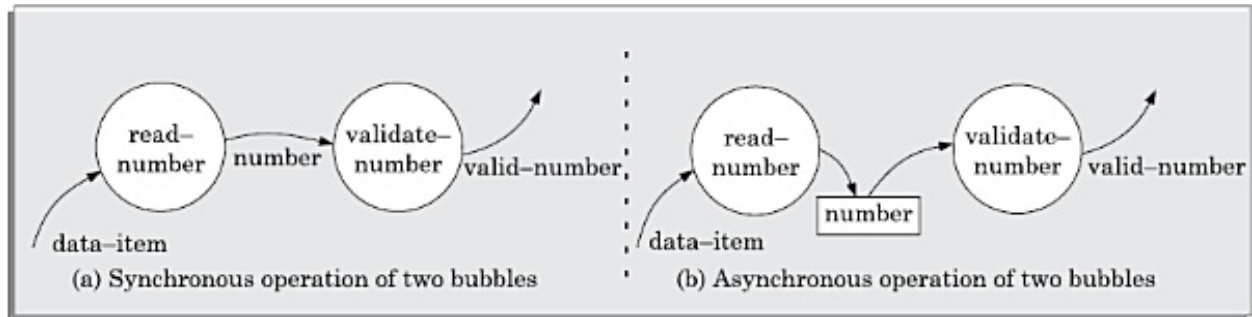


Figure 6.3: Synchronous and asynchronous data flow.

Data dictionary

Every DFD model of a system must be accompanied by a data dictionary. A data dictionary lists all data items that appear in a DFD model. The data items listed include all data flows and the contents of all data stores appearing on all the DFDs in a DFD model. Please remember that the DFD model of a system typically consists of several DFDs, viz., level 0 DFD, level 1 DFD, level 2 DFDs, etc., as shown in Figure 6.4 discussed in new subsection. However, a single data dictionary should capture all the data appearing in all the DFDs constituting the DFD model of a system.

A data dictionary lists the purpose of all data items and the definition of all composite data items in terms of their component data items.

For example, a data dictionary entry may represent that the data grossPay consists of the components regularPay and overtimePay.

$$grossPay = regularPay + overtimePay$$

For the smallest units of data items, the data dictionary simply lists their name and their type. Composite data items are expressed in terms of the component data items using certain operators. The operators using which a composite data item can be expressed in terms of its component data items are discussed subsequently.

The dictionary plays a very important role in any software development process, especially for the following reasons:

- A data dictionary provides a standard terminology for all relevant data for use by the developers working in a project. A consistent vocabulary for data items is very important, since in large projects different developers of the project have a tendency to use different terms to refer to the same data, which unnecessarily causes confusion.

- The data dictionary helps the developers to determine the definition of different data structures in terms of their component elements while implementing the design.
- The data dictionary helps to perform impact analysis. That is, it is possible to determine the effect of some data on various processing activities and vice versa. Such impact analysis is especially useful when one wants to check the impact of changing an input value type, or a bug in some functionality, etc.

For large systems, the data dictionary can become extremely complex and voluminous. Even moderate-sized projects can have thousands of entries in the data dictionary. It becomes extremely difficult to maintain a voluminous dictionary manually. Computer-aided software engineering (CASE) tools come handy to overcome this problem. Most CASE tools usually capture the data items appearing in a DFD as the DFD is drawn, and automatically generate the data dictionary. As a result, the designers do not have to spend almost any effort in creating the data dictionary. These CASE tools also support some query language facility to query about the definition and usage of data items. For example, queries may be formulated to determine which data item affects which processes, or a process affects which data items, or the definition and usage of specific data items, etc. Query handling is facilitated by storing the data dictionary in a relational database management system (RDBMS).

Data definition

Composite data items can be defined in terms of primitive data items using the following data definition operators.

- $+$: denotes composition of two data items, e.g. $a+b$ represents data a and b .
- $[,]$: represents selection, i.e. any one of the data items listed inside the square bracket can occur. For example, $[a,b]$ represents either a occurs or b occurs.
- $()$: the contents inside the bracket represent optional data which may or may not appear.
 $a+(b)$ represents either a or $a+b$ occurs.
- $\{\}$: represents iterative data definition, e.g. $\{name\}5$ represents five name data.
 $\{name\}^*$ represents zero or more instances of name data.
- $=$: represents equivalence, e.g. $a=b+c$ means that a is a composite data item

comprising of both b and c.

/* */: Anything appearing within /* and */ is considered as comment.

6.3 DEVELOPING THE DFD MODEL OF A SYSTEM

A DFD model of a system graphically represents how each input data is transformed to its corresponding output data through a hierarchy of DFDs.

The DFD model of a problem consists of many of DFDs and a single data dictionary.

The DFD model of a system is constructed by using a hierarchy of DFDs (see Figure 6.4). The top level DFD is called the level 0 DFD or the context diagram. This is the most abstract (simplest) representation of the system (highest level). It is the easiest to draw and understand. At each successive lower level DFDs, more and more details are gradually introduced. To develop a higher-level DFD model, processes are decomposed into their subprocesses and the data flow among these subprocesses are identified.

To develop the data flow model of a system, first the most abstract representation (highest level) of the problem is to be worked out. Subsequently, the lower level DFDs are developed. Level 0 and Level 1 consist of only one DFD each. Level 2 may contain up to 7 separate DFDs, and level 3 up to 49 DFDs, and so on. However, there is only a single data dictionary for the entire DFD model. All the data names appearing in all DFDs are populated in the data dictionary and the data dictionary contains the definitions of all the data items.

6.3.1 Context Diagram

The context diagram is the most abstract (highest level) data flow representation of a system. It represents the entire system as a single bubble. The bubble in the context diagram is annotated with the name of the software system being developed (usually a noun). This is the only bubble in a DFD model, where a noun is used for naming the bubble. The bubbles at all other levels are annotated with verbs according to the main function performed by the bubble. This is expected since the purpose of the context diagram is to capture the context of the system rather than its functionality. As an example of a context diagram, consider the context diagram a software developed to automate the book keeping activities of a supermarket (see Figure 6.10). The context diagram has been labelled as 'Supermarket software'.

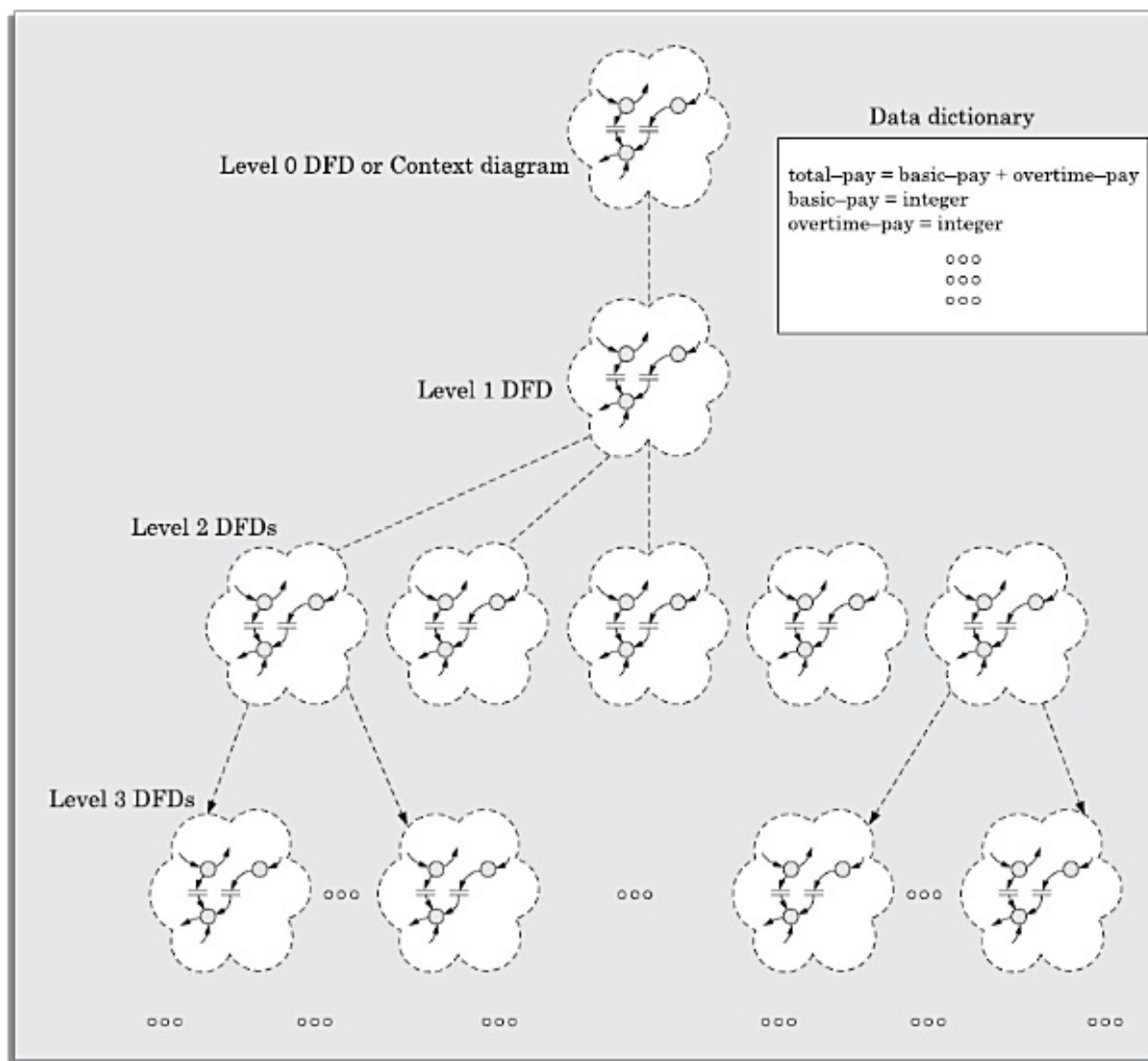


Figure 6.4: DFD model of a system consists of a hierarchy of DFDs and a single data dictionary.

The context diagram establishes the context in which the system operates; that is, who are the users, what data do they input to the system, and what data they received by the system.

The name context diagram of the level 0 DFD is justified because it represents the context in which the system would exist; that is, the external entities who would interact with the system and the specific data items that they would be supplying the system and the data items they would be receiving from the system. The various external entities with which the system interacts and the data flow occurring between the system and the external entities are represented. The data input to the system and the data output from the system are represented as incoming and outgoing arrows. These data flow arrows should be annotated with the corresponding data

names.

To develop the context diagram of the system, we have to analyse the SRS document to identify the different types of users who would be using the system and the kinds of data they would be inputting to the system and the data they would be receiving from the system. Here, the term users of the system also includes any external systems which supply data to or receive data from the system.

6.3.2 Level 1 DFD

The level 1 DFD usually contains three to seven bubbles. That is, the system is represented as performing three to seven important functions. To develop the level 1 DFD, examine the high-level functional requirements in the SRS document. If there are three to seven high-level functional requirements, then each of these can be directly represented as a bubble in the level 1 DFD. Next, examine the input data to these functions and the data output by these functions as documented in the SRS document and represent them appropriately in the diagram.

What if a system has more than seven high-level requirements identified in the SRS document? In this case, some of the related requirements have to be combined and represented as a single bubble in the level 1 DFD. These can be split appropriately in the lower DFD levels. If a system has less than three high-level functional requirements, then some of the high-level requirements need to be split into their subfunctions so that we have roughly about five to seven bubbles represented on the diagram. We illustrate construction of level 1 DFDs in Examples 6.1 to 6.4.

Decomposition

Each bubble in the DFD represents a function performed by the system. The bubbles are decomposed into subfunctions at the successive levels of the DFD model. Decomposition of a bubble is also known as factoring or exploding a bubble. Each bubble at any level of DFD is usually decomposed to anything three to seven bubbles. A few bubbles at any level make that level superfluous. For example, if a bubble is decomposed to just one bubble or two bubbles, then this decomposition becomes trivial and redundant. On the other hand, too many bubbles (i.e. more than seven bubbles) at any level of a DFD makes the DFD model hard to understand. Decomposition of a bubble should be carried

on until a level is reached at which the function of the bubble can be described using a simple algorithm.

We can now describe how to go about developing the DFD model of a system more systematically.

1. Construction of context diagram: Examine the SRS document to determine:

- Different high-level functions that the system needs to perform.
- Data input to every high-level function.
- Data output from every high-level function.
- Interactions (data flow) among the identified high-level functions.

Represent these aspects of the high-level functions in a diagrammatic form. This would form the top-level data flow diagram (DFD), usually called the DFD 0.

Construction of level 1 diagram: Examine the high-level functions described in the SRS document. If there are three to seven high-level requirements in the SRS document, then represent each of the high-level function in the form of a bubble. If there are more than seven bubbles, then some of them have to be combined. If there are less than three bubbles, then some of these have to be split.

Construction of lower-level diagrams: Decompose each high-level function into its constituent subfunctions through the following set of activities:

- Identify the different subfunctions of the high-level function.
- Identify the data input to each of these subfunctions.
- Identify the data output from each of these subfunctions.
- Identify the interactions (data flow) among these subfunctions.

Represent these aspects in a diagrammatic form using a DFD.

Recursively repeat Step 3 for each subfunction until a subfunction can be represented by using a simple algorithm.

Numbering of bubbles

It is necessary to number the different bubbles occurring in the DFD. These numbers help in uniquely identifying any bubble in the DFD from its bubble number. The bubble at the context level is usually assigned the number 0 to indicate that it is the 0 level DFD. Bubbles at level 1 are numbered, 0.1, 0.2, 0.3, etc. When a bubble numbered x is

decomposed, its children bubble are numbered x.1, x.2, x.3, etc. In this numbering scheme, by looking at the number of a bubble we can unambiguously determine its level, its ancestors, and its successors.

Balancing DFDs

The DFD model of a system usually consists of many DFDs that are organised in a hierarchy. In this context, a DFD is required to be balanced with respect to the corresponding bubble of the parent DFD.

The data that flow into or out of a bubble must match the data flow at the next level of DFD. This is known as balancing a DFD.

We illustrate the concept of balancing a DFD in Figure 6.5. In the level 1 DFD, data items d1 and d3 flow out of the bubble 0.1 and the data item d2 flows into the bubble 0.1 (shown by the dotted circle). In the next level, bubble 0.1 is decomposed into three DFDs (0.1.1,0.1.2,0.1.3). The decomposition is balanced, as d1 and d3 flow out of the level 2 diagram and d2 flows in. Please note that dangling arrows (d1,d2,d3) represent the data flows into or out of a diagram.

How far to decompose?

A bubble should not be decomposed any further once a bubble is found to represent a simple set of instructions. For simple problems, decomposition up to level 1 should suffice. However, large industry standard problems may need decomposition up to level 3 or level 4. Rarely, if ever, decomposition beyond level 4 is needed.

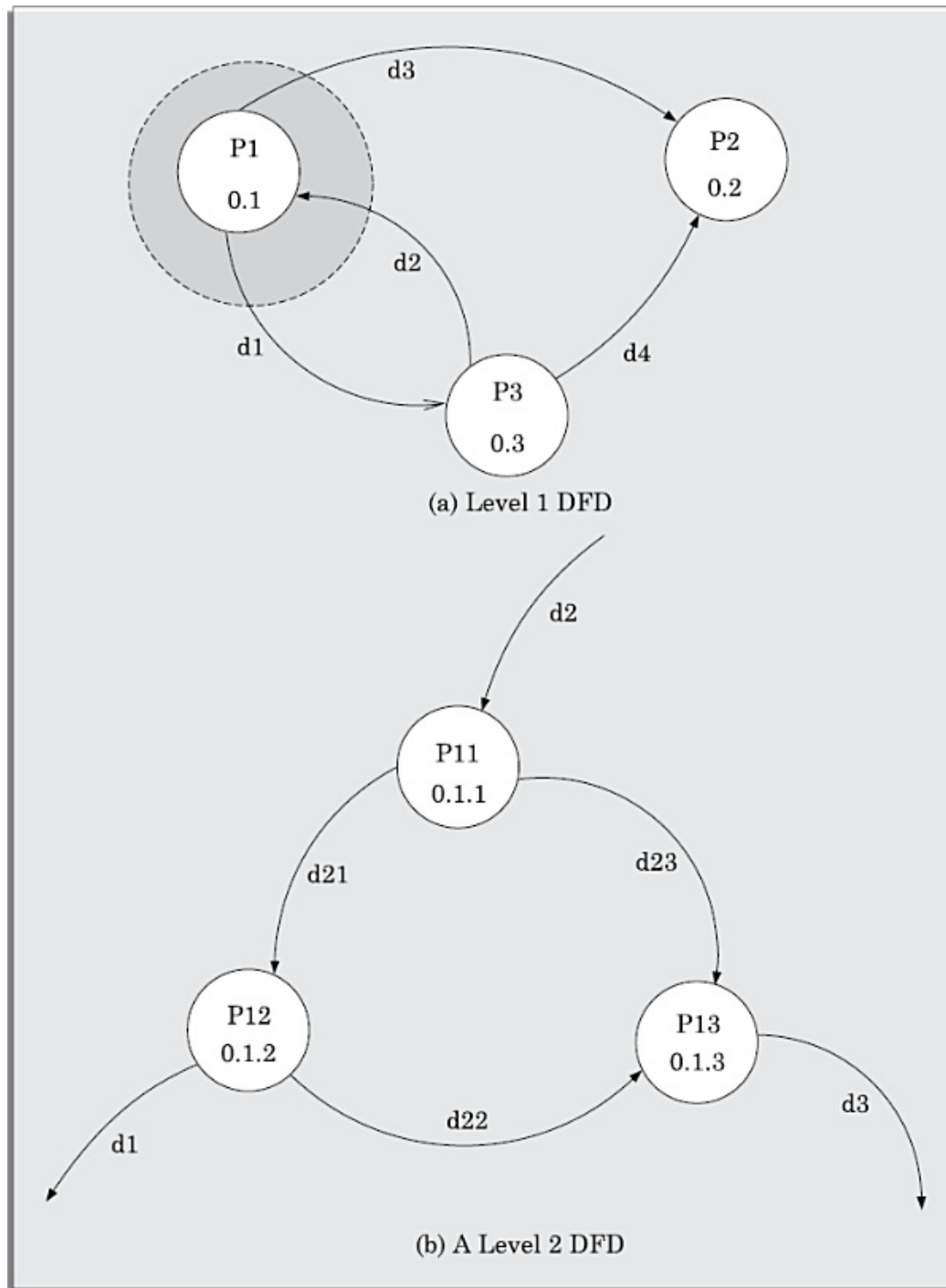


Figure 6.5: An example showing balanced decomposition.

Commonly made errors while constructing a DFD model

Although DFDs are simple to understand and draw, students and practitioners alike encounter similar types of problems while modelling software problems using DFDs. While learning from experience is a powerful thing, it is an expensive pedagogical technique in the business world. It is therefore useful to understand the different types of mistakes that beginners usually make while constructing the DFD model

of systems, so that you can consciously try to avoid them. The errors are as follows:

- Many beginners commit the mistake of drawing more than one bubble in the context diagram. Context diagram should depict the system as a single bubble.
- Many beginners create DFD models in which external entities appearing at all levels of DFDs. All external entities interacting with the system should be represented only in the context diagram. The external entities should not appear in the DFDs at any other level.
- It is a common oversight to have either too few or too many bubbles in a DFD. Only three to seven bubbles per diagram should be allowed. This also means that each bubble in a DFD should be decomposed three to seven bubbles in the next level.
- Many beginners leave the DFDs at the different levels of a DFD model unbalanced.
- A common mistake committed by many beginners while developing a DFD model is attempting to represent control information in a DFD.

It is important to realise that a DFD represents only data flow, and it does not represent any control information.

The following are some illustrative mistakes of trying to represent control aspects such as:

Illustration 1. A book can be searched in the library catalog by inputting its name. If the book is available in the library, then the details of the book are displayed. If the book is not listed in the catalog, then an error message is generated. While developing the DFD model for this simple problem, many beginners commit the mistake of drawing an arrow (as shown in Figure 6.6) to indicate that the error function is invoked after the search book. But, this is a control information and should not be shown on the DFD.

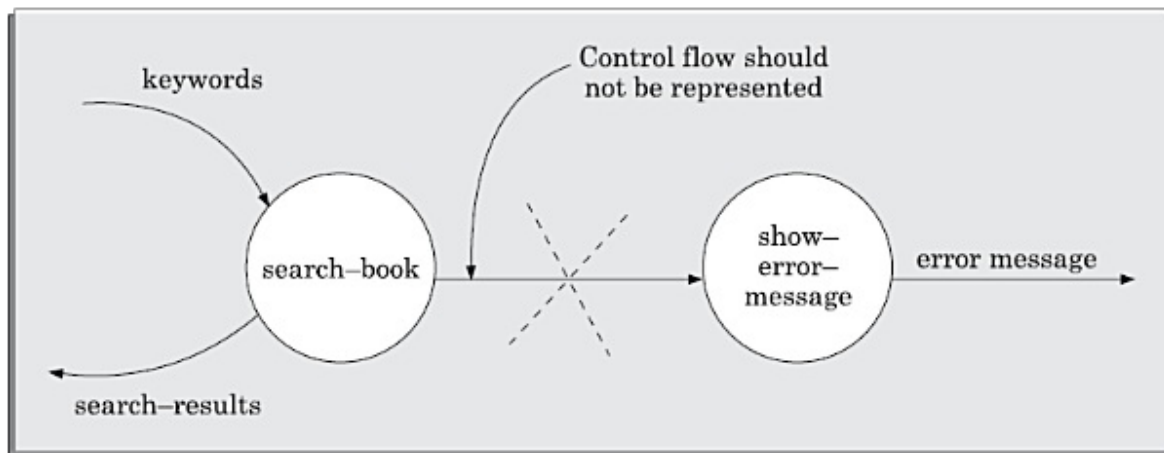


Figure 6.6: It is incorrect to show control information on a DFD.

Illustration 2. Another type of error occurs when one tries to represent when or in what order different functions (processes) are invoked. A DFD similarly should not represent the conditions under which different functions are invoked.

Illustration 3. If a bubble A invokes either the bubble B or the bubble C depending upon some conditions, we need only to represent the data that flows between bubbles A and B or bubbles A and C and not the conditions depending on which the two modules are invoked.

- A data flow arrow should not connect two data stores or even a data store with an external entity. Thus, data cannot flow from a data store to another data store or to an external entity without any intervening processing. As a result, a data store should be connected only to bubbles through data flow arrows.
- All the functionalities of the system must be captured by the DFD model. No function of the system specified in the SRS document of the system should be overlooked.
- Only those functions of the system specified in the SRS document should be represented. That is, the designer should not assume functionality of the system not specified by the SRS document and then try to represent them in the DFD.
- Incomplete data dictionary and data dictionary showing incorrect composition of data items are other frequently committed mistakes.
- The data and function names must be intuitive. Some students and even practicing developers use meaningless symbolic data names such as a,b,c, etc. Such names hinder understanding the DFD model.

- Novices usually clutter their DFDs with too many data flow arrow. It becomes difficult to understand a DFD if any bubble is associated with more than seven data flows. When there are too many data flowing in or out of a DFD, it is better to combine these data items into a high-level data item. Figure 6.7 shows an example concerning how a DFD can be simplified by combining several data flows into a single high-level data flow.

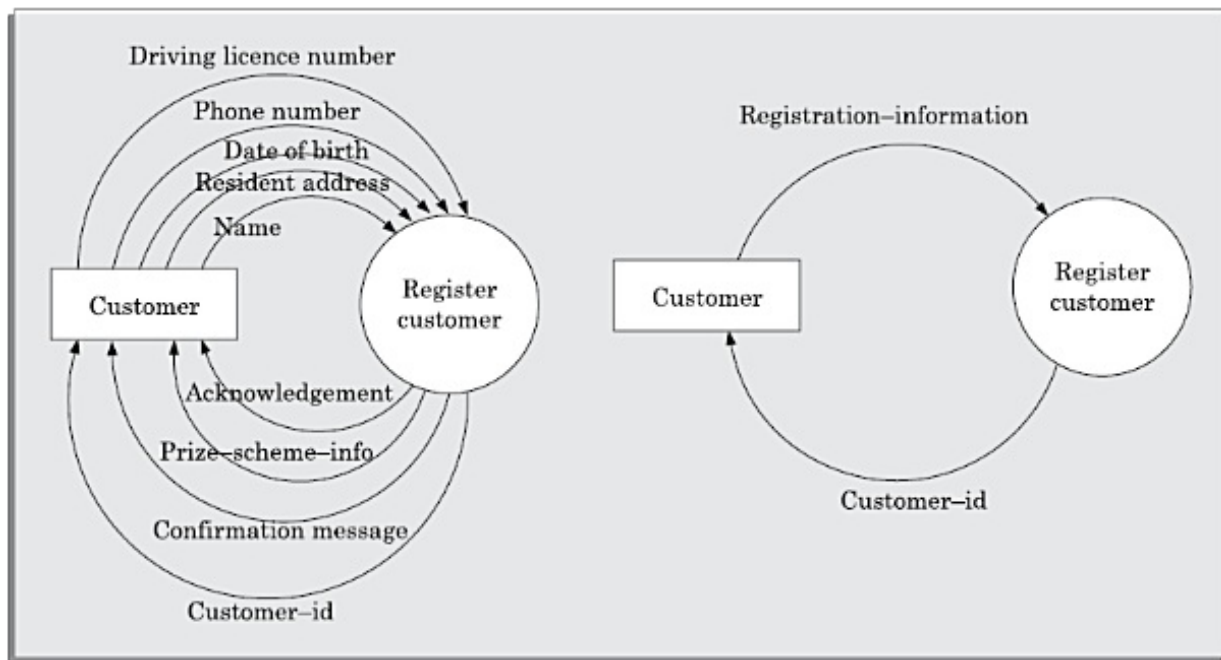


Figure 6.7: Illustration of how to avoid data cluttering.

We now illustrate the structured analysis technique through a few examples.

Example 6.1 (RMS Calculating Software) A software system called RMS calculating software would read three integral numbers from the user in the range of -1000 and $+1000$ and would determine the root mean square (RMS) of the three input numbers and display it.

In this example, the context diagram is simple to draw. The system accepts three integers from the user and returns the result to him. This has been shown in Figure 6.8(a). To draw the level 1 DFD, from a cursory analysis of the problem description, we can see that there are four basic functions that the system needs to perform—accept the input numbers from the user, validate the numbers, calculate the root mean square of the input numbers and, then display the result. After representing these four functions in Figure 6.8(b), we observe that the calculation of root mean square essentially consists of the functions—calculate the squares of the input numbers,

calculate the mean, and finally calculate the root. This decomposition is shown in the level 2 DFD in Figure 6.8(c).

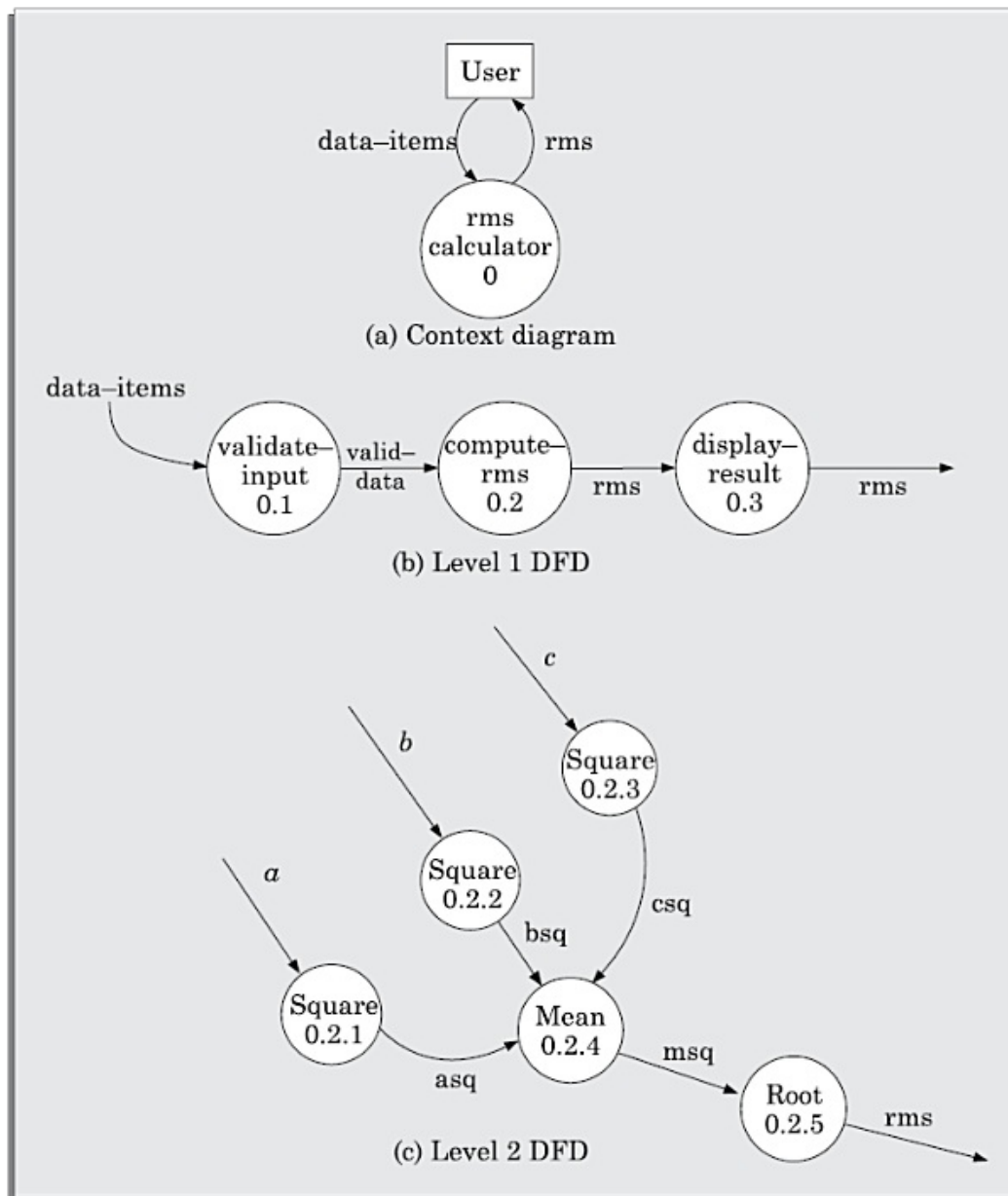


Figure 6.8: Context diagram, level 1, and level 2 DFDs for Example 6.1.

Data dictionary for the DFD model of Example 6.1

data-items: {integer}3
rms: float
valid-data:data-items
a: integer
b: integer
c: integer
asq: integer

bsq: integer
csq: integer
msq: integer

Example 6.1 is an almost trivial example and is only meant to illustrate the basic methodology. Now, let us perform the structured analysis for a more complex problem.

Example 6.2 (Tic-Tac-Toe Computer Game) Tic-tac-toe is a computer game in which a human player and the computer make alternate moves on a 3×3 square. A move consists of marking a previously unmarked square. The player who is first to place three consecutive marks along a straight line (i.e., along a row, column, or diagonal) on the square wins. As soon as either of the human player or the computer wins, a message congratulating the winner should be displayed. If neither player manages to get three consecutive marks along a straight line, and all the squares on the board are filled up, then the game is drawn. The computer always tries to win a game.

The context diagram and the level 1 DFD are shown in Figure 6.9.

Data dictionary for the DFD model of Example 6.2

move: integer /* number between 1 to 9 */
display: game+result
game: board
board: {integer}9
result: ["computer won", "human won", "drawn"]

Example 6.3 (Supermarket Prize Scheme) A super market needs to develop a software that would help it to automate a scheme that it plans to introduce to encourage regular customers. In this scheme, a customer would have first register by supplying his/her residence address, telephone number, and the driving license number. Each customer who registers for this scheme is assigned a unique customer number (CN) by the computer. A customer can present his CN to the check out staff when he makes any purchase. In this case, the value of his purchase is credited against his CN. At the end of each year, the supermarket intends to award surprise gifts to 10 customers who make the highest total purchase over the year. Also, it intends to award a 22 caret gold coin to every customer whose purchase exceeded Rs. 10,000. The entries against the CN are reset on the last day of every year after the prize winners' lists are generated.

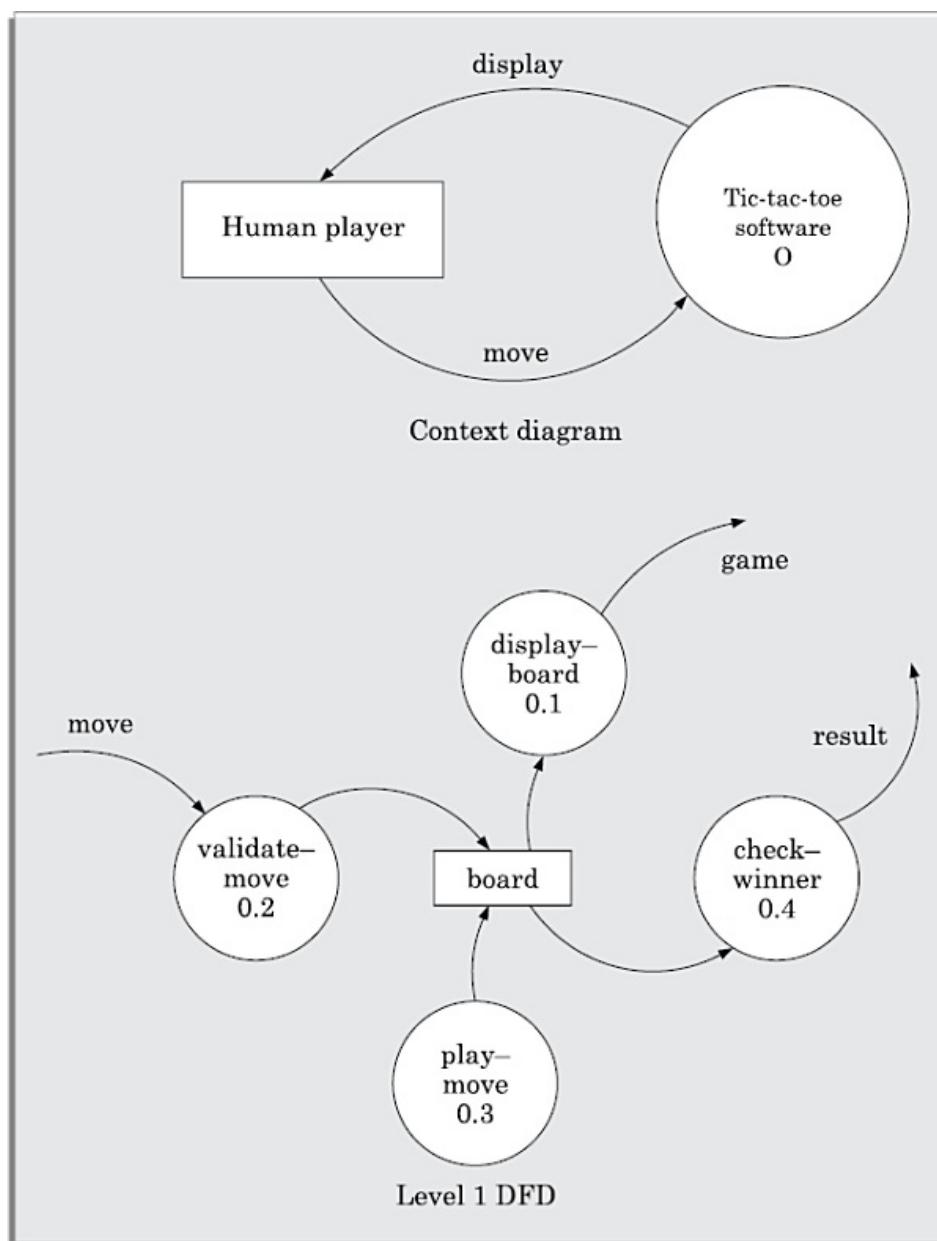


Figure 6.9: Context diagram and level 1 DFDs for Example 6.2.

The context diagram for the supermarket prize scheme problem of Example 6.3 is shown in Figure 6.10. The level 1 DFD in Figure 6.11. The level 2 DFD in Figure 6.12.

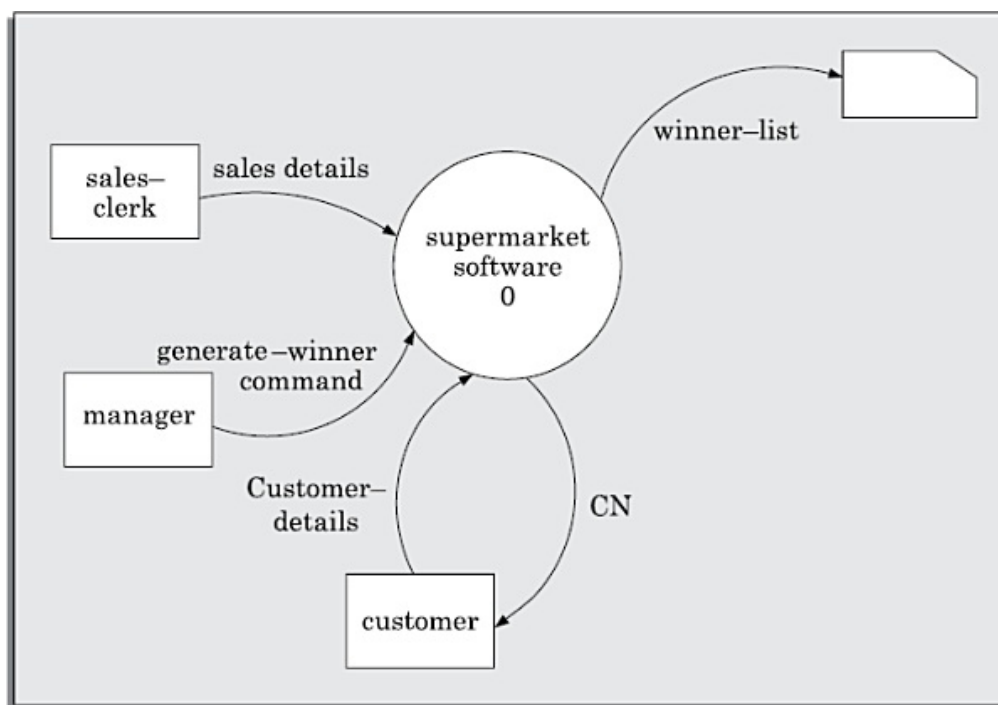


Figure 6.10: Context diagram for Example 6.3.

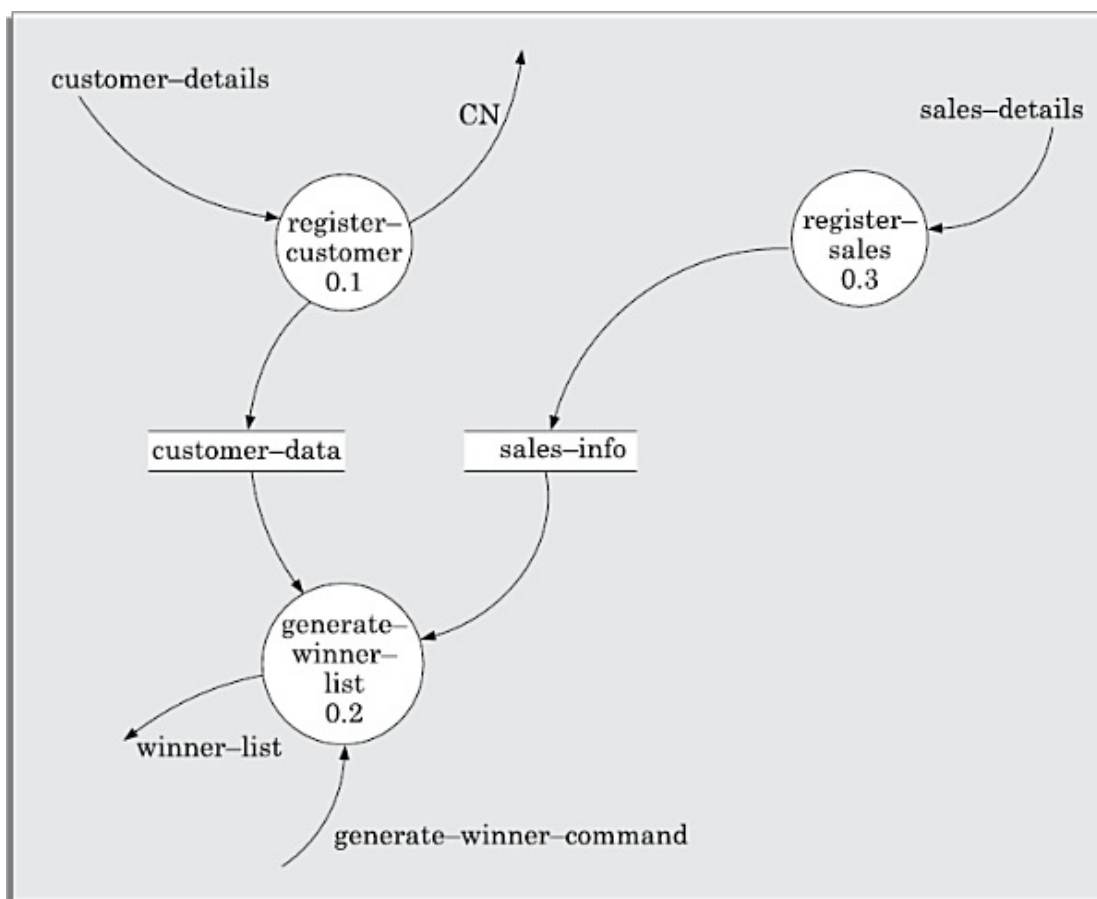


Figure 6.11: Level 1 diagram for Example 6.3.

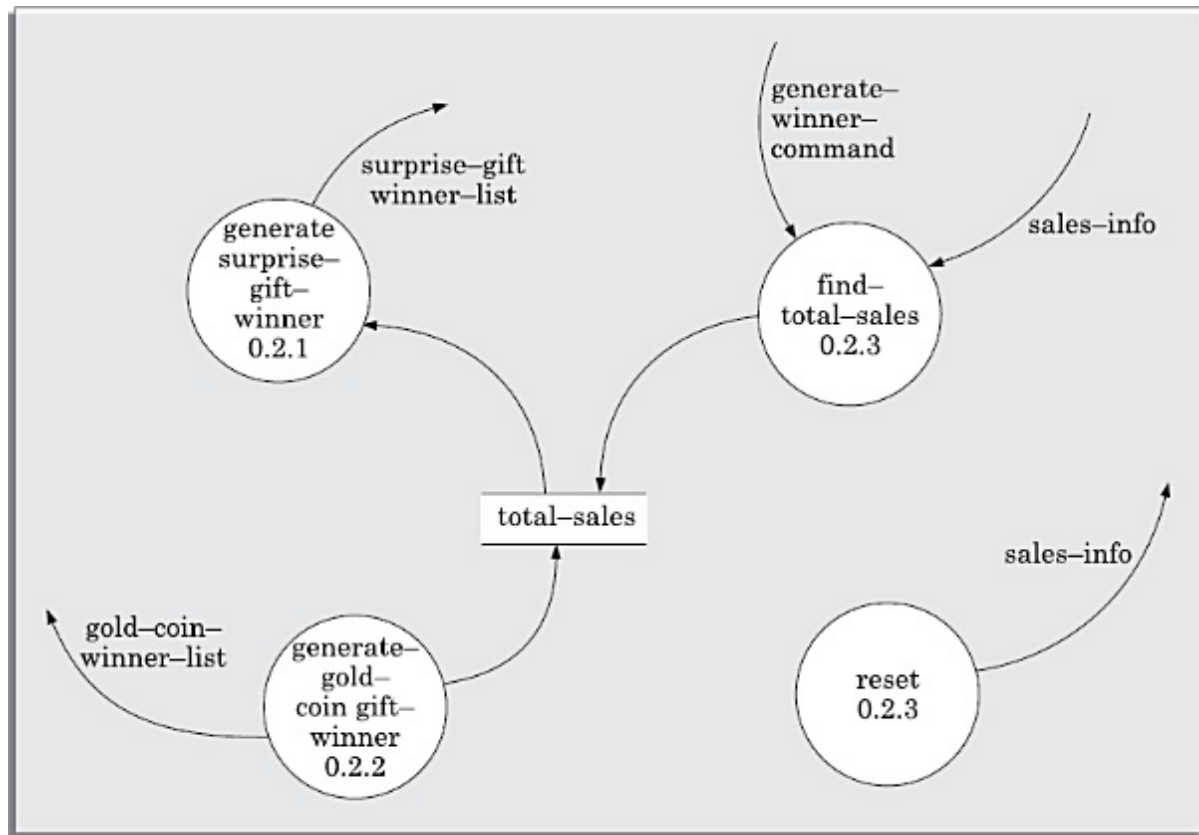


Figure 6.12: Level 2 diagram for Example 6.3.

Data dictionary for the DFD model of Example 6.3

address: name+house#+street#+city+pin

sales-details: {item+amount}* + CN

CN: integer

customer-data: {address+CN}*

sales-info: {sales-details}*

winner-list: surprise-gift-winner-list + gold-coin-winner-list

surprise-gift-winner-list: {address+CN}*

gold-coin-winner-list: {address+CN}*

gen-winner-command: command

total-sales: {CN+integer}*

Observations: The following observations can be made from the Example 6.3.

1. The fact that the customer is issued a manually prepared customer identity card or that the customer hands over the identity card each time he makes a purchase has not been shown in the DFD. This is because these are item transfers occurring outside the computer.
2. The data generate-winner-list in a way represents control information

(that is, command to the software) and no real data. We have included it in the DFD because it simplifies the structured design process as we shall realize after we practise solving a few problems. We could have also as well done without the generate-winner-list data, but this could have a bit complicated the design.

3. Observe in Figure 6.11 that we have two separate stores for the customer data and sales data. Should we have combined them into a single data store? The answer is—No, we should not. If we had combined them into a single data store, the structured design that would be carried out based on this model would become complicated. Customer data and sales data have very different characteristics. For example, customer data once created, does not change. On the other hand, the sales data changes frequently and also the sales data is reset at the end of a year, whereas the customer data is not.

Example 6.4 (Trading-house Automation System (TAS)) A trading house wants us to develop a computerized system that would automate various book-keeping activities associated with its business. The following are the salient features of the system to be developed:

- The trading house has a set of regular customers. The customers place orders with it for various kinds of commodities. The trading house maintains the names and addresses of its regular customers. Each of these regular customers should be assigned a unique customer identification number (CIN) by the computer. The customers quote their CIN on every order they place.
- Once order is placed, as per current practice, the accounts department of the trading house first checks the credit-worthiness of the customer. The credit-worthiness of the customer is determined by analysing the history of his payments to different bills sent to him in the past. After automation, this task has be done by the computer.
- If a customer is not credit-worthy, his orders are not processed any further and an appropriate order rejection message is generated for the customer.
- If a customer is credit-worthy, the items that he has ordered are checked against the list of items that the trading house deals with. The items in the order which the trading house does not deal with, are not processed any further and an appropriate apology message for the

customer for these items is generated.

- The items in the customer's order that the trading house deals with are checked for availability in the inventory. If the items are available in the inventory in desired quantity, then:
 - A bill is with the forwarding address of the customer is printed.
 - A material issue slip is printed. The customer can produce this material issue slip at the store house and take delivery of the items.
 - Inventory data is adjusted to reflect the sale to the customer.
- If any of the ordered items are not available in the inventory in sufficient quantity to satisfy the order, then these out-of-stock items along with the quantity ordered by the customer and the CIN are stored in a "pending-order" file for further processing to be carried out when the purchase department issues the "generate indent" command.
- The purchase department should be allowed to periodically issue commands to generate indents. When a command to generate indents is issued, the system should examine the "pending-order" file to determine the orders that are pending and determine the total quantity required for each of the items. It should find out the addresses of the vendors who supply these items by examining a file containing vendor details and then should print out indents to these vendors.
- The system should also answer managerial queries regarding the statistics of different items sold over any given period of time and the corresponding quantity sold and the price realised.

The context diagram for the trading house automation problem is shown in Figure 6.13. The level 1 DFD in Figure 6.14.

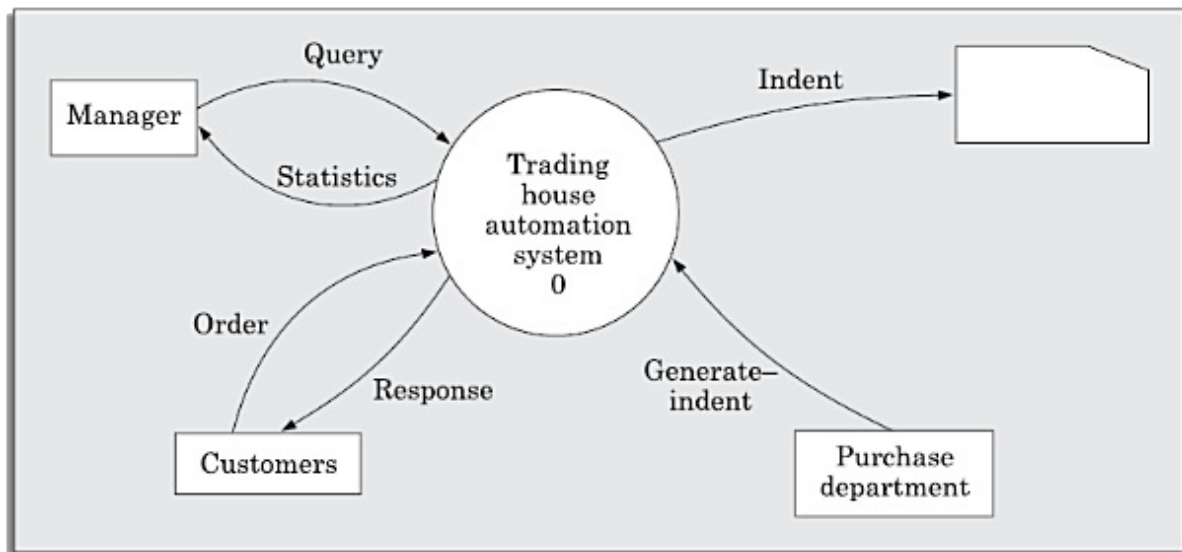


Figure 6.13: Context diagram for Example 6.4.

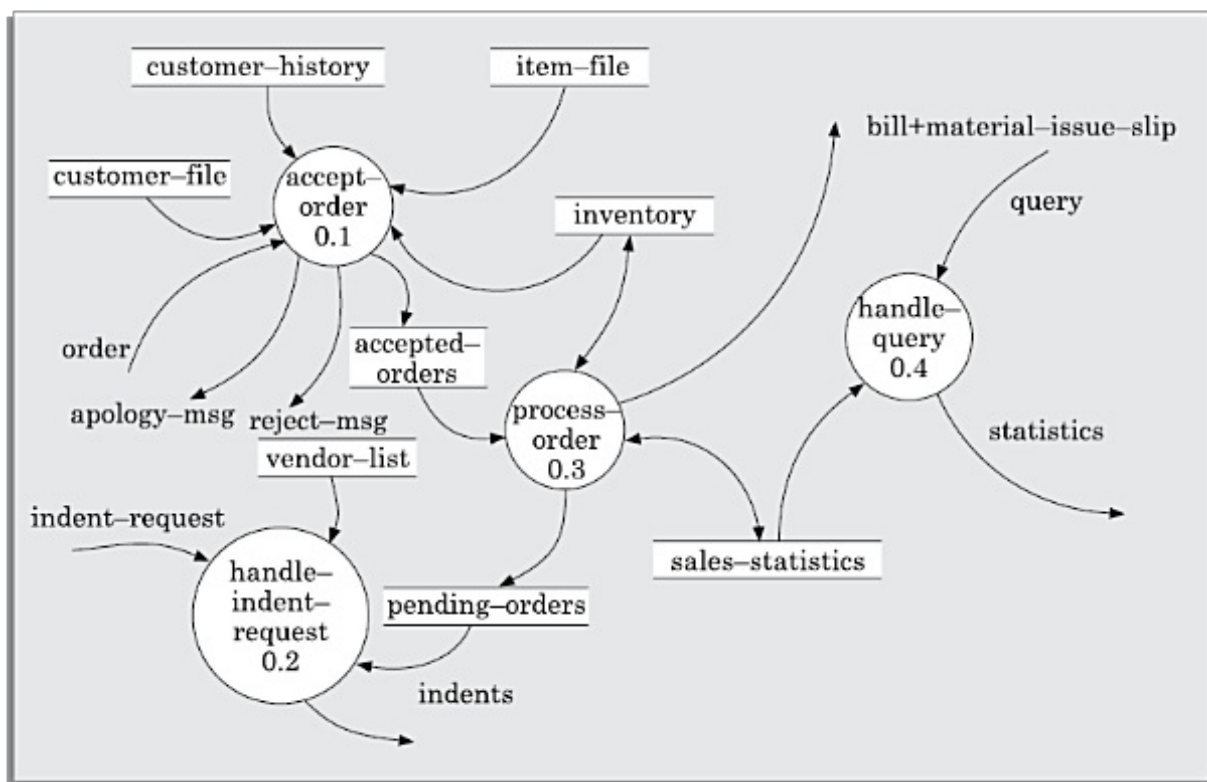


Figure 6.14: Level 1 DFD for Example 6.4.

Data dictionary for the DFD model of Example 6.4

response: [bill + material-issue-slip, reject-msg, apology-msg]

query: period /* query from manager regarding sales statistics*/

period: [date+date, month, year, day]

date: year + month + day year: integer

month: integer day: integer customer-id: integer

order: customer-id + {items + quantity}* + order#

accepted-order: order /* ordered items available in inventory */
reject-msg: order + message /* rejection message */
pending-orders: customer-id + order# + {items+quantity}*
customer-address: name+house#+street#+city+pin
name: string
house#: string
street#: string
city: string
pin: integer
customer-id: integer
customer-file: {customer-address}* + customer-id
bill: {item + quantity + price}* + total-amount + customer-address +
order#
material-issue-slip: message + item + quantity + customer-address
message: string
statistics: {item + quantity + price }*
sales-statistics: {statistics}* + date
quantity: integer
order#: integer /* unique order number generated by the program */
price: integer
total-amount: integer
generate-indent: command
indent: {item+quantity}* + vendor-address
indents: {indent}*
vendor-address: customer-address
vendor-list: {vendor-address}*
item-file: {item}*
item: string
indent-request: command

Observations: The following observations can be made from Example 6.4.

1. In a DFD, if two data stores deal with different types of data, e.g. one type of data is invariant with time whereas another varies with time, (e.g. vendor address, and inventory data) it is a good idea to represent them as separate data stores.

If two types of data always get updated at the same time, they should be stored in a single data store. Otherwise, separate data stores should be used for them.

The inventory data changes each time supply arrives and the inventory

is updated or an item is sold, whereas the vendor data remains unchanged.

2. If we are developing the DFD model of a process which is already being manually carried out, then the names of the registers being maintained in the manual process would appear as data stores in the DFD model. For example, if TAS is currently being manually carried out, then normally there would registers corresponding to accepted orders, pending orders, vendor list, etc.
3. We can observe that DFDs enable a software developer to develop the data domain and functional domain model of the system at the same time. As the DFD is refined into greater levels of detail, the analyst performs an implicit functional decomposition. At the same time, the DFD refinement automatically results in refinement of corresponding data items.
4. The data that are maintained in physical registers in manual processing, become data stores in the DFD representation. Therefore, to determine which data should be represented as a data store, it is useful to try to imagine whether a set of data items would be maintained in a register in a manual system.

Example 6.5 (Personal Library Software) Perform structured analysis for the personal library software of Example 6.5.

The context diagram is shown in Figure 6.15.

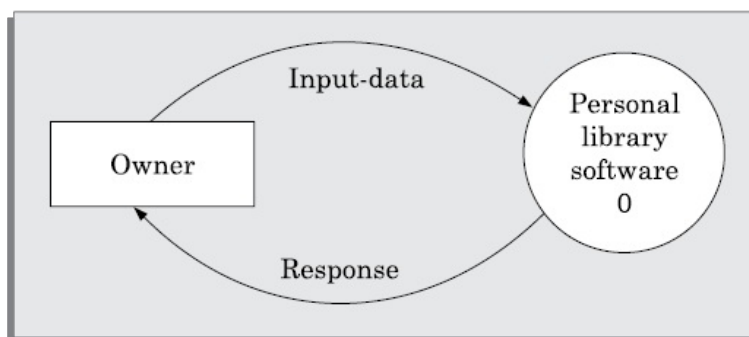
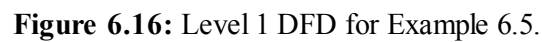
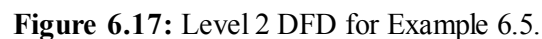


Figure 6.15: Context diagram for Example 6.5.

The level 1 DFD is shown in Figure 6.16.



Data dictionary for the DFD model of Example 6.5



```
input-data: friend-reg-data + own-book-data + stat-request + borrowed-book-data
```

response: friend-reg-conf-msg + own-book-response + stat-response + borrowed-book-response
own-book-data: query-details + own-book-details + query-outstanding-books-option + return-own book-
details + reg-own-book-data
own-book-response: query-book-response + issue-book-msg + friend-details + return-book- msg +
serial#.
borrowed-book-data: borrowed-book-details + book-return-details + display-books-option borrowed-book-
response: reg-msg + unreg-msg + borrowed-books-list
friend-reg-data: name + address + landline# + mobile#
own-book-details: friend-reg-data + book-title + data-of-issue
return-own-book-details: book-title + date-of-return
friend-details: name + address + landline# + mobile# + book-list
borrowed-book-details: book-title + borrow-date
serial#: integer

Observation: Observe that since there are more than seven functional requirements for the personal library software, related requirements have been combined to have only five bubbles in the level 1 diagram. Only level 2 DFD has been shown, since the other DFDs are trivial and need not be drawn.

Shortcomings of the DFD model

DFD models suffer from several shortcomings. The important shortcomings of DFD models are the following:

- Imprecise DFDs leave ample scope to be imprecise. In the DFD model, we judge the function performed by a bubble from its label. However, a short label may not capture the entire functionality of a bubble. For example, a bubble named find-book-position has only intuitive meaning and does not specify several things, e.g. what happens when some input information is missing or is incorrect. Further, the find-book-position bubble may not convey anything regarding what happens when the required book is missing.
- Not-well defined control aspects are not defined by a DFD. For instance, the order in which inputs are consumed and outputs are produced by a bubble is not specified. A DFD model does not specify the order in which the different bubbles are executed. Representation of such aspects is very important for modelling real-time systems.
- Decomposition: The method of carrying out decomposition to arrive at the successive levels and the ultimate level to which decomposition is carried out are highly subjective and depend on the choice and judgment of the analyst. Due to this reason, even for the same problem, several alternative DFD representations are possible. Further, many times it is not possible to say which DFD representation is

superior or preferable to another one.

- Improper data flow diagram: The data flow diagramming technique does not provide any specific guidance as to how exactly to decompose a given function into its subfunctions and we have to use subjective judgment to carry out decomposition.

6.3.3 Extending DFD Technique to Make it Applicable to Real-time Systems

In a real-time system, some of the high-level functions are associated with deadlines. Therefore, a function must not only produce correct results but also should produce them by some prespecified time. For real-time systems, execution time is an important consideration for arriving at a correct design. Therefore, explicit representation of control and event flow aspects are essential. One of the widely accepted techniques for extending the DFD technique to real-time system analysis is the Ward and Mellor technique [1985]. In the Ward and Mellor notation, a type of process that handles only control flows is introduced. These processes representing control processing are denoted using dashed bubbles. Control flows are shown using dashed lines/arrows.

Unlike Ward and Mellor, Hatley and Pirbhai [1987] show the dashed and solid representations on separate diagrams. To be able to separate the data processing and the control processing aspects, a control flow diagram (CFD) is defined. This reduces the complexity of the diagrams. In order to link the data processing and control processing diagrams, a notational reference (solid bar) to a control specification is used. The CSPEC describes the following:

- The effect of an external event or control signal.
- The processes that are invoked as a consequence of an event.

Control specifications represents the behavior of the system in two different ways:

- It contains a state transition diagram (STD). The STD is a sequential specification of behaviour.
- It contains a program activation table (PAT). The PAT is a combinatorial specification of behaviour. PAT represents invocation

sequence of bubbles in a DFD.

6.4 STRUCTURED DESIGN

The aim of structured design is to transform the results of the structured analysis (that is, the DFD model) into a structure chart. A structure chart represents the software architecture. The various modules making up the system, the module dependency (i.e. which module calls which other modules), and the parameters that are passed among the different modules. The structure chart representation can be easily implemented using some programming language. Since the main focus in a structure chart representation is on module structure of a software and the interaction among the different modules, the procedural aspects (e.g. how a particular functionality is achieved) are not represented.

The basic building blocks using which structure charts are designed are as following:

Rectangular boxes: A rectangular box represents a module. Usually, every rectangular box is annotated with the name of the module it represents.

Module invocation arrows: An arrow connecting two modules implies that during program execution control is passed from one module to the other in the direction of the connecting arrow. However, just by looking at the structure chart, we cannot say whether a modules calls another module just once or many times. Also, just by looking at the structure chart, we cannot tell the order in which the different modules are invoked.

Data flow arrows: These are small arrows appearing alongside the module invocation arrows. The data flow arrows are annotated with the corresponding data name. Data flow arrows represent the fact that the named data passes from one module to the other in the direction of the arrow.

Library modules: A library module is usually represented by a rectangle with double edges. Libraries comprise the frequently called modules. Usually, when a module is invoked by many other modules, it is made into a library module.

Selection: The diamond symbol represents the fact that one module of several modules connected with the diamond symbol is invoked depending on the outcome of the condition attached with the diamond symbol.

Repetition: A loop around the control flow arrows denotes that the respective modules are invoked repeatedly.

In any structure chart, there should be one and only one module at the top, called the root. There should be at most one control relationship between any two modules in the structure chart. This means that if module A invokes module B, module B cannot invoke module A. The main reason behind this restriction is that we can consider the different modules of a structure chart to be arranged in layers or levels. The principle of abstraction does not allow lower-level modules to be aware of the existence of the high-level modules. However, it is possible for two higher-level modules to invoke the same lower-level module. An example of a properly layered design and another of a poorly layered design are shown in Figure 6.18.

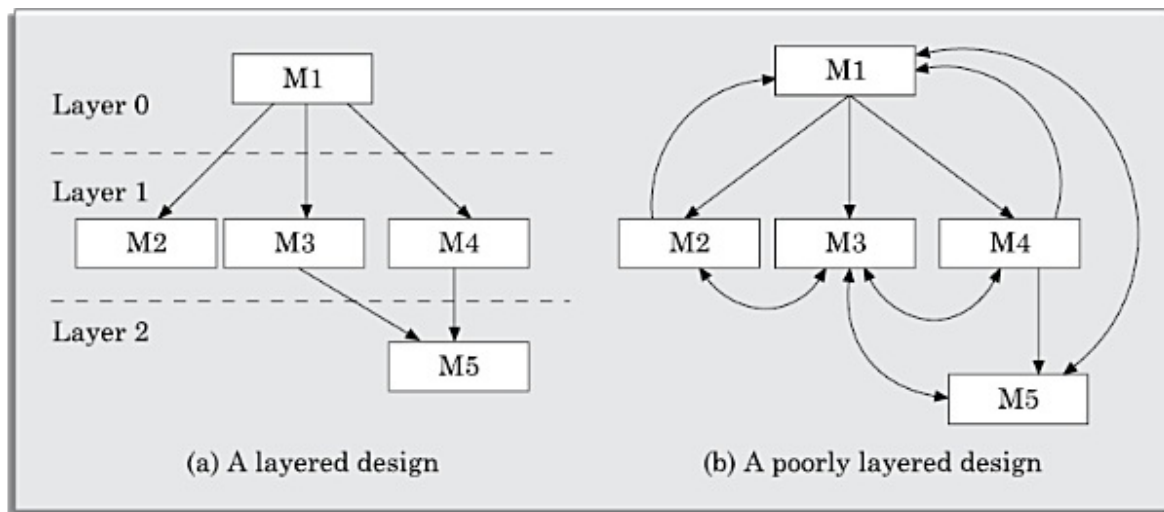


Figure 6.18: Examples of properly and poorly layered designs.

Flow chart *versus* structure chart

We are all familiar with the flow chart representation of a program. Flow chart is a convenient technique to represent the flow of control in a program. A structure chart differs from a flow chart in three principal ways:

- It is usually difficult to identify the different modules of a program from its flow chart representation.
- Data interchange among different modules is not represented in a flow chart.
- Sequential ordering of tasks that is inherent to a flow chart is suppressed in a structure chart.

6.4.1 Transformation of a DFD Model into Structure Chart

Systematic techniques are available to transform the DFD representation of a problem into a module structure represented by as a structure chart. Structured design provides two strategies to guide transformation of a DFD into a structure chart:

- Transform analysis
- Transaction analysis

Normally, one would start with the level 1 DFD, transform it into module representation using either the transform or transaction analysis and then proceed toward the lower level DFDs.

At each level of transformation, it is important to first determine whether the transform or the transaction analysis is applicable to a particular DFD.

Whether to apply transform or transaction processing?

Given a specific DFD of a model, how does one decide whether to apply transform analysis or transaction analysis? For this, one would have to examine the data input to the diagram. The data input to the diagram can be easily spotted because they are represented by dangling arrows. If all the data flow into the diagram are processed in similar ways (i.e. if all the input data flow arrows are incident on the same bubble in the DFD) then transform analysis is applicable. Otherwise, transaction analysis is applicable. Normally, transform analysis is applicable only to very simple processing.

Please recollect that the bubbles are decomposed until it represents a very simple processing that can be implemented using only a few lines of code. Therefore, transform analysis is normally applicable at the lower levels of a DFD model. Each different way in which data is processed corresponds to a separate transaction. Each transaction corresponds to a functionality that lets a user perform a meaningful piece of work using the software.

Transform analysis

Transform analysis identifies the primary functional components (modules) and the input and output data for these components. The first step in transform analysis is to divide the DFD into three types of parts:

- Input.
- Processing.
- Output.

The input portion in the DFD includes processes that transform input data from physical (e.g, character from terminal) to logical form (e.g. internal tables, lists, etc.). Each input portion is called an afferent branch.

The output portion of a DFD transforms output data from logical form to physical form. Each output portion is called an efferent branch. The remaining portion of a DFD is called central transform.

In the next step of transform analysis, the structure chart is derived by drawing one functional component each for the central transform, the afferent and efferent branches. These are drawn below a root module, which would invoke these modules.

Identifying the input and output parts requires experience and skill. One possible approach is to trace the input data until a bubble is found whose output data cannot be deduced from its inputs alone. Processes which validate input are not central transforms. Processes which sort input or filter data from it are central transforms. The first level of structure chart is produced by representing each input and output unit as a box and each central transform as a single box.

In the third step of transform analysis, the structure chart is refined by adding subfunctions required by each of the high-level functional components. Many levels of functional components may be added. This process of breaking functional components into subcomponents is called factoring. Factoring includes adding read and write modules, error-handling modules, initialisation and termination process, identifying consumer modules etc. The factoring process is continued until all bubbles in the DFD are represented in the structure chart.

Example 6.6 Draw the structure chart for the RMS software of Example 6.1.

By observing the level 1 DFD of Figure 6.8, we can identify validate-input as the afferent branch and write-output as the efferent branch. The remaining (i.e., compute-rms) as the central transform. By applying the step 2 and step 3 of transform analysis, we get the structure chart shown in Figure 6.19.

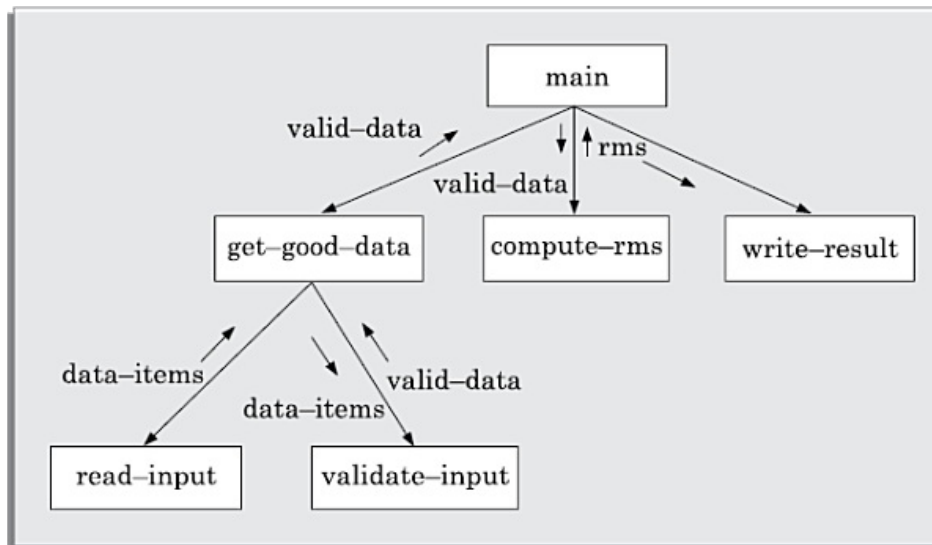


Figure 6.19: Structure chart for Example 6.6.

Example 6.7 Draw the structure chart for the tic-tac-toe software of Example 6.2.

The structure chart for the Tic-tac-toe software is shown in Figure 6.20. Observe that the check-game-status bubble, though produces some outputs, is not really responsible for converting logical data to physical data. On the other hand, it carries out the processing involving checking game status. That is the main reason, why we have considered it as a central transform and not as an efferent type of module.

Transaction analysis

Transaction analysis is an alternative to transform analysis and is useful while designing transaction processing programs. A transaction allows the user to perform some specific type of work by using the software. For example, 'issue book', 'return book', 'query book', etc., are transactions.

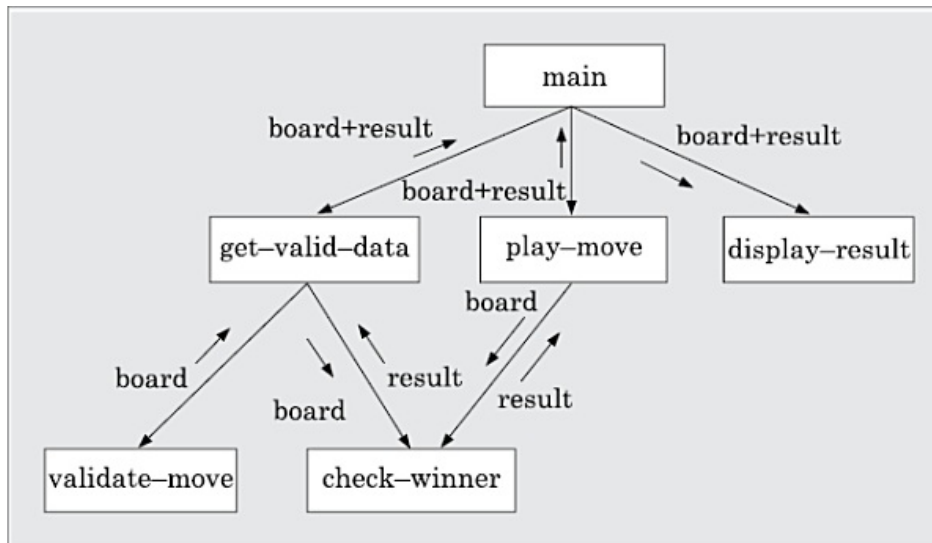


Figure 6.20: Structure chart for Example 6.7.

As in transform analysis, first all data entering into the DFD need to be identified. In a transaction-driven system, different data items may pass through different computation paths through the DFD. This is in contrast to a transform centered system where each data item entering the DFD goes through the same processing steps. Each different way in which input data is processed is a transaction. A simple way to identify a transaction is the following. Check the input data. The number of bubbles on which the input data to the DFD are incident defines the number of transactions. However, some transactions may not require any input data. These transactions can be identified based on the experience gained from solving a large number of examples.

For each identified transaction, trace the input data to the output. All the traversed bubbles belong to the transaction. These bubbles should be mapped to the same module on the structure chart. In the structure chart, draw a root module and below this module draw each identified transaction as a module. Every transaction carries a tag identifying its type. Transaction analysis uses this tag to divide the system into transaction modules and a transaction-center module.

Example 6.8 Draw the structure chart for the Supermarket Prize Scheme software of Example 6.3.

The structure chart for the Supermarket Prize Scheme software is shown in Figure 6.21.

Example 6.9 Draw the structure chart for the *trade-house automation system* (TAS) software of Example 6.4.

The structure chart for the *trade-house automation system* (TAS) software of *****ebook converter DEMO - www.ebook-converter.com*****

Example 6.4 is shown in Figure 6.22.

By observing the level 1 DFD of Figure 6.14, we can see that the data input to the diagram are handled by different bubbles and therefore transaction analysis is applicable to this DFD. Input data to this DFD are handled in three different ways (accept-order, accept-indent-request, and handle-query), we have three different transactions corresponding to these as shown in Figure 6.22.

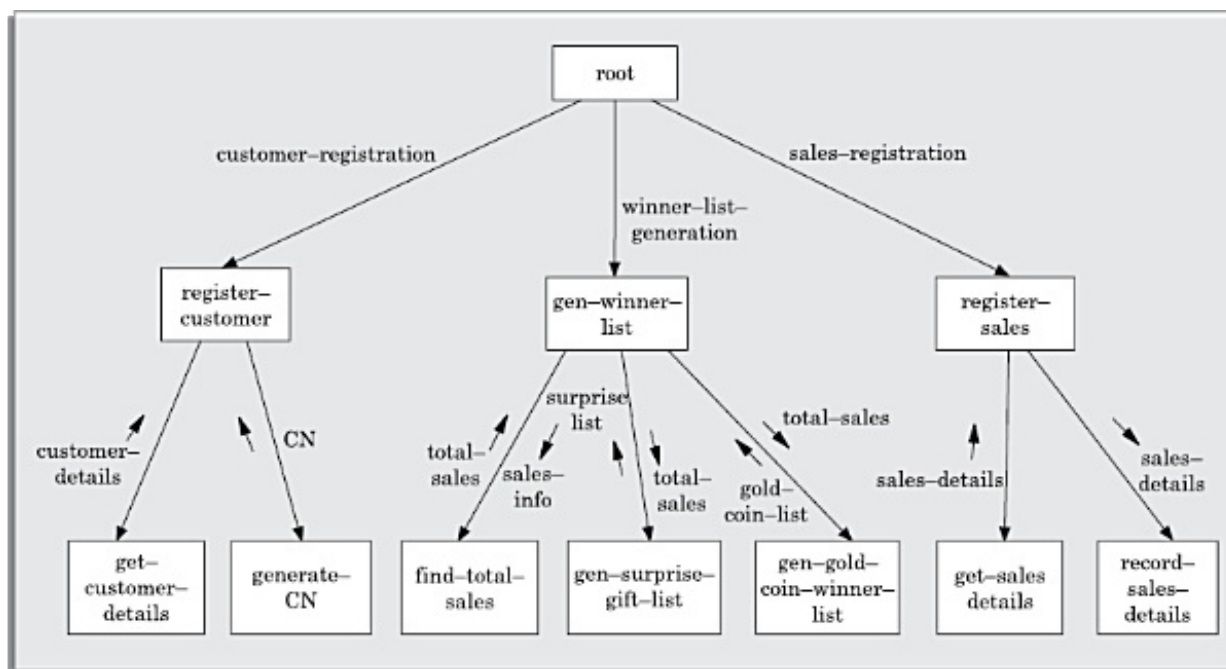


Figure 6.21: Structure chart for Example 6.8.

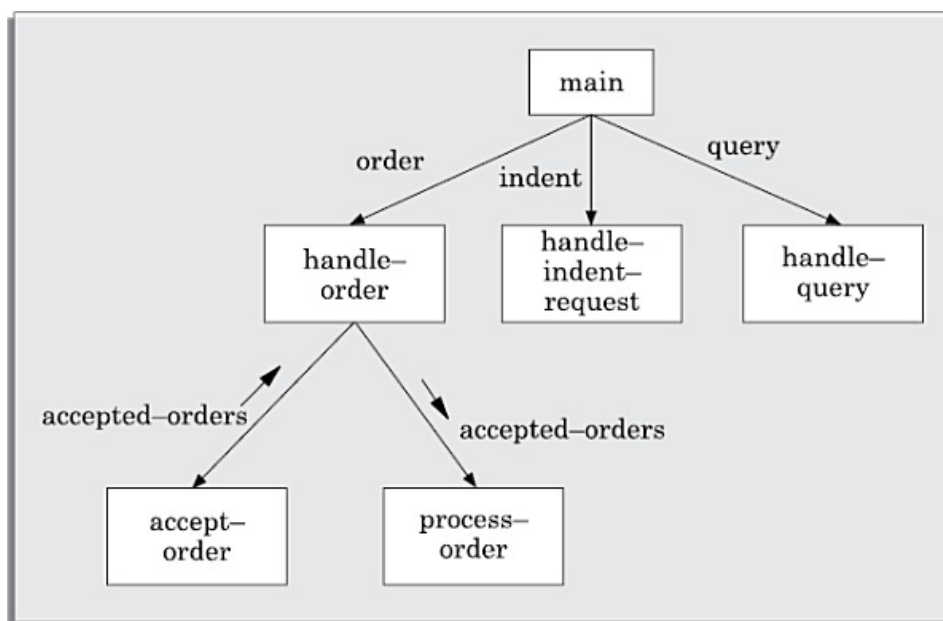


Figure 6.22: Structure chart for Example 6.9.

Word of caution

We should view transform and transaction analyses as guidelines, rather than rules. We should apply these guidelines in the context of the problem and handle the pathogenic cases carefully.

Example 6.10 Draw the structure chart for the personal library software of Example 6.6.

The structure chart for the personal library software is shown in Figure 6.23.

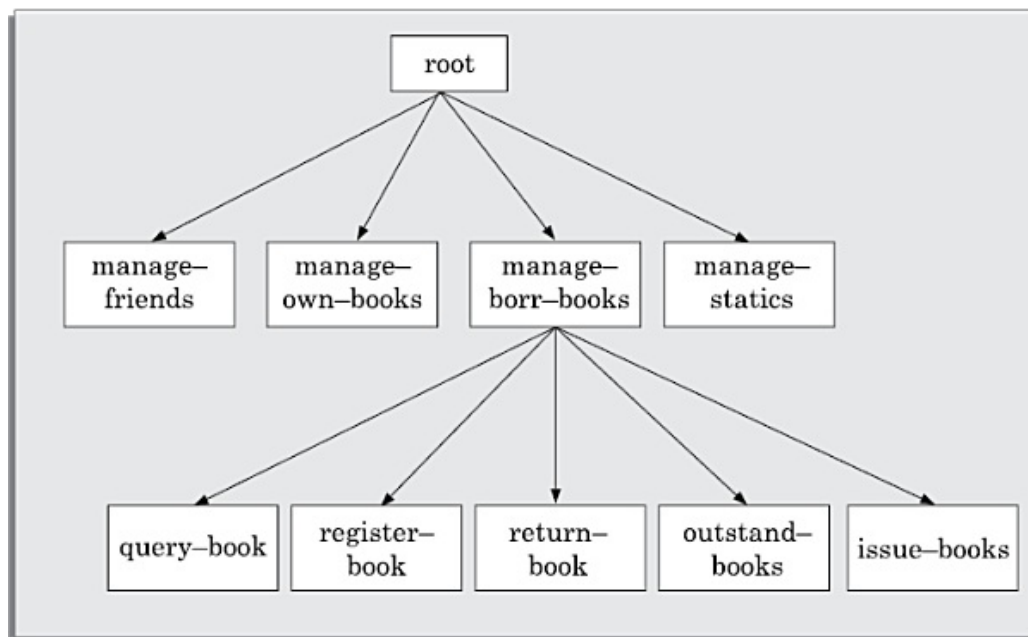


Figure 6.23: Structure chart for Example 6.10.

6.5 DETAILED DESIGN

During detailed design the pseudo code description of the processing and the different data structures are designed for the different modules of the structure chart. These are usually described in the form of module specifications (MSPEC). MSPEC is usually written using structured English. The MSPEC for the non-leaf modules describe the different conditions under which the responsibilities are delegated to the lower-level modules. The MSPEC for the leaf-level modules should describe in algorithmic form how the primitive processing steps are carried out. To develop the MSPEC of a module, it is usually necessary to refer to the DFD model and the SRS document to determine the functionality of the module.

6.6 DESIGN REVIEW

Chapter

7

OBJECT MODELLING USING UML

In recent years, the object-oriented software development style has become very popular and is at present being widely used in industry as well as in academic circles. Since its inception in the early eighties, the object technology has made rapid progress. From a modest beginning in the early eighties, the advancements to the object technology gathered momentum in the nineties and the technology is now nearing maturity. Considering the widespread use and popularity of the object technology in both industry and academia, it is important to learn this technology well.

It is well known that mastering an object-oriented programming language such as Java or C++ rarely equips one with the skills necessary to develop good quality object-oriented software—it is important to learn the object-oriented design skills well. Once a good design has been arrived at, it is easy to code it using an object-oriented language. It has now even become possible to automatically generate much of the code from the design by using a CASE tool. In order to arrive at a satisfactory object-oriented design (OOD) solution to a problem, it is necessary to create several types of models. But, one may ask: “What has modelling got anything to do with designing?” Let us answer this question in the following:

A model is constructed by focusing only on a few aspects of the problem and ignoring the rest. The model of a problem is called an analysis model. On the other hand, the model of the solution (code) is called the design model. The design model is usually obtained by carrying out iterative refinements to the analysis model using a design methodology.

Note that any design is a model of the solution, whereas any model of the problem is an analysis model. In this chapter, we shall discuss how to document a model using a modelling language. In the subsequent chapter, we shall discuss a design process that can be used to iteratively refine an

analysis model into a design model.

In the context of model construction, we need to carefully understand the distinction between a modelling language and a design process, since we shall use these two terms frequently in our discussions.

Modelling language: A modelling language consists of a set of notations using which design and analysis models are documented.

Design process: A design process addresses the following issue: "Given a problem description, how to systematically work out the design solution to the problem?" In other words, a design process consists of a step by step procedure (or recipe) using which a problem description can be converted into a design solution. A design process is, at times, also referred to as a design methodology. In this text, we shall use the terms design process and design methodology interchangeably.

A model can be documented using a modelling language such as unified modelling language (UML). Over the last decade, UML has become immensely popular. UML has also been accepted by ISO as a standard for modelling object-oriented systems. In this Chapter, we primarily discuss the syntax and semantics of UML. However, before discussing the nitty-gritty of the syntax and semantics of UML, we review a few basic concepts and terminologies that have come to be associated with object-orientation.

7.1 BASIC OBJECT-ORIENTATION CONCEPTS

The principles of object-orientation have been founded on a few simple concepts. These concepts are pictorially shown in Figure 7.1. After discussing these basic concepts, we examine a few related technical terms.

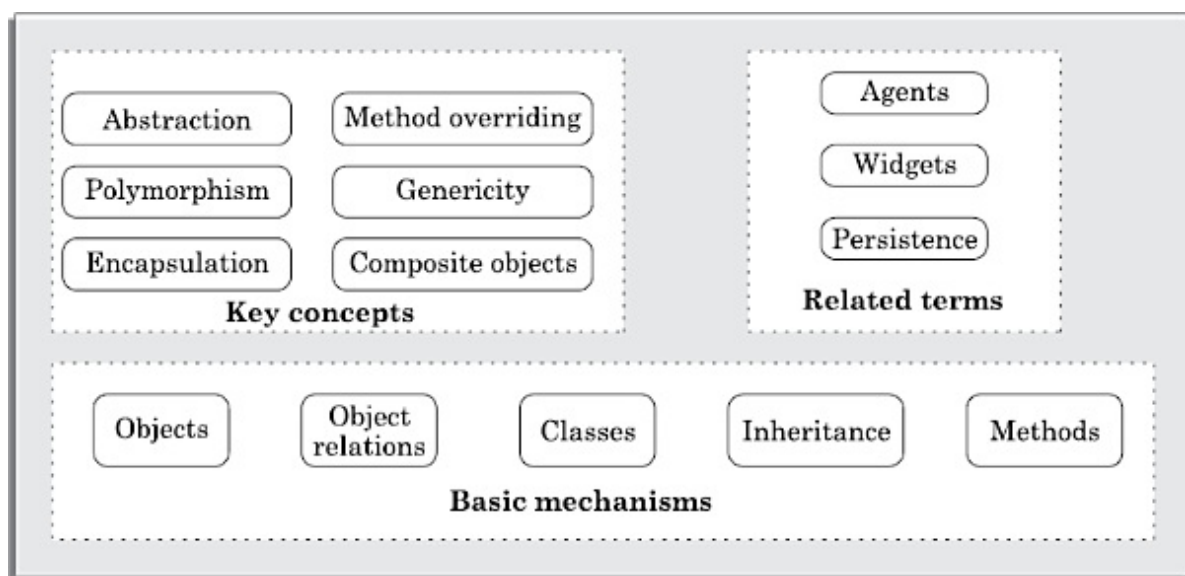


Figure 7.1: Important concepts used in the object-oriented approach.

7.1.1 Basic Concepts

A few important concepts that form the corner stones of the object-oriented paradigm have pictorially been shown in Figure 7.1. In the following sections and subsections, we discuss these concepts in more detail.

Objects

In the object-oriented approach, it is convenient to imagine the working of a software in terms of a set of interacting objects. This is analogous to the way object manipulations take place in a real-world manual system for getting some work done. For example, consider a manually operated library system. For issuing a book, an issue register needs to be filled up and then the return date needs to be stamped on the book. In an object-oriented library automation software, analogous activities involving the book object and the issue register object take place.

Each object in an object-oriented program usually represents a tangible real-world entity such as a library member, a book, an issue register, etc. However while solving a problem, it becomes advantageous at times to consider certain conceptual entities (e.g., a scheduler, a controller, etc.) as objects as well. This simplifies the solution and helps to arrive at a good design.

A key advantage of considering a system as a set of objects is the following:

When the system is analysed, developed, and implemented in terms of objects, it becomes easy to understand the design and the implementation of the system, since objects provide an excellent decomposition of a large problem into small parts.

Each object essentially consists of some data that is private to the object and a set of functions (termed as operations or methods) that operate on those data. This aspect has pictorially been illustrated in Figure 7.2. Observe that the data of the object can only be accessed by the methods of the object. Consequently, the only access point to the data for the external objects is through the invocation of the methods of the object. In fact, the methods of an object have the sole authority to operate on the data that is private to the object. In other words, no object can directly access the data of any other object. Therefore, each object can be thought of as hiding its internal data from other objects. However, an object can access the private

data of another object by invoking the methods supported by that object. This mechanism of hiding data from other objects is popularly known as the principle of data hiding or data abstraction. Data hiding promotes high cohesion and low coupling among objects, and therefore is considered to be an important principle that can help one to arrive at a reasonably good design.

As already mentioned, each object stores some data and supports certain operations on the stored data. As an example, consider the libraryMember object of a library automation application. The private data of a libraryMember object can be the following:

- name of the member
- membership number
- address
- phone number
- e-mail address
- date when admitted as a member
- membership expiry date
- books outstanding

The operations supported by a libraryMember object can be the following:

- issue-book
- find-books-outstanding
- find-books-overdue
- return-book
- find-membership-details

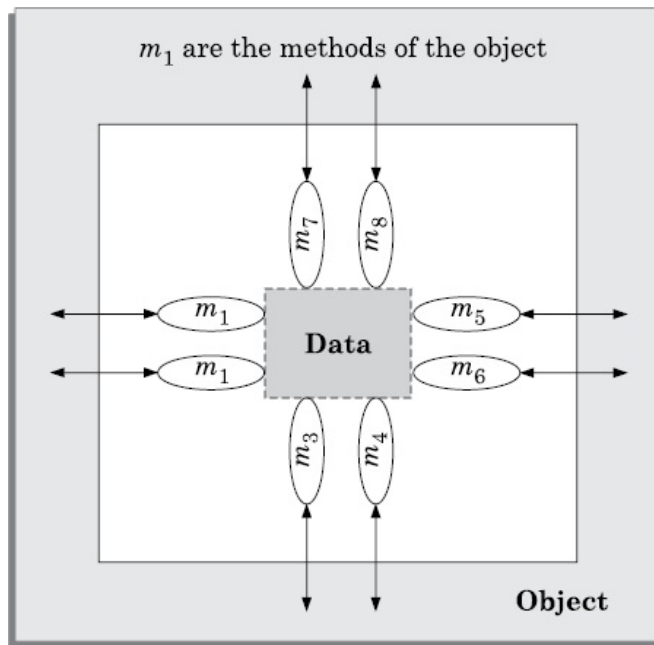


Figure 7.2: A model of an object

The data stored internally in an object are called its attributes, and the operations supported by an object are called its methods.

Though the terminologies associated with object-orientation are simple and well-defined, a word of caution here: the term 'object' is often used rather loosely in the literature and also in this text. Often an 'object' would mean a single entity. However, at other times, we shall use it to refer to a group of similar objects (class). In this text, usually the context of use would resolve the ambiguity, if any.

Is it really true that an object-oriented program consists of objects only?

So far, we claimed that an object-oriented program consists of objects only and the program execution involves through interaction of these objects. However, a program written using in the present programming languages such as Java, C++, etc. work through interaction of objects and primitive data. These programming languages consider that primitive data are not objects and distinctions are made between these in several ways. To get perspective on this, we need to examine a bit of programming history.

The first object-oriented programming language was Smalltalk. It was developed in the 1970s at the Xerox research center in USA. It was a pure object-oriented language in the sense that applications developed in this language consisted of objects only. In other words, no distinction was made

between objects and instances of primitive types of data (int, float, etc.). This was intended to gain better programmer acceptance. Small talk did not find widespread acceptance among the programmers, since it required the programmers to learn and use an entirely different program development paradigm. However, in the later generation programming languages such as C++ and Java, instances of primitive data types were treated differently from objects. For example, in these languages, objects are passed as reference arguments to methods, whereas primitive data types are passed by value. The motivation behind this distinction was to make the object-oriented languages appear as a small extension to procedural languages rather than bringing in any radically new approach. In this light, C++ was designed to retain the procedural approach of C and only extend it with the object-orientation constructs. Of course, different object-oriented constructs were translated into C code by a preprocessor. This made it easy for the procedural programmers to migrate to C++. This was possibly an important reason why C++ gained much more popularity compared to a pure object-oriented programming language. At present, object-oriented programming languages such as Java and C++ distinguish primitive data types from objects and treat them very differently.

Class

Similar objects constitute a class. That is, all the objects constituting a class possess similar attributes and methods. For example, the set of all library members would constitute the class `LibraryMember` in a library automation application. In this case, each library member object has attributes such as `member name`, `membership number`, `member address`, etc. and also has methods such as `issue-book`, `return-book`, etc. Once we define a class, it can be used as a template for object creation.

Let us now investigate the important question as to whether a class is an abstract data type (ADT). To be able to answer this question, we must first be aware of the basic definition of an ADT. We first discuss the same in a nutshell. There are two things that are inherent to an ADT—abstract data and data type. In programming language theory, a data type identifies a group of variables having a particular behaviour. A data type can be instantiated to create a variable. For example, `int` is a type in C++ language. When we write an instruction `int i;` we actually create an instance of `int`. From this, we can infer the following—An ADT is a type where the data contained in each

instantiated entity is abstracted (hidden) from other entities. Let us now examine whether a class supports the two mechanisms of an ADT.

Abstract data: The data of an object can be accessed only through its methods. In other words, the exact way data is stored internally (stack, array, queue, etc.) in the object is abstracted out (not known to the other objects).

Data type: In programming language terminology, a data type defines a collection of data values and a set of predefined operations on those values. Thus, a data type can be instantiated to create a variable of that type. We can instantiate a class into objects. Therefore, a class is a data type.

It can be inferred from the above discussions that a class is an ADT. But, an ADT need not be a class, since to be a class they need to support the inheritance and other object-orientation properties.

Methods

The operations (such as `create`, `issue`, `return`, etc.) supported by an object are implemented in the form of methods. Notice that we are distinguishing between the terms operation and method. Though the terms 'operation' and 'method' are sometimes used interchangeably, there is a technical difference between these two terms which we explain in the following.

An operation is a specific responsibility of a class, and the responsibility is implemented using a method. However, it is at times useful to have multiple methods to implement a responsibility. In this case, all the methods share the same name (that is, the name of the operation), but parameter list of each method is required to be different for enabling the compiler to determine the exact method to be bound on a method call. We therefore say that in this case, the method name is overloaded with multiple implementations of the operation. As an example, consider the following. Suppose one of the responsibilities of a class named `Circle` is to create instances of itself. Assume that the class provides three definitions for the `create` operation—`int create()`, `int create(int radius)` and `int create(float x, float y, int radius);`. In this case, we say that `create` is an overloaded method.

The implementation of a responsibility of a class through multiple methods with the same method name is called method overloading.

Methods are the only means available to other objects in a software for accessing and manipulating the data of another object. The set of valid

messages to an object constitutes its protocol. Let us now try to understand the distinction between a message and a method.

In Smalltalk, an object could request the services (i.e., invoke methods) of other objects by sending messages to them. The idea was that the mechanism of message passing would lead to weak coupling among objects. Though this was an important feature of Smalltalk, yet the programmers who were trying to migrate from procedural programming to object-oriented programming, found it to be a paradigm shift and therefore difficult to accept. Subsequently, C++ implemented message passing by method invocation (similar to a function call). This was rapidly accepted by the programmers. Later object-oriented languages such as Java have followed the same trait of retaining the method invocation feature, that is normally associated with the procedural languages.

7.1.2 Class Relationships

Classes in a programming solution can be related to each other in the following four ways:

- Inheritance
- Association and link
- Aggregation and composition
- Dependency

In the following subsection, we discuss these different types of relationships that can exist among classes.

Inheritance

The inheritance feature allows one to define a new class by incrementally extending the features of an existing class. The original class is called the base class (also called superclass or parent class) and the new class obtained through inheritance is called the derived class (also called a subclass or a child class). The derived class is said to inherit the features of the base class. An example of inheritance has been shown in Figure 7.3. In Figure 7.3, observe that the classes `Faculty`, `Students`, and `Staff` have been derived from the base class `LibraryMember` through an inheritance relationship (note the special type of arrow that has been used to draw it). The inheritance relation between library member and faculty can alternatively be expressed as the following—A faculty is a type of library member. So, the inheritance relationship is also at times called is a relation.

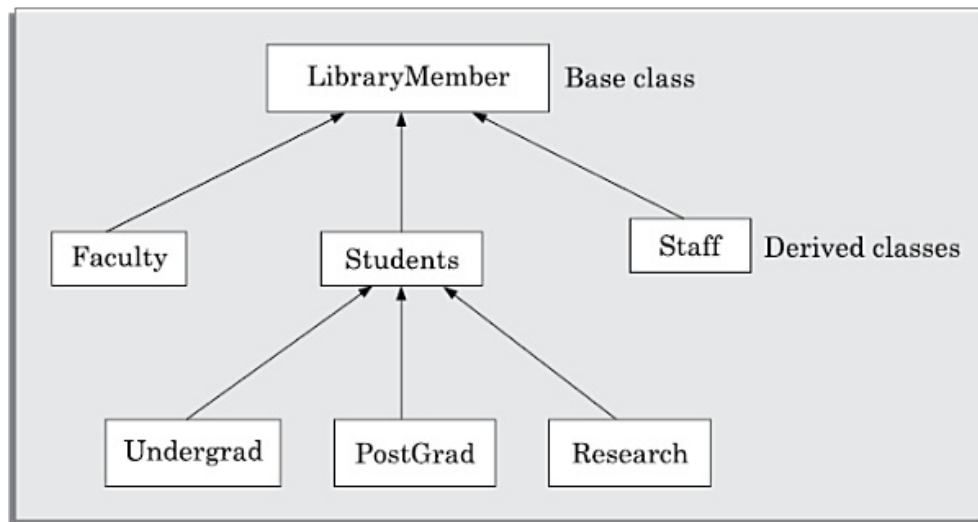


Figure 7.3: Library information system example.

A base class is said to be a generalisation of its derived classes. This means that the base class should contain only those properties (i.e., data and methods) that are common to all its derived classes. For example, in Figure 7.3 each of the derived classes supports the issue-book method, and this method is supported by the base class as well. In other words, each derived class is a specialised base class that extends the base class functionalities in certain ways.

Each derived class can be considered as a specialisation of its base class because it modifies or extends the basic properties of the base class in certain ways. Therefore, the inheritance relationship can be viewed as a generalisation-specialisation relationship.

Observe that in Figure 7.3 the classes `Faculty`, `Student`, and `Staff` are all special types of library members. Several things are common among the members. These include attributes such as membership id, member name and address, date on which books issued out, etc. However, for the different categories of members, the issue procedure differs since different types members are issued books for different durations. We can convey as much by saying that the classes `Faculty`, `Staff`, and `Students` are special types of `LibraryMember` classes. Using the inheritance relationship, different classes can be arranged in a class hierarchy (or class tree) as shown in Figure 7.3.

In addition to inheriting all the data and methods of the base class, a derived class usually defines some new data and methods. A derived class may even redefine a method which already exists in the base class. Redefinition of a method which already exists in its base class is termed as

method overriding.

When a new definition of a method that existed in the base class is provided in a derived class, the method is said to be overridden in the derived class.

The inheritance relationship existing among certain classes in a library automation system is shown in Figure 7.3. As shown, `LibraryMember` is the base class for the derived classes `Faculty`, `Student`, and `Staff`. Similarly, `Student` is the base class for the derived classes `Undergrad`, `Postgrad`, and `Research`. Each derived class inherits all the data and methods of the base class, and defines some additional data and methods or modifies some of the inherited data and methods. The inheritance relationship has been represented in Figure 7.3 using a directed arrow drawn from a derived class to its base class. We now illustrate how the method of a base class is overridden by the derived classes. In Figure 7.3, the base class `LibraryMember` might define the following data—`member name`, `address`, and `library membership number`. Though `faculty`, `student`, and `staff` classes inherit these data, they need to redefine their respective `issueBook` methods because for the specific library that we are modelling, the number of books that can be borrowed and the duration of loan are different for different categories of library members.

Inheritance is a basic mechanism that every object-oriented programming language needs to support. If a language supports ADTs, but does not support inheritance, then it is called an object-based language and not object-oriented. An example of an object-based programming language is Ada.

Now let us try to understand why do we need the inheritance relationship in the first place. Can't we program as well without using the inheritance relationship?

Two important advantages of using the inheritance mechanism in programming include code reuse and simplicity of program design.

Let us examine how code reuse and simplicity of design come about while using the inheritance mechanism. If certain methods or data are present in several classes, then instead of defining these methods and data in each of the classes separately, these methods and data are defined only once in the base class and then inherited by each of its subclasses. For example, in the Library Information System example of Figure 7.4, each category of member (that is, `Faculty`, `Student`, and `Staff`) need to store the following data

—member-name, member-address, and membership-number. Therefore, these data are defined in the base class `LibraryMember` and are inherited by all its subclasses. Another advantage that accrues from the use of the inheritance mechanism is the conceptual simplification brought about through the reduction of the number of independent features of the different classes and incremental understanding of the different classes becomes possible. Thus, inheritance can be considered as a use of the abstraction mechanism we discussed in Chapter 1. The class at the root of an inheritance hierarchy (e.g. `LibraryMember` in Figure 7.3) is the simplest to understand—as it has the least number of data and method members compared to all other classes in the hierarchy. The classes at the leaf-level of the inheritance hierarchy have the maximum number of features (data and method members) because they inherit features of all their ancestors, and therefore turn out to be the toughest to understand.

In a large class hierarchy, it is easier to first understand the root class and then to recursively understand its children classes in the hierarchy until the leaf level classes are reached.

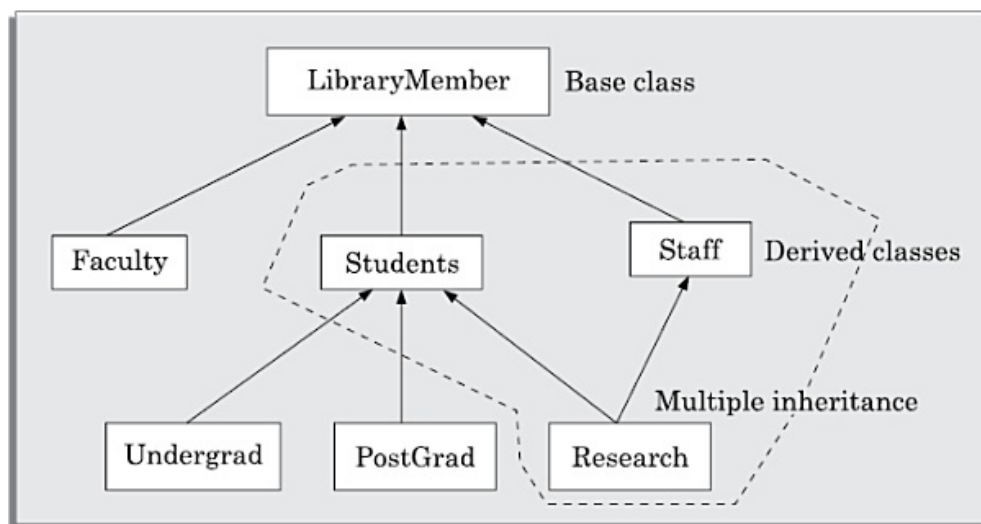


Figure 7.4: An example of multiple inheritance.

Multiple inheritance

Construction of the class relationships for a given problem consists of identifying and representing four types of relations—inheritance, composition/aggregation, association, and dependency. However, at times it may so happen that some features of a class are similar to one class and a few other features of the class are similar to those of another class. In this case, it would be useful if the class could be allowed to inherit features from both the classes. Using the multiple inheritance feature, a class can inherit

features from multiple base classes.

Multiple inheritance is a mechanism by which a subclass can inherit attributes and methods from more than one base class.

Consider the following example of a class that is derived from two base classes through the use of the multiple inheritance mechanism. In this example, in an academic institute research students can also be employed as staff of the institute, then some of the characteristics of the `Research` class are similar to the `Student` class (e.g., every student would have a roll number) while some other characteristics (e.g., having a basic salary and employee number, etc.) might be similar to the `Staff` class. Using multiple inheritance the class `Research` can inherit features from both the classes `Student` and `Staff`. In Figure 7.4, we have shown the class `Research` to be derived from both the `Student` and `Staff` classes by drawing inheritance arrows to both the parent classes of the `Research` class.

Association and link

Association is a common type of relation among classes. When two classes are associated, they can take each others help (i.e. invoke each others methods) to serve user requests. More technically, we can say that if one class is associated with another bidirectionally, then the corresponding objects of the two classes know each others ids (identities). As a result, it becomes possible for the object of one class to invoke the methods of the corresponding object of the other class.

Consider the following example. A `Student` can register in one `Elective` subject. In this example, the class `Student` is associated with the class `ElectiveSubject`. Therefore, an `ElectiveSubject` object (e.g. `MachineLearning`) would know the ids of all `Student` objects that have registered for the Subject and can invoke their methods such as `printName`, `printRoll` and `enterGrade`. This relationship has been represented in Figure 6(a). When an object knows some other object, it must internally store its id. For example, for an object of `ElectiveSubject` class `e1` to invoke the `printName()` method one of the registered students `s1`, it must execute the code `s1.printName()`. Thus, it should have stored the id `s1` of the registered student as an attribute. The association relationship can either be bidirectional or unidirectional. That is, both the associated classes know each other (store each others ids). We have graphically shown this

association between Student class and ElectiveSubject in Figure 7.5(a).

Consider another example of association between two classes: Library Member borrows Books. Here, borrows is the association between the class LibraryMember and the class Book. The association relation would imply that given a book, it would be possible to determine the borrower and vice versa.

n-ary association

Binary association between classes is very commonly encountered in design problems. However, there can be situations where three or more different classes can be involved in an association. As an example of a ternary association, consider the following—A person books a ticket for a certain show. Here, an association exists among the classes Person, Ticket, and Show. This example of ternary association relationship has pictorially been shown in Figure 7.5(b).

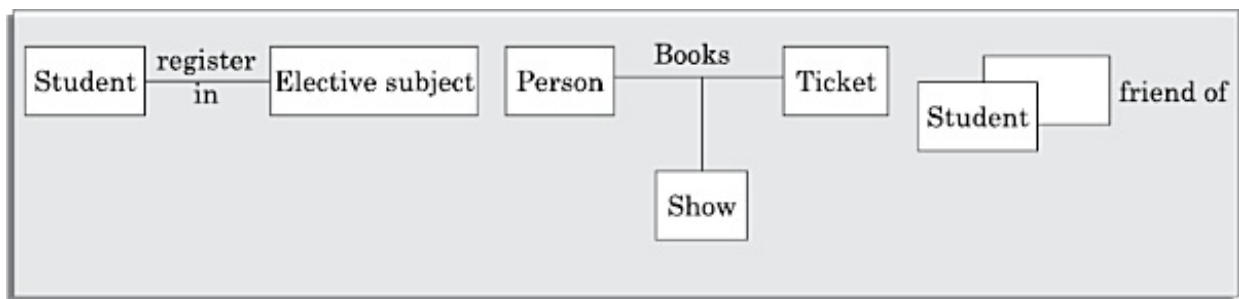


Figure 7.5: Example of (a) binary (b) ternary (c) unary association.

A class can have an association relationship with itself. This is called recursive association or unary association. As an example, consider the following—two students may be friends. Here, an association named friendship exists among pairs of objects of the Student class. This has pictorially been shown in Figure 7.5(c).

In unary association, two (or more) different objects of the same class are linked by the association relationship.

When two classes are associated, the relationship between two objects of the corresponding classes is called a link.

An association describes a group of similar links. Alternatively, we can say that a link can be considered as an instance of an association relation. Let us now try to identify the association relation from the text description of a problem.

Example 7.1 Consider the following extract from a problem description. Identify the association relation among classes and the corresponding

association links among objects from an analysis of the description. "A person works for a company. Ram works for Infosys. Hari works for TCS."

Answer: In this example, an association relationship named works for exists between the classes Person and Company. Ram works for Infosys, this implies that a link exists between the object Ram and the object Infosys. Similarly, a works for link exists between the objects Hari and TCS.

During a run of the system, new links can get formed among the objects of the associated classes and some existing links may get dissolved. Note that some objects may not have any association link to any of the objects of the associated class. Some objects of the associated classes may have not have links. For example in course of time, Ram may resign from Infosys and join Wipro. In this case, the link between Ram and Infosys breaks and a link between Ram and Wipro gets formed. But, suppose Ram does not join any other job after leaving Infosys. In this case, Ram does not have works for link with any Company object. Thus, links are time varying (or dynamic) in nature. Association relationship between two classes is static in nature.

If two classes are associated, then the association relationship exists at all points of time. In contrast, links between objects are dynamic in nature. Links between the objects of the associated classes can get formed and dissolved as the program executes.

An association between two classes simply means that zero or more links may be present among the objects of the associated classes at any time during execution.

Mathematically, a link can be considered to be a tuple. Consider the following example. "Amit has borrowed the book Graph Theory." Here, a link named borrowed has got established between the objects Amit and the Graph Theory book. This link can also be expressed as the ordered pair of object instances {Amit, Graph Theory}.

Composition and aggregation

Composition and aggregation represent part/whole relationships among objects. Objects which contain other objects are called composite objects. As an example, consider the following—A Book object can have upto ten Chapters. In this case, a Book object is said to be composed of upto ten Chapter objects. The composition/aggregation relationship can also be read as follows—A Book has upto ten Chapter objects (shown in Figure

7.6). For this reason, the composition/aggregation relationship is also known as has a relationship. Aggregation/composition can occur in a hierarchy of levels. That is, an object contained in another object may itself contain some other object. Composition and aggregation relationships cannot be reflexive. That is, an object cannot contain an object of the same type as itself.

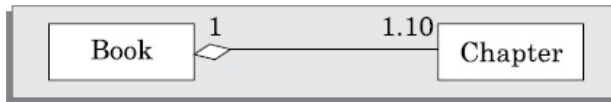


Figure 7.6: Example of aggregation relationship.

Dependency

A class is said to be dependent on another class, if any changes to the latter class necessitates a change to be made to the dependent class.

A dependency relation between two classes shows that any change made to the independent class would require the corresponding change to be made to the dependent class.

Dependencies among classes may arise due to various causes. Two important reasons for dependency to exist between two classes are the following:

- A method of a class takes an object of another class as an argument.
- A class implements an interface class. In this case, dependency arises due to the following reason. If some properties of the interface class are changed, then a change becomes necessary to the class implementing the interface class as well.

Abstract class

Classes that are not intended to produce instances of themselves are called abstract classes. In other words, an abstract class cannot be instantiated. If an abstract class cannot be instantiated to create objects, then what is the use of defining an abstract class? Abstract classes merely exist so that behaviour common to a variety of classes can be factored into one common location, where they can be defined once. Definition of an abstract class helps to push reusable code up in the class hierarchy, thereby enhancing code reuse.

By using abstract classes, code reuse can be enhanced and the effort required to develop software brought down.

Abstract classes usually support generic methods. These methods help to standardise the method names and input and output parameters in the derived classes. The subclasses of the abstract classes are expected to provide the concrete implementations for these methods. For example, in a library automation software `Issuable` can be an abstract class (see Figure 7.7) and the concrete classes `Book`, `Journal`, and `CD` are derived from the `Issuable` class. The `Issuable` class may define several generic methods such as `issue`. Since the issue procedures for books, journals, and CDs would be different, the `issue` method would have to be overridden in the `Book`, `Journal`, and `CD` classes. Though this does not help in code reuse, but helps to have standardised implementations of the `issue` method across different concrete classes. Observe that `Issuable` is an abstract class and cannot be instantiated. On the other hand, `Book`, `Journal`, and `CD` are concrete classes and can be instantiated to create objects.

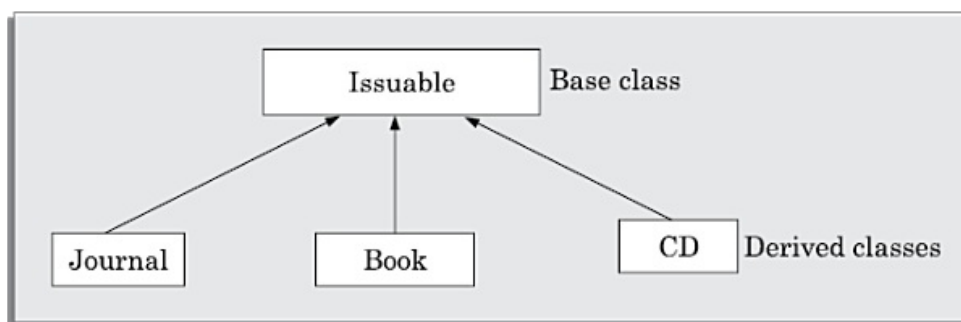


Figure 7.7: An example of an abstract class.

7.1.3 How to Identify Class Relationships?

Suppose we want to write the code for a simple programming problem. How do we identify the classes and their relationships from this description, so that we can write the necessary code? This can be done by a careful analysis of the sentences given in the problem description. The nouns in a sentence often denote the classes. On the other hand, the relationships among classes are usually indicated by the presence of certain key words. In the following, we give examples of a few key words (shown in *italics*) that indicate the specific relationships among two classes A and B:

Composition

- B is a permanent part of A
- A is made up of Bs

- A is a permanent collection of Bs

Aggregation

- B is a part of A
- A contains B
- A is a collection of Bs

Inheritance

- A is a kind of B
- A is a specialisation of B
- A behaves like B

Association

- A delegates to B
- A needs help from B
- A collaborates with B. Here collaborates with can be any of a large variety of collaborations that are possible among classes such as employs, credits, precedes, succeeds, etc.

7.1.4 Other Key Concepts

We now discuss a few other key concepts used in the object-oriented program development approaches:

Abstraction

Let us first recapitulate how the abstraction mechanism works (we had already discussed this basic mechanism in Section 1.3.2). Abstraction is the selective examination of certain aspects of a problem while ignoring all the remaining aspects of a problem. In other words, the main purpose of using the abstraction mechanism is to consider only those aspects of the problem that are relevant to a given purpose and to suppress all aspects of the problem that are not relevant.

The abstraction mechanism allows us to represent a problem in a simpler way by considering only those aspects that are relevant to some purpose and omitting all other details that are irrelevant.

Many different abstractions of the same problem can be constructed depending on the purpose for which the abstractions are made. The abstraction mechanism cannot only help the development engineers to understand and appreciate a problem better while working out a solution, but can also help better comprehension of a system design by the maintenance team. Abstraction is supported in two different ways in an object-oriented designs (OODs). These are the following:

Feature abstraction: A class hierarchy can be viewed as defining several levels (hierarchy) of abstraction, where each class is an abstraction of its subclasses. That is, every class is a simplified (abstract) representation of its derived classes and retains only those features that are common to all its children classes and ignores the rest of the features. Thus, the inheritance mechanism can be thought of as providing feature abstraction.

Data abstraction: An object itself can be considered as a data abstraction entity, because it abstracts out the exact way in which it stores its various private data items and it merely provides a set of methods to other objects to access and manipulate these data items. In other words, we can say that data abstraction implies that each object hides (abstracts away) from other objects the exact way in which it stores its internal information. This helps in developing good quality programs, as it causes objects to have low coupling with each other, since they do not directly access any data belonging to each other. Each object only provides a set of methods, which other objects can use for accessing and manipulating this private information of the object. For example, a stack object might store its internal data either in the form of an array of values or in the form of a linked list. Other objects would not know how exactly this object has stored its data (i.e. data is abstracted) and how it internally manipulates its data. What they would know is the set of methods such as push, pop, and top-of-stack that it provides to the other objects for accessing and manipulating the data.

An important advantage of the principle of data abstraction is that it reduces coupling among various objects, Therefore, it leads to a reduction of the overall complexity of a design, and helps in easy maintenance and code reuse.

Abstraction is a powerful mechanism for reducing the perceived complexity of software designs. Analysis of the data collected from several software development projects shows that software productivity is inversely proportional to the perceived software complexity. Therefore, implicit use of abstraction, as it takes place in object-oriented development, is a promising

way of increasing productivity of the software developers.

Encapsulation

The data of an object is encapsulated within its methods. To access the data internal to an object, other objects have to invoke its methods, and cannot directly access the data. This concept is schematically shown in Figure 7.8. Observe from Figure 7.8 that there is no way for an object to access the data private to another object, other than by invoking its methods. Encapsulation offers the following three important advantages:

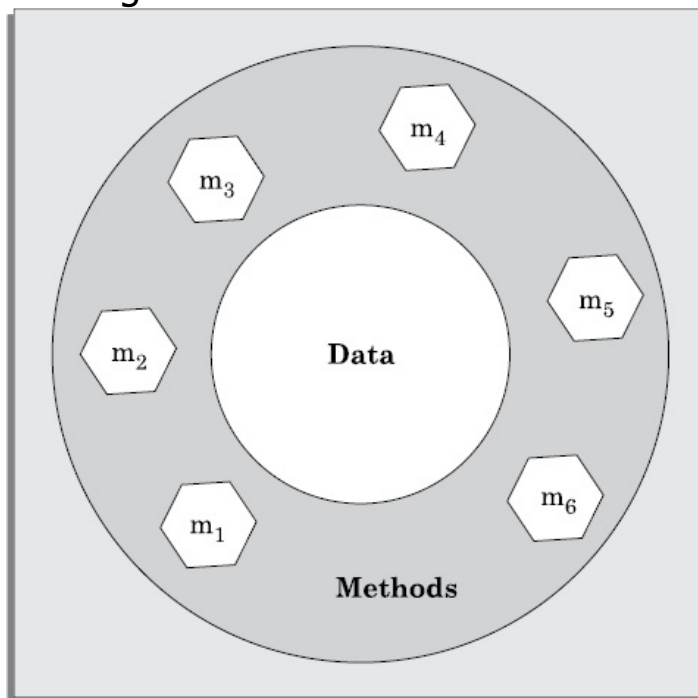


Figure 7.8: Schematic representation of the concept of encapsulation.

Protection from unauthorised data access: The encapsulation feature protects an object's variables from being accidentally corrupted by other objects. This protection includes protection from unauthorised access and also protection from the problems that arise from concurrent access to data such as deadlock and inconsistent values.

Data hiding: Encapsulation implies that the internal structure data of an object are hidden, so that all interactions with the object are simple and standardised. This facilitates reuse of a class across different projects. Furthermore, if the internal data or the method body of a class are modified, other classes are not affected. This leads to easier maintenance and bug correction.

Weak coupling: Since objects do not directly change each others internal

data, they are weakly coupled. Weak coupling among objects enhances understandability of the design since each object can be studied and understood in isolation from other objects.

Polymorphism

Polymorphism literally means poly (many) morphism (forms). Remember that in Chemistry, diamond, graphite, and coal are called polymorphic forms of carbon. That is, though diamond, coal, and graphite are essentially carbon, they behave very differently. In an analogous manner in the object-oriented paradigm, polymorphism denotes that an object may respond (behave) very differently even when the same operation is invoked on it depending on the exact polymorphic object to which the call gets bound.

There are two main types of polymorphisms in object-orientation:

Static polymorphism: Static polymorphism occurs when multiple methods implement the same operation. In this type of polymorphism, when a method is called (same method name but different parameter types), different behaviour (actions) would be observed. This type of polymorphism is also referred to as static binding, because the exact method to be bound on a method call is determined at compiled-time (statically). Let us try to understand static binding through the following example. Suppose a class named Circle has three definitions for the create operation—`int create()`, `int create(int radius)`, and `int create(float x, float y, int radius)`. Recollect that when multiple methods implement the same operation, then the mechanism used is called method overloading. (Notice the difference between an operation and a method.) When the same operation (e.g. create) is implemented by multiple methods, the method name is said to be overloaded. One definition of the create operation does not take any argument (`create()`) and creates a circle with default parameters. The second definition takes the center point and the radius of the circle as its parameters (`create(float x, float y, float radius)`). Assume that in both the above cases, the fill style would be set to the default value “no fill”. The third definition of the create operation takes the center point, the radius, and a fill style as its input. When the create method is invoked, depending on the parameters given in the invocation, the matching method can be easily determined during compilation by examining the parameter list of the call and the call would get be statically bound. If `create` method is invoked with no parameters, then a default circle

would be created. If only the center and radius are supplied, then an appropriate circle would be created with no fill type, and so on. The definition of the `Circle` class with the overloaded `create` method is shown in Figure 7.9. **Dynamic polymorphism:** Dynamic polymorphism is also called dynamic binding. In dynamic binding, the exact method that would be invoked (bound) on a method call can only be known at the run time (dynamically) and cannot be determined at compile time. That is, the exact behaviour that would be produced on a method call cannot be predicted at compile time and can only be observed at run time.

```
class Circle{
    private float x, y, radius;
    private int fillType;

    public create();
    public create(float x, float y, float radius);
    public create(float x, float y, float radius, int fillType);
}
```

Figure 7.9: Circle class with overloaded create method.

Let us now explain how dynamic binding works in object-oriented programs. Dynamic binding is based on two important concepts:

- Assignment of an object to another compatible object.
- Method overriding in a class hierarchy.

Assignment to compatible of objects

In object-orientation, objects of the derived classes are compatible with the objects of the base class. That is, an object of the derived class can be assigned to an object of the base class, but not vice versa. Also an object cannot be assigned to an object of a sibling class or an object of a totally unrelated class for obvious reasons. This is an important principle in object-orientation and is known as the Liskov Substitution principle. To understand this principle, recollect that a derived class defines some extra attributes and assignment of an object of the base class to an object of the derived class can leave those attributes undefined.

Method overriding

We have already explained the method overriding principle in which a

derived class provides a new definition to a method of the base class.

Let us now understand how dynamic binding works by making use of the above two mechanisms. Suppose we have defined a class hierarchy of different geometric shapes for a graphical drawing package as shown in Figure 7.10. As can be seen from the figure, `Shape` is an abstract class and the classes `Circle`, `Rectangle`, and `Line` are directly derived from it and further the classes `Ellipse`, `Square` and `Cube` have been derived to form an inheritance hierarchy. Now, suppose the `draw` method is declared in the `Shape` class and is overridden in each derived class. Further, suppose a set of different types of `Shape` objects have been created one by one. By Liskov's substitution principle, the created `Shape` objects can be stored in an array of type `Shape`. If the different types of geometric objects making up a drawing are stored in an array of type `Shape`, then a call to the `draw` method for each object would take care to display the appropriate drawing element. That is, the same `draw` call to a `Shape` object would take care of drawing the appropriate drawing object. Observe that due to dynamic binding, a call to the `draw` method of the `shape` class takes care of displaying the appropriate drawing object residing in the `shape` array. This is illustrated in the code segment shown in Figure 7.11.

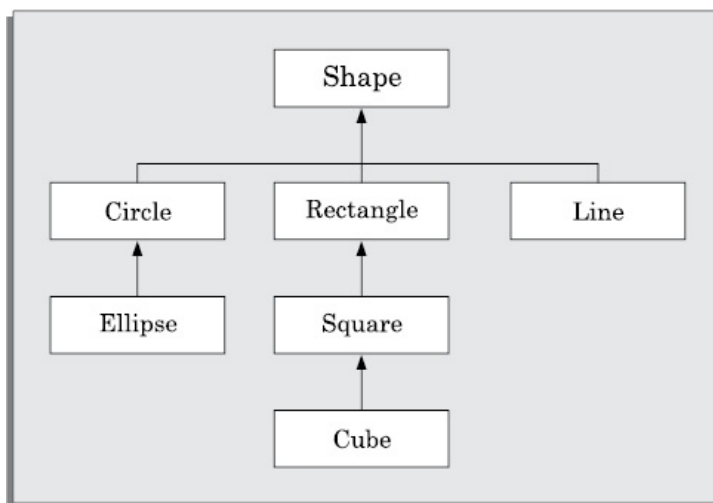


Figure 7.10: Class hierarchy of geometric objects.

Traditional code	Object-oriented code
<pre>Shape s[100]; for(i=0;i<100;i++){ if(s[i]==Circle) then draw_circle(); else if(shape==Rectangle) then draw_rectangle(); — — — }</pre>	<pre>Shape s[100]; for(i=0;i<100;i++) shape.draw();</pre>

Figure 7.11: Traditional code versus object-oriented code incorporating the dynamic binding feature.

We analyse the advantage of polymorphism by comparing the code segments of an object-oriented program and a traditional program for drawing various graphic objects on the screen (shown in Figure 7.11). Using dynamic binding, a programmer can invoke the generic method of an object and leave the exact way in which this message would be handled would be decided` depending on the object that is currently assigned to the receiving object. With dynamic binding, new derived objects can be added with minimal changes to a program.

The principal advantage of dynamic binding is that it leads to elegant programming and facilitates code reuse and maintenance.

It can be easily seen from Figure 7.11 that the use of dynamic binding, the object-oriented code is much more concise, understandable, and intellectually appealing as compared to equivalent procedural code. Further, suppose that in the example program segment, it is later found necessary to handle a new graphics drawing primitive, say ellipse. Then, the procedural code has to be changed by adding a new if-then-else clause. However, in case of an object-oriented program, the code need not change, only a new class called Ellipse has to be derived in the Shape hierarchy.

We can now summarise the mechanism of dynamic binding as follows:

Even when the method of an object of the base class is invoked, an appropriate overridden method of a derived class would be invoked depending on the exact object that may have been assigned at the run-time to the object of the base class.

Genericity

Genericity is the ability to parameterise class definitions. For example, while defining a class stack of different types of elements such as integer stack, character stack, floating-point stack, etc.; genericity permits us to define a generic class of type stack and later instantiate it either as an integer stack, a character stack, or a floating-point stack as may be required. This can be achieved by assigning a suitable value to a parameter used in the generic class definition.

7.1.5 Related Technical Terms

In the following, we discuss a few terms related to object-orientation as follows:

Persistence

Objects usually get destroyed once a program completes its execution. Persistent objects are stored permanently. That is, they live across different executions. An object can be made persistent by maintaining copies of the object in a secondary storage or in a database.

Agents

A passive object is one that performs some action only when requested through invocation of some of its methods. An agent (also called an active object), on the other hand, monitors events occurring in the application and takes actions autonomously. Agents are used in applications such as monitoring exceptions. For example, in a database application such as accounting, an agent may monitor the balance sheet and would alert the user whenever inconsistencies arise in a balance sheet due to some improper transaction taking place.

Widget

The term widget stands for window object. A widget is a primitive object used for graphical user interface (GUI) design. More complex graphical user interface design primitives (widgets) can be derived from the basic widget using the inheritance mechanism. A widget maintains internal data such as the geometry of the window, back ground and fore ground colors of the window, cursor shape and size, etc. The methods supported by a widget manipulate the stored data and carry out operations such as resize window, iconify window, destroy window, etc.

Widgets are becoming the standard components of GUI design. This has given rise to the technique of component-based user interface development. We shall discuss more about widgets and component-based user interface development in Chapter 9 where we discuss GUI design.

7.1.6 Advantages and Disadvantages of OOD

As is true for any other technique, OOD has its own peculiar set of advantages and disadvantages. We briefly review these in the following.

Advantages of OOD

In the last couple of decades since OOD has come into existence, it has found widespread acceptance in industry as well as in academic circles. The main reason for the popularity of OOD is that it holds out the following promises:

- Code and design reuse
- Increased productivity
- Ease of testing and maintenance
- Better code and design understandability enabling development of large programs

Out of all the above mentioned advantages, it is usually agreed that the chief advantage of OOD is improved productivity—which comes about due to a variety of factors, such as the following:

- Code reuse by the use of predeveloped class libraries
- Code reuse due to inheritance
- Simpler and more intuitive abstraction, i.e., better management of inherent problem and code complexity
- Better problem decomposition

Several research results indicate that when companies start to develop software using the object-oriented paradigm, the first few projects incur higher costs than the traditionally developed projects. This is possibly due to getting used to a new technique and building up the class libraries that can be reused in the subsequent projects. After completion of a few projects, cost

saving becomes possible. According to experience reports, well-established object-oriented development environment can help to reduce development costs by as much as 20 per cent to 50 per cent over a traditional development environment.

Disadvantages of OOD

The following are some of the prominent disadvantages inherent to the object paradigm:

- The principles of abstraction, data hiding, inheritance, etc. do incur run time overhead due to the additional code that gets generated on account of these features. This causes an project-oriented program to run a little slower than an equivalent procedural program.
- An important consequence of object-orientation is that the data that is centralised in a procedural implementation, gets scattered across various objects in an object-oriented implementation. Therefore, the spatial locality of data becomes weak and this leads to higher cache miss ratios and consequently to larger memory access times. This finally shows up as increased program run time.

As we can see, increased run time is the principal disadvantage of object-orientation and higher productivity is the major advantage. In the present times, computers have become remarkably fast, and a small run time overhead is not an issue at all. Consequently, the advantages of OOD overshadow the disadvantages.

7.2 UNIFIED MODELLING LANGUAGE (UML)

As the name itself implies, UML is a language for documenting models. As is the case with any other language, UML has its syntax (a set of basic symbols and sentence formation rules) and semantics (meanings of basic symbols and sentences). It provides a set of basic graphical notations (e.g. rectangles, lines, ellipses, etc.) that can be combined in certain ways to document the design and analysis results.

It is important to remember that UML is neither a system design or development methodology by itself, nor is tied to any specific methodology. UML is merely a language for documenting models. Before the advent of UML, every design methodology not only prescribed entirely different design steps, but each was tied to some specific design modelling language. For example,

OMT methodology had its own design methodology and had its own unique set of notations. So was the case with Booch's methodology, and so on. This situation made it hard for someone familiar with one methodology to understand the design solutions developed and documented using another methodology. In general, reuse of design solutions across different methodologies was hard. UML was intended to address this problem that was inherent to the modelling techniques that existed.

UML can be used to document object-oriented analysis and design results that have been obtained using any methodology.

One of the objectives of the developers of UML was to keep the notations of UML independent of any specific design methodology, so that it can be used along with any specific design methodology. In this respect, UML is different from its predecessors (e.g., OMT, Booch's methodology, etc.) where the notations supported by the modelling languages were closely tied to the corresponding design methodologies.

7.2.1 Origin of UML

In the late eighties and early nineties, there was a proliferation of object-oriented design techniques and notations. Many of these had become extremely popular and were widely used. However, the notations they used and the specific design paradigms that they advocated, differed from each other in major ways. With so many popular techniques to choose from, it was not very uncommon to find different project teams in the same organisation using different methodologies and documenting their object-oriented analysis and design results using different notations. These diverse notations used for documenting design solutions gave rise to a lot of confusion among the team members and made it extremely difficult to reuse designs across projects and communicating ideas across project teams.

UML was developed to standardise the large number of object-oriented modelling notations that existed in the early nineties. The principal ones in use those days include the following:

- OMT [Rumbaugh 1991]
- Booch's methodology [Booch 1991]
- OOSE [Jacobson 1992]
- Odell's methodology [Odell 1992]

- Shlaer and Mellor methodology[Shlaer 1992]

Needless to say that UML has borrowed many concepts from these modeling techniques. Concepts and notations from especially the first three methodologies have heavily been drawn upon. The influence of various object modeling techniques on UML is shown schematically in Figure 7.12. As shown in Figure 7.12, OMT had the most profound influence on UML.

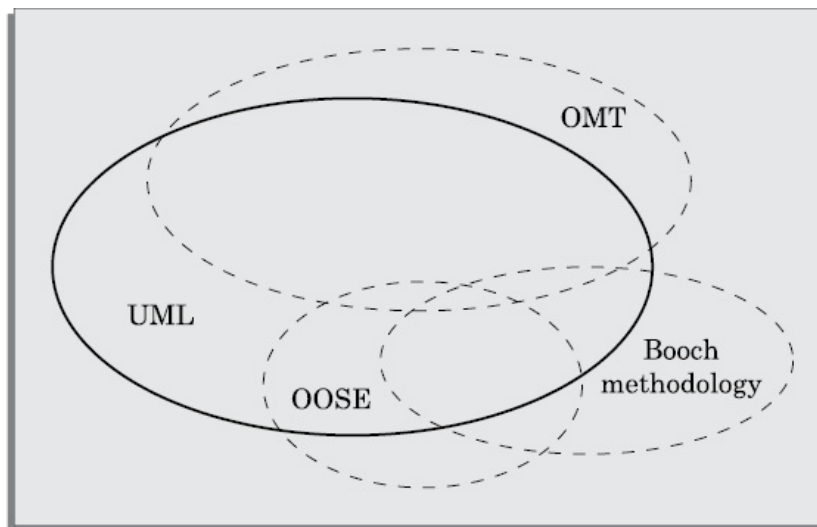


Figure 7.12: Schematic representation of the impact of different object modelling techniques on UML.

UML was adopted by object management group (OMG) as a de facto standard in 1997. Actually, OMG is not a standards formulating body, but is an association of industries that tries to facilitate early formulation of standards. OMG aims to promote consensus notations and techniques with the hope that if the usage becomes wide-spread, then they would automatically become standards. For more information on OMG, see www.omg.org. With widespread use of UML, ISO adopted UML a standard (ISO 19805) in 2005, and with this UML has become an official standard; this has further enhanced the use of UML.

UML is more complex than its antecedents. This is only natural and expected because it is intended to be more comprehensive and applicable to a wider gamut of problems than any of the notations that existed before UML. UML contains an extensive set of notations to help document several aspects (views) of a design solution through many types of diagrams. UML has successfully been used to model both large and small problems. The elegance of UML, its adoption by OMG, and subsequently by ISO as well as a strong industry backing have helped UML to find wide spread acceptance. UML is now being used in academic and research institutions as well as in large

number of software development projects world-wide. It is interesting to note that the use of UML is not restricted to the software industry alone. As an example of UML's use outside the software development problems, some car manufacturers are planning to use UML for their "build-to-order" initiative.

Many of the UML notations are difficult to draw by hand on a paper and are best drawn using a CASE tool such as Rational Rose[©] (see www.rational.com) or MagicDraw (www.magicdraw.com). Now several free UML CASE tools are also available on the web. Most of the available CASE tools help to refine an initial object model to final design, and these also automatically generate code templates in a variety of languages, once the UML models have been constructed.

7.2.2 Evolution of UML

Since the release of UML 1.0 in 1997, UML continues to evolve (see Figure 7.13) with feedback from practitioners and academicians to make it applicable to different system development situations. Almost every year several new releases (shown as UML 1.X in Figure 7.13) were announced. A major milestone in the evolution of UML was the release of UML 2.0 in the year 2007. Since the use of embedded applications is increasing rapidly, there was popular demand to extend UML to support the special concepts and notations required to develop embedded applications. UML 2.0 was an attempt to make UML applicable to the development of concurrent and embedded systems. For this, many new features such as events, ports, and frames in sequence diagrams were introduced. We briefly discuss these developments in this chapter.

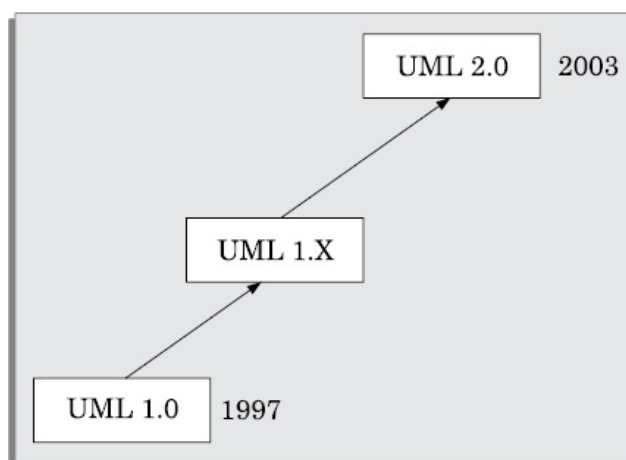


Figure 7.13: Evolution of UML.

What is a model?

Before we discuss the features of UML in detail, it is important to understand what exactly is meant by a model, and why is it necessary to create a model.

A model is an abstraction of a real problem (or situation), and is constructed by leaving out unnecessary details. This reduces the problem complexity and makes it easy to understand the problem (or situation).

A model is a simplified version of a real system. It is useful to think of a model as capturing aspects important for some application while omitting (or abstracting out) the rest. As we had already pointed out in Chapter 1, as the size of a problem increases, the perceived complexity increases exponentially due to human cognitive limitations. Therefore, to develop a good understanding of any problem, it is necessary to construct a model of the problem. Modelling has turned out to be a very essential tool in software design and helps to effectively handle the complexity in a problem. These models that are first constructed are the models of the problem. A design methodology essentially transform these analysis models into a design model through iterative refinements.

Different types of models are obtained based on the specific aspects of the actual system that are ignored while constructing the model. To understand this, let us consider the models constructed by an architect of a large building. While constructing the frontal view of a large building (elevation plan), the architect ignores aspects such as floor plan, strength of the walls, details of the inside architecture, etc. While constructing the floor plan, he completely ignores the frontal view (elevation plan), site plan, thermal and lighting characteristics, etc. of the building.

A model in the context of software development can be graphical, textual, mathematical, or program code-based. Graphical models are very popular because they are easy to understand and construct. UML is primarily a graphical modeling tool. However, there are certain modelling situations (discussed later in this Chapter), for which in addition to the graphical UML models, separate textual explanations are required to accompany the graphical models.

Why construct a model?

An important reason behind constructing a model is that it helps to manage the complexity in a problem and facilitates arriving at good solutions and at the same time helps to reduce the design costs. The initial model of a problem is called an analysis model. The analysis model of a problem can be refined into a design model using a design

methodology. Once models of a system have been constructed, these can be used for a variety of purposes during software development, including the following:

- Analysis
- Specification
- Design
- Coding
- Visualisation and understanding of an implementation.
- Testing, etc.

Since a model can be used for a variety of purposes, it is reasonable to expect that the models would vary in detail depending on the purpose for which these are being constructed. For example, a model developed for initial analysis and specification should be very different from the one used for design. A model that is constructed for analysis and specification would not show any of the design decisions that would be made later on during the design stage. On the other hand, a model constructed for design purposes should capture all the design decisions. Therefore, it is a good idea to explicitly mention the purpose for which a model has been developed.

We now discuss the different types of UML diagrams and the notations used to develop these diagrams.

7.3 UML DIAGRAMS

In this section, we discuss the diagrams supported by UML 1.0. Later in Section 7.9.2, we discuss the changes to UML 1.0 brought about by UML 2.0. UML 1.0 can be used to construct nine different types of diagrams to capture five different views of a system. Just as a building can be modelled from several views (or perspectives) such as ventilation perspective, electrical perspective, lighting perspective, heating perspective, etc.; the different UML diagrams provide different perspectives of a software system to be developed and facilitate a comprehensive understanding of the system. Each perspective focuses on some specific aspect and ignores the rest. Some may ask, why construct several models from different perspectives—why not just construct one model that captures all perspectives? The answer to this is the following:

If a single model is made to capture all the required perspectives, then it would be as complex as the original problem, and would be of little use.

Once a system has been modelled from all the required perspectives, the constructed models can be refined to get the actual implementation of the system.

UML diagrams can capture the following views (models) of a system:

- User's view
- Structural view
- Behavioural view
- Implementation view
- Environmental view

Figure 7.14 shows the different views that the UML diagrams can document. Observe that the users' view is shown as the central view. This is because based on the users' view, all other views are developed and all views need to conform to the user's view. Most of the object oriented analysis and design methodologies, including the one we are going to discuss in Chapter 8 require us to iterate among the different views several times to arrive at the final design. We first provide a brief overview of the different views of a system which can be documented using UML. In the subsequent sections, the diagrams used to realize the important views are discussed.

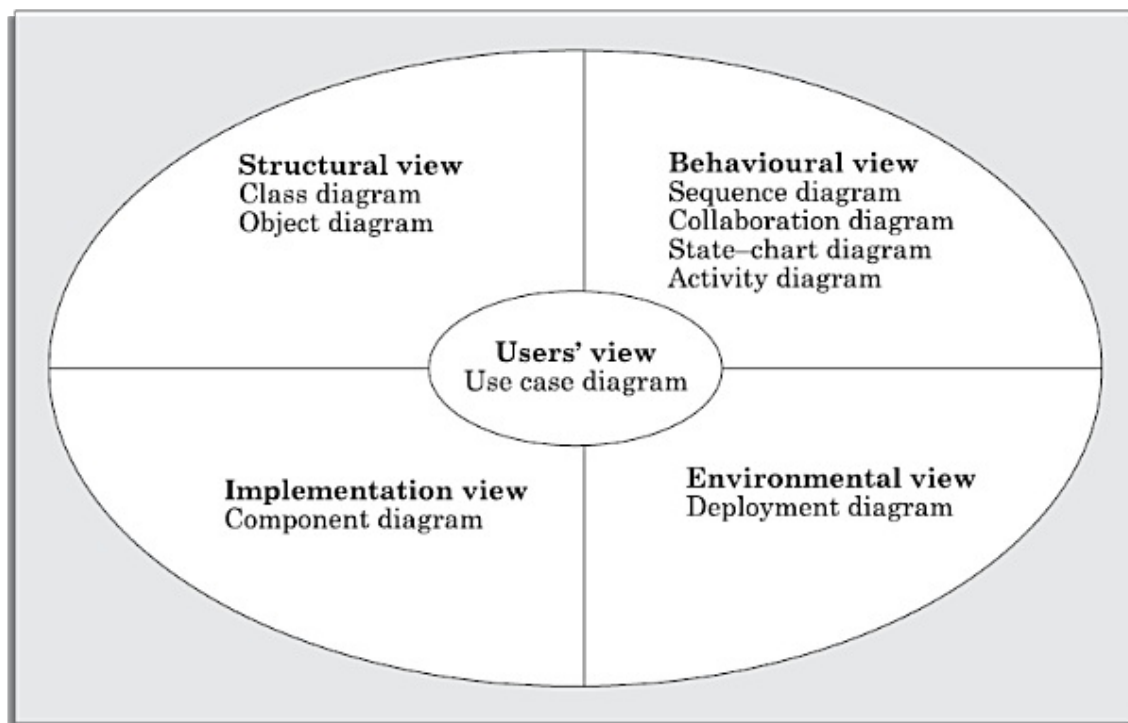


Figure 7.14: Different types of diagrams and views supported in UML.

Users' view

This view defines the functionalities made available by the system to its users.

The users' view captures the view of the system in terms of the functionalities offered by the system to its users.

The users' view is a black-box view of the system where the internal structure, the dynamic behaviour of different system components, the implementation etc. are not captured. The users' view is very different from all other views in the sense that it is a functional model¹ compared to all other views that are essentially object models.²

The users' view can be considered as the central view and all other views are required to conform to this view. This thinking is in fact the crux of any user centric development style. It is indeed remarkable that even for object-oriented development, we need a functional view. That is because, after all, a user considers a system as providing a set of functionalities.

Structural view

The structural view defines the structure of the problem (or the solution) in terms of the kinds of objects (classes) important to the understanding of the working of a system and to its implementation. It also captures the relationships among the classes (objects).

The structural model is also called the static model, since the structure of a system does not change with time.

Behavioural view

The behavioural view captures how objects interact with each other in time to realise the system behaviour. The system behaviour captures the time-dependent (dynamic) behaviour of the system. It therefore constitutes the dynamic model of the system.

Implementation view

This view captures the important components of the system and their interdependencies. For example, the implementation view might show the GUI part, the middleware, and the database part as the different parts and also would capture their interdependencies.

Environmental view

This view models how the different components are implemented on different pieces of hardware.

For any given problem, should one construct all the views using all the diagrams provided by UML? The answer is No. For a simple system, the use case model, class diagram, and one of the interaction diagrams may be sufficient. For a system in which the objects undergo many state changes, a state chart diagram may be necessary. For a system, which is implemented on a large number of hardware components, a deployment diagram may be necessary. So, the type of models to be constructed depends on the problem at hand. Rosenberg provides an analogy [Ros 2000] saying that “Just like you do not use all the words listed in the dictionary while writing a prose, you do not use all the UML diagrams and modeling elements while modeling a system.”

7.4 USE CASE MODEL

The use case model for any system consists of a set of use cases.

Intuitively, the use cases represent the different ways in which a system can be used by the users.

A simple way to find all the use cases of a system is to ask the question —“What all can the different categories of users do by using the system?” Thus, for the library information system (LIS), the use cases could be:

- issue-book
- query-book
- return-book
- create-member
- add-book, etc.

Roughly speaking, the use cases correspond to the high-level functional requirements that we discussed in Chapter 4. We can also say that the use cases partition the system behaviour into transactions, such that each transaction performs some useful action from the user’s point of view. Each transaction, to complete, may involve multiple message exchanges between the user and the system.

The purpose of a use case is to define a piece of coherent behaviour without revealing the internal structure of the system. The use cases do not mention any specific algorithm to be used nor the internal data representation, internal structure of the software. A use case typically

involves a sequence of interactions between the user and the system. Even for the same use case, there can be several different sequences of interactions. A use case consists of one main line sequence and several alternate sequences. The main line sequence represents the interactions between a user and the system that normally take place. The mainline sequence is the most frequently occurring sequence of interaction. For example, in the mainline sequence of the withdraw cash use case supported by a bank ATM would be—the user inserts the ATM card, enters password, selects the amount withdraw option, enters the amount to be withdrawn, completes the transaction, and collects the amount. Several variations to the main line sequence (called alternate sequences) may also exist. Typically, a variation from the mainline sequence occurs when some specific conditions hold. For the bank ATM example, consider the following variations or alternate sequences:

- Password is invalid.
- The amount to be withdrawn exceeds the account balance.

The mainline sequence and each of the alternate sequences corresponding to the invocation of a use case is called a scenario of the use case.

A use case can be viewed as a set of related scenarios tied together by a common goal. The main line sequence and each of the variations are called scenarios or instances of the use case. Each scenario is a single path of user events and system activity.

Normally, each use case is independent of the other use cases. However, implicit dependencies among use cases may exist because of dependencies that may exist among use cases at the implementation level due to factors such as shared resources, objects, or functions. For example, in the Library Automation System example, `renew-book` and `reserve-book` are two independent use cases. But, in actual implementation of `renew-book`, a check is to be made to see if any book has been reserved by a previous execution of the `reserve-book` use case. Another example of dependence among use cases is the following. In the Bookshop Automation Software, `update-inventory` and `sale-book` are two independent use cases. But, during execution of `sale-book` there is an implicit dependency on `update-inventory`. Since when sufficient quantity is unavailable in the inventory, `sale-book` cannot operate until the inventory is replenished using `update-inventory`.

The use case model is an important analysis and design artifact. As already

*****ebook converter DEMO - www.ebook-converter.com*****

mentioned, other UML models must conform to this model in any use case-driven (also called as the user-centric) analysis and development approach. It should be remembered that the “use case model” is not really an object-oriented model according to a strict definition of the term.

In contrast to all other types of UML diagrams, the use case model represents a functional or process model of a system.

7.4.1 Representation of Use Cases

A use case model can be documented by drawing a use case diagram and writing an accompanying text elaborating the drawing. In the use case diagram, each use case is represented by an ellipse with the name of the use case written inside the ellipse. All the ellipses (i.e. use cases) of a system are enclosed within a rectangle which represents the system boundary. The name of the system being modeled (e.g., library information system) appears inside the rectangle.

The different users of the system are represented by using stick person icons. Each stick person icon is referred to as an actor.³ An actor is a role played by a user with respect to the system use. It is possible that the same user may play the role of multiple actors. An actor can participate in one or more use cases. The line connecting an actor and the use case is called the communication relationship. It indicates that an actor makes use of the functionality provided by the use case.

Both human users and external systems can be represented by stick person icons. When a stick person icon represents an external system, it is annotated by the stereotype <<external system>>.

At this point, it is necessary to explain the concept of a stereotype in UML. One of the main objectives of the creators of the UML was to restrict the number of primitive symbols in the language. It was clear to them that when a language has a large number of primitive symbols, it becomes very difficult to learn use. To convince yourself, consider that English with 26 alphabets is much easier to learn and use compared to the Chinese language that has thousands of symbols. In this context, the primary objective of stereotype is to reduce the number of different types of symbols that one needs to learn.

The stereotype construct when used to annotate a basic symbol, can give slightly different meaning to the basic symbol— thereby eliminating the need to have several symbols whose meanings differ slightly from each other.

Just as you stereotype your friends as studious, jovial, serious, etc. stereotyping can be used to give special meaning to any basic UML construct. We shall, later on, see how other UML constructs can be stereotyped. We can stereotype the stick person icon symbol to denote an external system. If the developers of UML had assigned a separate symbol to denote things such as an external system, then the number of basic symbols one would have to learn and remember while using UML would have increased significantly. This would have certainly made learning and using UML much more difficult.

You can draw a rectangle around the use cases, called the system boundary box, to indicate the scope of your system. Anything within the box represents functionality that is in scope and anything outside the box is not. However, drawing the system boundary is optional.

We now give a few examples to illustrate how use cases of a system can be documented.

Example 7.2 The use case model for the Tic-tac-toe game software is shown in Figure 7.15. This software has only one use case, namely, “play move”. Note that we did not name the use case “get-user-move”, as “get-user-move” would be inappropriate because this would represent the developer’s perspective of the use case. The use cases should be named from the users’ perspective.

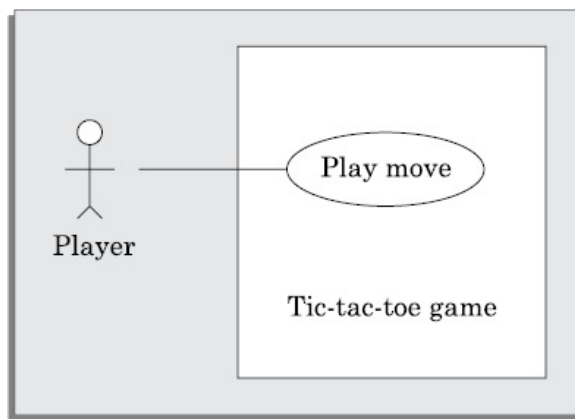


Figure 7.15: Use case model for Example 7.2.

Text description

Each ellipse in a use case diagram, by itself conveys very little information, other than giving a hazy idea about the use case. Therefore, every use case diagram should be accompanied by a text description. The text description should define the details of the interaction between the user and the computer as well as other relevant aspects of the use case. It should include all the behaviour

associated with the use case in terms of the mainline sequence, various alternate sequences, the system responses associated with the use case, the exceptional conditions that may occur in the behaviour, etc. The behaviour description is often written in a conversational style describing the interactions between the actor and the system. The text description may be informal, but some structuring is helpful. The following are some of the information which may be included in a use case text description in addition to the mainline sequence, and the alternate scenarios.

Contact persons: This section lists of personnel of the client organisation with whom the use case was discussed, date and time of the meeting, etc.

Actors: In addition to identifying the actors, some information about actors using a use case which may help the implementation of the use case may be recorded.

Pre-condition: The preconditions would describe the state of the system before the use case execution starts.

Post-condition: This captures the state of the system after the use case has successfully completed.

Non-functional requirements : This could contain the important constraints for the design and implementation, such as platform and environment conditions, qualitative statements, response time requirements, etc.

Exceptions, error situations: This contains only the domain-related errors such as lack of user's access rights, invalid entry in the input fields, etc. Obviously, errors that are not domain related, such as software errors, need not be discussed here.

Sample dialogs: These serve as examples illustrating the use case.

Specific user interface requirements : These contain specific requirements for the user interface of the use case. For example, it may contain forms to be used, screen shots, interaction style, etc.

Document references: This part contains references to specific domain-related documents which may be useful to understand the system operation.

Example 7.3 The use case diagram of the Super market prize scheme described in example 6.3 is shown in Figure 7.16.

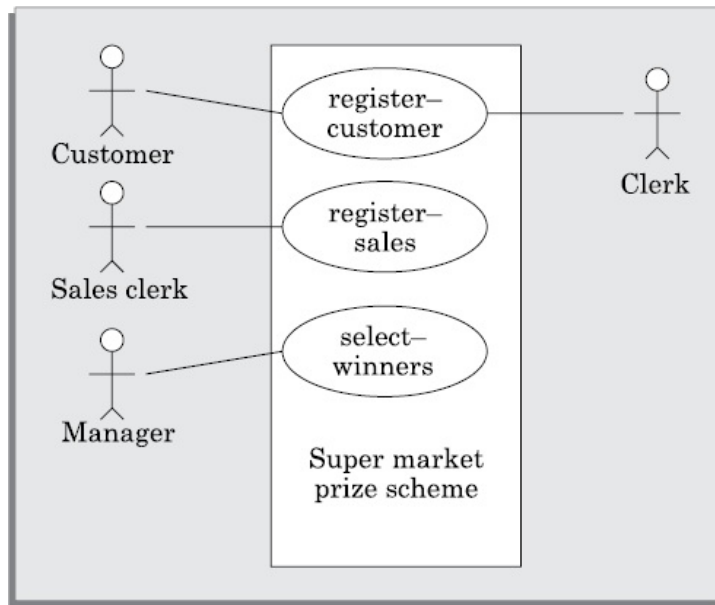


Figure 7.16: Use case model for Example 7.3.

Text description

U1: register-customer: Using this use case, the customer can register himself by providing the necessary details.

Scenario 1: Mainline sequence

1. Customer: select register customer option
2. System: display prompt to enter name, address, and telephone number.
3. Customer: enter the necessary values
4. System: display the generated id and the message that the customer has successfully been registered.

Scenario 2: At step 4 of mainline sequence

4. System: displays the message that the customer has already registered.

Scenario 3: At step 4 of mainline sequence

4. System: displays message that some input information have not been entered. The system displays a prompt to enter the missing values.

U2: register-sales: Using this use case, the clerk can register the details of the purchase made by a customer.

Scenario 1: Mainline sequence

1. Clerk: selects the register sales option.

2. System: displays prompt to enter the purchase details and the id of the customer.

3. Clerk: enters the required details.

4 :System: displays a message of having successfully registered the sale.

U3: select-winners. Using this use case, the manager can generate the winner list.

Scenario 2: Mainline sequence

1. Manager: selects the select-winner option.

2. System: displays the gold coin and the surprise gift winner list.

7.4.2 Why Develop the Use Case Diagram?

If you examine a use case diagram, the utility of the use cases represented by the ellipses would become obvious. They along with the accompanying text description serve as a type of requirements specification of the system and the model based on which all other models are developed. In other words, the use case model forms the core model to which all other models must conform. But, what about the actors (stick person icons)? What way are they useful to system development? One possible use of identifying the different types of users (actors) is in implementing a security mechanism through a login system, so that each actor can invoke only those functionalities to which he is entitled to. Another important use is in designing the user interface in the implementation of the use case targetted for each specific category of users who would use the use case. Another possible use is in preparing the documentation (e.g. users' manual) targeted at each category of user. Further, actors help in identifying the use cases and understanding the exact functioning of the system.

7.4.3 How to Identify the Use Cases of a System?

Identification of the use cases involves brain storming and reviewing the SRS document. Typically, the high-level requirements specified in the SRS document correspond to the use cases. In the absence of a well-formulated SRS document, a popular method of identifying the use cases is actor-based. This involves first identifying the different types of actors and their usage of the system. Subsequently, for each actor the

different functions that they might initiate or participate are identified. For example, for a Library Automation System, the categories of users can be members, librarian, and the accountant. Each user typically focuses on a set of functionalities. For example, the member typically concerns himself with book issue, return, and renewal aspects. The librarian concerns himself with creation and deletion of the member and book records. The accountant concerns itself with the amount collected from membership fees and the expenses aspects.

7.4.4 Essential Use Case versus Real Use Case

Essential use cases are created during early requirements elicitation. These are also early problem analysis artifacts. They are independent of the design decisions and tend to be correct over long periods of time.

Real use cases describe the functionality of the system in terms of its actual current design committed to specific input/output technologies. Therefore, the real use cases can be developed only after the design decisions have been made. Real use cases are a design artifact. However, sometimes organisations commit to development contracts that include the detailed user interface specifications. In such cases, there is no distinction between the essential use case and the real use case.

7.4.5 Factoring of Commonality among Use Cases

It is often desirable to factor use cases into component use cases. All use cases need not be factored. In fact, factoring of use cases are required under two situations as follows:

- Complex use cases need to be factored into simpler use cases. This would not only make the behaviour associated with the use case much more comprehensible, but also make the corresponding interaction diagrams more tractable. Without decomposition, the interaction diagrams for complex use cases may become too large to be accommodated on a single standard-sized (A4) paper.
- Use cases need to be factored whenever there is common behaviour across different use cases. Factoring would make it possible to define such behaviour only once and reuse it wherever required.

It is desirable to factor out common usage such as error handling from a set of use cases. This makes analysis of the class design much simpler and

elegant. However, a word of caution here. Factoring of use cases should not be done except for achieving the above two objectives. From the design point of view, it is not advantageous to break up a use case into many smaller parts just for the sake of it. UML offers three factoring mechanisms as discussed further.

Generalisation

Use case generalisation can be used when you have one use case that is similar to another, but does something slightly differently or something more. Generalisation works the same way with use cases as it does with classes. The child use case inherits the behaviour and meaning of the present use case. The notation is the same too (See Figure 7.17). It is important to remember that the base and the derived use cases are separate use cases and should have separate text descriptions.

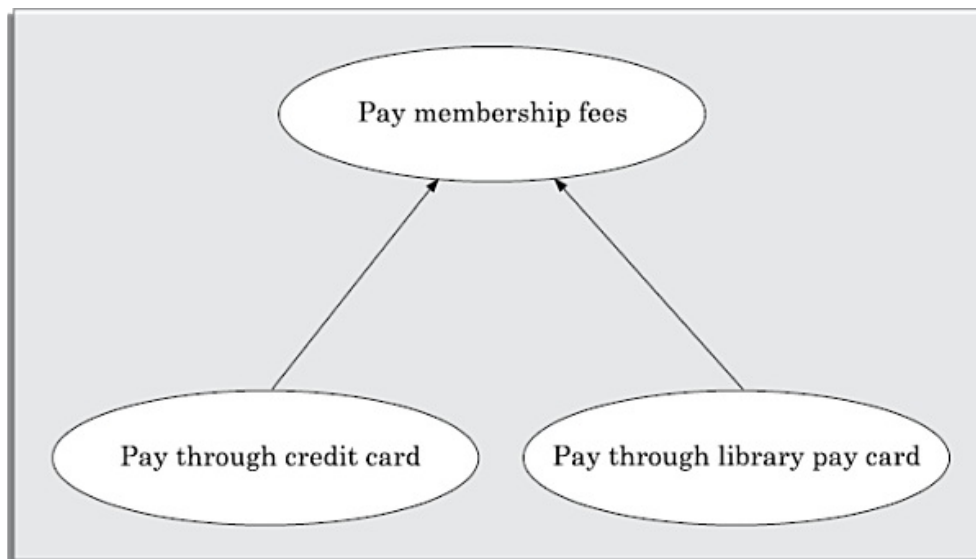


Figure 7.17: Representation of use case generalisation.

Includes

The includes relationship in the older versions of UML (prior to UML 1.1) was known as the uses relationship. The includes relationship implies one use case includes the behaviour of another use case in its sequence of events and actions. The includes relationship is appropriate when you have a chunk of behaviour that is similar across a number of use cases. The factoring of such behaviour will help in not repeating the specification and implementation across different use cases. Thus, the includes relationship explores the issue of reuse by factoring out the commonality across use cases. It can also be gainfully employed to

decompose a large and complex use case into more manageable parts.

As shown in Figure 7.18, the includes relationship is represented using a predefined stereotype <<include>>. In the includes relationship, a base use case compulsorily and automatically includes the behaviour of the common use case. As shown in example Figure 7.19, the use cases *issue-book* and *renew-book* both include *check-reservation* use case. The base use case may include several use cases. In such cases, it may interleave their associated common use cases together. The common use case becomes a separate use case and independent text description should be provided for it.



Figure 7.18: Representation of use case inclusion.

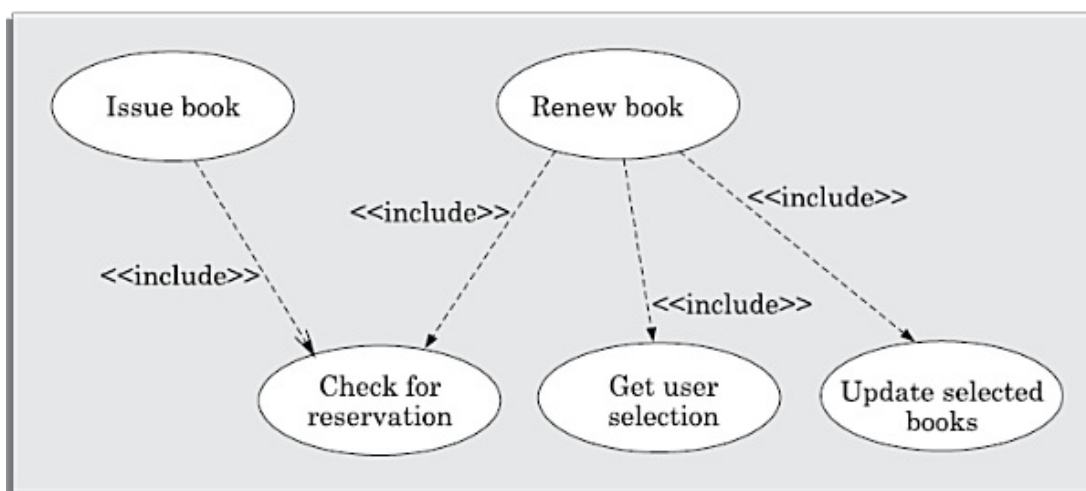


Figure 7.19: Example of use case inclusion.

Extends

The main idea behind the extends relationship among use cases is that it allows you show optional system behaviour. An optional system behaviour is executed only if certain conditions hold, otherwise the optional behaviour is not executed. This relationship among use cases is also predefined as a stereotype as shown in Figure 7.20.

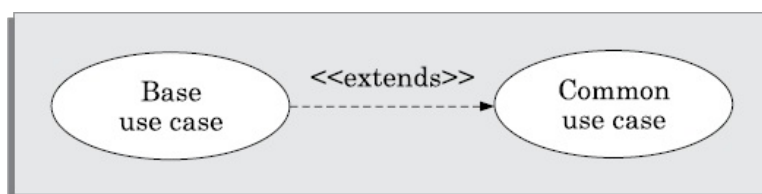


Figure 7.20: Example of use case extension.

The extends relationship is similar to generalisation. But unlike generalisation, the extending use case can add additional behaviour only at an extension point only when certain conditions are satisfied. The extension points are points within the use case where variation to the mainline (normal) action sequence may occur. The extends relationship is normally used to capture alternate paths or scenarios.

Organisation

When the use cases are factored, they are organised hierarchically. The high-level use cases are refined into a set of smaller and more refined use cases as shown in Figure 7.21. Top-level use cases are super-ordinate to the refined use cases. The refined use cases are sub-ordinate to the top-level use cases. Note that only the complex use cases should be decomposed and organised in a hierarchy. It is not necessary to decompose the simple use cases.

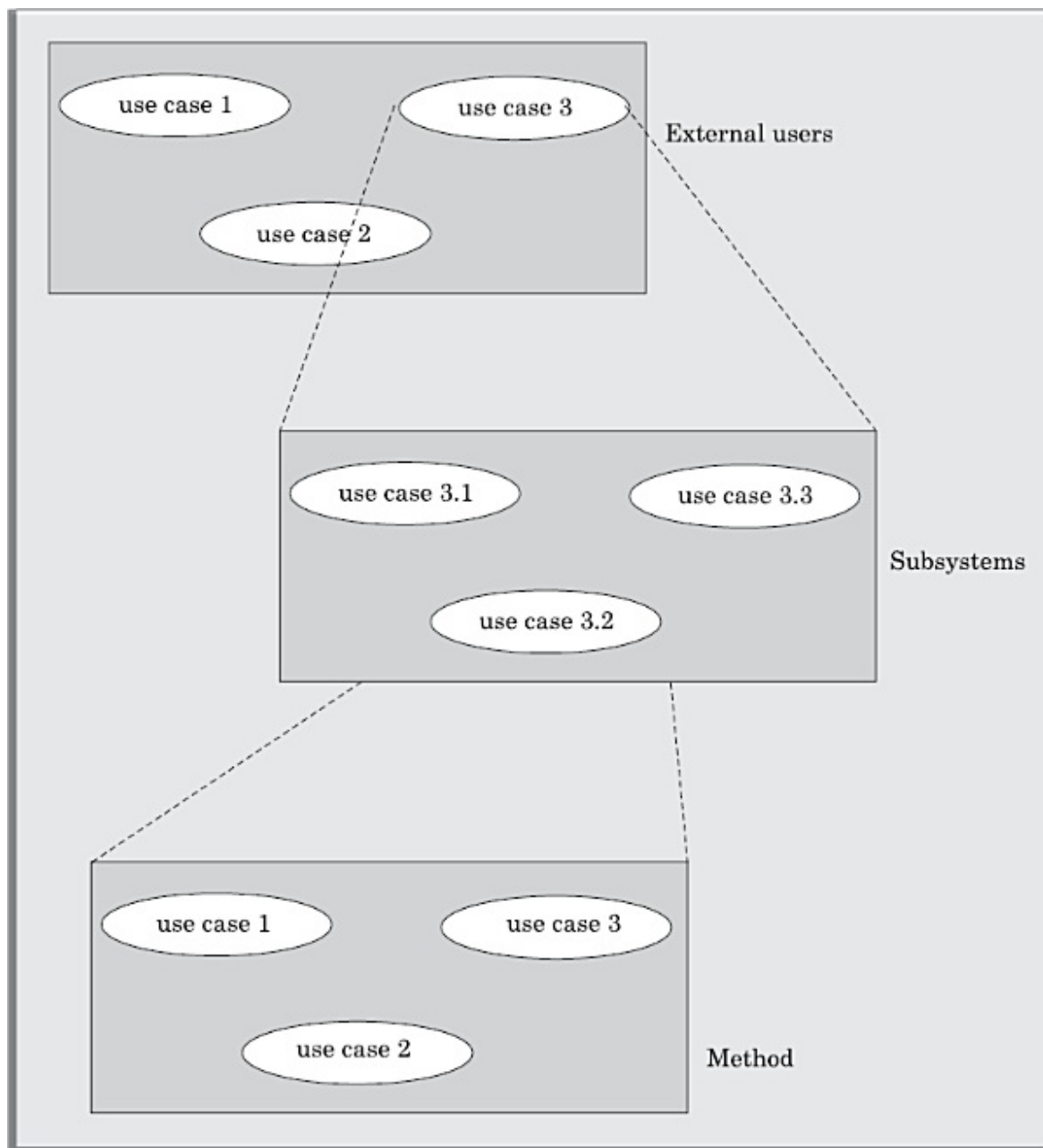


Figure 7.21: Hierarchical organisation of use cases.

The functionality of a super-ordinate use case is traceable to its subordinate use cases. Thus, the functionality provided by the super-ordinate use cases is composite of the functionality of the sub-ordinate use cases.

At the highest level of the use case model, only the fundamental use cases are shown. The focus is on the application context. Therefore, this level is also referred to as the context diagram. In the context diagram, the system limits are emphasised. In the top-level diagram, only those use cases with which external users interact are shown. The topmost use cases specify the complete services offered by the system to the external users of the system. The subsystem-level use cases specify the services offered by the

subsystems. Any number of levels involving the subsystems may be utilized. In the lowest level of the use case hierarchy, the class-level use cases specify the functional fragments or operations offered by the classes.

7.4.6 USE CASE PACKAGING

Packaging is the mechanism provided by UML to handle complexity. When we have too many use cases in the top-level diagram, we can package the related use cases so that at best 6 or 7 packages are present at the top level diagram. Any modeling element that becomes large and complex can be broken up into packages. Please note that you can put any element of UML (including another package) in a package diagram. The symbol for a package is a folder. Just as you organise a large collection of documents in a folder, you organise UML elements into packages. An example of packaging use cases is shown in Figure 7.22.

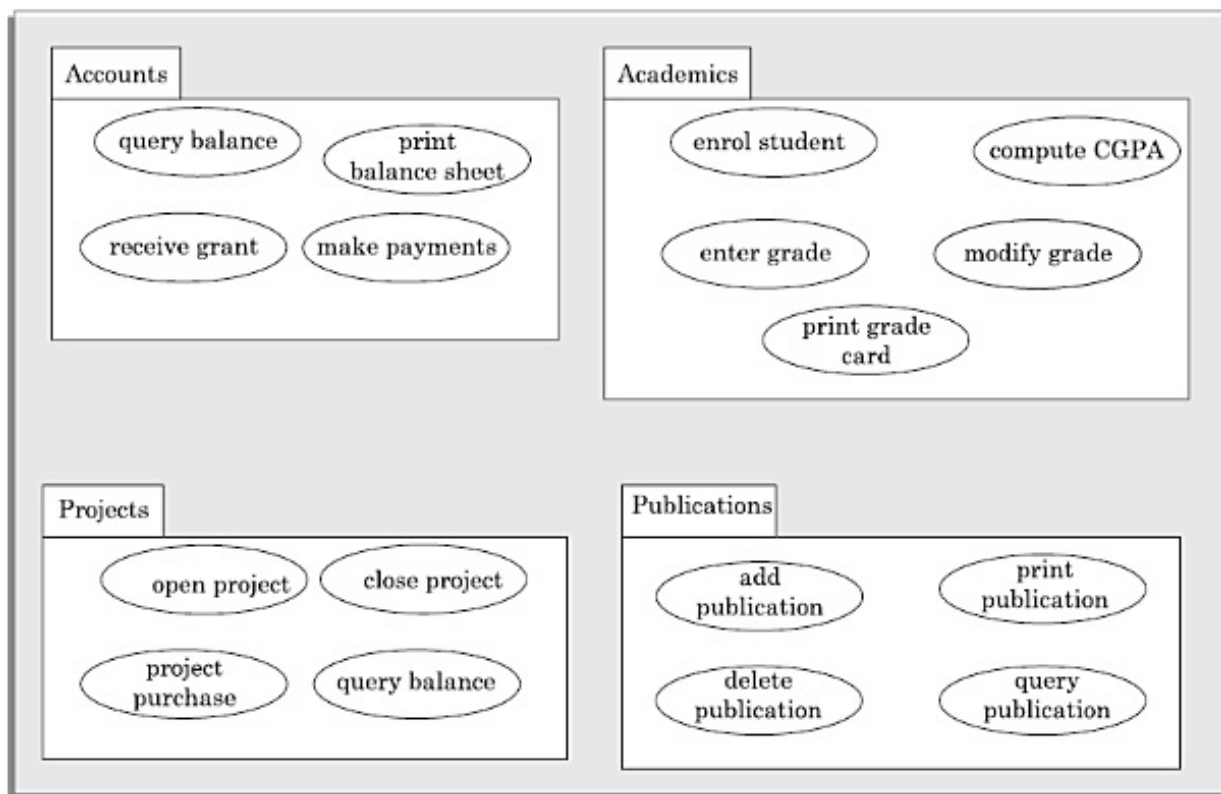


Figure 7.22: Use case packaging.

7.5 CLASS DIAGRAMS

A class diagram describes the static structure of a system. It shows how a system is structured rather than how it behaves. The static structure of a system comprises a number of class diagrams and their dependencies. The main constituents of a class diagram are classes and

their relationships—generalisation, aggregation, association, and various kinds of dependencies. We now discuss the UML syntax for representation of the classes and their relationships.

Classes

The classes represent entities with common features, i.e., attributes and operations. Classes are represented as solid outline rectangles with compartments. Classes have a mandatory name compartment where the name is written centered in boldface. The class name is usually written using mixed case convention and begins with an uppercase (e.g. `LibraryMember`). Object names on the other hand, are written using a mixed case convention, but starts with a small case letter (e.g., `studentMember`). Class names are usually chosen to be singular nouns. An example of various representations of a class are shown in Figure 7.23.

Classes have optional attributes and operations compartments. A class may appear on several diagrams. Its attributes and operations are suppressed on all but one diagram. But, one may wonder why there are so many representations for a class! The answer is that these different notations are used depending on the amount of information about a class is available. At the start of the design process, only the names of the classes is identified. This is the most abstract representation for the class. Later in the design process the methods for the class and the attributes are identified and the other more concrete notations are used.

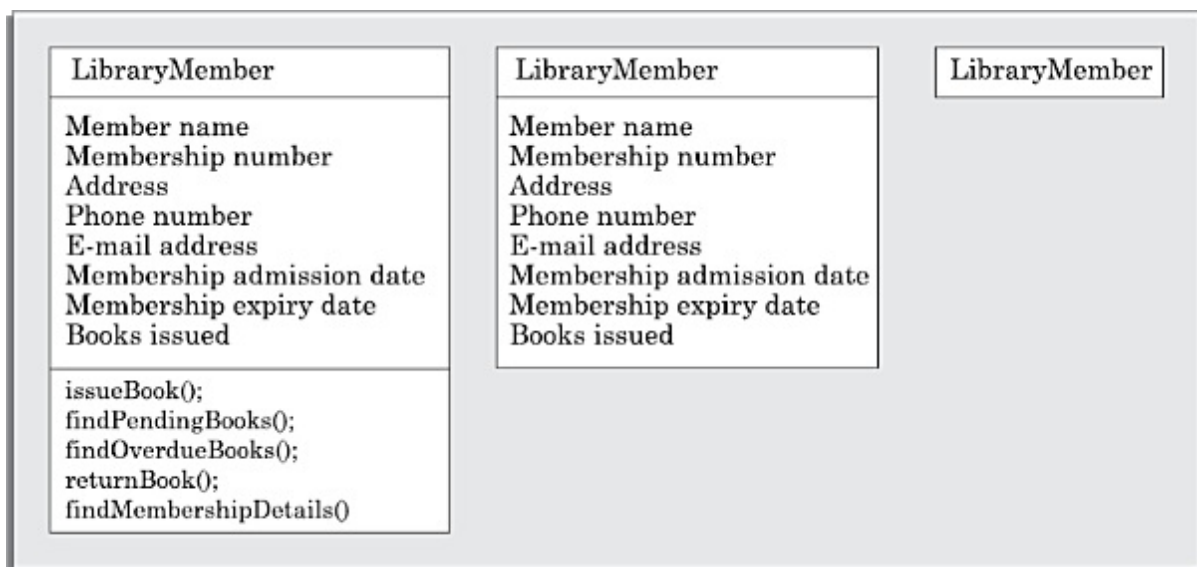


Figure 7.23: Different representations of the `LibraryMember` class.

Attributes

An attribute is a named property of a class. It represents the kind of data that an object might contain. Attributes are listed with their names, and may optionally contain specification of their type (that is, their class, e.g., Int, Book, Employee, etc.), an initial value, and constraints. Attribute names are written left-justified using plain type letters, and the names should begin with a lower case letter.

Attribute names may be followed by square brackets containing a multiplicity expression, e.g. sensorStatus[10]. The multiplicity expression indicates the number of attributes per instance of the class. An attribute without square brackets must hold exactly one value. The type of an attribute is written by following the attribute name with a colon and the type name, (e.g., sensorStatus[1]:Int).

The attribute name may be followed by an initialisation expression. The initialisation expression can consist of an equal sign and an initial value that is used to initialise the attributes of the newly created objects, e.g. sensorStatus[1]:Int=0.

Operation: The operation names are typically left justified, in plain type, and always begin with a lower case letter. Abstract operations are written in italics.⁴ (Remember that abstract operations are those for which the implementation is not provided during the class definition.) The parameters of a function may have a kind specified. The kind may be "in" indicating that the parameter is passed into the operation; or "out" indicating that the parameter is only returned from the operation; or "inout" indicating that the parameter is used for passing data into the operation and getting result from the operation. The default is "in".

An operation may have a return type consisting of a single return type expression, e.g., issueBook(in bookName):Boolean. An operation may have a class scope (i.e., shared among all the objects of the class) and is denoted by underlining the operation name.

Often a distinction is made between the terms operation and method. An operation is something that is supported by a class and invoked by objects of other classes. There can be multiple methods implementing the same operation. We have pointed out earlier that this is called static polymorphism. The method names can be the same; however, it should be possible to distinguish among the methods by examining their parameters. Thus, the terms operation and method are distinguishable only when there is

polymorphism. When there is only a single method implementing an operation, the terms method and operation are indistinguishable and can be used interchangeably.

Association

Association between two classes is represented by drawing a straight line between the concerned classes. Figure 7.24 illustrates the graphical representation of the association relation. The name of the association is written along side the association line. An arrowhead may be placed on the association line to indicate the reading direction of the association. The arrowhead should not be misunderstood to be indicating the direction of a pointer implementing an association. On each side of the association relation, the multiplicity is noted as an individual number or as a value range. The multiplicity indicates how many instances of one class are associated with the other. Value ranges of multiplicity are noted by specifying the minimum and maximum value, separated by two dots, e.g. 1..5. An asterisk is used as a wild card and means many (zero or more). The association of Figure 7.24 should be read as "Many books may be borrowed by a LibraryMember". Usually, associations (and links) appear as verbs in the problem statement.

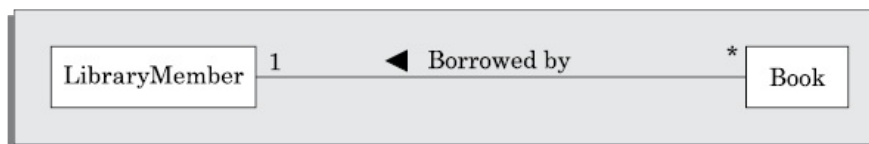


Figure 7.24: Association between two classes.

Associations are usually realised by assigning appropriate reference attributes to the classes involved. Thus, associations can be implemented using pointers from one object class to another. Links and associations can also be implemented by using a separate class that stores which objects of a class are linked to which objects of another class. Some CASE tools use the role names of the association relation for the corresponding automatically generated attribute.

Aggregation

Aggregation is a special type of association relation where the involved classes are not only associated to each other, but a whole-part relationship exists between them. That is, the aggregate object not only knows the addresses of its parts and therefore invoke the methods of its parts, but also takes the responsibility of creating and destroying

its parts. An example of aggregation, a book register is an aggregation of book objects. Books can be added to the register and deleted as and when required.

Aggregation is represented by an empty diamond symbol at the aggregate end of a relationship. An example of the aggregation relationship has been shown in Fig 7.25. The figure represents the fact that a document can be considered as an aggregation of paragraphs. Each paragraph can in turn be considered as aggregation of lines. Observe that the number 1 is annotated at the diamond end, and a * is annotated at the other end. This means that one document can have many paragraphs. On the other hand, if we wanted to indicate that a document consists of exactly 10 paragraphs, then we would have written number 10 in place of the (*).

The aggregation relationship cannot be reflexive (i.e. recursive). That is, an object cannot contain objects of the same class as itself. Also, the aggregation relation is not symmetric. That is, two classes A and B cannot contain instances of each other. However, the aggregation relationship can be transitive. In this case, aggregation may consist of an arbitrary number of levels. As an example of a transitive aggregation relationship, please see Figure 7.25.

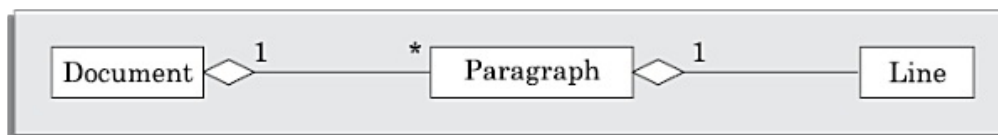


Figure 7.25: Representation of aggregation.

Composition

Composition is a stricter form of aggregation, in which the parts are existence-dependent on the whole. This means that the life of the parts cannot exist outside the whole. In other words, the lifeline of the whole and the part are identical. When the whole is created, the parts are created and when the whole is destroyed, the parts are destroyed.

A typical example of composition is an order object where after placing the order, no item in the order cannot be changed. If any changes to any of the order items are required after the order has been placed, then the entire order has to be cancelled and a new order has to be placed with the changed items. In this case, as soon as an order object is created, all the order items in it are created and as soon as the order object is destroyed, all order items in it are also destroyed. That is, the life of the components (order items) is the same as the aggregate (order). The composition relationship is

represented as a filled diamond drawn at the composite-end. An example of the composition relationship is shown in Figure 7.26.

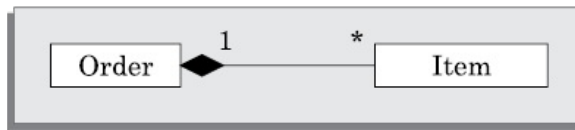


Figure 7.26: Representation of composition.

Aggregation versus Composition: Both aggregation and composition represent part/whole relationships. When the components can dynamically be added and removed from the aggregate, then the relationship is aggregation. If the components cannot be dynamically added/delete then the components are have the same life time as the composite. In this case, the relationship is represented by composition.

As an example, consider the example of an order consisting many order items. If the order once placed, the items cannot be changed at all. In this case, the order is a composition of order items. However, if order items can be changed (added, delete, and modified) after the order has been placed, then aggregation relation can be used to model it.

Inheritance

The inheritance relationship is represented by means of an empty arrow pointing from the subclass to the superclass. The arrow may be directly drawn from the subclass to the superclass. Alternatively, when there are many subclasses of a base class, the inheritance arrow from the subclasses may be combined to a single line (see Figure 7.27) and is labelled with the aspect of the class that is abstracted.

The direct arrows allow flexibility in laying out the diagram and can easily be drawn by hand. The combined arrows emphasise the collectivity of the subclasses, when specialisation has been done on the basis of some discriminator. In the example of Figure 7.27, issuable and reference are the discriminators. The various subclasses of a superclass can then be differentiated by means of the discriminator. The set of subclasses of a class having the same discriminator is called a partition. It is often helpful to mention the discriminator during modelling, as these become documented design decisions.

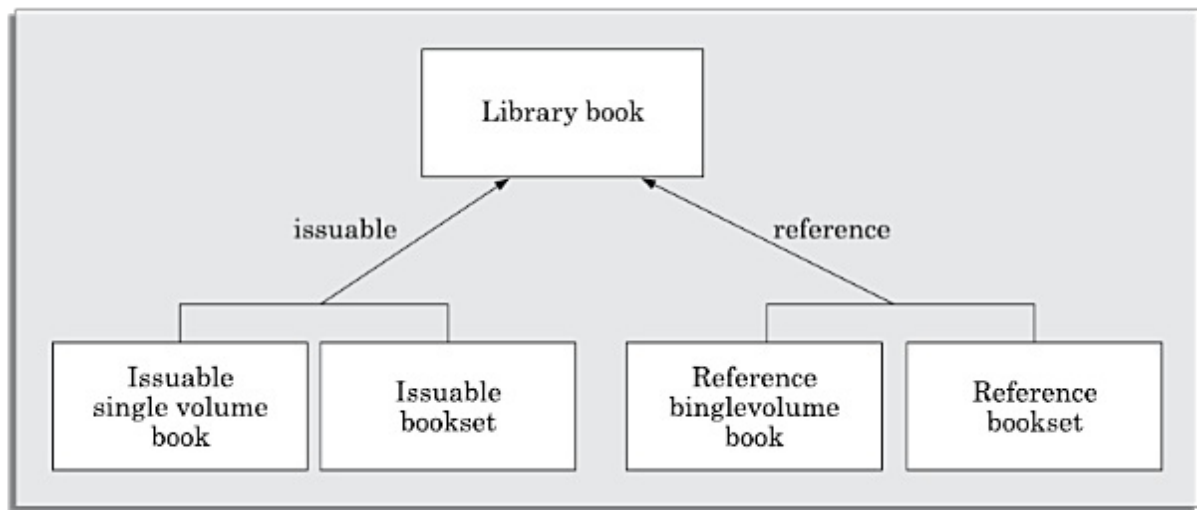


Figure 7.27: Representation of the inheritance relationship.

Dependency

A dependency relationship is shown as a dotted arrow (see Figure 7.28) that is drawn from the dependent class to the independent class.



Figure 7.28: Representation of dependence between classes.

Constraints

A constraint describes a condition or an integrity rule. Constraints are typically used to describe the permissible set of values of an attribute, to specify the pre- and post-conditions for operations, to define certain ordering of items, etc. For example, to denote that the books in a library are sorted on ISBN number we can annotate the book class with the constraint

{sorted}. UML allows you to use any free form expression to describe constraints. The only rule is that they are to be enclosed within braces. Constraints can be expressed using informal English. However, UML also provides object constraint language (OCL) to specify constraints. In OCL the constraints are specified a semi-formal language, and therefore it is more amenable to automatic processing as compared to the informal constraints enclosed within {}. The interested reader is referred to [Rumbaugh1999].

Object diagrams

Object diagrams shows the snapshot of the objects in a system at a point in

time. Since it shows instances of classes, rather than the classes themselves, it is often called as an instance diagram. The objects are drawn using rounded rectangles (see Figure 7.29).

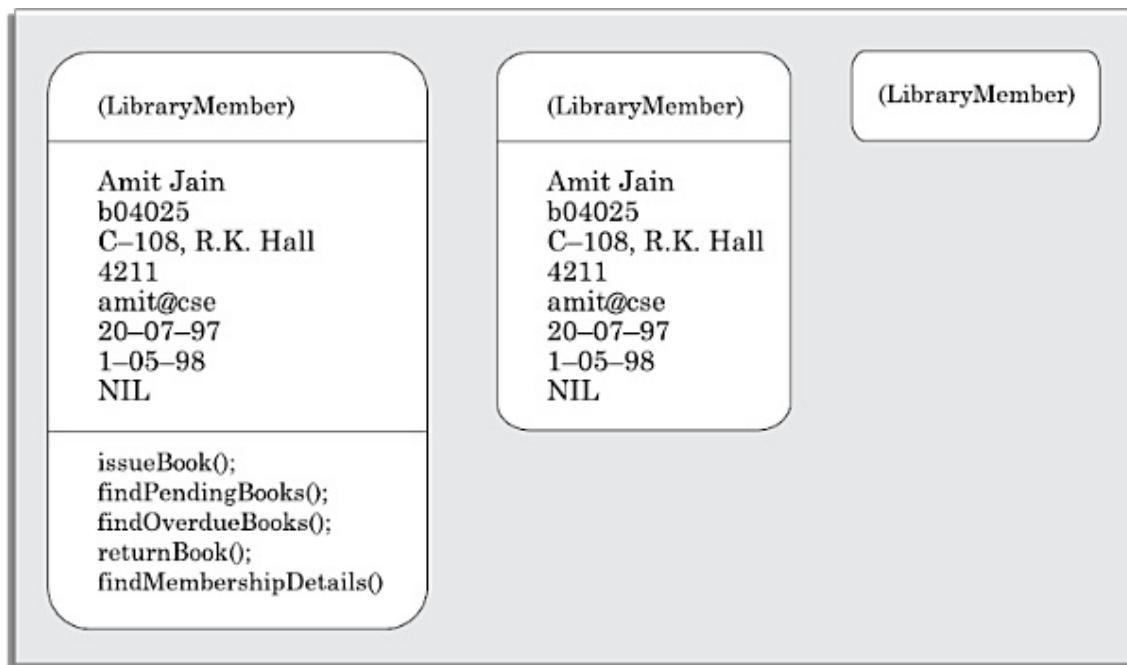


Figure 7.29: Different representations of a LibraryMember object.

An object diagram may undergo continuous change as execution proceeds. For example, links may get formed between objects and get broken. Objects may get created and destroyed, and so on. Object diagrams are useful to explain the working of a system.

7.6 INTERACTION DIAGRAMS

When a user invokes one of the functions supported by a system, the required behaviour is realised through the interaction of several objects in the system. Interaction diagrams, as their name itself implies, are models that describe how groups of objects interact among themselves through message passing to realise some behaviour.

Typically, each interaction diagram realises the behaviour of a single use case.

Sometimes, especially for complex use cases, more than one interaction diagrams may be necessary to capture the behaviour. An interaction diagram shows a number of example objects and the messages that are passed between the objects within the use case.

There are two kinds of interaction diagrams—sequence diagrams and collaboration diagrams. These two diagrams are equivalent in the sense that

any one diagram can be derived automatically from the other. However, they are both useful. These two actually portray different perspectives of behaviour of a system and different types of inferences can be drawn from them. The interaction diagrams play a major role in any effective object-oriented design process. We discuss this issue in Chapter 8.

Sequence diagram

A sequence diagram shows interaction among objects as a two dimensional chart. The chart is read from top to bottom. The objects participating in the interaction are shown at the top of the chart as boxes attached to a vertical dashed line. Inside the box the name of the object is written with a colon separating it from the name of the class and both the name of the object and the class are underlined. This signifies that we are referring any arbitrary instance of the class. For example, in Figure 7.30 :Book represents any arbitrary instance of the Book class.

An object appearing at the top of the sequence diagram signifies that the object existed even before the time the use case execution was initiated. However, if some object is created during the execution of the use case and participates in the interaction (e.g., a method call), then the object should be shown at the appropriate place on the diagram where it is created.

The vertical dashed line is called the object's lifeline. Any point on the lifeline implies that the object exists at that point. Absence of lifeline after some point indicates that the object ceases to exist after that point in time, particular point of time. Normally, at the point if an object is destroyed, the lifeline of the object is crossed at that point and the lifeline for the object is not drawn beyond that point. A rectangle called the activation symbol is drawn on the lifeline of an object to indicate the points of time at which the object is active. Thus an activation symbol indicates that an object is active as long as the symbol (rectangle) exists on the lifeline. Each message is indicated as an arrow between the lifelines of two objects. The messages are shown in chronological order from the top to the bottom. That is, reading the diagram from the top to the bottom would show the sequence in which the messages occur.

Each message is labelled with the message name. Some control information can also be included. Two important types of control information are:

- A condition (e.g., [invalid]) indicates that a message is sent, only if the condition is true.
- An iteration marker shows that the message is sent many times to multiple receiver objects as would happen when you are iterating over a collection or the elements of an array. You can also indicate the basis of the iteration, e.g., [for every book object].

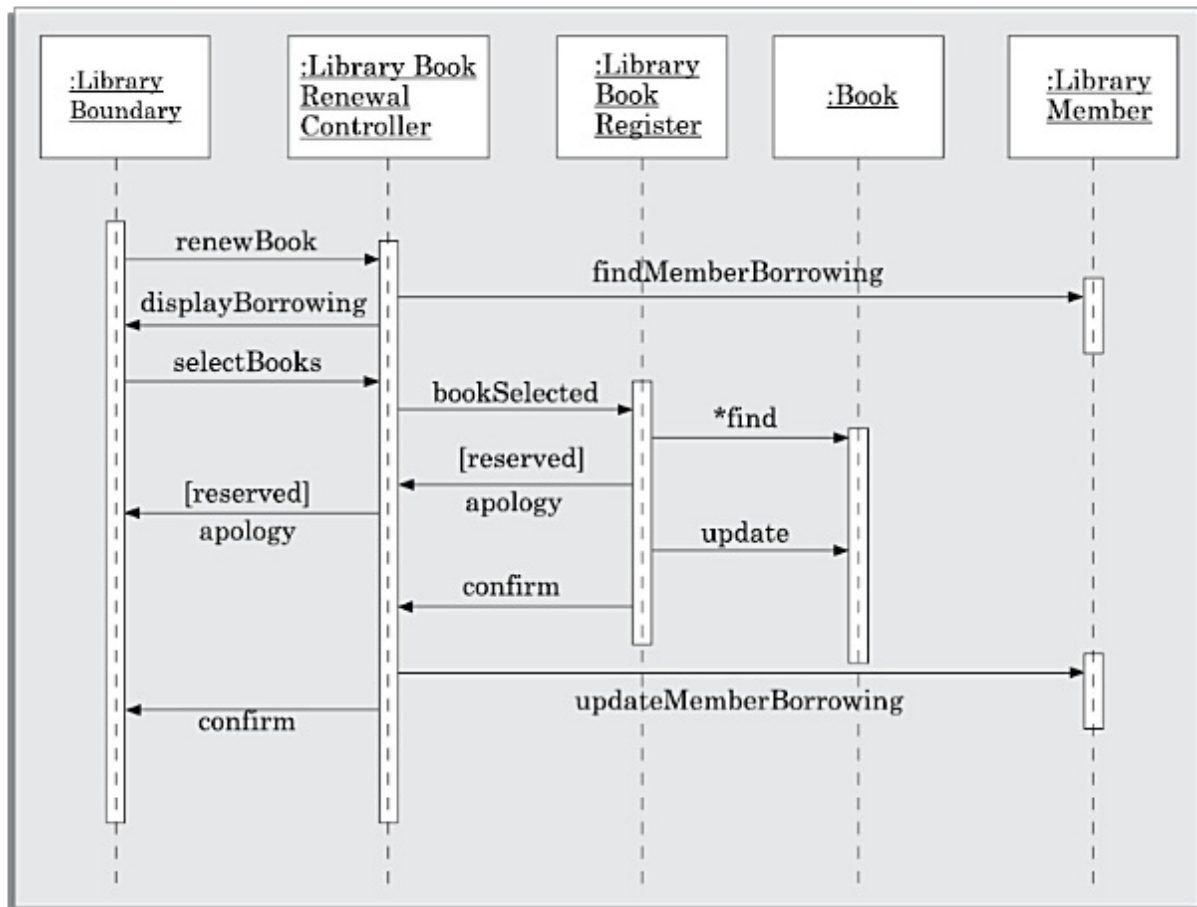


Figure 7.30: Sequence diagram for the renew book use case

The sequence diagram for the book renewal use case for the Library Automation Software is shown in Figure 7.30. Observe that the exact objects which participate to realise the renew book behaviour and the order in which they interact can be clearly inferred from the sequence diagram. The development of the sequence diagram in the development methodology (discussed in Chapter 8) would help us to determine the responsibilities that must be assigned to the different classes; i.e., what methods should be supported by each class.

Collaboration diagram

A collaboration diagram shows both structural and behavioural aspects

explicitly. This is unlike a sequence diagram which shows only the behavioural aspects. The structural aspect of a collaboration diagram consists of objects and links among them indicating association. In this diagram, each object is also called a collaborator. The behavioural aspect is described by the set of messages exchanged among the different collaborators.

The link between objects is shown as a solid line and can be used to send messages between two objects. The message is shown as a labelled arrow placed near the link. Messages are prefixed with sequence numbers because they are the only way to describe the relative sequencing of the messages in this diagram.

The collaboration diagram for the example of Figure 7.30 is shown in Figure 7.31. Use of the collaboration diagrams in our development process would be to help us to determine which classes are associated with which other classes.

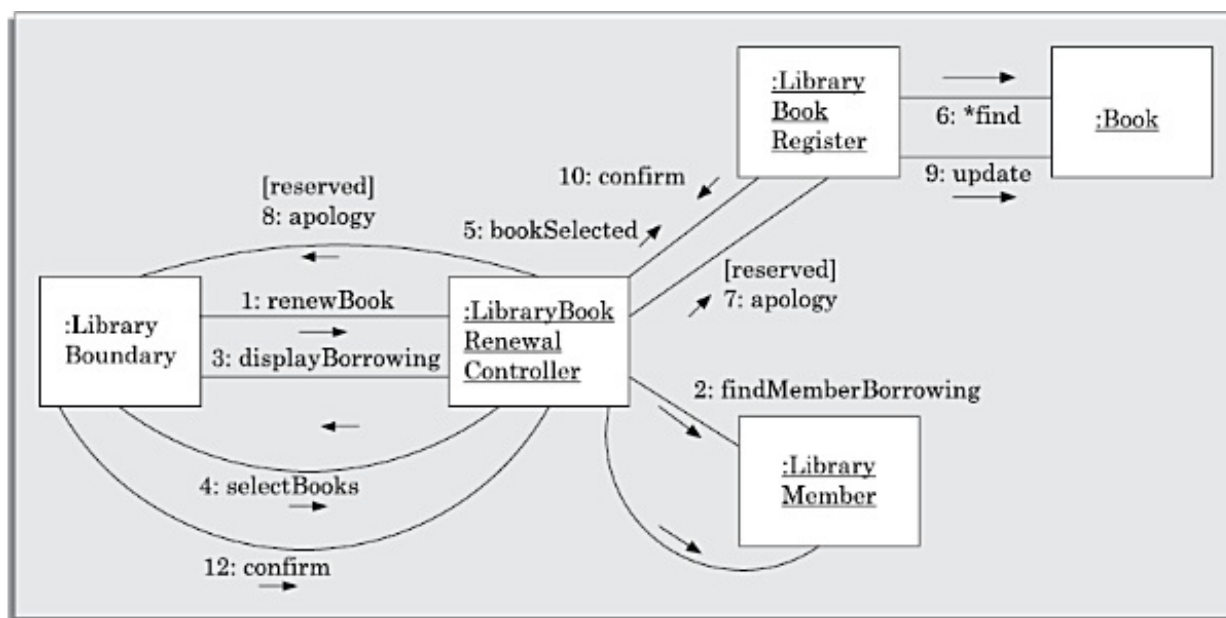


Figure 7.31: Collaboration diagram for the renew book use case.

7.7 ACTIVITY DIAGRAM

The activity diagram is possibly one modelling element which was not present in any of the predecessors of UML. No such diagrams were present either in the works of Booch, Jacobson, or Rumbaugh. It has possibly been based on the event diagram of Odell [1992] though the notation is very different from that used by Odell.

The activity diagram focuses on representing various activities or chunks of processing and their sequence of activation. The activities in general may not

correspond to the methods of classes. An activity is a state with an internal action and one or more outgoing transitions which automatically follow the termination of the internal activity. If an activity has more than one outgoing transitions, then exact situation under which each is executed must be identified through appropriate conditions.

Activity diagrams are similar to the procedural flow charts. The main difference is that activity diagrams support description of parallel activities and synchronisation aspects involved in different activities.

Parallel activities are represented on an activity diagram by using swim lanes. Swim lanes enable you to group activities based on who is performing them, e.g., academic department vs. hostel office. Thus swim lanes subdivide activities based on the responsibilities of some components. The activities in a swim lanes can be assigned to some model elements, e.g. classes or some component, etc. For example, in Figure 7.32 the swim lane corresponding to the academic section, the activities that are carried out by the academic section and the specific situation in which these are carried out are shown.

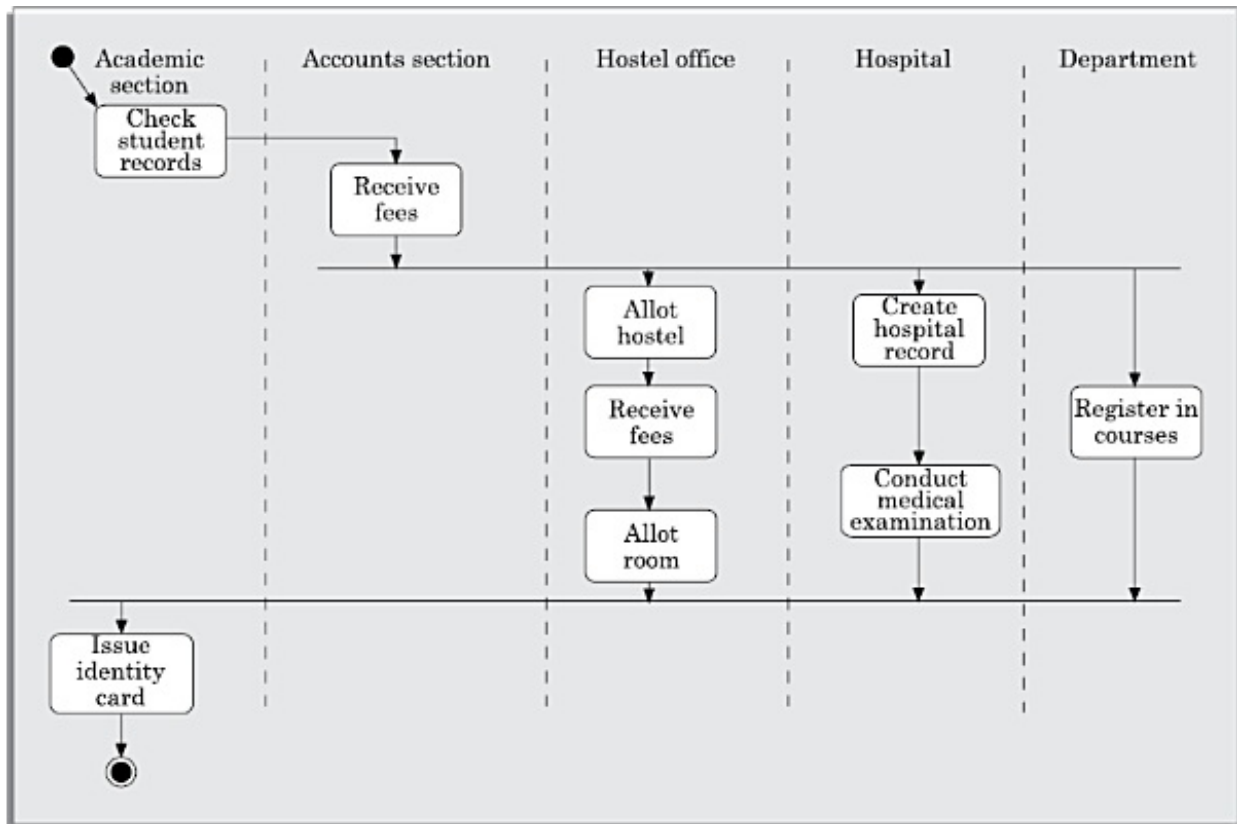


Figure 7.32: Activity diagram for student admission procedure at IIT.

Activity diagrams are normally employed in business process modelling. This is carried out during the initial stages of requirements analysis and specification. Activity diagrams can be very useful to understand complex

processing activities involving the roles played by many components. Besides helping the developer to understand the complex processing activities, these diagrams can also be used to develop interaction diagrams which help to allocate activities (responsibilities) to classes.

The student admission process in IIT is shown as an activity diagram in Figure 7.32. This shows the part played by different components of the Institute in the admission procedure. After the fees are received at the account section, parallel activities start at the hostel office, hospital, and the Department. After all these activities complete (this is a synchronisation issue and is represented as a horizontal line), the identity card can be issued to a student by the Academic section.

7.8 STATE CHART DIAGRAM

A state chart diagram is normally used to model how the state of an object changes in its life time. State chart diagrams are good at describing how the behaviour of an object changes across several use case executions. However, if we are interested in modelling some behaviour that involves several objects collaborating with each other, state chart diagram is not appropriate. We have already seen that such behaviour is better modelled using sequence or collaboration diagrams. State chart diagrams are based on the finite state machine (FSM) formalism. An FSM consists of a finite number of states corresponding to those of the object being modelled. The object undergoes state changes when specific events occur. The FSM formalism existed long before the object-oriented technology and has been used for a wide variety of applications. Apart from modelling, it has even been used in theoretical computer science as a generator for regular languages.

Why state chart?

A major disadvantage of the FSM formalism is the state explosion problem. The number of states becomes too many and the model too complex when used to model practical systems. This problem is overcome in UML by using state charts. The state chart formalism was proposed by David Harel [1990]. A state chart is a hierarchical model of a system and introduces the concept of a composite state (also called nested state).

Actions are associated with transitions and are considered to be processes that occur quickly and are not interruptible. Activities are associated with

states and can take longer. An activity can be interrupted by an event.

Basic elements of a state chart

The basic elements of the state chart diagram are as follows:

Initial state: This is represented as a filled circle.

Final state: This is represented by a filled circle inside a larger circle.

State: These are represented by rectangles with rounded corners.

Transition: A transition is shown as an arrow between two states. Normally, the name of the event which causes the transition is placed along side the arrow. You can also assign a guard to the transition. A guard is a Boolean logic condition. The transition can take place only if the guard evaluates to true. The syntax for the label of the transition is shown in 3 parts—[guard]event/action.

An example state chart for the order object of the Trade House Automation software is shown in Figure 7.33. Observe that from Rejected order state, there is an automatic and implicit transition to the end state. Such transitions are called pseudo transitions.

7.9 POSTSCRIPT

UML has gained rapid acceptance among practitioners and academicians over a short time and has proved its utility in arriving at good design solutions to software development problems.

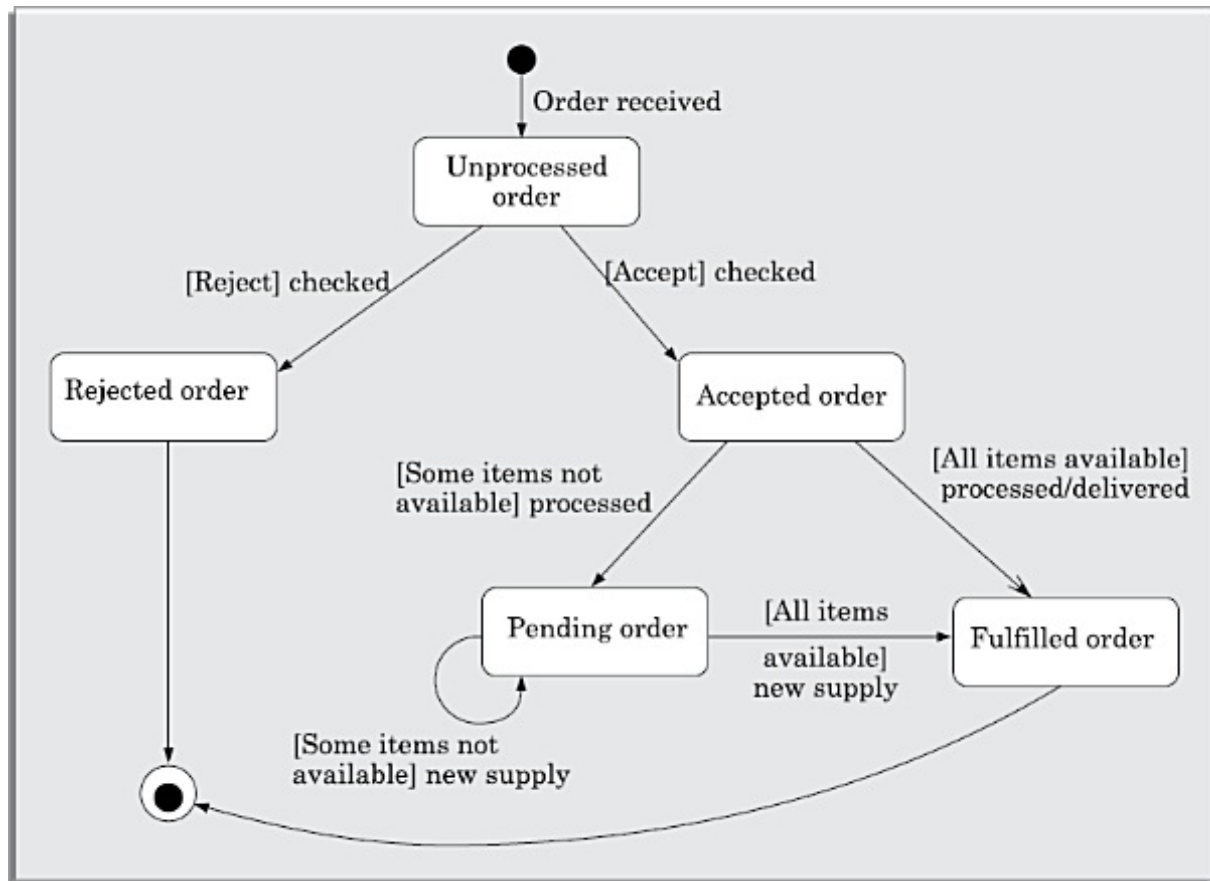


Figure 7.33: State chart diagram for an order object.

In this text, we have kept our discussions on UML to a bare minimum and have concentrated only on those aspects that are necessary to solve moderate sized traditional software design problems.

Before concluding this chapter, we give an overview of some of the aspects that we had chosen to leave out. We first discuss the package and deployment diagrams. Since UML has undergone a significant change with the release of UML 2.0 in 2003. We briefly mention the highlights of the improvements brought about UML 2.0 over the UML 1.X which was our focus so far. This significant revision was necessitated to make UML applicable to the development of software for emerging embedded and telecommunication domains.

7.9.1 Package, Component, and Deployment Diagrams

In the following subsections we provide a brief overview of the package, component, and deployment diagrams:

Package diagram

A package is a grouping of several classes. In fact, a package diagram can be

used to group any UML artifacts. We had already discussed packaging of use cases in Section 7.4.6. Packages are popular way of organising source code files. Java packages are a good example which can be modelled using a package diagram. Such package diagrams show the different class groups (packages) and their inter dependencies. These are very useful to document organisation of source files for large projects that have a large number of program files. An example of a package diagram has been shown in Figure 7.34.

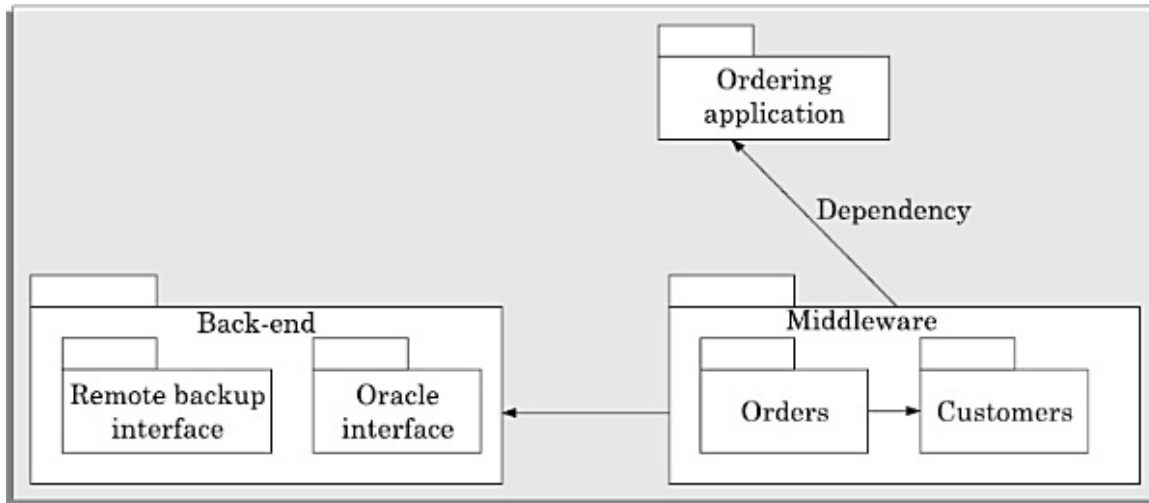


Figure 7.34: An example package diagram.

Note, that a package may contain further packages.

Component diagram

A component represents a piece of software that can be independently purchased, upgraded, and integrated into an existing software. A component diagram can be used to represent the physical structure of an implementation in terms of the various components of the system. A component diagram is typically used to achieve the following purposes:

- Organise source code to be able to construct executable releases.
- Specify dependencies among different components.

A package diagram can be used to provide a high-level view of each component in terms the different classes it contains.

Deployment diagram

The deployment diagram shows the environmental view of a system. That is, it captures the environment in which the software solution is implemented. In other words, a deployment diagram shows how a

software system will be physically deployed in the hardware environment. That is, which component will execute on which hardware component and how they will they communicate with each other. Since the diagram models the run time architecture of an application, this diagram can be very useful to the system's operation staff.

The environmental view provided by the deployment diagram is important for complex and large software solutions that run on hardware systems comprising multiple components. In this case, deployment diagram provides an overview of how the different components are distributed among the different hardware components of the system.

7.9.2 UML 2.0

UML 1.X lacked a few specialised capabilities that made it difficult to use in some non- traditional domains. Some of the features that prominently lacked in UML 1.X include lack of support for representation of the following—concurrent execution of methods, development domain, asynchronous messages, events, ports, and active objects. In many applications, including the embedded and telecommunication software development, capability to model timing requirements using a timing diagram was urgently required to make UML applicable in these important segments of software development. Further, certain changes were required to support interoperability among UML-based CASE tools using XML metadata interchange (XMI).

UML 2.0 defines thirteen types of diagrams, divided into three categories as follows:

Structure diagrams: These include the class diagram, object diagram, component diagram, composite structure diagram, package diagram, and deployment diagram.

Behaviour diagrams: These diagrams include the use case diagram, activity diagram, and state machine diagram.

Interaction diagrams: These diagrams include the sequence diagram, communication diagram, timing diagram, and interaction overview diagram. The collaboration diagram of UML 1.X has been renamed in UML 2.0 as communication diagram. This renaming was necessary as the earlier name was somewhat misleading, it shows the communications among the classes during the execution of a use case rather than showing collaborative problem solving.

Though a large number of new features have been introduced in UML 2.0 as compared to 1.X, in the following subsections, we discuss only two of the enhancements in UML2.0 through combined fragments and composite structure diagram.

Combined fragments in sequence diagrams

A combined fragment is a construct that has been introduced in UML 2.0 to allow description of various control and logic structures in a more visually apparent and concise manner. It also allows representation of concurrent execution behaviour such as that takes place in a multithreaded execution situation.

Let us now understand the anatomy of a combined fragment and its use. A combined fragment divides a sequence diagram into a number of areas or fragments that have different behaviour (see Figure 7.35). A combined fragment appears over an area of a sequence diagram to make certain control and logic aspects visually clear. As shown in Figure 7.35, a combined fragment consists of many fragments and an operator shown at the top left corner. Each fragment can be associated with a guard (a Boolean expression). We now discuss these components of a combined fragment:

Fragment: A fragment in a sequence diagram is represented by a box, and encloses a portion of the interactions within a sequence diagram. Each fragment is also known as an interaction operand. An interaction operand may contain an optional guard condition, which is also called an interaction constraint. The behaviour specified in an interaction operand is executed only if its guard condition evaluates to true.

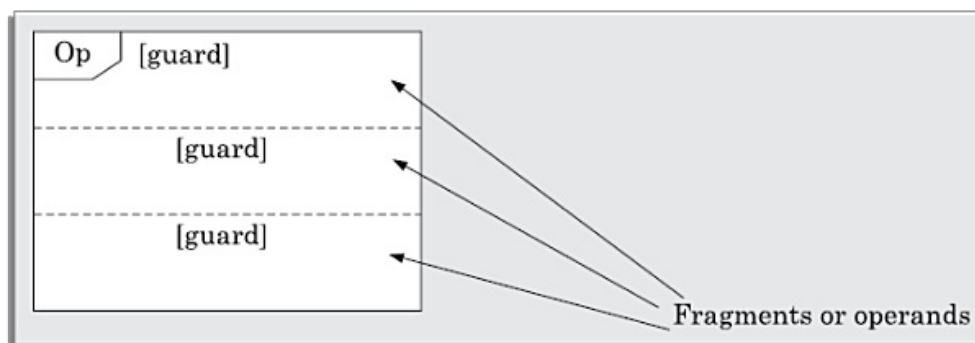


Figure 7.35: Anatomy of a combined fragment in UML 2.0.

Operator: A combined fragment is associated with one operator called interaction operator that is shown at the top left corner of the fragment. The operator indicates the type of fragment. The type of logic operator along with the guards in the fragment defines the behaviour of the

combined fragment. A combined fragment can also contain nested combined fragments or interaction uses containing additional conditional structures that represent more complex structures that affect the flow of messages.

Some of the important operators of a combined fragment are the following:

alt: This operator indicates that among multiple fragments, only the one whose guard is true will execute.

opt: An optional fragment that will execute only if the guard is true.

par: This operator indicates that various fragments can execute at the same time.

loop: A loop operator indicates that the various fragments may execute multiple times and the guard indicates the basis of iteration, meaning that the execution would continue until the guard turns false.

region: It defines a critical region in which only one thread can execute.

An example of a combined fragment has been shown in Figure 7.36.

Composite structure diagram

The composite structure diagram lets you define how a class is defined by a further structure of classes and the communication paths between these parts. Some new core constructs such as parts, ports and connectors are introduced.

Part: The concept of parts makes possible the description of the internal structure of a class.

Port: The concept of a port makes it possible to describe connection points formally. These are addressable, which means that signals can be sent to them.

Connector: Connectors can be used to specify the communication links between two or more parts.

Chapter 10

CODING AND TESTING

In this chapter, we will discuss the coding and testing phases of the software life cycle.

Coding is undertaken once the design phase is complete and the design documents have been successfully reviewed.

In the coding phase, every module specified in the design document is coded and unit tested. During unit testing, each module is tested in isolation from other modules. That is, a module is tested independently as and when its coding is complete.

After all the modules of a system have been coded and unit tested, the integration and system testing phase is undertaken.

Integration and testing of modules is carried out according to an integration plan. The integration plan, according to which different modules are integrated together, usually envisages integration of modules through a number of steps. During each integration step, a number of modules are added to the partially integrated system and the resultant system is tested. The full product takes shape only after all the modules have been integrated together. System testing is conducted on the full product. During system testing, the product is tested against its requirements as recorded in the SRS document.

We had already pointed out in Chapter 2 that testing is an important phase in software development and typically requires the maximum effort among all the development phases. Usually, testing of a professional software is carried out using a large number of test cases. It is usually the case that many of the different test cases can be executed in parallel by different team members. Therefore, to reduce the testing time, during the testing phase the largest manpower (compared to all other life cycle phases) is deployed. In a typical development organisation, at any time, the maximum number of software

engineers can be found to be engaged in testing activities. It is not very surprising then that in the software industry there is always a large demand for software test engineers. However, many novice engineers bear the wrong impression that testing is a secondary activity and that it is intellectually not as stimulating as the activities associated with the other development phases.

Over the years, the general perception of testing as monkeys typing in random data and trying to crash the system has changed. Now testers are looked upon as masters of specialised concepts, techniques, and tools.

As we shall soon realize, testing a software product is as much challenging as initial development activities such as specifications, design, and coding. Moreover, testing involves a lot of creative thinking.

In this Chapter, we first discuss some important issues associated with the activities undertaken in the coding phase. Subsequently, we focus on various types of program testing techniques for procedural and object-oriented programs.

10.1 CODING

The input to the coding phase is the design document produced at the end of the design phase. Please recollect that the design document contains not only the high-level design of the system in the form of a module structure (e.g., a structure chart), but also the detailed design. The detailed design is usually documented in the form of module specifications where the data structures and algorithms for each module are specified. During the coding phase, different modules identified in the design document are coded according to their respective module specifications. We can describe the overall objective of the coding phase to be the following.

The objective of the coding phase is to transform the design of a system into code in a high-level language, and then to unit test this code.

Normally, good software development organisations require their programmers to adhere to some well-defined and standard style of coding which is called their coding standard. These software development organisations formulate their own coding standards that suit them the most, and require their developers to follow the standards rigorously because of the significant business advantages it offers. The main advantages of adhering to a standard style of coding are the following:

- A coding standard gives a uniform appearance to the codes written by different engineers.
- It facilitates code understanding and code reuse.
- It promotes good programming practices.

A coding standard lists several rules to be followed during coding, such as the way variables are to be named, the way the code is to be laid out, the error return conventions, etc. Besides the coding standards, several coding guidelines are also prescribed by software companies. But, what is the difference between a coding guideline and a coding standard?

It is mandatory for the programmers to follow the coding standards. Compliance of their code to coding standards is verified during code inspection. Any code that does not conform to the coding standards is rejected during code review and the code is reworked by the concerned programmer. In contrast, coding guidelines provide some general suggestions regarding the coding style to be followed but leave the actual implementation of these guidelines to the discretion of the individual developers.

After a module has been coded, usually code review is carried out to ensure that the coding standards are followed and also to detect as many errors as possible before testing. It is important to detect as many errors as possible during code reviews, because reviews are an efficient way of removing errors from code as compared to defect elimination using testing. We first discuss a few representative coding standards and guidelines. Subsequently, we discuss code review techniques. We then discuss software documentation in Section 10.3.

10.1.1 Coding Standards and Guidelines

Good software development organisations usually develop their own coding standards and guidelines depending on what suits their organisation best and based on the specific types of software they develop. To give an idea about the types of coding standards that are being used, we shall only list some general coding standards and guidelines that are commonly adopted by many software development organisations, rather than trying to provide an exhaustive list.

Representative coding standards

Rules for limiting the use of globals: These rules list what types of data can be declared global and what cannot, with a view to limit the data that needs to be defined with global scope.

Standard headers for different modules: The header of different modules should have standard format and information for ease of understanding and maintenance. The following is an example of header format that is being used in some companies:

- Name of the module.
- Date on which the module was created.
- Author's name.
- Modification history.
- Synopsis of the module. This is a small writeup about what the module does.
- Different functions supported in the module, along with their input/output parameters.
- Global variables accessed/modified by the module.

Naming conventions for global variables, local variables, and constant identifiers: A popular naming convention is that variables are named using mixed case lettering. Global variable names would always start with a capital letter (e.g., GlobalData) and local variable names start with small letters (e.g., localData). Constant names should be formed using capital letters only (e.g., CONSTDATA).

Conventions regarding error return values and exception handling mechanisms: The way error conditions are reported by different functions in a program should be standard within an organisation. For example, all functions while encountering an error condition should either return a 0 or 1 consistently, independent of which programmer has written the code. This facilitates reuse and debugging.

Representative coding guidelines: The following are some representative coding guidelines that are recommended by many software development organisations. Wherever necessary, the rationale behind these guidelines is also mentioned.

Do not use a coding style that is too clever or too difficult to understand: Code should be easy to understand. Many inexperienced engineers actually take pride in writing cryptic and incomprehensible code. Clever coding can obscure meaning of the code and reduce code understandability; thereby making maintenance and debugging difficult and expensive.

Avoid obscure side effects: The side effects of a function call include modifications to the parameters passed by reference, modification of global variables, and I/O operations. An obscure side effect is one that is not obvious from a casual examination of the code. Obscure side effects make it difficult to understand a piece of code. For example, suppose the value of a global variable is changed or some file I/O is performed obscurely in a called module. That is, this is difficult to infer from the function's name and header information. Then, it would be really hard to understand the code.

Do not use an identifier for multiple purposes: Programmers often use the same identifier to denote several temporary entities. For example, some programmers make use of a temporary loop variable for also computing and storing the final result. The rationale that they give for such multiple use of variables is memory efficiency, e.g., three variables use up three memory locations, whereas when the same variable is used for three different purposes, only one memory location is used. However, there are several things wrong with this approach and hence should be avoided. Some of the problems caused by the use of a variable for multiple purposes are as follows:

- Each variable should be given a descriptive name indicating its purpose. This is not possible if an identifier is used for multiple purposes. Use of a variable for multiple purposes can lead to confusion and make it difficult for somebody trying to read and understand the code.
- Use of variables for multiple purposes usually makes future enhancements more difficult. For example, while changing the final computed result from integer to float type, the programmer might subsequently notice that it has also been used as a temporary loop variable that cannot be a float type.

Code should be well-documented: As a rule of thumb, there should be at least one comment line on the average for every three source lines of code.

Length of any function should not exceed 10 source lines: A lengthy function is usually very difficult to understand as it probably has a large number of variables and carries out many different types of computations. For the same reason, lengthy functions are likely to have disproportionately larger number of bugs.

Do not use GO TO statements: Use of GO TO statements makes a program

*****ebook converter DEMO - www.ebook-converter.com*****

unstructured. This makes the program very difficult to understand, debug, and maintain.

10.2 CODE REVIEW

Testing is an effective defect removal mechanism. However, testing is applicable to only executable code. Review is a very effective technique to remove defects from source code. In fact, review has been acknowledged to be more cost-effective in removing defects as compared to testing. Over the years, review techniques have become extremely popular and have been generalised for use with other work products.

Code review for a module is undertaken after the module successfully compiles. That is, all the syntax errors have been eliminated from the module. Obviously, code review does not target to design syntax errors in a program, but is designed to detect logical, algorithmic, and programming errors. Code review has been recognised as an extremely cost-effective strategy for eliminating coding errors and for producing high quality code.

The reason behind why code review is a much more cost-effective strategy to eliminate errors from code compared to testing is that reviews directly detect errors. On the other hand, testing only helps detect failures and significant effort is needed to locate the error during debugging.

The rationale behind the above statement is explained as follows. Eliminating an error from code involves three main activities—testing, debugging, and then correcting the errors. Testing is carried out to detect if the system fails to work satisfactorily for certain types of inputs and under certain circumstances. Once a failure is detected, debugging is carried out to locate the error that is causing the failure and to remove it. Of the three testing activities, debugging is possibly the most laborious and time consuming activity. In code inspection, errors are directly detected, thereby saving the significant effort that would have been required to locate the error.

Normally, the following two types of reviews are carried out on the code of a module:

- Code inspection.
- Code walkthrough.

The procedures for conduction and the final objectives of these two review techniques are very different. In the following two subsections, we discuss

these two code review techniques.

10.2.1 Code Walkthrough

Code walkthrough is an informal code analysis technique. In this technique, a module is taken up for review after the module has been coded, successfully compiled, and all syntax errors have been eliminated. A few members of the development team are given the code a couple of days before the walkthrough meeting. Each member selects some test cases and simulates execution of the code by hand (i.e., traces the execution through different statements and functions of the code).

The main objective of code walkthrough is to discover the algorithmic and logical errors in the code.

The members note down their findings of their walkthrough and discuss those in a walkthrough meeting where the coder of the module is present.

Even though code walkthrough is an informal analysis technique, several guidelines have evolved over the years for making this naive but useful analysis technique more effective. These guidelines are based on personal experience, common sense, several other subjective factors. Therefore, these guidelines should be considered as examples rather than as accepted rules to be applied dogmatically. Some of these guidelines are following:

- The team performing code walkthrough should not be either too big or too small. Ideally, it should consist of between three to seven members.
- Discussions should focus on discovery of errors and avoid deliberations on how to fix the discovered errors.
- In order to foster co-operation and to avoid the feeling among the engineers that they are being watched and evaluated in the code walkthrough meetings, managers should not attend the walkthrough meetings.

10.2.2 Code Inspection

During code inspection, the code is examined for the presence of some common programming errors. This is in contrast to the hand simulation of code execution carried out during code walkthroughs. We can state the principal aim of the code inspection to be the following:

The principal aim of code inspection is to check for the presence of some common types of errors that usually creep into code due to programmer mistakes and oversights and to check whether coding standards have been adhered to.

The inspection process has several beneficial side effects, other than finding errors. The programmer usually receives feedback on programming style, choice of algorithm, and programming techniques. The other participants gain by being exposed to another programmer's errors.

As an example of the type of errors detected during code inspection, consider the classic error of writing a procedure that modifies a formal parameter and then calls it with a constant actual parameter. It is more likely that such an error can be discovered by specifically looking for this kinds of mistakes in the code, rather than by simply hand simulating execution of the code. In addition to the commonly made errors, adherence to coding standards is also checked during code inspection.

Good software development companies collect statistics regarding different types of errors that are commonly committed by their engineers and identify the types of errors most frequently committed. Such a list of commonly committed errors can be used as a checklist during code inspection to look out for possible errors.

Following is a list of some classical programming errors which can be checked during code inspection:

- Use of uninitialised variables.
- Jumps into loops.
- Non-terminating loops.
- Incompatible assignments.
- Array indices out of bounds.
- Improper storage allocation and deallocation.
- Mismatch between actual and formal parameter in procedure calls.
- Use of incorrect logical operators or incorrect precedence among operators.
- Improper modification of loop variables.
- Comparison of equality of floating point values.
- Dangling reference caused when the referenced memory has not been allocated.

10.2.3 Clean Room Testing

Clean room testing was pioneered at IBM. This type of testing relies

heavily on walkthroughs, inspection, and formal verification. The programmers are not allowed to test any of their code by executing the code other than doing some syntax testing using a compiler. It is interesting to note that the term cleanroom was first coined at IBM by drawing analogy to the semiconductor fabrication units where defects are avoided by manufacturing in an ultra-clean atmosphere.

This technique reportedly produces documentation and code that is more reliable and maintainable than other development methods relying heavily on code execution-based testing. The main problem with this approach is that testing effort is increased as walkthroughs, inspection, and verification are time consuming for detecting all simple errors. Also testing-based error detection is efficient for detecting certain errors that escape manual inspection.

10.3 SOFTWARE DOCUMENTATION

When a software is developed, in addition to the executable files and the source code, several kinds of documents such as users' manual, software requirements specification (SRS) document, design document, test document, installation manual, etc., are developed as part of the software engineering process. All these documents are considered a vital part of any good software development practice. Good documents are helpful in the following ways:

- Good documents help enhance understandability of code. As a result, the availability of good documents help to reduce the effort and time required for maintenance.
- Documents help the users to understand and effectively use the system.
- Good documents help to effectively tackle the manpower turnover¹ problem. Even when an engineer leaves the organisation, and a new engineer comes in, he can build up the required knowledge easily by referring to the documents.
- Production of good documents helps the manager to effectively track the progress of the project. The project manager would know that some measurable progress has been achieved, if the results of some pieces of work has been documented and the same has been reviewed.

Different types of software documents can broadly be classified into the following:

Internal documentation: These are provided in the source code itself.

External documentation: These are the supporting documents such as SRS document, installation document, user manual, design document, and test document.

We discuss these two types of documentation in the next section.

10.3.1 Internal Documentation

Internal documentation is the code comprehension features provided in the source code itself. Internal documentation can be provided in the code in several forms. The important types of internal documentation are the following:

- Comments embedded in the source code.
- Use of meaningful variable names.
- Module and function headers.
- Code indentation.
- Code structuring (i.e., code decomposed into modules and functions).
- Use of enumerated types.
- Use of constant identifiers.
- Use of user-defined data types.

Out of these different types of internal documentation, which one is the most valuable for understanding a piece of code?

Careful experiments suggest that out of all types of internal documentation, meaningful variable names is most useful while trying to understand a piece of code.

The above assertion, of course, is in contrast to the common expectation that code commenting would be the most useful. The research finding is obviously true when comments are written without much thought. For example, the following style of code commenting is not much of a help in understanding the code.

```
a=10; /* a made 10 */
```

A good style of code commenting is to write to clarify certain non-obvious aspects of the working of the code, rather than cluttering the code with trivial comments. Good software development organisations usually ensure good internal documentation by appropriately formulating their coding standards

and coding guidelines. Even when a piece of code is carefully commented, meaningful variable names has been found to be the most helpful in understanding the code.

10.3.2 External Documentation

External documentation is provided through various types of supporting documents such as users' manual, software requirements specification document, design document, test document, etc. A systematic software development style ensures that all these documents are of good quality and are produced in an orderly fashion.

An important feature that is required of any good external documentation is consistency with the code. If the different documents are not consistent, a lot of confusion is created for somebody trying to understand the software. In other words, all the documents developed for a product should be up-to-date and every change made to the code should be reflected in the relevant external documents. Even if only a few documents are not up-to-date, they create inconsistency and lead to confusion. Another important feature required for external documents is proper understandability by the category of users for whom the document is designed. For achieving this, Gunning's fog index is very useful. We discuss this next.

Gunning's fog index

Gunning's fog index (developed by Robert Gunning in 1952) is a metric that has been designed to measure the readability of a document. The computed metric value (fog index) of a document indicates the number of years of formal education that a person should have, in order to be able to comfortably understand that document. That is, if a certain document has a fog index of 12, any one who has completed his 12th class would not have much difficulty in understanding that document.

The Gunning's fog index of a document D can be computed as follows:

$$\text{fog}(D) = 0.4 \times \left(\frac{\text{words}}{\text{sentences}} \right) + \text{per cent of words having 3 or more syllables}$$

Observe that the fog index is computed as the sum of two different factors. The first factor computes the average number of words per sentence (total number of words in the document divided by the total number of sentences). This factor therefore accounts for the common observation that long sentences are difficult to understand. The second factor measures the percentage of complex words in the document. Note that a syllable is a group

of words that can be independently pronounced. For example, the word "sentence" has three syllables ("sen", "ten", and "ce"). Words having more than three syllables are complex words and presence of many such words hamper readability of a document.

Example 10.1 Consider the following sentence: "The Gunning's fog index is based on the premise that use of short sentences and simple words makes a document easy to understand." Calculate its Fog index.

The fog index of the above example sentence is

$$0.4 \square (23/1) + (4/23) \square 100 = 26$$

If a users' manual is to be designed for use by factory workers whose educational qualification is class 8, then the document should be written such that the Gunning's fog index of the document does not exceed 8.

10.4 TESTING

The aim of program testing is to help realiseidentify all defects in a program. However, in practice, even after satisfactory completion of the testing phase, it is not possible to guarantee that a program is error free. This is because the input data domain of most programs is very large, and it is not practical to test the program exhaustively with respect to each value that the input can assume. Consider a function taking a floating point number as argument. If a tester takes 1sec to type in a value, then even a million testers would not be able to exhaustively test it after trying for a million number of years. Even with this obvious limitation of the testing process, we should not underestimate the importance of testing. We must remember that careful testing can expose a large percentage of the defects existing in a program, and therefore provides a practical way of reducing defects in a system.

10.4.1 Basic Concepts and Terminologies

In this section, we will discuss a few basic concepts in program testing on which our subsequent discussions on program testing would be based.

How to test a program?

Testing a program involves executing the program with a set of test inputs and observing if the program behaves as expected. If the

program fails to behave as expected, then the input data and the conditions under which it fails are noted for later debugging and error correction. A highly simplified view of program testing is schematically shown in Figure 10.1. The tester has been shown as a stick icon, who inputs several test data to the system and observes the outputs produced by it to check if the system fails on some specific inputs. Unless the conditions under which a software fails are noted down, it becomes difficult for the developers to reproduce a failure observed by the testers. For examples, a software might fail for a test case only when a network connection is enabled.

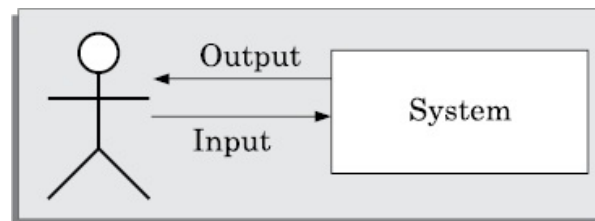


Figure 10.1: A simplified view of program testing.

Terminologies

As is true for any specialised domain, the area of software testing has come to be associated with its own set of terminologies. In the following, we discuss a few important terminologies that have been standardised by the IEEE Standard Glossary of Software Engineering Terminology [IEEE90]:

- A **mistake** is essentially any programmer action that later shows up as an incorrect result during program execution. A programmer may commit a mistake in almost any development activity. For example, during coding a programmer might commit the mistake of not initializing a certain variable, or might overlook the errors that might arise in some exceptional situations such as division by zero in an arithmetic operation. Both these mistakes can lead to an incorrect result.
- An **error** is the result of a mistake committed by a developer in any of the development activities. Among the extremely large variety of errors that can exist in a program. One example of an error is a call made to a wrong function.

The terms error, fault, bug, and defect are considered to be synonyms in the area of

program testing.

Though the terms error, fault, bug, and defect are all used interchangeably by the program testing community. Please note that in the domain of hardware testing, the term fault is used with a slightly different connotation [IEEE90] as compared to the terms error and bug.

Example 10.2 Can a designer's mistake give rise to a program error? Give an example of a designer's mistake and the corresponding program error.

Answer: Yes, a designer's mistake give rise to a program error. For example, a requirement might be overlooked by the designer, which can lead to it being overlooked in the code as well.

- A **failure** of a program essentially denotes an incorrect behaviour exhibited by the program during its execution. An incorrect behaviour is observed either as an incorrect result produced or as an inappropriate activity carried out by the program. Every failure is caused by some bugs present in the program. In other words, we can say that every software failure can be traced to some bug or other present in the code. The number of possible ways in which a program can fail is extremely large. Out of the large number of ways in which a program can fail, in the following we give three randomly selected examples:
 - The result computed by a program is 0, when the correct result is 10.
 - A program crashes on an input.
 - A robot fails to avoid an obstacle and collides with it.

It may be noted that mere presence of an error in a program code may not necessarily lead to a failure during its execution.

Example 10.3 Give an example of a program error that may not cause any failure.

Answer: Consider the following C program segment:

```
int markList[1:10]; /* mark list of 10 students*/
int roll;           /* student roll number*/
...
if(roll>0)
    markList[roll]=mark;
else
    markList[roll]=0;
```

In the above code, if the variable roll assumes zero or some negative value

under some circumstances, then an array index out of bound type of error would result. However, it may be the case that for all allowed input values the variable roll is always assigned positive values. Then, the else clause is unreachable and no failure would occur. Thus, even if an error is present in the code, it does not show up as an error since it is unreachable for normal input values.

Explanation: An array index out of bound type of error is said to occur, when the array index variable assumes a value beyond the array bounds.

- A **test case** is a triplet [I , S, R], where I is the data input to the program under test, S is the state of the program at which the data is to be input, and R is the result expected to be produced by the program. The state of a program is also called its execution mode. As an example, consider the different execution modes of a certain text editor software. The text editor can at any time during its execution assume any of the following execution modes—edit, view, create, and display. In simple words, we can say that a test case is a set of test inputs, the mode in which the input is to be applied, and the results that are expected during and after the execution of the test case.

An example of a test case is—[input: "abc", state: edit, result: abc is displayed], which essentially means that the input abc needs to be applied in the edit mode, and the expected result is that the string abc would be displayed.

- A **test scenario** is an abstract test case in the sense that it only identifies the aspects of the program that are to be tested without identifying the input, state, or output. A test case can be said to be an implementation of a test scenario. In the test case, the input, output, and the state at which the input would be applied is designed such that the scenario can be executed. An important automatic test case design strategy is to first design test scenarios through an analysis of some program abstraction (model) and then implement the test scenarios as test cases.
- A **test script** is an encoding of a test case as a short program. Test scripts are developed for automated execution of the test cases.
- A test case is said to be a **positive test case** if it is designed to test whether the software correctly performs a required functionality. A test

case is said to be **negative test case**, if it is designed to test whether the software carries out something, that is not required of the system. As one example each of a positive test case and a negative test case, consider a program to manage user login. A positive test case can be designed to check if a login system validates a user with the correct user name and password. A negative test case in this case can be a test case that checks whether the the login functionality validates and admits a user with wrong or bogus login user name or password.

- A **test suite** is the set of all test that have been designed by a tester to test a given program.
- **Testability** of a requirement denotes the extent to which it is possible to determine whether an implementation of the requirement conforms to it in both functionality and performance. In other words, the testability of a requirement is the degree to which an implementation of it can be adequately tested to determine its conformance to the requirement.

Example 10.4 Suppose two programs have been written to implement essentially the same functionality. How can you determine which of these is more testable?

Answer: A program is more testable, if it can be adequately tested with less number of test cases. Obviously, a less complex program is more testable. The complexity of a program can be measured using several types of metrics such as number of decision statements used in the program. Thus, a more testable program should have a lower structural complexity metric.

- A **failure mode** of a software denotes an observable way in which it can fail. In other words, all failures that have similar observable symptoms, constitute a failure mode. As an example of the failure modes of a software, consider a railway ticket booking software that has three failure modes—failing to book an available seat, incorrect seat booking (e.g., booking an already booked seat), and system crash.
- **Equivalent faults** denote two or more bugs that result in the system failing in the same failure mode. As an example of equivalent faults, consider the following two faults in C language—division by zero and illegal memory access errors. These two are equivalent faults, since each of these leads to a program crash.

Verification versus validation

The objectives of both verification and validation techniques are very similar since both these techniques are designed to help remove errors in a software. In spite of the apparent similarity between their objectives, the underlying principles of these two bug detection techniques and their applicability are very different. We summarise the main differences between these two techniques in the following:

- Verification is the process of determining whether the output of one phase of software development conforms to that of its previous phase; whereas validation is the process of determining whether a fully developed software conforms to its requirements specification. Thus, the objective of verification is to check if the work products produced after a phase conform to that which was input to the phase. For example, a verification step can be to check if the design documents produced after the design step conform to the requirements specification. On the other hand, validation is applied to the fully developed and integrated software to check if it satisfies the customer's requirements.
- The primary techniques used for verification include review, simulation, formal verification, and testing. Review, simulation, and testing are usually considered as informal verification techniques. Formal verification usually involves use of theorem proving techniques or use of automated tools such as a model checker. On the other hand, validation techniques are primarily based on product testing. Note that we have categorised testing both under program verification and validation. The reason being that unit and integration testing can be considered as verification steps where it is verified whether the code is as per the module and module interface specifications. On the other hand, system testing can be considered as a validation step where it is determined whether the fully developed code is as per its requirements specification.
- Verification does not require execution of the software, whereas validation requires execution of the software.
- Verification is carried out during the development process to check if the development activities are proceeding alright, whereas validation is carried out to check if the right as required by the customer has been developed.

We can therefore say that the primary objective of the verification steps are to determine whether the steps in product development are being carried out alright, whereas validation is carried out towards the end of the development process to determine whether the right product has been developed.

- Verification techniques can be viewed as an attempt to achieve phase containment of errors. Phase containment of errors has been acknowledged to be a cost-effective way to eliminate program bugs, and is an important software engineering principle. The principle of detecting errors as close to their points of commitment as possible is known as phase containment of errors. Phase containment of errors can reduce the effort required for correcting bugs. For example, if a design problem is detected in the design phase itself, then the problem can be taken care of much more easily than if the error is identified, say, at the end of the testing phase. In the later case, it would be necessary not only to rework the design, but also to appropriately redo the relevant coding as well as the system testing activities, thereby incurring higher cost.

While verification is concerned with phase containment of errors, the aim of validation is to check whether the deliverable software is error free.

We can consider the verification and validation techniques to be different types of bug filters. To achieve high product reliability in a cost-effective manner, a development team needs to perform both verification and validation activities. The activities involved in these two types of bug detection techniques together are called the “V and V” activities.

Based on the above discussions, we can conclude that:

Error detection techniques = Verification techniques + Validation techniques

Example 10.5 Is it at all possible to develop a highly reliable software, using validation techniques alone? If so, can we say that all verification techniques are redundant?

Answer: It is possible to develop a highly reliable software using validation techniques alone. However, this would cause the development cost to increase drastically. Verification techniques help achieve phase containment of errors and provide a means to cost-effectively remove bugs.

10.4.2 Testing Activities

Testing involves performing the following main activities:

Test suite design: The set of test cases using which a program is to be tested is designed possibly using several test case design techniques. We discuss a few important test case design techniques later in this Chapter.

Running test cases and checking the results to detect failures: Each test case is run and the results are compared with the expected results. A mismatch between the actual result and expected results indicates a failure. The test cases for which the system fails are noted down for later debugging.

Locate error: In this activity, the failure symptoms are analysed to locate the errors. For each failure observed during the previous activity, the statements that are in error are identified.

Error correction: After the error is located during debugging, the code is appropriately changed to correct the error.

The testing activities have been shown schematically in Figure 10.2. As can be seen, the test cases are first designed, the test cases are run to detect failures. The bugs causing the failure are identified through debugging, and the identified error is corrected. Of all the above mentioned testing activities, debugging often turns out to be the most time-consuming activity.

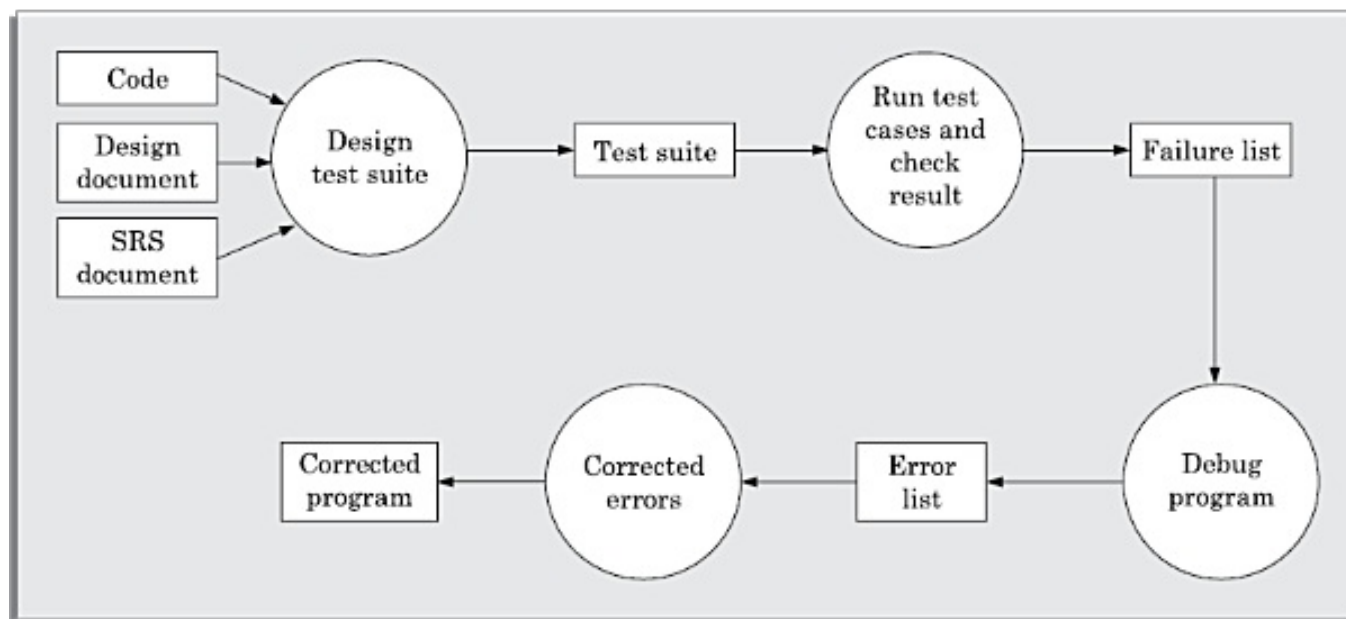


Figure 10.2: Testing process.

10.4.3 Why Design Test Cases?

Before discussing the various test case design techniques, we need to convince ourselves on the following question. Would it not be sufficient to test a software using a large number of random input values? Why design

test cases? The answer to this question—this would be very costly and at the same time very ineffective way of testing due to the following reasons:

When test cases are designed based on random input data, many of the test cases do not contribute to the significance of the test suite, That is, they do not help detect any additional defects not already being detected by other test cases in the suite.

Testing a software using a large collection of randomly selected test cases does not guarantee that all (or even most) of the errors in the system will be uncovered. Let us try to understand why the number of random test cases in a test suite, in general, does not indicate of the effectiveness of testing. Consider the following example code segment which determines the greater of two integer values x and y. This code segment has a simple programming error:

```
if (x>y) max = x;
else max = x;
```

For the given code segment, the test suite $\{(x=3,y=2);(x=2,y=3)\}$ can detect the error, whereas a larger test suite $\{(x=3,y=2);(x=4,y=3);(x=5,y=1)\}$ does not detect the error. All the test cases in the larger test suite help detect the same error, while the other error in the code remains undetected. So, it would be incorrect to say that a larger test suite would always detect more errors than a smaller one, unless of course the larger test suite has also been carefully designed. This implies that for effective testing, the test suite should be carefully designed rather than picked randomly.

We have already pointed out that exhaustive testing of almost any non-trivial system is impractical due to the fact that the domain of input data values to most practical software systems is either extremely large or countably infinite. Therefore, to satisfactorily test a software with minimum cost, we must design a minimal test suite that is of reasonable size and can uncover as many existing errors in the system as possible. To reduce testing cost and at the same time to make testing more effective, systematic approaches have been developed to design a small test suite that can detect most, if not all failures.

A minimal test suite is a carefully designed set of test cases such that each test case helps detect different errors. This is in contrast to testing using some random input values.

There are essentially two main approaches to systematically design test cases:

- Black-box approach
- White-box (or glass-box) approach

In the black-box approach, test cases are designed using only the functional specification of the software. That is, test cases are designed solely based on an analysis of the input/out behaviour (that is, functional behaviour) and does not require any knowledge of the internal structure of a program. For this reason, black-box testing is also known as functional testing. On the other hand, designing white-box test cases requires a thorough knowledge of the internal structure of a program, and therefore white-box testing is also called structural testing. Black- box test cases are designed solely based on the input-output behaviour of a program. In contrast, white-box test cases are based on an analysis of the code. These two approaches to test case design are complementary. That is, a program has to be tested using the test cases designed by both the approaches, and one testing using one approach does not substitute testing using the other.

10.4.4 Testing in the Large versus Testing in the Small

A software product is normally tested in three levels or stages:

- Unit testing
- Integration testing
- System testing

During unit testing, the individual functions (or units) of a program are tested.

Unit testing is referred to as testing in the small, whereas integration and system testing are referred to as testing in the large.

After testing all the units individually, the units are slowly integrated and tested after each step of integration (integration testing). Finally, the fully integrated system is tested (system testing). Integration and system testing are known as testing in the large.

Often beginners ask the question—“Why test each module (unit) in isolation first, then integrate these modules and test, and again test the integrated set of modules—why not just test the integrated set of modules once thoroughly?” The answer to this question is the following—There are two main reasons to it. First while testing a module, other modules with which this module needs to interface may not be ready. Moreover, it is

always a good idea to first test the module in isolation before integration because it makes debugging easier. If a failure is detected when an integrated set of modules is being tested, it would be difficult to determine which module exactly has the error.

In the following sections, we discuss the different levels of testing. It should be borne in mind in all our subsequent discussions that unit testing is carried out in the coding phase itself as soon as coding of a module is complete. On the other hand, integration and system testing are carried out during the testing phase.

10.5 UNIT TESTING

Unit testing is undertaken after a module has been coded and reviewed. This activity is typically undertaken by the coder of the module himself in the coding phase. Before carrying out unit testing, the unit test cases have to be designed and the test environment for the unit under test has to be developed. In this section, we first discuss the environment needed to perform unit testing.

Driver and stub modules

In order to test a single module, we need a complete environment to provide all relevant code that is necessary for execution of the module. That is, besides the module under test, the following are needed to test the module:

- The procedures belonging to other modules that the module under test calls.
- Non-local data structures that the module accesses.
- A procedure to call the functions of the module under test with appropriate parameters.

Modules required to provide the necessary environment (which either call or are called by the module under test) are usually not available until they too have been unit tested. In this context, stubs and drivers are designed to provide the complete environment for a module so that testing can be carried out.

Stub: The role of stub and driver modules is pictorially shown in Figure 10.3. A stub procedure is a dummy procedure that has the same I/O parameters as the function called by the unit under test but has a highly simplified

behaviour. For example, a stub procedure may produce the expected behaviour using a simple table look up mechanism.

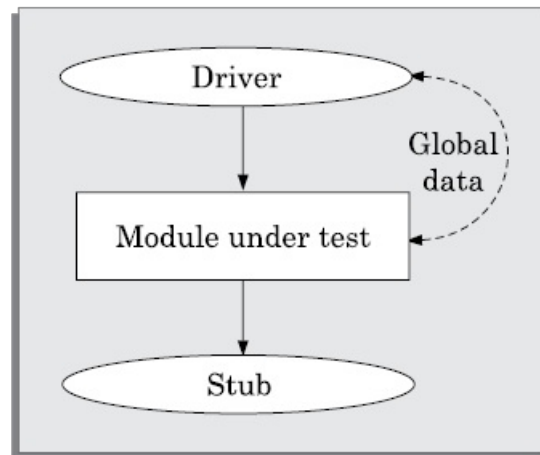


Figure 10.3: Unit testing with the help of driver and stub modules.

Driver: A driver module should contain the non-local data structures accessed by the module under test. Additionally, it should also have the code to call the different functions of the unit under test with appropriate parameter values for testing.

10.6 BLACK-BOX TESTING

In black-box testing, test cases are designed from an examination of the input/output values only and no knowledge of design or code is required. The following are the two main approaches available to design black box test cases:

- Equivalence class partitioning
- Boundary value analysis

In the following subsections, we will elaborate these two test case design techniques.

10.6.1 Equivalence Class Partitioning

In the equivalence class partitioning approach, the domain of input values to the program under test is partitioned into a set of equivalence classes. The partitioning is done such that for every input data belonging to the same equivalence class, the program behaves similarly.

The main idea behind defining equivalence classes of input data is that testing the code with any one value belonging to an equivalence class is as good as testing the

code with any other value belonging to the same equivalence class.

Equivalence classes for a unit under test can be designed by examining the input data and output data. The following are two general guidelines for designing the equivalence classes:

1. If the input data values to a system can be specified by a range of values, then one valid and two invalid equivalence classes need to be defined. For example, if the equivalence class is the set of integers in the range 1 to 10 (i.e., $[1,10]$), then the invalid equivalence classes are $[-\infty,0]$, $[11,+\infty]$.
2. If the input data assumes values from a set of discrete members of some domain, then one equivalence class for the valid input values and another equivalence class for the invalid input values should be defined. For example, if the valid equivalence classes are $\{A,B,C\}$, then the invalid equivalence class is $\square - \{A,B,C\}$, where $\square$ is the universe of possible input values.

In the following, we illustrate equivalence class partitioning-based test case generation through four examples.

Example 10.6 For a software that computes the square root of an input integer that can assume values in the range of 0 and 5000. Determine the equivalence classes and the black box test suite.

Answer: There are three equivalence classes—The set of negative integers, the set of integers in the range of 0 and 5000, and the set of integers larger than 5000. Therefore, the test cases must include representatives for each of the three equivalence classes. A possible test suite can be: $\{-5,500,6000\}$.

Example 10.7 Design the equivalence class test cases for a program that reads two integer pairs (m_1, c_1) and (m_2, c_2) defining two straight lines of the form $y=mx+c$. The program computes the intersection point of the two straight lines and displays the point of intersection.

Answer: The equivalence classes are the following:

- Parallel lines ($m_1 = m_2, c_1 \neq c_2$)
- Intersecting lines ($m_1 \neq m_2$)
- Coincident lines ($m_1 = m_2, c_1 = c_2$)

Now, selecting one representative value from each equivalence class, we get the required equivalence class test suite $\{(2,2)(2,5),(5,5)(7,7), (10,10)$

(10,10)}.

Example 10.8 Design equivalence class partitioning test suite for a function that reads a character string of size less than five characters and displays whether it is a palindrome.

Answer: The equivalence classes are the leaf level classes shown in Figure 10.4. The equivalence classes are palindromes, non-palindromes, and invalid inputs. Now, selecting one representative value from each equivalence class, we have the required test suite: {abc,aba,abcdef}.

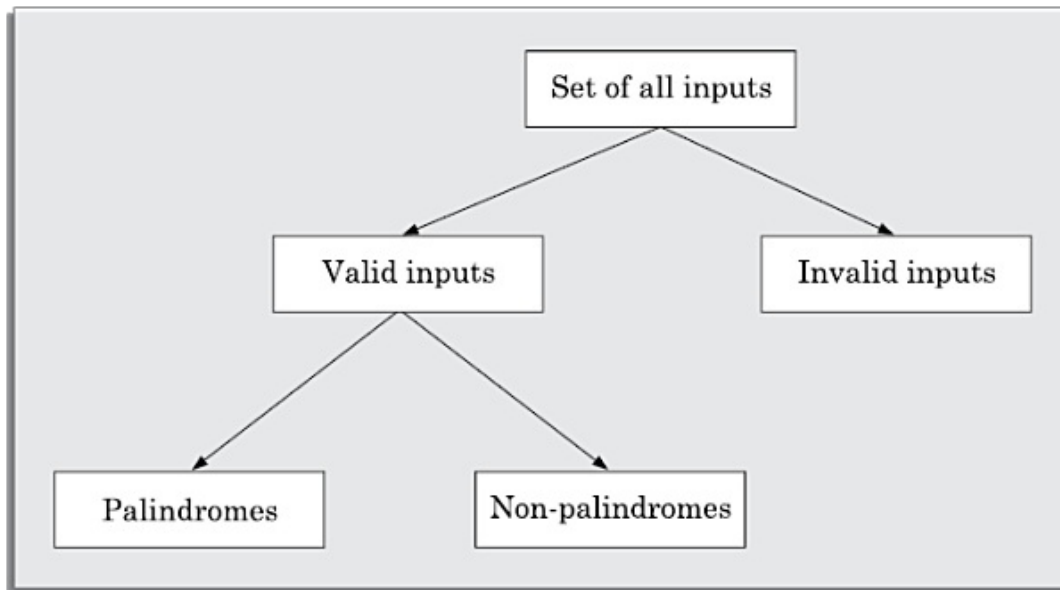


Figure 10.4: Equivalence classes for Example 10.6.

10.6.2 Boundary Value Analysis

A type of programming error that is frequently committed by programmers is missing out on the special consideration that should be given to the values at the boundaries of different equivalence classes of inputs. The reason behind programmers committing such errors might purely be due to psychological factors. Programmers often fail to properly address the special processing required by the input values that lie at the boundary of the different equivalence classes. For example, programmers may improperly use $<$ instead of $<=$, or conversely $<=$ for $<$, etc.

Boundary value analysis-based test suite design involves designing test cases using the values at the boundaries of different equivalence classes.

To design boundary value test cases, it is required to examine the equivalence classes to check if any of the equivalence classes contains a range of values. For those equivalence classes that are not a range of values

(i.e., consist of a discrete collection of values) no boundary value test cases can be defined. For an equivalence class that is a range of values, the boundary values need to be included in the test suite. For example, if an equivalence class contains the integers in the range 1 to 10, then the boundary value test suite is {0,1,10,11}.

Example 10.9 For a function that computes the square root of the integer values in the range of 0 and 5000, determine the boundary value test suite.

Answer: There are three equivalence classes—The set of negative integers, the set of integers in the range of 0 and 5000, and the set of integers larger than 5000. The boundary value-based test suite is: {0,-1,5000,5001}.

Example 10.10 Design boundary value test suite for the function described in Example 10.6.

Answer: The equivalence classes have been showed in Figure 10.5. There is a boundary between the valid and invalid equivalence classes. Thus, the boundary value test suite is {abcdefg, abcdef}.

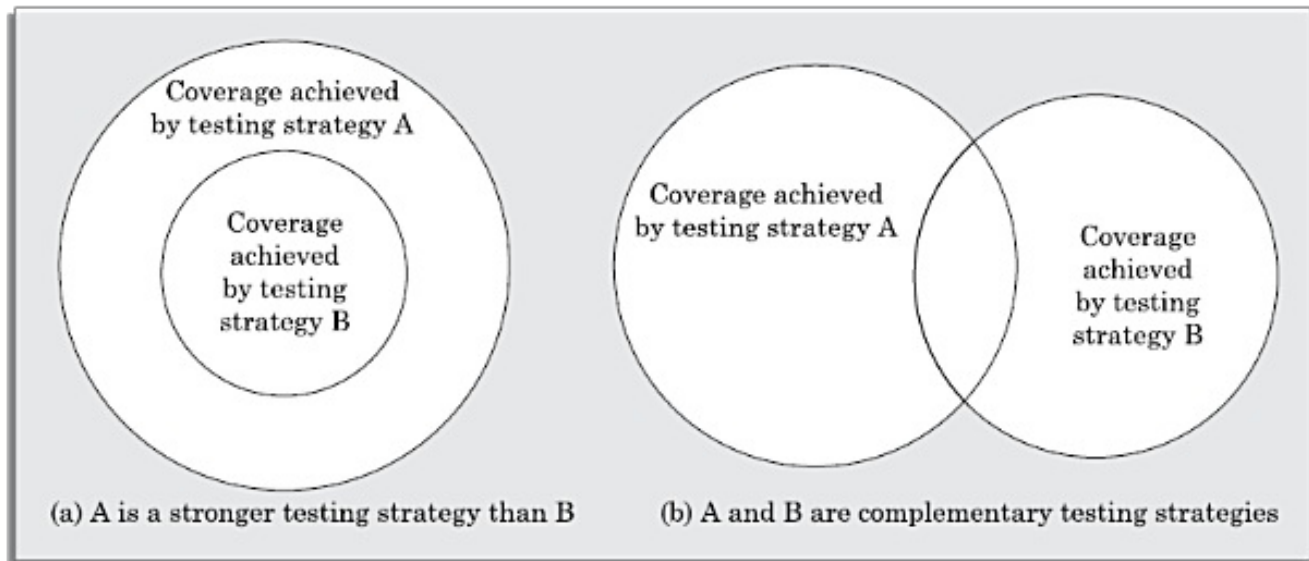


Figure 10.5: CFG for (a) sequence, (b) selection, and (c) iteration type of constructs.

10.6.3 Summary of the Black-box Test Suite Design Approach

We now summarise the important steps in the black-box test suite design approach:

- Examine the input and output values of the program.
- Identify the equivalence classes.
- Design equivalence class test cases by picking one representative

value from each equivalence class.

- Design the boundary value test cases as follows. Examine if any equivalence class is a range of values. Include the values at the boundaries of such equivalence classes in the test suite.

The strategy for black-box testing is intuitive and simple. For black-box testing, the most important step is the identification of the equivalence classes. Often, the identification of the equivalence classes is not straightforward. However, with little practice one would be able to identify all equivalence classes in the input data domain. Without practice, one may overlook many equivalence classes in the input data set. Once the equivalence classes are identified, the equivalence class and boundary value test cases can be selected almost mechanically.

10.7 WHITE-BOX TESTING

White-box testing is an important type of unit testing. A large number of white-box testing strategies exist. Each testing strategy essentially designs test cases based on analysis of some aspect of source code and is based on some heuristic. We first discuss some basic concepts associated with white-box testing, and follow it up with a discussion on specific testing strategies.

10.7.1 Basic Concepts

A white-box testing strategy can either be coverage-based or fault-based.

Fault-based testing

A fault-based testing strategy targets to detect certain types of faults. These faults that a test strategy focuses on constitutes the **fault model** of the strategy. An example of a fault-based strategy is mutation testing, which is discussed later in this section.

Coverage-based testing

A coverage-based testing strategy attempts to execute (or cover) certain elements of a program. Popular examples of coverage-based testing strategies are statement coverage, branch coverage, multiple condition coverage, and path coverage-based testing.

Testing criterion for coverage-based testing

A coverage-based testing strategy typically targets to execute (i.e., cover) certain program elements for discovering failures.

The set of specific program elements that a testing strategy targets to execute is called the testing criterion of the strategy.

For example, if a testing strategy requires all the statements of a program to be executed at least once, then we say that the testing criterion of the strategy is statement coverage. We say that a test suite is adequate with respect to a criterion, if it covers all elements of the domain defined by that criterion.

Stronger versus weaker testing

We have mentioned that a large number of white-box testing strategies have been proposed. It therefore becomes necessary to compare the effectiveness of different testing strategies in detecting faults. We can compare two testing strategies by determining whether one is stronger, weaker, or complementary to the other.

A white-box testing strategy is said to be stronger than another strategy, if the stronger testing strategy covers all program elements covered by the weaker testing strategy, and the stronger strategy additionally covers at least one program element that is not covered by the weaker strategy.

When none of two testing strategies fully covers the program elements exercised by the other, then the two are called complementary testing strategies. The concepts of stronger, weaker, and complementary testing are schematically illustrated in Figure 10.6. Observe in Figure 10.6(a) that testing strategy A is stronger than B since B covers only a proper subset of elements covered by A. On the other hand, Figure 10.6(b) shows A and B are complementary testing strategies since some elements of A are not covered by B and vice versa.

If a stronger testing has been performed, then a weaker testing need not be carried out.

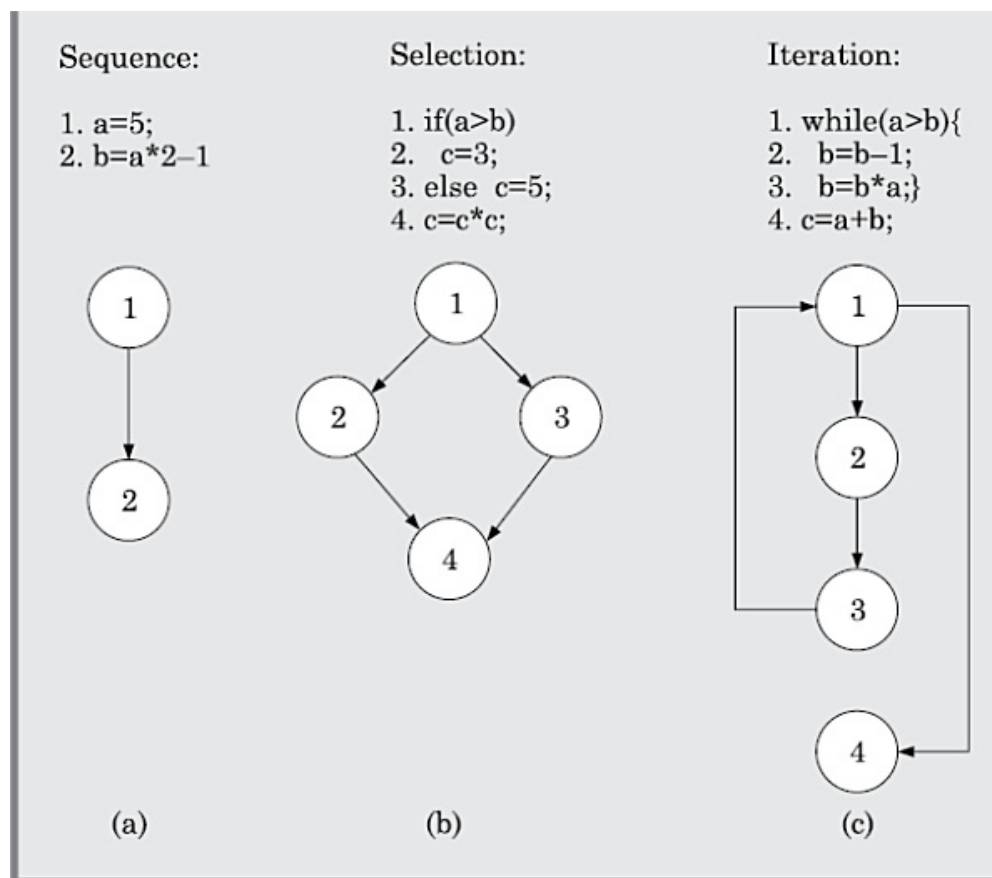


Figure 10.6: Illustration of stronger, weaker, and complementary testing strategies.

A test suite should, however, be enriched by using various complementary testing strategies.

We need to point out that coverage-based testing is frequently used to check the quality of testing achieved by a test suite. It is hard to manually design a test suite to achieve a specific coverage for a non-trivial program.

10.7.2 Statement Coverage

The statement coverage strategy aims to design test cases so as to execute every statement in a program at least once.

The principal idea governing the statement coverage strategy is that unless a statement is executed, there is no way to determine whether an error exists in that statement.

It is obvious that without executing a statement, it is difficult to determine whether it causes a failure due to illegal memory access, wrong result computation due to improper arithmetic operation, etc. It can however be pointed out that a weakness of the statement-coverage strategy is that executing a statement once and observing that it behaves properly for one

input value is no guarantee that it will behave correctly for all input values. Never the less, statement coverage is a very intuitive and appealing testing technique. In the following, we illustrate a test suite that achieves statement coverage.

Example 10.11 Design statement coverage-based test suite for the following Euclid's GCD computation program:

```
int computeGCD(x,y)
    int x,y;
{
    1 while (x != y){
    2 if (x>y) then
    3 x=x-y;
    4 else y=y-x;
    5 }
    6 return x;
}
```

Answer: To design the test cases for the statement coverage, the conditional expression of the `while` statement needs to be made true and the conditional expression of the `if` statement needs to be made both true and false. By choosing the test set $\{(x = 3, y = 3), (x = 4, y = 3), (x = 3, y = 4)\}$, all statements of the program would be executed at least once.

10.7.3 Branch Coverage

A test suite satisfies branch coverage, if it makes each branch condition in the program to assume true and false values in turn. In other words, for branch coverage each branch in the CFG representation of the program must be taken at least once, when the test suite is executed. Branch testing is also known as edge testing, since in this testing scheme, each edge of a program's control flow graph is traversed at least once.

Example 10.12 For the program of Example 10.11, determine a test suite to achieve branch coverage.

Answer: The test suite $\{(x = 3, y = 3), (x = 3, y = 2), (x = 4, y = 3), (x = 3, y = 4)\}$ achieves branch coverage.

It is easy to show that branch coverage-based testing is a stronger testing than statement coverage-based testing. We can prove this by showing that branch coverage ensures statement coverage, but not vice versa.

Theorem 10.1 Branch coverage-based testing is stronger than statement coverage-based testing.

Proof: We need to show that (a) branch coverage ensures statement coverage, and (b) statement coverage does not ensure branch coverage.

(a) Branch testing would guarantee statement coverage since every statement must belong to some branch (assuming that there is no unreachable code).

(b) To show that statement coverage does not ensure branch coverage, it would be sufficient to give an example of a test suite that achieves statement coverage, but does not cover at least one branch. Consider the following code, and the test suite {5}.

```
if (x > 2) x += 1;
```

The test suite would achieve statement coverage. However, it does not achieve branch coverage, since the condition ($x > 2$) is not made false by any test case in the suite.

10.7.4 Multiple Condition Coverage

In the multiple condition (MC) coverage-based testing, test cases are designed to make each component of a composite conditional expression to assume both true and false values. For example, consider the composite conditional expression $((c_1 \text{ .and. } c_2) \text{ .or. } c_3)$. A test suite would achieve MC coverage, if all the component conditions c_1 , c_2 and c_3 are each made to assume both true and false values. Branch testing can be considered to be a simplistic condition testing strategy where only the compound conditions appearing in the different branch statements are made to assume the true and false values. It is easy to prove that condition testing is a stronger testing strategy than branch testing. For a composite conditional expression of n components, 2^n test cases are required for multiple condition coverage. Thus, for multiple condition coverage, the number of test cases increases exponentially with the number of component conditions. Therefore, multiple condition coverage-based testing technique is practical only if n (the number of conditions) is small.

Example 10.13 Give an example of a fault that is detected by multiple condition coverage, but not by branch coverage.

Answer: Consider the following C program segment:

```
if(temperature>150 || temperature>50)
    setWarningLightOn();
```

The program segment has a bug in the second component condition, it should have been `temperature<50`. The test suite `{temperature=160, temperature=40}` achieves branch coverage. But, it is not able to check that `setWarningLightOn();` should not be called for temperature values within 150 and 50.

10.7.5 Path Coverage

A test suite achieves path coverage if it executes each linearly independent paths (or basis paths) at least once. A linearly independent path can be defined in terms of the control flow graph (CFG) of a program. Therefore, to understand path coverage-based testing strategy, we need to first understand how the CFG of a program can be drawn.

Control flow graph (CFG)

A control flow graph describes how the control flows through the program. We can define a control flow graph as the following:

A control flow graph describes the sequence in which the different instructions of a program get executed.

In order to draw the control flow graph of a program, we need to first number all the statements of a program. The different numbered statements serve as nodes of the control flow graph (see Figure 10.5). There exists an edge from one node to another, if the execution of the statement representing the first node can result in the transfer of control to the other node.

More formally, we can define a CFG as follows. A CFG is a directed graph consisting of a set of nodes and edges (N, E) , such that each node $n \in N$ corresponds to a unique program statement and an edge exists between two nodes if control can transfer from one node to the other.

We can easily draw the CFG for any program, if we know how to represent the sequence, selection, and iteration types of statements in the CFG. After all, every program is constructed by using these three types of constructs only. Figure 10.5 summarises how the CFG for these three types of constructs can be drawn. The CFG representation of the sequence and decision types of statements is straight forward. Please note carefully how the CFG for the loop

(iteration) construct can be drawn. For iteration type of constructs such as the while construct, the loop condition is tested only at the beginning of the loop and therefore always control flows from the last statement of the loop to the top of the loop. That is, the loop construct terminates from the first statement (after the loop is found to be false) and does not at any time exit the loop at the last statement of the loop. Using these basic ideas, the CFG of the program given in Figure 10.7(a) can be drawn as shown in Figure 10.7(b).

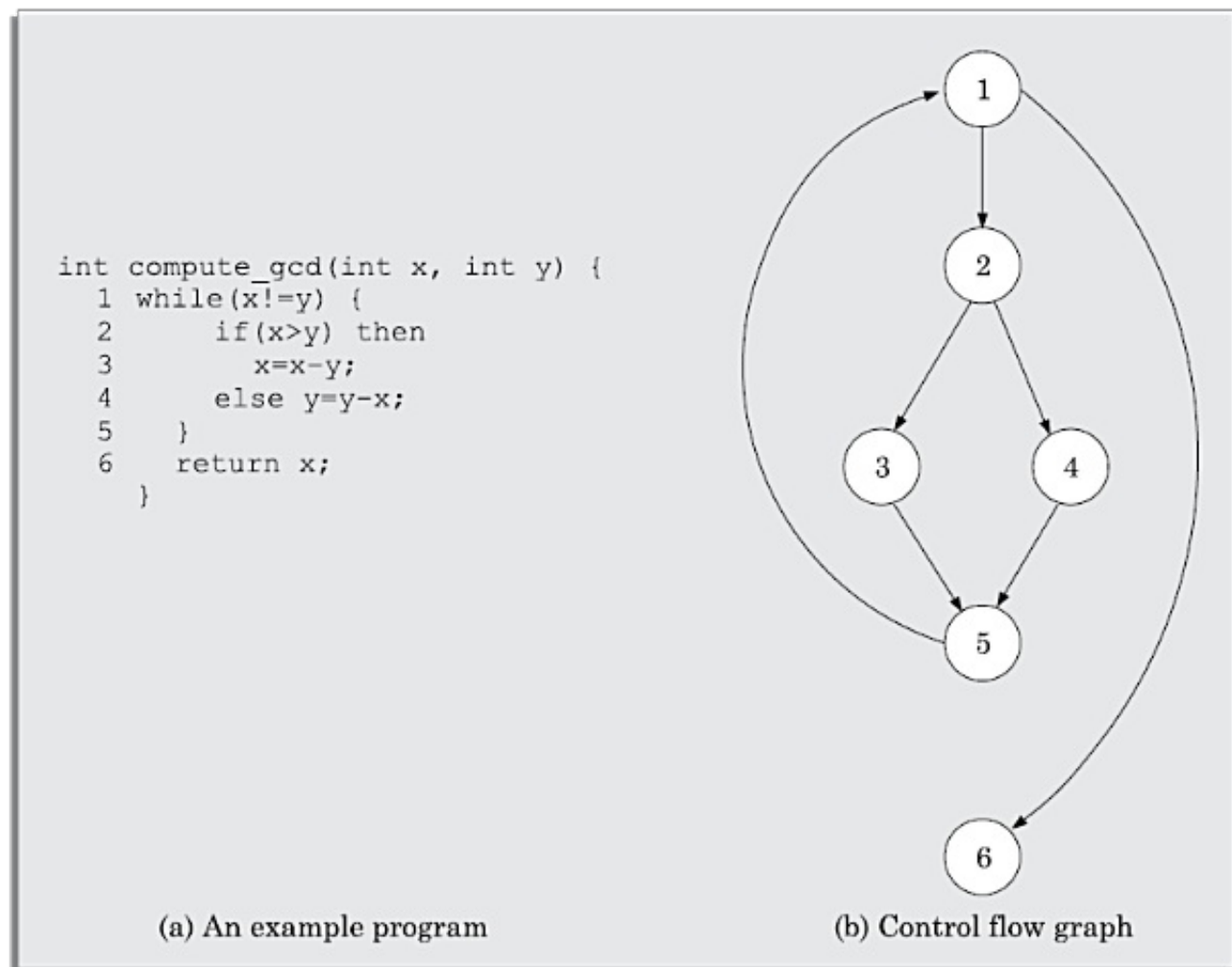


Figure 10.7: Control flow diagram of an example program.

Path

A path through a program is any node and edge sequence from the start node to a terminal node of the control flow graph of a program. Please note that a program can have more than one terminal nodes when it contains multiple exit or return type of statements. Writing test cases to cover all paths of a typical program is impractical since there can be an infinite number of paths through a program in presence of loops. For example, in Figure 10.5(c), there can be an infinite number of paths

such as 12314, 12312314, 12312312314, etc. If coverage of all paths is attempted, then the number of test cases required would become infinitely large. For this reason, path coverage testing does not try to cover all paths, but only a subset of paths called linearly independent paths (or basis paths). Let us now discuss what are linearly independent paths and how to determine these in a program.

Linearly independent set of paths (or basis path set)

A set of paths for a given program is called linearly independent set of paths (or the set of basis paths or simply the basis set), if each path in the set introduces at least one new edge that is not included in any other path in the set. Please note that even if we find that a path has one new node compared to all other linearly independent paths, then this path should also be included in the set of linearly independent paths. This is because, any path having a new node would automatically have a new edge. An alternative definition of a linearly independent set of paths [McCabe76] is the following:

If a set of paths is linearly independent of each other, then no path in the set can be obtained through any linear operations (i.e., additions or subtractions) on the other paths in the set.

According to the above definition of a linearly independent set of paths, for any path in the set, its subpath cannot be a member of the set. In fact, any arbitrary path of a program, can be synthesized by carrying out linear operations on the basis paths. Possibly, the name basis set comes from the observation that the paths in the basis set form the “basis” for all the paths of a program. Please note that there may not always exist a unique basis set for a program and several basis sets for the same program can usually be determined.

Even though it is straight forward to identify the linearly independent paths for simple programs, for more complex programs it is not easy to determine the number of independent paths. In this context, McCabe’s cyclomatic complexity metric is an important result that lets us compute the number of linearly independent paths for any arbitrary program. McCabe’s cyclomatic complexity defines an upper bound for the number of linearly independent paths through a program. Also, the McCabe’s cyclomatic complexity is very simple to compute. Though the McCabe’s metric does not directly identify the linearly independent paths, but it provides us with a practical way of determining approximately how many paths to look for.

10.7.6 McCabe's Cyclomatic Complexity Metric

M McCabe obtained his results by applying graph-theoretic techniques to the control flow graph of a program. McCabe's cyclomatic complexity defines an upper bound on the number of independent paths in a program. We discuss three different ways to compute the cyclomatic complexity. For structured programs, the results computed by all the three methods are guaranteed to agree.

Method 1: Given a control flow graph G of a program, the cyclomatic complexity $V(G)$ can be computed as:

$$V(G) = E - N + 2$$

where, N is the number of nodes of the control flow graph and E is the number of edges in the control flow graph.

For the CFG of example shown in Figure 10.7, $E = 7$ and $N = 6$. Therefore, the value of the Cyclomatic complexity $= 7 - 6 + 2 = 3$.

Method 2: An alternate way of computing the cyclomatic complexity of a program is based on a visual inspection of the control flow graph as follows—In this method, the cyclomatic complexity $V(G)$ for a graph G is given by the following expression:

$$V(G) = \text{Total number of non-overlapping bounded areas} + 1$$

In the program's control flow graph G , any region enclosed by nodes and edges can be called as a bounded area. This is an easy way to determine the McCabe's cyclomatic complexity. But, what if the graph G is not planar (i.e., however you draw the graph, two or more edges always intersect). Actually, it can be shown that control flow representation of structured programs always yields planar graphs. But, presence of GOTO's can easily add intersecting edges. Therefore, for non-structured programs, this way of computing the McCabe's cyclomatic complexity does not apply.

The number of bounded areas in a CFG increases with the number of decision statements and loops. Therefore, the McCabe's metric provides a quantitative measure of testing difficulty and the ultimate reliability of a program. Consider the CFG example shown in Figure 10.7. From a visual examination of the CFG the number of bounded areas is 2. Therefore the cyclomatic complexity, computed with this method is also $2+1=3$. This method provides a very easy way of computing the cyclomatic complexity of CFGs, just from a visual examination of the CFG. On the other hand, the method for computing CFGs can easily be automated. That is, the McCabe's metric computations methods 1 and 3 can be easily coded into a program

that can be used to automatically determine the cyclomatic complexities of arbitrary programs.

Method 3: The cyclomatic complexity of a program can also be easily computed by computing the number of decision and loop statements of the program. If N is the number of decision and loop statements of a program, then the McCabe's metric is equal to $N + 1$.

How is path testing carried out by using computed McCabe's cyclomatic metric value?

Knowing the number of basis paths in a program does not make it any easier to design test cases for path coverage, only it gives an indication of the minimum number of test cases required for path coverage. For the CFG of a moderately complex program segment of say 20 nodes and 25 edges, you may need several days of effort to identify all the linearly independent paths in it and to design the test cases. It is therefore impractical to require the test designers to identify all the linearly independent paths in a code, and then design the test cases to force execution along each of the identified paths. In practice, for path testing, usually the tester keeps on forming test cases with random data and executes those until the required coverage is achieved. A testing tool such as a dynamic program analyser (see Section 10.8.2) is used to determine the percentage of linearly independent paths covered by the test cases that have been executed so far. If the percentage of linearly independent paths covered is below 90 per cent, more test cases (with random inputs) are added to increase the path coverage. Normally, it is not practical to target achievement of 100 per cent path coverage, since, the McCabe's metric is only an upper bound and does not give the exact number of paths.

Steps to carry out path coverage-based testing

The following is the sequence of steps that need to be undertaken for deriving the path coverage-based test cases for a program:

1. Draw control flow graph for the program.
2. Determine the McCabe's metric $V(G)$.
3. Determine the cyclomatic complexity. This gives the minimum number of test cases required to achieve path coverage.
4. repeat

Test using a randomly designed set of test cases.
Perform dynamic analysis to check the path coverage achieved.
until at least 90 per cent path coverage is achieved.

Uses of McCabe's cyclomatic complexity metric

Beside its use in path testing, cyclomatic complexity of programs has many other interesting applications such as the following:

Estimation of structural complexity of code: McCabe's cyclomatic complexity is a measure of the structural complexity of a program. The reason for this is that it is computed based on the code structure (number of decision and iteration constructs used). Intuitively, the McCabe's complexity metric correlates with the difficulty level of understanding a program, since one understands a program by understanding the computations carried out along all independent paths of the program.

Cyclomatic complexity of a program is a measure of the psychological complexity or the level of difficulty in understanding the program.

In view of the above result, from the maintenance perspective, it makes good sense to limit the cyclomatic complexity of the different functions to some reasonable value. Good software development organisations usually restrict the cyclomatic complexity of different functions to a maximum value of ten or so. This is in contrast to the computational complexity that is based on the execution of the program statements.

Estimation of testing effort: Cyclomatic complexity is a measure of the maximum number of basis paths. Thus, it indicates the minimum number of test cases required to achieve path coverage. Therefore, the testing effort and the time required to test a piece of code satisfactorily is proportional to the cyclomatic complexity of the code. To reduce testing effort, it is necessary to restrict the cyclomatic complexity of every function to seven.

Estimation of program reliability: Experimental studies indicate there exists a clear relationship between the McCabe's metric and the number of errors latent in the code after testing. This relationship exists possibly due to the correlation of cyclomatic complexity with the structural complexity of code. Usually the larger is the structural complexity, the more difficult it is to test and debug the code.

10.7.7 Data Flow-based Testing

Data flow based testing method selects test paths of a program

according to the definitions and uses of different variables in a program. Consider a program P . For a statement numbered S of P , let

$DEF(S) = \{X / \text{statement } S \text{ contains a definition of } X \}$ and

$USES(S) = \{X / \text{statement } S \text{ contains a use of } X \}$

For the statement S: $a=b+c$;; $DEF(S)=\{a\}$, $USES(S)=\{b, c\}$. The definition of variable X at statement S is said to be live at statement S1 , if there exists a path from statement S to statement S1 which does not contain any definition of X .

All definitions criterion is a test coverage criterion that requires that an adequate test set should cover all definition occurrences in the sense that, for each definition occurrence, the testing paths should cover a path through which the definition reaches a use of the definition. All use criterion requires that all uses of a definition should be covered. Clearly, all-uses criterion is stronger than all-definitions criterion. An even stronger criterion is all definition-use-paths criterion, which requires the coverage of all possible definition-use paths that either are cycle-free or have only simple cycles. A simple cycle is a path in which only the end node and the start node are the same.

The definition-use chain (or DU chain) of a variable X is of the form $[X, S, S1]$, where S and S1 are statement numbers, such that $X \in DEF(S)$ and $X \in USES(S1)$, and the definition of X in the statement S is live at statement S1 . One simple data flow testing strategy is to require that every DU chain be covered at least once. Data flow testing strategies are especially useful for testing programs containing nested if and loop statements.

10.7.8 Mutation Testing

All white-box testing strategies that we have discussed so far, are coverage-based testing techniques. In contrast, mutation testing is a fault-based testing technique in the sense that mutation test cases are designed to help detect specific types of faults in a program. In mutation testing, a program is first tested by using an initial test suite designed by using various white box testing strategies that we have discussed. After the initial testing is complete, mutation testing can be taken up.

The idea behind mutation testing is to make a few arbitrary changes to a program at a time. Each time the program is changed, it is called a mutated program and the change effected is called a mutant. An underlying assumption behind mutation testing is that all programming errors can be

expressed as a combination of simple errors. A mutation operator makes specific changes to a program. For example, one mutation operator may randomly delete a program statement. A mutant may or may not cause an error in the program. If a mutant does not introduce any error in the program, then the original program and the mutated program are called equivalent programs.

A mutated program is tested against the original test suite of the program. If there exists at least one test case in the test suite for which a mutated program yields an incorrect result, then the mutant is said to be dead, since the error introduced by the mutation operator has successfully been detected by the test suite. If a mutant remains alive even after all the test cases have been exhausted, the test suite is enhanced to kill the mutant. However, it is not this straightforward. Remember that there is a possibility of a mutated program to be an equivalent program. When this is the case, it is futile to try to design a test case that would identify the error.

An important advantage of mutation testing is that it can be automated to a great extent. The process of generation of mutants can be automated by predefining a set of primitive changes that can be applied to the program. These primitive changes can be simple program alterations such as—deleting a statement, deleting a variable definition, changing the type of an arithmetic operator (e.g., + to -), changing a logical operator (`and` to `or`) changing the value of a constant, changing the data type of a variable, etc. A major pitfall of the mutation-based testing approach is that it is computationally very expensive, since a large number of possible mutants can be generated.

Mutation testing involves generating a large number of mutants. Also each mutant needs to be tested with the full test suite. Obviously therefore, mutation testing is not suitable for manual testing. Mutation testing is most suitable to be used in conjunction of some testing tool that should automatically generate the mutants and run the test suite automatically on each mutant. At present, several test tools are available that automatically generate mutants for a given program.

10.8 DEBUGGING

After a failure has been detected, it is necessary to first identify the program statement(s) that are in error and are responsible for the failure, the error can then be fixed. In this Section, we shall summarise the important approaches that are available to identify the error locations. Each of these approaches has its own advantages and

disadvantages and therefore each will be useful in appropriate circumstances. We also provide some guidelines for effective debugging.

10.8.1 Debugging Approaches

The following are some of the approaches that are popularly adopted by the programmers for debugging:

Brute force method

This is the most common method of debugging but is the least efficient method. In this approach, print statements are inserted throughout the program to print the intermediate values with the hope that some of the printed values will help to identify the statement in error. This approach becomes more systematic with the use of a symbolic debugger (also called a source code debugger), because values of different variables can be easily checked and break points and watch points can be easily set to test the values of variables effortlessly. Single stepping using a symbolic debugger is another form of this approach, where the developer mentally computes the expected result after every source instruction and checks whether the same is computed by single stepping through the program.

Backtracking

This is also a fairly common approach. In this approach, starting from the statement at which an error symptom has been observed, the source code is traced backwards until the error is discovered. Unfortunately, as the number of source lines to be traced back increases, the number of potential backward paths increases and may become unmanageably large for complex programs, limiting the use of this approach.

Cause elimination method

In this approach, once a failure is observed, the symptoms of the failure (i.e., certain variable is having a negative value though it should be positive, etc.) are noted. Based on the failure symptoms, the causes which could possibly have contributed to the symptom is developed and tests are conducted to eliminate each. A related technique of identification of the error from the error symptom is the software fault tree analysis.

Program slicing

This technique is similar to back tracking. In the backtracking approach, one often has to examine a large number of statements. However, the search space is reduced by defining slices. A slice of a program for a particular variable and at a particular statement is the set of source lines preceding this statement that can influence the value of that variable [Mund2002]. Program slicing makes use of the fact that an error in the value of a variable can be caused by the statements on which it is data dependent.

10.8.2 Debugging Guidelines

Debugging is often carried out by programmers based on their ingenuity and experience. The following are some general guidelines for effective debugging:

- Many times debugging requires a thorough understanding of the program design. Trying to debug based on a partial understanding of the program design may require an inordinate amount of effort to be put into debugging even for simple problems.
- Debugging may sometimes even require full redesign of the system. In such cases, a common mistakes that novice programmers often make is attempting not to fix the error but its symptoms.
- One must be beware of the possibility that an error correction may introduce new errors. Therefore after every round of error-fixing, regression testing (see Section 10.13) must be carried out.

10.9 PROGRAM ANALYSIS TOOLS

A program analysis tool usually is an automated tool that takes either the source code or the executable code of a program as input and produces reports regarding several important characteristics of the program, such as its size, complexity, adequacy of commenting, adherence to programming standards, adequacy of testing, etc. We can classify various program analysis tools into the following two broad categories:

- Static analysis tools
- Dynamic analysis tools

These two categories of program analysis tools are discussed in the following subsection.

10.9.1 Static Analysis Tools

Static program analysis tools assess and compute various characteristics of a program without executing it. Typically, static analysis tools analyse the source code to compute certain metrics characterising the source code (such as size, cyclomatic complexity, etc.) and also report certain analytical conclusions. These also check the conformance of the code with the prescribed coding standards. In this context, it displays the following analysis results:

- To what extent the coding standards have been adhered to?
- Whether certain programming errors such as uninitialised variables, mismatch between actual and formal parameters, variables that are declared but never used, etc., exist? A list of all such errors is displayed.

Code review techniques such as code walkthrough and code inspection discussed in Sections 10.2.1 and 10.2.2 can be considered as static analysis methods since those target to detect errors based on analysing the source code. However, strictly speaking, this is not true since we are using the term static program analysis to denote automated analysis tools. On the other hand, a compiler can be considered to be a type of a static program analysis tool.

A major practical limitation of the static analysis tools lies in their inability to analyse run-time information such as dynamic memory references using pointer variables and pointer arithmetic, etc. In a high level programming languages, pointer variables and dynamic memory allocation provide the capability for dynamic memory references. However, dynamic memory referencing is a major source of programming errors in a program.

Static analysis tools often summarise the results of analysis of every function in a polar chart known as Kiviat Chart. A Kiviat Chart typically shows the analysed values for cyclomatic complexity, number of source lines, percentage of comment lines, Halstead's metrics, etc.

10.9.2 Dynamic Analysis Tools

Dynamic program analysis tools can be used to evaluate several program

characteristics based on an analysis of the run time behaviour of a program. These tools usually record and analyse the actual behaviour of a program while it is being executed. A dynamic program analysis tool (also called a dynamic analyser) usually collects execution trace information by instrumenting the code. Code instrumentation is usually achieved by inserting additional statements to print the values of certain variables into a file to collect the execution trace of the program. The instrumented code when executed, records the behaviour of the software for different test cases.

An important characteristic of a test suite that is computed by a dynamic analysis tool is the extent of coverage achieved by the test suite.

After a software has been tested with its full test suite and its behaviour recorded, the dynamic analysis tool carries out a post execution analysis and produces reports which describe the coverage that has been achieved by the complete test suite for the program. For example, the dynamic analysis tool can report the statement, branch, and path coverage achieved by a test suite. If the coverage achieved is not satisfactory more test cases can be designed, added to the test suite, and run. Further, dynamic analysis results can help eliminate redundant test cases from a test suite.

Normally the dynamic analysis results are reported in the form of a histogram or pie chart to describe the structural coverage achieved for different modules of the program. The output of a dynamic analysis tool can be stored and printed easily to provide evidence that thorough testing has been carried out.

10.10 INTEGRATION TESTING

Integration testing is carried out after all (or at least some of) the modules have been unit tested. Successful completion of unit testing, to a large extent, ensures that the unit (or module) as a whole works satisfactorily. In this context, the objective of integration testing is to detect the errors at the module interfaces (call parameters). For example, it is checked that no parameter mismatch occurs when one module invokes the functionality of another module. Thus, the primary objective of integration testing is to test the module interfaces, i.e., there are no errors in parameter passing, when one module invokes the functionality of another module.

The objective of integration testing is to check whether the different modules of a program interface with each other properly.

During integration testing, different modules of a system are integrated in a planned manner using an integration plan. The integration plan specifies the steps and the order in which modules are combined to realise the full system. After each integration step, the partially integrated system is tested.

An important factor that guides the integration plan is the module dependency graph.

We have already discussed in Chapter 6 that a structure chart (or module dependency graph) specifies the order in which different modules call each other. Thus, by examining the structure chart, the integration plan can be developed. Any one (or a mixture) of the following approaches can be used to develop the test plan:

- Big-bang approach to integration testing
- Top-down approach to integration testing
- Bottom-up approach to integration testing
- Mixed (also called sandwiched) approach to integration testing

In the following subsections, we provide an overview of these approaches to integration testing.

Big-bang approach to integration testing

Big-bang testing is the most obvious approach to integration testing. In this approach, all the modules making up a system are integrated in a single step. In simple words, all the unit tested modules of the system are simply linked together and tested. However, this technique can meaningfully be used only for very small systems. The main problem with this approach is that once a failure has been detected during integration testing, it is very difficult to localise the error as the error may potentially lie in any of the modules. Therefore, debugging errors reported during big-bang integration testing are very expensive to fix. As a result, big-bang integration testing is almost never used for large programs.

Bottom-up approach to integration testing

Large software products are often made up of several subsystems. A subsystem might consist of many modules which communicate among each other through well-defined interfaces. In bottom-up integration testing, first the modules for the each subsystem are integrated. Thus, the subsystems can be integrated separately and independently.

The primary purpose of carrying out the integration testing a subsystem is to test whether the interfaces among various modules making up the subsystem work satisfactorily. The test cases must be carefully chosen to exercise the interfaces in all possible manners.

In a pure bottom-up testing no stubs are required, and only test-drivers are required. Large software systems normally require several levels of subsystem testing, lower-level subsystems are successively combined to form higher-level subsystems. The principal advantage of bottom-up integration testing is that several disjoint subsystems can be tested simultaneously. Another advantage of bottom-up testing is that the low-level modules get tested thoroughly, since they are exercised in each integration step. Since the low-level modules do I/O and other critical functions, testing the low-level modules thoroughly increases the reliability of the system. A disadvantage of bottom-up testing is the complexity that occurs when the system is made up of a large number of small subsystems that are at the same level. This extreme case corresponds to the big-bang approach.

Top-down approach to integration testing

Top-down integration testing starts with the root module in the structure chart and one or two subordinate modules of the root module. After the top-level 'skeleton' has been tested, the modules that are at the immediately lower layer of the 'skeleton' are combined with it and tested. Top-down integration testing approach requires the use of program stubs to simulate the effect of lower-level routines that are called by the routines under test. A pure top-down integration does not require any driver routines. An advantage of top-down integration testing is that it requires writing only stubs, and stubs are simpler to write compared to drivers. A disadvantage of the top-down integration testing approach is that in the absence of lower-level routines, it becomes difficult to exercise the top-level routines in the desired manner since the lower level routines usually perform input/output (I/O) operations.

Mixed approach to integration testing

The mixed (also called sandwiched) integration testing follows a combination of top-down and bottom-up testing approaches. In top-down approach, testing can start only after the top-level modules have been coded and unit tested. Similarly, bottom-up testing can start only

after the bottom level modules are ready. The mixed approach overcomes this shortcoming of the top-down and bottom-up approaches. In the mixed testing approach, testing can start as and when modules become available after unit testing. Therefore, this is one of the most commonly used integration testing approaches. In this approach, both stubs and drivers are required to be designed.

10.10.1 Phased versus Incremental Integration Testing

Big-bang integration testing is carried out in a single step of integration. In contrast, in the other strategies, integration is carried out over several steps. In these later strategies, modules can be integrated either in a phased or incremental manner. A comparison of these two strategies is as follows:

- In incremental integration testing, only one new module is added to the partially integrated system each time.
- In phased integration, a group of related modules are added to the partial system each time.

Obviously, phased integration requires less number of integration steps compared to the incremental integration approach. However, when failures are detected, it is easier to debug the system while using the incremental testing approach since the errors can easily be traced to the interface of the recently integrated module. Please observe that a degenerate case of the phased integration testing approach is big-bang testing.

10.11 TESTING OBJECT-ORIENTED PROGRAMS

During the initial years of object-oriented programming, it was believed that object-orientation would, to a great extent, reduce the cost and effort incurred on testing. This thinking was based on the observation that object-orientation incorporates several good programming features such as encapsulation, abstraction, reuse through inheritance, polymorphism, etc., thereby chances of errors in the code is minimised. However, it was soon realised that satisfactory testing object-oriented programs is much more difficult and requires much more cost and effort as compared to testing similar procedural programs. The main reason behind this situation is that various object-oriented features introduce additional complications and scope of new types of bugs that are

present in procedural programs. Therefore additional test cases are needed to be designed to detect these. We examine these issues as well as some other basic issues in testing object-oriented programs in the following subsections.

10.11.1 What is a Suitable Unit for Testing

Object-oriented Programs?

For procedural programs, we had seen that procedures are the basic units of testing. That is, first all the procedures are unit tested. Then various tested procedures are integrated together and tested. Thus, as far as procedural programs are concerned, procedures are the basic units of testing. Since methods in an object-oriented program are analogous to procedures in a procedural program, can we then consider the methods of object-oriented programs as the basic unit of testing? Weyuker studied this issue and postulated his anticomposition axiom as follows:

Adequate testing of individual methods does not ensure that a class has been satisfactorily tested.

The main intuitive justification for the anticomposition axiom is the following. A method operates in the scope of the data and other methods of its object. That is, all the methods share the data of the class. Therefore, it is necessary to test a method in the context of these. Moreover, objects can have significant number of states. The behaviour of a method can be different based on the state of the corresponding object. Therefore, it is not enough to test all the methods and check whether they can be integrated satisfactorily. A method has to be tested with all the other methods and data of the corresponding object. Moreover, a method needs to be tested at all the states that the object can assume. As a result, it is improper to consider a method as the basic unit of testing an object-oriented program.

An object is the basic unit of testing of object-oriented programs.

Thus, in an object oriented program, unit testing would mean testing each object in isolation. During integration testing (called cluster testing in the object-oriented testing literature) various unit tested objects are integrated and tested. Finally, system-level testing is carried out.

10.11.2 Do Various Object-orientation Features Make Testing Easy?

In this section, we discuss the implications of different object-orientation features in testing.

Encapsulation: We had discussed in Chapter 7 that the encapsulation feature helps in data abstraction, error isolation, and error prevention. However, as far as testing is concerned, encapsulation is not an obstacle to testing, but leads to difficulty during debugging. Encapsulation prevents the tester from accessing the data internal to an object. Of course, it is possible that one can require classes to support state reporting methods to print out all the data internal to an object. Thus, the encapsulation feature though makes testing difficult, the difficulty can be overcome to some extent through use of appropriate state reporting methods.

Inheritance: The inheritance feature helps in code reuse and was expected to simplify testing. It was expected that if a class is tested thoroughly, then the classes that are derived from this class would need only incremental testing of the added features. However, this is not the case.

Even if the base class class has been thoroughly tested, the methods inherited from the base class need to be tested again in the derived class.

The reason for this is that the inherited methods would work in a new context (new data and method definitions). As a result, correct behaviour of a method at an upper level, does not guarantee correct behaviour at a lower level. Therefore, retesting of inherited methods needs to be followed as a rule, rather as an exception.

Dynamic binding: Dynamic binding was introduced to make the code compact, elegant, and easily extensible. However, as far as testing is concerned all possible bindings of a method call have to be identified and tested. This is not easy since the bindings take place at run-time.

Object states: In contrast to the procedures in a procedural program, objects store data permanently. As a result, objects do have significant states. The behaviour of an object is usually different in different states. That is, some methods may not be active in some of its states. Also, a method may act differently in different states. For example, when a book has been issued out in a library information system, the book reaches the issuedOut state. In this state, if the issue method is invoked, then it may not exhibit its normal behaviour.

In view of the discussions above, testing an object in only one of its states is not enough. The object has to be tested at all its possible states. Also,

whether all the transitions between states (as specified in the object model) function properly or not should be tested. Additionally, it needs to be tested that no extra (sneak) transitions exist, neither are there extra states present other than those defined in the state model. For state-based testing, it is therefore beneficial to have the state model of the objects, so that the conformance of the object to its state model can be tested.

10.11.3 Why are Traditional Techniques Considered Not Satisfactory for Testing Object-oriented Programs?

We have already seen that in traditional procedural programs, procedures are the basic unit of testing. In contrast, objects are the basic unit of testing for object-oriented programs. Besides this, there are many other significant differences as well between testing procedural and object-oriented programs. For example, statement coverage-based testing which is popular for testing procedural programs is not meaningful for object-oriented programs. The reason is that inherited methods have to be retested in the derived class. In fact, the different object-oriented features (inheritance, polymorphism, dynamic binding, state-based behaviour, etc.) require special test cases to be designed compared to the traditional testing as discussed in Section 10.11.4. The various object-orientation features are explicit in the design models, and it is usually difficult to extract from and analysis of the source code. As a result, the design model is a valuable artifact for testing object-oriented programs. Test cases are designed based on the design model. Therefore, this approach is considered to be intermediate between a fully white-box and a fully black-box approach, and is called a grey-box approach. Please note that grey-box testing is considered important for object-oriented programs. This is in contrast to testing procedural programs.

10.11.4 Grey-Box Testing of Object-oriented Programs

As we have already mentioned, model-based testing is important for object-oriented programs, as these test cases help detect bugs that are specific to the object-orientation constructs.

For object-oriented programs, several types of test cases can be designed based on the design models of object-oriented programs. These are called the grey-box test cases.

The following are some important types of grey-box testing that can be carried on based on UML models:

State-model-based testing

State coverage: Each method of an object are tested at each state of the object.

State transition coverage: It is tested whether all transitions depicted in the state model work satisfactorily.

State transition path coverage: All transition paths in the state model are tested.

Use case-based testing

Scenario coverage: Each use case typically consists of a mainline scenario and several alternate scenarios. For each use case, the mainline and all alternate sequences are tested to check if any errors show up.

Class diagram-based testing

Testing derived classes: All derived classes of the base class have to be instantiated and tested. In addition to testing the new methods defined in the derived class, the inherited methods must be retested.

Association testing: All association relations are tested.

Aggregation testing: Various aggregate objects are created and tested.

Sequence diagram-based testing

Method coverage: All methods depicted in the sequence diagrams are covered. **Message path coverage:** All message paths that can be constructed from the sequence diagrams are covered.

10.11.5 Integration Testing of Object-oriented Programs

There are two main approaches to integration testing of object-oriented programs:

- Thread-based
- Use based

Thread-based approach: In this approach, all classes that need to collaborate to realise the behaviour of a single use case are integrated and tested. After all the required classes for a use case are integrated and tested,

another use case is taken up and other classes (if any) necessary for execution of the second use case to run are integrated and tested. This is continued till all use cases have been considered.

Use-based approach: Use-based integration begins by testing classes that either need no service from other classes or need services from at most a few other classes. After these classes have been integrated and tested, classes that use the services from the already integrated classes are integrated and tested. This is continued till all the classes have been integrated and tested.

10.12 SYSTEM TESTING

After all the units of a program have been integrated together and tested, system testing is taken up.

System tests are designed to validate a fully developed system to assure that it meets its requirements. The test cases are therefore designed solely based on the SRS document.

The system testing procedures are the same for both object-oriented and procedural programs, since system test cases are designed solely based on the SRS document and the actual implementation (procedural or object-oriented) is immaterial.

There are essentially three main kinds of system testing depending on who carries out testing:

1. **Alpha Testing:** Alpha testing refers to the system testing carried out by the test team within the developing organisation.
2. **Beta Testing:** Beta testing is the system testing performed by a select group of friendly customers.
3. **Acceptance Testing:** Acceptance testing is the system testing performed by the customer to determine whether to accept the delivery of the system.

In each of the above types of system tests, the test cases can be the same, but the difference is with respect to who designs test cases and carries out testing.

The system test cases can be classified into functionality and performance test cases.

Before a fully integrated system is accepted for system testing, smoke testing is performed. Smoke testing is done to check whether at least the

main functionalities of the software are working properly. Unless the software is stable and at least the main functionalities are working satisfactorily, system testing is not undertaken.

The functionality tests are designed to check whether the software satisfies the functional requirements as documented in the SRS document. The performance tests, on the other hand, test the conformance of the system with the non-functional requirements of the system. We have already discussed how to design the functionality test cases by using a black-box approach (in Section 10.5 in the context of unit testing). So, in the following subsection we discuss only smoke and performance testing.

10.12.1 Smoke Testing

Smoke testing is carried out before initiating system testing to ensure that system testing would be meaningful, or whether many parts of the software would fail. The idea behind smoke testing is that if the integrated program cannot pass even the basic tests, it is not ready for a vigorous testing. For smoke testing, a few test cases are designed to check whether the basic functionalities are working. For example, for a library automation system, the smoke tests may check whether books can be created and deleted, whether member records can be created and deleted, and whether books can be loaned and returned.

10.12.2 Performance Testing

Performance testing is an important type of system testing.

Performance testing is carried out to check whether the system meets the non-functional requirements identified in the SRS document.
--

There are several types of performance testing corresponding to various types of non-functional requirements. For a specific system, the types of performance testing to be carried out on a system depends on the different non-functional requirements of the system documented in its SRS document. All performance tests can be considered as black-box tests.

Stress testing

Stress testing is also known as endurance testing. Stress testing evaluates system performance when it is stressed for short periods of time. Stress tests are black-box tests which are designed to impose a range of abnormal and even illegal input conditions so as to stress the

capabilities of the software. Input data volume, input data rate, processing time, utilisation of memory, etc., are tested beyond the designed capacity. For example, suppose an operating system is supposed to support fifteen concurrent transactions, then the system is stressed by attempting to initiate fifteen or more transactions simultaneously. A real-time system might be tested to determine the effect of simultaneous arrival of several high-priority interrupts.

Stress testing is especially important for systems that under normal circumstances operate below their maximum capacity but may be severely stressed at some peak demand hours. For example, if the corresponding non-functional requirement states that the response time should not be more than twenty secs per transaction when sixty concurrent users are working, then during stress testing the response time is checked with exactly sixty users working simultaneously.

Volume testing

Volume testing checks whether the data structures (buffers, arrays, queues, stacks, etc.) have been designed to successfully handle extraordinary situations. For example, the volume testing for a compiler might be to check whether the symbol table overflows when a very large program is compiled.

Configuration testing

Configuration testing is used to test system behaviour in various hardware and software configurations specified in the requirements. Sometimes systems are built to work in different configurations for different users. For instance, a minimal system might be required to serve a single user, and other extended configurations may be required to serve additional users during configuration testing. The system is configured in each of the required configurations and depending on the specific customer requirements, it is checked if the system behaves correctly in all required configurations.

Compatibility testing

This type of testing is required when the system interfaces with external systems (e.g., databases, servers, etc.). Compatibility aims to check whether the interfaces with the external systems are performing as required. For instance, if the system needs to communicate with a large

database system to retrieve information, compatibility testing is required to test the speed and accuracy of data retrieval.

Regression testing

This type of testing is required when a software is maintained to fix some bugs or enhance functionality, performance, etc. Regression testing is also discussed in Section 10.13.

Recovery testing

Recovery testing tests the response of the system to the presence of faults, or loss of power, devices, services, data, etc. The system is subjected to the loss of the mentioned resources (as discussed in the SRS document) and it is checked if the system recovers satisfactorily. For example, the printer can be disconnected to check if the system hangs. Or, the power may be shut down to check the extent of data loss and corruption.

Maintenance testing

This addresses testing the diagnostic programs, and other procedures that are required to help maintenance of the system. It is verified that the artifacts exist and they perform properly.

Documentation testing

It is checked whether the required user manual, maintenance manuals, and technical manuals exist and are consistent. If the requirements specify the types of audience for which a specific manual should be designed, then the manual is checked for compliance of this requirement.

Usability testing

Usability testing concerns checking the user interface to see if it meets all user requirements concerning the user interface. During usability testing, the display screens, messages, report formats, and other aspects relating to the user interface requirements are tested. A GUI being just being functionally correct is not enough. Therefore, the GUI has to be checked against the checklist we discussed in Sec. 9.5.6.

Security testing

Security testing is essential for software that handle or process confidential data that is to be guarded against pilfering. It needs to be tested whether the system is fool-proof from security attacks such as intrusion by hackers. Over the last few years, a large number of security testing techniques have been proposed, and these include password cracking, penetration testing, and attacks on specific ports, etc.

10.12.3 Error Seeding

Sometimes customers specify the maximum number of residual errors that can be present in the delivered software. These requirements are often expressed in terms of maximum number of allowable errors per line of source code. The error seeding technique can be used to estimate the number of residual errors in a software.

Error seeding, as the name implies, it involves seeding the code with some known errors. In other words, some artificial errors are introduced (seeded) into the program. The number of these seeded errors that are detected in the course of standard testing is determined. These values in conjunction with the number of unseeded errors detected during testing can be used to predict the following aspects of a program:

- The number of errors remaining in the product.
- The effectiveness of the testing strategy.

Let N be the total number of defects in the system, and let n of these defects be found by testing.

Let S be the total number of seeded defects, and let s of these defects be found during testing. Therefore, we get:

$$\frac{n}{N} = \frac{s}{S}$$

or

$$N = S \times \frac{n}{s}$$

Defects still remaining in the program after testing can be given by:

$$N - n = n \times \frac{(S - 1)}{s}$$

Error seeding works satisfactorily only if the kind seeded errors and their frequency of occurrence matches closely with the kind of defects that actually exist. However, it is difficult to predict the types of errors that exist in a software. To some extent, the different categories of errors that are latent